

키워드

1. 시제품을 끊임없이 제작하며 사이클을 반복하는 개발 방법론으로, 워터폴과 대조적이며, 소프트웨어 개발을 넘어 기업 경영 전반에서 사용되고 있다. 고객의 변화하는 요구사항과 환경 변화에 능동적인 이 소프트웨어 개발 방법론을 쓰시오. [20-2](#)

답: 애자일(Agile)

2. 소프트웨어 공학에서 리팩토링(Refactoring)을 하는 목적에 대해 간략히 서술하시오. [20-3](#)

답: 리팩토링의 목적은 프로그램을 쉽게 이해하고 수정하여 빠르게 개발할 수 있도록 하기 위함이다.

3. 요구사항 확인에 대한 다음 설명에서 괄호(①, ②)에 들어갈 알맞은 용어를 쓰시오. [21-1](#)

- (①) 요구사항은 시스템이 무엇을 하는지, 어떤 기능을 하는지 등 사용자가 시스템을 통해 제공받기를 원하는 기능이나 시스템이 반드시 수행해야 하는 기능을 의미한다.
- (②) 요구사항은 품질이나 제약사항과 관련된 요구사항으로, 시스템의 장비 구성, 성능, 인터페이스, 테스트, 보안 등의 요구사항을 말한다.

답: ① 기능 ② 비기능

4. UML(Unified Modeling Language)에 관한 다음 설명에서 괄호에 공통으로 들어갈 알맞은 용어를 쓰시오. [21-3](#)

() 다이어그램은 UML 다이어그램 중 객체(Object)들을 ()로 추상화하여 표현하는 다이어그램으로 대표적인 구조적 다이어그램이다. ()는 각각의 객체들이 갖는 속성과 메소드를 표현한 것으로 3개의 구획으로 나뉘 이름, 속성, 메소드를 표기한다.

답: 클래스(Class)

5. UML에 대한 다음 설명에서 괄호(①~③)에 들어갈 알맞은 용어를 쓰시오. [22-3](#)

UML은 시스템 분석, 설계, 구현 등 시스템 개발 과정에서 시스템 개발자와 고객 또는 개발자 상호 간의 의사소통이 원활하게 이루어지도록 표준화한 대표적인 객체지향 모델링 언어로, 사물, (①), 다이어그램으로 이루어져 있다.

- (①)는 사물과 사물 사이의 연관성을 표현하는 것으로, 연관, 집합, 포함, 일반화 등 다양한 형태의 (①)가 존재한다.
- (②)는 UML에 표현되는 사물의 하나로, 객체가 갖는 속성과 동작을 표현한다. 일반적으로 직사각형으로 표현하며, 직사각형 안에 이름, 속성, 동작을 표기한다.
- (③)는 (②)와 같은 UML에 표현되는 사물의 하나로, (②)나 컴포넌트의 동작을 모아놓은 것이며, 외부적으로 가시화되는 행동을 표현한다. 단독으로 사용되는 경우는 없으며, (③) 구현을 위한 (②) 또는 컴포넌트와 함께 사용된다.

답: ① 관계(Relationship) ② 클래스(Class) ③ 인터페이스(Interface)

6. UML의 관계(Relationships)에 관한 다음 설명에서 각 번호(①, ②)에 들어갈 알맞은 용어를 <보기>에서 찾아 쓰시오. [21-3](#)

관계(Relationships)는 사물과 사물 사이의 연관성을 표현하는 것이다.

① - 하나의 사물이 다른 사물에 포함되어 있는 관계로, 전체와 부분으로 구분되어지며 서로 독립적이다.

② - 상위 모듈이 하위 모듈보다 더 일반적인 개념을 가지고 있으며, 하위 모듈이 상위 모듈보다 더 구체적인 개념을 가진다.

<보기>

- | | | |
|------------------|---------------|---------------|
| • Association | • Aggregation | • Composition |
| • Generalization | • Dependency | • Realization |

답: ① **Aggregation** ② **Generalization**

[해설]

① **Aggregation (집합 관계)**: 한 객체가 다른 객체를 포함하는 '전체와 부분(Whole-Part)'의 관계를 나타냅니다. 전체 객체와 부분 객체는 서로 독립적인 생명 주기를 가집니다. 즉, 전체 객체가 소멸하더라도 부분 객체는 사라지지 않고 독립적으로 존재할 수 있습니다. (예: 컴퓨터와 마우스, 프린터의 관계)

반면, <보기>에 있는 *Composition(합성 관계)*은 부분 객체가 전체 객체에 완전히 종속되어 생명 주기를 함께하는 강한 결합 관계를 뜻하므로 문맥상 '서로 독립적이다'라는 설명에는 Aggregation이 올바릅니다.

② **Generalization (일반화 관계)**: 하나의 사물이 다른 사물에 비해 더 일반적인 개념인지 구체적인 개념인지를 나타내는 관계입니다. 흔히 객체 지향의 '상속(Inheritance)' 관계를 의미합니다. 상위 모듈(부모 클래스)은 하위 모듈보다 더 추상적이고 일반적인 개념(예: 자동차)을 가지며, 하위 모듈(자식 클래스)은 상위 모듈보다 더 구체적인 개념(예: 전기차, 스포츠카)을 가집니다. 따라서 설명에 부합하는 용어는 Generalization입니다.

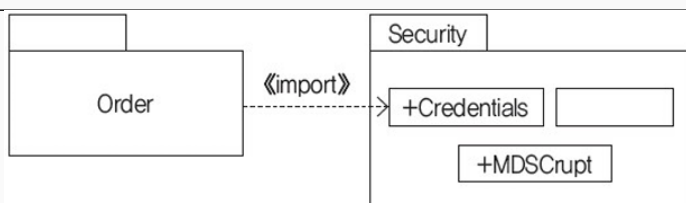
③ **Association (연관 관계)**: 두 개 이상의 객체가 서로 관련되어 있음을 나타내는 가장 일반적인 관계입니다.

④ **Composition (포 관계)**: 집약 관계보다 더 강한 결합으로, 부분이 전체에 완전히 포함되어 생명 주기를 함께하는 관계입니다. (예: 스마트폰과 그 내부 부품)

⑤ **Dependency (의존 관계)**: 한 클래스의 메서드가 다른 클래스의 객체를 매개변수로 받는 등, 짧은 시간 동안만 필요에 의해 다른 클래스를 참조하는 형태의 일시적인 관계입니다.

⑥ **Realization (실체화 관계)**: 인터페이스(Interface)와 이를 실제로 구현한 클래스 간의 관계를 뜻합니다.

7. UML을 이용한 다이어그램 중 다음 그림에 해당하는 다이어그램을 쓰시오. [20-4](#), [23-3](#), [25-3](#)



() 다이어그램

()은(는) UML 정적 모델링의 하나로, 관련있는 객체들을 하나로 묶어 상위 개념으로 추상화한 것이다. 위의 그림과 같이 유스케이스나 클래스 등의 요소들을 그룹화하여 의존 관계를 표현하며, 대규모 시스템에서 주요 요소 간의 종속성을 파악하는 데 사용한다. 시스템의 구조를 간략하게 표현할 수 있고 의존 관계를 명확하게 파악할 수 있어, 불필요한 의존 관계를 제거하거나 간략화함으로써 시스템의 복잡도를 낮추는 곳에도 사용할 수 있다.

답: **패키지 다이어그램(Package Diagram)**

8. LOC 기법에 의하여 예측된 총 라인 수가 30,000라인, 개발에 참여할 프로그래머 5명, 프로그래머들의 평균 생산성이 월간 300라인일 때 개발에 소요되는 기간을 계산식과 함께 쓰시오. [20-1](#)

답:

- 계산식 : $(30,000 / 300) / 5 = 20$
- 답 : 20개월

[해설] LOC 기법에서 개발 기간은 '노력(인월) / 투입 인원'이고, 노력(인월)은 'LOC(총 라인 수) / 1인당 월평균 생산 코드 라인 수'이므로 $(LOC / 1인당 월평균 생산 코드 라인 수) / 투입 인원$ 에 값을 대입하여 답을 구할 수 있습니다.

9. 데이터베이스의 스키마(Schema)에 대해 간략히 서술하시오. [20-3](#)

답: 스키마는 데이터베이스의 구조와 제약 조건에 관한 전반적인 명세를 기술한 것이다.

10. 데이터베이스 설계에 대한 다음 설명에서 괄호(①~③)에 들어갈 알맞은 용어를 쓰시오. [21-1](#)

1. (①): 논리적 구조로 표현된 데이터를 디스크 등의 저장장치에 저장할 수 있는 데이터로 변환하는 과정으로, 파일의 저장 구조 및 액세스 경로를 결정하며, 테이블 정의서 및 명세서가 산출된다.
2. (②): 현실 세계에 대한 인식을 추상적 개념으로 표현하는 과정으로, 개념 스키마 모델링과 트랜잭션 모델링을 수행하며, 요구 조건 명세를 E-R 다이어그램으로 작성한다.
3. (③): 현실의 자료를 특정 DBMS가 지원하는 자료구조로 변환하는 과정으로, 트랜잭션의 인터페이스를 설계하고, 정규화를 통해 스키마를 평가 및 정제한다.

답: ① 물리적 설계 ② 개념적 설계 ③ 논리적 설계

11. 다음은 데이터베이스 구축까지의 과정을 나열한 것이다. 괄호(①~⑤)에 들어갈 알맞은 용어를 <보기>에서 찾아 쓰시오. [20-2, 23-2, 26-1](#)



<보기>

- | | | | |
|------------|-------------|----------|------------|
| • 개념적 설계 | • 데이터베이스 구현 | • 물리적 설계 | • 요구 조건 분석 |
| • 인터페이스 설계 | • 논리 스키마 설계 | • 논리적 설계 | • 트랜잭션 작성 |

답: ① 요구 조건 분석 ② 개념적 설계 ③ 논리적 설계 ④ 물리적 설계 ⑤ 데이터베이스 구현

[해설]

데이터베이스 설계(Design) 5단계: 요 -> 개 -> 논 -> 물 -> 구 순서로 진행됩니다.

- 요구 조건 분석: 사용자의 요구사항 수집, 요구조건 명세서 작성
- 개념적 설계: 현실 세계의 개념화, E-R 다이어그램(개념 스키마) 생성, 트랜잭션 모델링
- 논리적 설계: 목표 DBMS에 맞는 파일 구조 매핑, 테이블(릴레이션) 스키마 변환, 정규화 수행
- 물리적 설계: 저장 구조, 테이블 스페이스, 인덱스(Index), 파티션, 접근 경로 등 성능 중심 설계
- 데이터베이스 구현: 선언된 스키마를 특정 DBMS의 DDL(데이터 정의어) 코드로 변환하여 실제 데이터베이스 및 테이블 생성

12. 데이터 모델의 구성 요소에 대한 다음 설명에서 괄호(①, ②)에 들어갈 알맞은 용어를 쓰시오. [21-1](#)

1. (①)은 데이터베이스에 저장된 실제 데이터를 처리하는 작업에 대한 명세서로서 데이터베이스를 조작하는 기본 도구에 해당한다.
2. (②)는 논리적으로 표현된 객체 타입들 간의 관계로서 데이터의 구성 및 정적 성질을 표현한다.
3. 제약 조건은 데이터베이스에 저장될 수 있는 실제 데이터의 논리적인 제약 조건을 의미한다.

답: ① 연산(Operation) ② 구조(Structure)

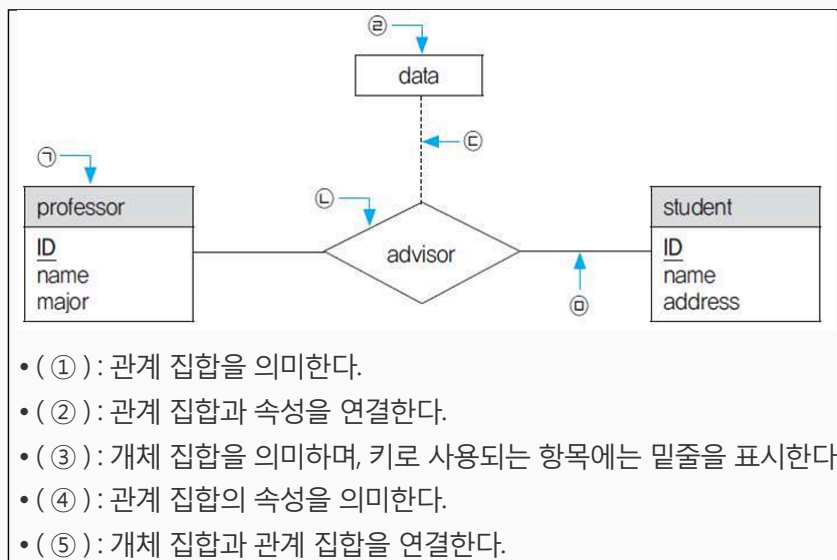
13. 다음 테이블에서 카디널리티(Cardinality)와 디그리(Degree)를 구하시오. [21-1, 24-2](#)

<회원>

ID	이름	거주지	신청강의
191-SR05	백영현	마포구	E01
024-RU09	차수인	관악구	S03
181-SQ03	허채빈	서대문구	E02
059-RL08	윤지호	광진구	S03
029-SX07	배서희	서대문구	E02

답: ① 카디널리티(Cardinality) : 5 ② 디그리(Degree) : 4

14. 다음 E-R 다이어그램을 참고하여 괄호(①~⑤)의 설명에 적합한 요소를 찾아 기호(㉠~㉤)로 쓰시오. [22-3](#)



답: ① ㉠ ② ㉡ ③ ㉢ ④ ㉣ ⑤ ㉤

15. 키(Key)에 대한 다음 설명에서 괄호(①, ②)에 들어갈 알맞은 용어를 쓰시오. [22-1](#)

키(Key)는 데이터베이스에서 조건에 만족하는 튜플을 찾거나 순서대로 정렬할 때 기준이 되는 속성을 말한다.

- 슈퍼키(Super Key)는 한 릴레이션 내에 있는 속성들의 집합으로 구성된 키로, 릴레이션을 구성하는 모든 튜플에 대해 (①)을 만족한다.
- 후보키(Candidate Key)는 릴레이션을 구성하는 속성들 중에서 튜플을 유일하게 식별하기 위해 사용되는 속성들의 부분집합으로, (①)과 (②)을 만족하는 특징이 있다.

답: ① 유일성(Unique) ② 최소성(Minimality)

16. 관계대수에 대한 다음 설명에서 괄호(①~⑤)에 들어갈 알맞은 용어를 쓰시오. [22-3](#)

관계대수는 관계형 데이터베이스에서 원하는 정보와 그 정보를 검색하기 위해서 어떻게 유도하는가를 기술하는 절차적인 언어이다. 관계대수에 사용되는 연산은 다음과 같다.

- 합집합(UNION)은 두 릴레이션에 존재하는 튜플의 합집합을 구하되, 결과로 생성된 릴레이션에서 중복되는 튜플은 제거되는 연산으로, 사용하는 기호는 (①)이다.

- 차집합(DIFFERENCE)은 두 릴레이션에 존재하는 튜플의 차집합을 구하는 연산으로, 사용하는 기호는 (②)이다.
- 교차곱(CARTESIAN PRODUCT)은 두 릴레이션에 있는 튜플들의 순서쌍을 구하는 연산으로, 사용하는 기호는 (③)이다.
- 프로젝트(PROJECT)는 주어진 릴레이션에서 속성 리스트(Attribute List)에 제시된 속성 값만을 추출하여 새로운 릴레이션을 만드는 연산으로, 사용하는 기호는 (④)이다.
- 조인(JOIN)은 공통 속성을 중심으로 두 개의 릴레이션을 하나로 합쳐서 새로운 릴레이션을 만드는 연산으로, 사용하는 기호는 (⑤)이다.

답: ① $\cup$ ② $-$ ③ $\times$ ④ π ⑤ $\bowtie$

17. 다음이 설명하고 있는 관계대수 연산자의 기호를 쓰시오. 20-3

릴레이션 A, B가 있을 때 릴레이션 B의 조건에 맞는 것들만 릴레이션 A에서 분리하여 프로젝션을 하는 연산이다.

답: $\div$ (나누기 기호)

18. 데이터베이스에 대한 다음 설명에서 괄호에 공통으로 들어갈 알맞은 용어를 쓰시오. 22-2

- ()은 관계 데이터의 연산을 표현하는 방법으로, 관계 데이터 모델의 제안자인 코드(E. F. Codd)가 수학의 술어 해석(Predicate Calculus)에 기반을 두고 관계 데이터베이스를 위해 제안했다.
- 원하는 정보가 무엇이라는 것만 정의하는 비절차적 특성을 지니며, 원하는 정보를 정의할 때 계산 수식을 사용한다.
- 튜플 해석식을 사용하는 튜플 ()과 도메인 해석식을 사용하는 도메인 ()으로 구분된다.

답: **관계해석(Relational Calculus)**

19. 데이터의 중복으로 인해 테이블 조작 시 문제가 발생하는 현상을 이상(Anomaly)이라고 한다. 이상 중 삭제 이상(Deletion Anomaly)에 대해 간략히 서술하시오. 22-1

답: **테이블에서 튜플을 삭제할 때 의도와는 상관없는 값들도 함께 삭제되는 현상이다.**

[해설]

삽입 이상(Insertion Anomaly)

릴레이션에 데이터를 삽입할 때 의도와는 상관없이 원하지 않은 값들도 함께 삽입되는 현상.

삭제 이상(Deletion Anomaly)

릴레이션에서 한 튜플을 삭제할 때 의도와는 상관없는 값들도 함께 삭제되는 연쇄가 일어나는 현상.

갱신 이상(Update Anomaly)

릴레이션에서 튜플에 있는 속성값을 갱신할 때 일부 튜플의 정보만 갱신되어 정보에 모순이 생기는 현상.

20. 데이터베이스의 이상(Anomaly)의 종류 3가지를 쓰시오. 20-4

답: **삽입 이상(Insertion Anomaly), 삭제 이상(Deletion Anomaly), 갱신 이상(Update Anomaly)**

21. 함수적 종속(Functional Dependency)에 대한 다음 설명에서 괄호(①~③)에 들어갈 알맞은 용어를 <보기>에서 찾아 기호(㉠~㉣)로 쓰시오. (단, 테이블 <R>의 속성 '학생'과 '학과'의 밑줄은 키(Key)임을 의미한다.) 22-2

<R>

학생

학과

성적

학년

이순신	컴퓨터공학	A+	2
이순신	전기공학	B	2
유관순	경제학	B+	1
강감찬	문예창작	C	3
강감찬	한국사	C+	3
홍길동	영문학	B	4

- 테이블 <R>에서 '성적'은 기본키인 {학생, 학과}에 대해 (①) Functional Dependency이다.
- 테이블 <R>에서 '학년'은 기본키인 {학생, 학과} 중 '학생'만으로 식별이 가능하므로 기본키에 대해 (②) Functional Dependency이다.
- 임의의 테이블에 속성 A, B, C가 있을 때, $A \rightarrow B$ 이고 $B \rightarrow C$ 일 때 $A \rightarrow C$ 인 관계는 (③) Functional Dependency이다.

<보기>

- ☐ Hybrid ☐ Multi Valued ☐ Transitive ☐ Full
☐ Defined ☐ Natural ☐ Relational ☐ Partial

답: ① ☒ (Full) ② ☒ (Partial) ③ ☒ (Transitive)

[해설]

- 복합키 전체에 종속 = **완전(Full)** 함수 종속
- 복합키 중 일부에만 종속 = **부분(Partial)** 함수 종속
- $A \rightarrow B \rightarrow C$ 구조의 다리 건너 종속 = **이행적(Transitive)** 함수 종속

22. 데이터베이스에 대한 다음 설명에서 괄호에 공통으로 들어갈 알맞은 답을 쓰시오. 21-2

테이블을 만들 때는 이상(Anomaly)을 방지하기 위해 데이터들의 중복성 및 종속성을 배제하는 정규화를 수행한다. 아래 그림은 부분 함수적 종속을 제거하여 제 () 정규형을 만드는 과정이다.

<Table R>

A(key)	B(key)	C	D
A345	1001	Seoul	Pmre
D347	1001	Busan	Preo
A210	1007	Gwangju	Ciqen
A345	1007	Seoul	Esto
B230	1007	Daegu	Loid
D347	1201	Busan	Drag

<Table R>의 함수적 종속 관계

$A, B \rightarrow C, D$
 $A \rightarrow C$



<Table R1>

A(key)	B(key)	D
A345	1001	Pmre
D347	1001	Preo

<Table R2>

A(key)	C
A345	Seoul
D347	Busan

A210	1007	Ciqen	A210	Gwangju
A345	1007	Esto	B230	Daegu
B230	1007	Loid		
D347	1201	Drag		

<Table R>의 경우, C는 key에 해당하는 A와 B 중 A에만 종속되는 부분 함수적 종속이다. 이 문제 해결을 위해 <Table R1>에서 C를 분리하여 <Table R1>과 <Table R2>로 만들면 제 () 정규형에 해당하는 테이블이 완성된다.

답: 2

23. 시스템의 성능을 향상시키고 개발 및 운영의 편의성 등을 높이기 위해 정규화된 데이터 모델을 의도적으로 통합, 중복, 분리하여 정규화 원칙을 위배하는 행위를 가리키는 용어를 쓰시오. [21-1](#)

답: **반정규화(Denormalization)**

24. 데이터베이스에서 비정규화(Denormalization)의 개념을 서술하시오. [20-1](#)

답: 비정규화는 정규화된 데이터 모델을 통합, 중복, 분리하는 과정으로, 의도적으로 정규화 원칙을 위배하는 행위이다.

25. 데이터베이스의 상태 변화를 일으키는 트랜잭션(Transaction)의 특성 중 원자성(Atomicity)에 대해 간략히 서술하시오. [21-2](#)

답: 원자성은 트랜잭션의 연산은 데이터베이스에 모두 반영되도록 완료되든지 아니면 전혀 반영되지 않도록 복구되어야 한다는 특성을 의미한다.

26. 다음은 트랜잭션(Transaction)의 특징이다. 괄호(①, ②)에 들어갈 알맞은 특징을 쓰시오. [20-1](#)

특징	설명
(①)	트랜잭션의 연산은 데이터베이스에 모두 반영되도록 완료(Commit)되든지 아니면 전혀 반영되지 않도록 복구(Rollback)되어야 한다. (All or Nothing)
일관성	트랜잭션이 그 실행을 성공적으로 완료하면 언제나 일관성 있는 데이터베이스 상태로 변환한다.
(②)	둘 이상의 트랜잭션이 동시에 병행 실행되는 경우 어느 하나의 트랜잭션 실행중에 다른 트랜잭션의 연산이 끼어들 수 없다.
지속성	성공적으로 완료된 트랜잭션의 결과는 시스템이 고장나더라도 영구적으로 반영되어야 한다.

답: ① **원자성(Atomicity)** ② **독립성(Isolation)**

27. 다음 설명에서 괄호에 공통으로 들어갈 알맞은 용어를 쓰시오. [21-3, 25-2](#)

파일의 구조는 파일을 구성하는 레코드들이 보조기억장치에 편성되는 방식을 의미하는 것으로, 크게 순차, (), 해싱으로 구분한다. () 파일 구조는 <값, 주소> 쌍으로 구성되는 데이터 구조를 활용하여 데이터에 접근하는 방식으로, 자기 디스크에서 주로 활용된다.

답: **색인(Index)**

28. 시스템 관리와 관련하여 다음의 설명이 의미하는 용어를 쓰시오. [20-2](#)

A는 한국IT 보안관제실에서 근무하게 되었다. A는 서비스 운용 중 외부 공격으로 인한 서버다운, 자연재해, 시스템 장애 등의 비상 상황에도 고객 응대 서비스를 정상적으로 수행하기 위해 구축한 시스템을 관리하는 업무를 수행한다. 이 용어는 위와 같은 비상 상황이 발생한 경우 "비상사태 또는 업무중단 시점부터 업무가 복구되어 다시 정상 가동 될 때까지의 시간"을 의미한다.

답: **RTO(목표복구시간)**

<RTO, RPO, RCO, RSO의 개념 비교>

구분	설명
RTO	Recovery Time Objectives: 재해복구 시간 목표(업무관점) 업무 중단 시점부터 복구되어 가동될 때 까지 시간 목표 예: 2시간, 7일, 1개월 구축 비용에 반비례하며, 재해 발생 손실에 비례
RPO	Recovery Point Objective: 재해복구 시점목표(데이터 관점) 재해 발생시 데이터손실을 수용 손실 허용 시점 예: 특정백업시점 데이터 복구, 전일 마감 백업 시점 복구
RCO	Recovery Communication Objective: 네트워크복구시간목표 주 영업점과 DR센터 간 네트워크 복구 수준 및 범위 예: 주요지점, 전 영업점 정보시스템 사용
RSO	Recovery Scope Objective: 재해복구 범위 목표 업무 중요도에 따른 복구 대상시스템 선정 예: 계정계, 정보계, 대외계 등

29. 데이터베이스 보안에 관련된 다음 설명에 해당하는 용어를 쓰시오. [21-1](#)

접근통제는 데이터가 저장된 객체와 이를 사용하려는 주체 사이의 정보 흐름을 제한하는 것이다. 이러한 접근통제에 관한 기술 중 ()는 데이터에 접근하는 사용자의 신원에 따라 접근 권한을 부여하여 제어하는 방식으로, 데이터의 소유자가 접근통제 권한을 지정하고 제어한다. 객체를 생성한 사용자가 생성된 객체에 대한 모든 권한을 부여 받고, 부여된 권한을 다른 사용자에게 허가할 수도 있다.

답: **DAC(임의 접근통제)**

30. 다음 설명에 해당하는 DB의 트랜잭션 연산을 <보기>에서 찾아 쓰시오. [22-1](#)

DBMS는 데이터베이스에 치명적인 손실이 발생했을 때 이를 복구하기 위해 데이터베이스의 처리 내용이나 이용 상황 등 상태 변화 시간을 흐름에 따라 기록한 로그를 생성한다.

- (①) : 데이터베이스가 비정상적으로 종료되었을 때, 디스크에 저장된 로그를 분석하여 트랜잭션의 시작(start)과 완료(commit)에 대한 기록이 있는 트랜잭션들의 작업을 재작업한다. 즉 로그를 이용하여 해당 데이터 항목에 대해 이전 값을 이후 값으로 변경하는 연산이다.
- (②) : 데이터베이스가 비정상적으로 종료되었을 때, 디스크에 저장된 로그를 분석하여 트랜잭션의 시작을 나타내는 'start'는 있지만 완료를 나타내는 'commit' 기록이 없는 트랜잭션들이 작업한 내용들을 모두 취소한다. 즉 로그를 이용하여 해당 데이터 항목에 대해 이후 값을 이전 값으로 변경한다.

<보기>

- | | | | |
|------------|------------|----------|----------|
| • ROLLBACK | • UNDO | • LOG | • COMMIT |
| • REDO | • RECOVERY | • BACKUP | • CHECK |

답: ① **REDO** ② **UNDO**

31. SW 인터페이스 구현에 관련된 다음 설명에서 괄호에 공통으로 들어갈 알맞은 용어를 쓰시오. [20-1](#)

웹 페이지의 기본 형식인 HTML의 문법이 각 웹 브라우저에서 상호 호환적이지 못하다는 문제와 SGML의 복잡함을 해결하기 위하여 개발된 ()은 다른 특수한 목적을 갖는 마크업 언어이다. 원활한 데이터의 연계를 위해 송·수신 시스템 간에 전송되는 데이터가 동일한 구조로 구성될 수 있도록 형태를 정의하는 역할을 수행하며, 다음과 같은 특징이 있다.

유니코드 문자 (Unicode Text)	텍스트 데이터 형식으로 유니코드를 사용하여 전 세계 언어를 지원한다.
() 파서(Parser)	대다수의 웹 브라우저가 해석을 위한 번역기(Parser)를 내장하고 있다.
마크업(Markup)과 내용(Content)	• () 문서의 문자들은 마크업과 내용으로 구분된다. • 일반적으로 마크업은 "<"로 시작하여 ">"로 끝나는 태그(Tag)를 의미하고, 그 외의 문자열은 내용에 해당한다.
엘리먼트(Element)	마크업과 내용으로 이루어지는 하나의 요소를 의미한다.

답: **XML(eXtensible Markup Language)**

32. 통합 구현과 관련하여 다음 설명의 괄호에 공통으로 들어갈 알맞은 용어를 쓰시오. 20-2

()는 HTTP, HTTPS, SMTP 등을 사용하여 xml 기반의 메시지를 네트워크 상에서 교환하는 프로토콜로, () envelope, 헤더(header), 바디(body) 등이 추가된 xml 문서이다. ()는 복잡하고 무거운 구조로 구성되어 있어 () 보다는 restful 프로토콜을 이용하기도 한다.

답: **SOAP(Simple Object Access Protocol)**

33. 웹 서비스(Web Service)와 관련된 다음 설명에 해당하는 용어를 쓰시오. 21-1

웹 서비스와 관련된 서식이나 프로토콜 등을 표준적인 방법으로 기술하고 게시하기 위한 언어로, XML로 작성되며 UDDI의 기초가 된다. SOAP, XML 스키마와 결합하여 인터넷에서 웹 서비스를 제공하기 위해 사용되며, 클라이언트는 이것을 통해 서버에서 어떠한 조작이 가능한지를 파악할 수 있다.

답: **WSDL(Web Services Description Language)**

34. 럼바우(Rumbaugh) 데이터 모델링에 대한 다음 설명에서 각 지문(㉠~㉢)에 해당하는 모델링을 <보기>에서 찾아 기호(㉠~㉢)로 쓰시오. 21-2

- ㉠ 다수의 프로세스들 간의 자료 흐름을 중심으로 처리 과정을 표현한 모델링 [예] 자료흐름도(DFD)
- ㉡ 시간의 흐름에 따른 객체들 간의 제어 흐름, 상호 작용, 동적 순서 등의 동적인 행위를 표현하는 모델링 [예] 상태 변화도(STD), 사건 추적도
- ㉢ 시스템에서 요구되는 객체를 찾아내어 속성과 연산 식별 및 객체들 간의 관계를 규정하여 표시하는 모델링 [예] ER 다이어그램(ERD)

<보기>

- | | | | |
|-------------|------------|---------------|----------------|
| ㉠ Operation | ㉡ Sequence | ㉢ Information | ㉣ Transaction |
| ㉤ Function | ㉥ I/O | ㉦ Dynamic | ㉧ Cause-Effect |

답: ㉠ **㉤ (Function)** ㉡ **㉦ (Dynamic)** ㉢ **㉢ (Information)**

35. 객체지향에 대한 다음 설명에 해당하는 용어를 <보기>에서 찾아 쓰시오. 22-2

- 자신이 사용하지 않는 인터페이스와 의존 관계를 맺거나 영향을 받지 않아야 한다는 객체지향 설계 원칙 중의 하나이다.
- 예를 들어 프린터, 팩스, 복사 기능을 가진 복합기의 경우 3가지 기능을 모두 가진 범용 인터페이스보다는 프린터

인터페이스, 팩스 인터페이스, 복사 인터페이스로 분리함으로써 하나의 기능 변경으로 인해 다른 기능이 영향을 받지 않도록 해야 한다.

<보기>

- SRP • SOLID • OCP • LSP • ISP • DIP • OTP • PNP

답: **ISP**

[해설]

객체지향 5대 원칙

- ① 단일 책임 원칙 (Single Responsibility Principle): 인터페이스 분리 원칙이란 객체는 자신이 사용하는 메서드에만 의존해야 한다는 법칙입니다. 객체가 사용하지 않는 메서드를 의존해서는 안 된다는 뜻이기도 합니다.
- ② 개방 폐쇄 원칙 (Open Closed Principle)
- ③ 리스코프 치환 원칙 (Liscov Substitution Principle)
- ④ 인터페이스 분리 원칙 (Interface Sergregation Principle)
- ⑤ 의존성 역전 원칙 (Dependency Inversion Principle)

36. 결합도(Coupling)의 종류 중 단순 처리 대상인 데이터만 전달되는 것이 아니라 어떻게 처리해야 하는지를 결정하는 제어 요소가 전달되는 경우의 결합도를 영문으로 쓰시오. [21-3](#)

답: **Control Coupling(제어 결합도)**

37. 데이터 모델의 구성 요소에 대한 다음 설명에서 괄호(①~③)에 들어갈 알맞은 용어를 <보기>에서 찾아 기호(㉠~㉣)로 쓰시오. [25-1](#)

- (①): 한 모듈이 다른 모듈의 내부 기능 및 그 내부 자료를 직접 참조하거나 수정할 때의 결합도이다.
- (②): 모듈 간의 인터페이스로 배열이나 레코드 등의 자료 구조가 전달될 때의 결합도이다.
- (③): 공유되는 공통 데이터 영역을 여러 모듈이 사용할 때의 결합도라 하며, 파라미터가 아닌 모듈 밖에 선언된 전역 변수를 사용하여 전역 변수를 갱신하는 방식으로 상호작용하는 때의 결합도이다.

<보기>

- ㉠ 자료 결합도 ㉡ 스템프 결합도 ㉢ 제어 결합도 ㉣ 공통 결합도
㉤ 내용 결합도 ㉥ 외부 결합도

답: ① **㉤** ② **㉡** ③ **㉣**

[해설]

㉣ 결합도(Coupling)

- 결합도는 모듈 간에 상호 의존하는 정도 또는 두 모듈 사이의 연관 관계
- 결합도가 약할수록 품질이 높고, 강할수록 품질이 낮음
- 결합도의 종류와 강도

내용 결합도	공통 결합도	외부 결합도	제어 결합도	스탬프 결합도	자료 결합도
--------	--------	--------	--------	---------	--------

결합도 강함 ←————→ 결합도 약함

종류	내용
내용 결합도 (Content Coupling)	한 모듈이 다른 모듈의 내부 기능 및 그 내부 자료를 직접 참조하거나 수정할 때의 결합도
공통(공유) 결합도 (Common Coupling)	공유되는 공통 데이터 영역을 여러 모듈이 사용할 때의 결합도
외부 결합도 (External Coupling)	어떤 모듈에서 선언한 데이터(변수)를 외부의 다른 모듈에서 참조할 때의 결합도

종류	내용
제어 결합도 (Control Coupling)	- 어떤 모듈이 다른 모듈 내부의 논리적인 흐름을 제어하기 위해 제어 신호나 제어 요소를 전달하는 결합도 P. 169로 - 하위 모듈에서 상위 모듈로 제어 신호가 이동하여 하위 모듈이 상위 모듈에게 처리 명령을 내리는 권리 전도 현상이 발생하게 됨
스탬프(검인) 결합도 (Stamp Coupling)	모듈 간의 인터페이스로 배열이나 레코드 등의 자료 구조가 전달될 때의 결합도 P. 200로
자료 결합도 (Data Coupling)	모듈 간의 인터페이스가 자료 요소로만 구성될 때의 결합도

◎ 응집도(Cohesion)

- 응집도는 모듈의 내부 요소들이 서로 관련되어 있는 정도
- 응집도가 강할수록 품질이 높고, 약할수록 품질이 낮음
- 응집도의 종류와 강도 [P. 225로](#)

기능적 응집도	순차적 응집도	교환적 응집도	절차적 응집도	시간적 응집도	논리적 응집도	우연적 응집도
---------	---------	---------	---------	---------	---------	---------

응집도 강함 ← [P. 225로](#) → 응집도 약함

종류	내용 P. 처음 위치로
기능적 응집도 (Functional Cohesion)	모듈 내부의 모든 기능 요소들이 단일 문제와 연관되어 수행될 경우의 응집도
순차적 응집도 (Sequential Cohesion)	모듈 내 하나의 활동으로부터 나온 출력 데이터를 그 다음 활동의 입력 데이터로 사용할 경우의 응집도
교환(통신)적 응집도 (Communication Cohesion)	동일한 입력과 출력을 사용하여 서로 다른 기능을 수행하는 구성 요소들이 모였을 경우의 응집도
절차적 응집도 (Procedural Cohesion)	모듈이 다수의 관련 기능을 가질 때 모듈 안의 구성 요소들이 그 기능을 순차적으로 수행할 경우의 응집도
시간적 응집도 (Temporal Cohesion)	특정 시간에 처리되는 몇 개의 기능을 모아 하나의 모듈로 작성할 경우의 응집도
논리적 응집도 (Logical Cohesion)	유사한 성격을 갖거나 특정 형태로 분류되는 처리 요소들로 하나의 모듈이 형성되는 경우의 응집도
우연적 응집도 (Coincidental Cohesion)	모듈 내부의 각 구성 요소들이 서로 관련 없는 요소로만 구성된 경우의 응집도

38. 소프트웨어 패키징이란 모듈별로 생성한 실행 파일들을 묶어 배포용 설치 파일을 만드는 것을 말한다. 소스 코드는 향후 관리를 고려하여 모듈화하여 패키징한다. 모듈화는 모듈 간 (①)의 최소화와 모듈 내 요소들의 (②)를 최대화 하는 것이 목표이다. 괄호(①, ②)에 들어갈 알맞은 용어를 쓰시오. [20-1](#)

답: ① **결합도(Coupling)** ② **응집도(Cohesion)**

39. 모듈에 대한 다음 설명에서 각 지문(①~③)에 해당하는 응집도(Cohesion)를 <보기>에서 찾아 기호(㉠~㉣)로 쓰시오. [21-2](#)

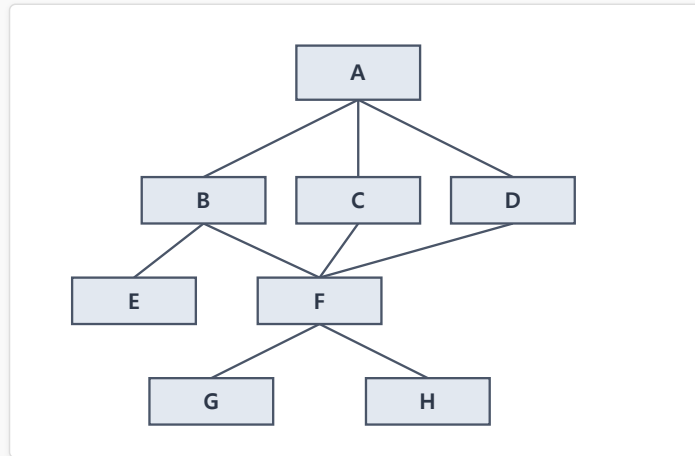
- | |
|---|
| ① 내부의 요소들이 기능적으로 연관성은 없으나, 순차적으로 실행될 때의 응집도
② 서로 다른 기능을 수행하지만 동일한 입력과 출력을 사용할 때의 응집도
③ 하나의 기능에 밀접하게 관련되어 있거나 연관되어 있을 때의 응집도 |
|---|

<보기>

- ㉠ 기능적 응집도 ㉡ 순차적 응집도 ㉢ 교환적 응집도 ㉣ 절차적 응집도

답: ① ㉡ ② ㉡ ③ ㉡

40. 다음의 모듈 관계를 표현한 시스템 구조도를 참고하여 모듈 F의 팬인(Fan-In)과 팬아웃(Fan-Out)을 구하시오. [22-2](#) [001로](#)



답:

- 팬인(Fan-In) : 3
- 팬아웃(Fan-Out) : 2

[해설]

- 팬인(Fan-In) : 해당 모듈로 들어오는 상위 모듈의 수 (B, C, D가 F로 들어옴 → 3)
- 팬아웃(Fan-Out) : 해당 모듈이 제어(호출)하는 하위 모듈의 수 (F가 G, H를 가리킴 → 2)

41. 네트워크에 관련된 다음 설명에 해당하는 용어를 쓰시오. [21-1](#)

모듈 간 통신 방식을 구현하기 위해 사용되는 대표적인 프로그래밍 인터페이스 집합으로, 복수의 프로세스를 수행하며 이뤄지는 프로세스 간 통신까지 구현이 가능하다. 대표적인 메소드에는 공유 메모리(Shared Memory), 소켓(Socket), 세마포어(Semaphores), 파이프와 네임드 파이프(Pipes&named Pipes), 메시지 큐잉(Message Queueing)이 있다.

답: **IPC(Inter-Process Communication)**

42. 다음 테스트 케이스를 참조하여 괄호에 들어갈 테스트 케이스의 구성 요소를 <보기>에서 찾아 쓰시오. [21-3, 25-3](#)

식별자_ID	테스트 항목	(①)	(②)	(③)
I_S_W10_35	로그인 기능	사용자 초기 화면	아이디(test_a01) 비밀번호(203ald5%ffa1)	로그인 성공
I_S_W10_36	로그인 기능	사용자 초기 화면	아이디(test_a01) 비밀번호(1234)	로그인 실패(1) - 비밀번호 불일치
I_S_W10_37	로그인 기능	사용자 초기 화면	아이디("") 비밀번호("")	로그인 실패(2) - 미입력

<보기>

- | | | | |
|-----------|----------|------------|------------|
| • 요구 절차 | • 의존성 여부 | • 테스트 데이터 | • 테스트 조건 |
| • 하드웨어 환경 | • 예상 결과 | • 소프트웨어 환경 | • 성공/실패 기준 |

답: ① 테스트 조건 ② 테스트 데이터 ③ 예상 결과

43. 디자인 패턴에 관한 다음 설명에서 괄호에 들어갈 알맞은 용어를 <보기>에서 찾아 쓰시오. [21-3](#)  **클릭!**

() 패턴은 객체 생성을 서브 클래스에서 처리하도록 분리하여 캡슐화한 패턴으로, 상위 클래스에서 인터페이스만 정의하고 실제 생성은 서브 클래스가 담당한다. 다른 이름으로 가상 생성자(Virtual Constructor) 패턴이라고도 불린다.

<보기>

- | | | | |
|-------------|--------------------|-------------------|-------------|
| • Singleton | • Abstract Factory | • Factory Method | • Prototype |
| • Facade | • Composite | • Template Method | • Builder |

답: **Factory Method**

44. 디자인 패턴에 대한 다음 설명에서 괄호(①, ②)에 들어갈 알맞은 용어를 <보기>에서 찾아 쓰시오. [22-3, 26-1](#)

- (①) 패턴은 구현부에서 추상층을 분리하여, 서로가 독립적으로 확장할 수 있도록 구성한 패턴으로, 기능과 구현을 두 개의 별도 클래스로 구현한다는 특징이 있다.
- (②) 패턴은 한 객체의 상태가 변화하면 객체에 상속된 다른 객체들에게 변화된 상태를 전달하는 패턴으로, 일대다의 의존성을 정의한다. 주로 분산된 시스템 간에 이벤트를 생성-발행(Publish)하고, 이를 수신(Subscribe)해야 할 때 이용한다.

<보기>

- | | | | | |
|-----------|------------------|------------|----------|----------|
| • Builder | • Factory Method | • Adapter | • Bridge | • Facade |
| • Proxy | • Observer | • Mediator | | |

답: ① **Bridge** ② **Observer**

44. 한 객체의 상태가 바뀌면 그 객체에 의존하는 다른 객체들에게 연락이 가서 자동으로 내용이 갱신되는 방식으로, 일대다의 의존성을 정의하는 패턴이다. 상호 작용을 하는 객체 사이에서는 가능하면 느슨하게 결합하는 이 패턴을 영문으로 쓰시오. [20-2](#)

답: **Observer**

45. 디자인 패턴에 관련된 다음 설명에서 괄호에 들어갈 알맞은 용어를 쓰시오. [21-2](#)  **클릭!**

디자인 패턴은 모듈 간의 관계 및 인터페이스를 설계할 때 참조할 수 있는 전형적인 해결 방식 또는 예제를 의미한다. 그 중 () 패턴은 클래스나 객체들이 서로 상호작용하는 방법이나 책임 분배 방법을 정의하는 패턴으로, Interpreter, Observer, Command 등이 그 예에 해당한다.

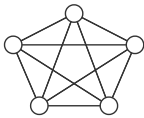
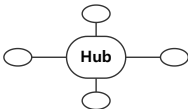
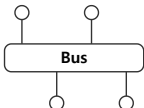
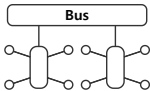
답: **행위(Behavioral)**

46. 기업 내 각종 애플리케이션 및 플랫폼 간의 정보 전달, 연계, 통합 등 상호 연동이 가능하게 해주는 솔루션으로, Point-to-Point, Hub&Spoke, Message Bus, Hybrid 등의 다양한 방식으로 구축이 가능한 모듈 연계 방법을 쓰시오. [21-1](#)

답: **EAI(Enterprise Application Integration)**

47. EAI(Enterprise Application Integration)의 구축 유형에 대한 설명이다. 괄호(①, ②) 안에 각각 들어갈 알맞은 유형을 쓰시오. [20-3](#)

EAI(Enterprise Application Integration)는 기업 내 각종 애플리케이션 및 플랫폼 간의 정보 전달, 연계, 통합 등 상호 연동이 가능하게 해주는 솔루션이다. 비즈니스 간 통합 및 연계성을 증대시켜 효율성 및 각 시스템 간의 확정성(Determinacy)을 높여 준다. EAI의 구축 유형은 다음과 같다.

(①)	- 가장 기본적인 애플리케이션 통합 방식으로, 애플리케이션을 1 : 1로 연결한다. - 변경 및 재사용이 어렵다.	
(②)	- 단일 접점인 허브 시스템을 통해 데이터를 전송하는 중앙 집중형 방식이다. - 확장 및 유지 보수가 용이하다. - 허브 장애 발생 시 시스템 전체에 영향을 미친다.	
Message Bus	- 애플리케이션 사이에 미들웨어를 두어 처리하는 방식이다. - 확장성이 뛰어나며 대용량 처리가 가능하다.	
Hybrid	- 그룹 내에서는 (②) 방식을, 그룹 간에는 Message Bus 방식을 사용한다. - 필요한 경우 한 가지 방식으로 EAI 구현이 가능하다. - 데이터 병목 현상을 최소화할 수 있다.	

답: ① Point to Point ② Hub & Spoke

48. 소프트웨어 인터페이스 구현에 관련된 다음 설명에서 괄호에 공통으로 들어갈 알맞은 용어를 쓰시오. [20-1](#)

()은 속성-값 쌍(Attribute-Value Pairs)으로 이루어진 데이터 객체를 전달하기 위해 사람이 읽을 수 있는 텍스트를 사용하는 개방형 표준 포맷이다. 비동기 처리에 사용되는 AJAX에서 XML을 대체하여 사용되고 있다. 다음은 ()이 가질 수 있는 자료 기본형이다.

숫자	정수와 실수를 표현한다.
문자열	유니코드 문자로 표현하며, 큰따옴표(" ")로 묶는다.
참/거짓	참(True) 또는 거짓(False)을 표현한다.
배열	다양한 요소들을 심표로 구분하여 표현하며, 대괄호([])로 묶는다.
객체	이름/값 쌍으로 표현하며, 중괄호({ })로 묶는다.
NULL	아직 알려지지 않거나 모르는 값을 표현하기 위한 자료형이다.

답: JSON(JavaScript Object Notation)

49. 클라이언트와 서버 간 자바스크립트 및 XML을 비동기 방식으로 처리하며, 전체 페이지를 새로 고치지 않고도 웹페이지 일부 영역만을 업데이트할 수 있도록 하는 기술을 의미하는 용어를 쓰시오. [20-2](#), [23-1](#), [25-2](#)

답: AJAX(Asynchronous JavaScript and XML)

50. 네트워크 트래픽에 대해 IP(Internet Protocol) 계층에서 IP 패킷 단위의 데이터 변조 방지 및 은닉 기능을 제공하는 네트워크 계층에서의 보안 통신 규약을 쓰시오. [20-2](#)

답: **IPSec(Internet Protocol Security)**

51. 인터페이스 구현 검증 도구에 대한 다음 설명에 해당하는 용어를 영문으로 쓰시오. [22-1](#)

Kent Beck과 Erich Gamma 등이 개발한 자바 프로그래밍 언어용 유닛 테스트 프레임워크로, xUnit 계열의 한 종류다. 같은 테스트 코드를 여러 번 작성하지 않게 도와주며, 테스트마다 예상 결과를 기억할 필요가 없는 자동화된 해법을 제공한다는 특징이 있다.

답: **JUnit**

52. 사용자 인터페이스(UI)에 대한 다음 설명에 해당하는 용어를 영문 약어로 쓰시오. [22-1](#)

사용자의 자연스러운 움직임을 통해 시스템과 상호작용하는 사용자 인터페이스(UI)로, 키보드나 마우스와 같이 조작을 배워야 하는 인공 제어 장치를 사용하는 인터페이스와 구분하기 위해 '자연스러운'이라는 표현을 사용한다. 시리(Siri), 빅스비(Bixby) 등과 같은 음성 비서에게 사용하는 자연어 명령어나 휴대폰이나 태블릿에서의 터치 등이 여기에 해당한다.

답: **NUI**

53. Windows, MacOS 등에서 사용하는 인터페이스로, 사용자가 명령어를 직접 입력하지 않고 키보드와 마우스 등을 이용하여 아이콘이나 메뉴를 선택하여 모든 작업을 수행하는 사용자 인터페이스를 쓰시오. [21-3](#)

답: **GUI(그래픽 사용자 인터페이스)**

54. UI(User Interface)의 설계 원칙 중 직관성에 대해 간략히 서술하시오. [20-3](#)

답: 직관성은 누구나 쉽게 이해하고 사용할 수 있어야 한다는 설계 원칙이다.

55. 사용자 인터페이스에 대한 다음 설명에서 괄호에 들어갈 알맞은 용어를 쓰시오. [20-2](#)

직관성	누구나 쉽게 이해하고 사용할 수 있어야 한다.
()	사용자의 목적을 정확하고 완벽하게 달성해야 한다.
학습성	누구나 쉽게 배우고 익힐 수 있어야 한다.
유연성	사용자의 요구사항을 최대한 수용하고 실수를 최소화해야 한다.

답: **유효성**

56. 인터페이스에 관련된 다음 설명에서 괄호(①, ②)에 들어갈 알맞은 용어를 쓰시오. [21-2](#)

- (①) : 사용자가 시스템이나 서비스를 이용하면서 느끼고 생각하게 되는 총체적인 감정 및 경험
- (②) : 사용자외 시스템 간의 상호작용이 원활하게 이뤄지도록 도와주는 장치나 소프트웨어
[예] CLI, GUI 등

답: ① **UX(사용자 경험)** ② **UI(사용자 인터페이스)**

57. 애플리케이션 테스트에서 사용되는 살충제 패러독스(Pesticide Paradox)의 개념을 간략히 설명하시오. [20-1](#)

답: 살충제 패러독스는 동일한 테스트 케이스로 동일한 테스트를 반복하면 더 이상 결함이 발견되지 않는 현상을 의미한다.

58. 애플리케이션을 실행하지 않고, 소스 코드에 대한 코딩 표준, 코딩 스타일, 코드 복잡도 및 남은 결함을 발견하기 위하여 사용하는 테스트를 쓰시오. [20-2](#)

답: 정적 분석(정적 테스트)

59. 테스트에 대한 다음 설명에서 각 지문(①~③)에 해당하는 커버리지(Coverage)를 <보기>에서 찾아 기호(㉠~㉣)로 쓰시오. [21-2](#)

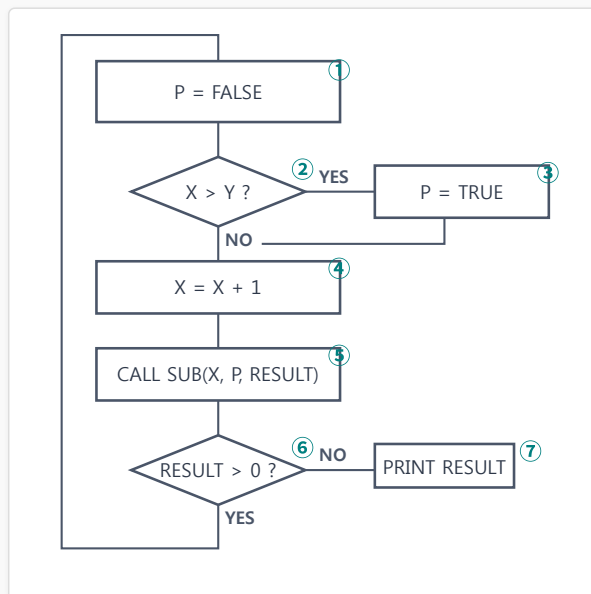
- | |
|---|
| ① 최소 한번은 모든 문장이 수행되도록 구성하는 검증 기준 |
| ② 조건식이 참(True)/거짓(False)일 때 수행되도록 구성하는 검증 기준 |
| ③ ②번과 달리 조건식에 상관없이 개별 조건이 참(True)/거짓(False)일 때 수행되도록 구성하는 검증 기준 |

<보기>

- | | | |
|----------------|-------------------|----------------|
| ㉠ 다중 조건 검증 기준 | ㉡ 선택 검증 기준 | ㉢ 조건 검증 기준 |
| ㉣ 결정(분기) 검증 기준 | ㉤ 결정(분기)/조건 검증 기준 | ㉥ 구문(문장) 검증 기준 |

답: ① ㉤ ② ㉢ ③ ㉤

60. 다음은 화이트박스 테스트의 프로그램 제어흐름이다. 다음 순서도를 참고하여 분기 커버리지로 구성할 테스트 케이스를 작성하시오. [20-3, 23-1, 25-2](#)



<작성예시>

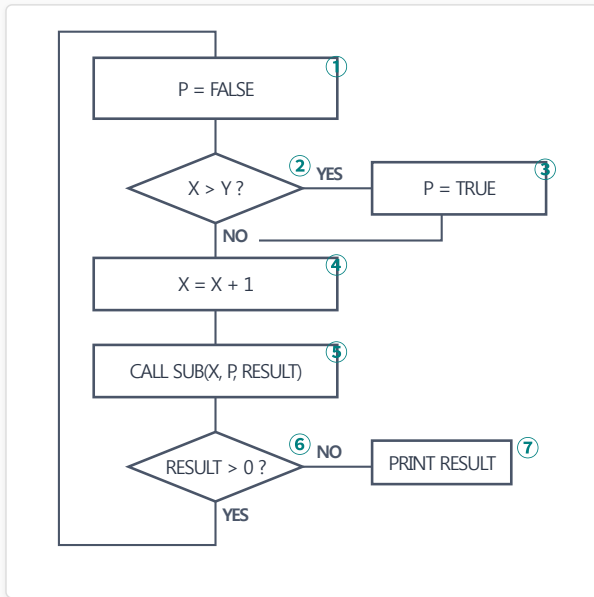
(①) → (②) → (④)

답:

- (①) → (②) → (③) → (④) → (⑤) → (⑥) → (⑦)
- (①) → (②) → (④) → (⑤) → (⑥) → (①)

[해설]

화이트박스 테스트의 검증 기준(Coverage) 중 분기 검증 기준(Branch Coverage)은 소스 코드의 모든 조건문이 한 번 이상 수행되도록 테스트 케이스를 설계하는 방법입니다.



61. 소프트웨어가 수행할 특정 기능을 알기 위해서 각 기능이 완전히 작동되는 것을 입증하는 테스트로, 동치 클래스 분해 및 경계값 분석을 이용하는 테스트 기법을 쓰시오. [20-3](#)

답: **블랙박스 테스트(Black Box Test)**

62. 테스트 기법 중 다음과 같이 '평가 점수표'를 미리 정해 놓은 후 각 영역의 경계에 해당하는 입력값을 넣고, 예상되는 출력값이 나오는지 실제 값과 비교하는 명세 기반 테스트 기법을 <보기>에서 찾아 쓰시오. [20-4, 22-3](#)

<평가 점수표>

<케이스>

평가점수	성적등급	테스트 케이스	1	2	3	4	5	6	7	8	9	10
90~100	A	입력값	-1	0	69	70	79	80	89	90	100	101
80~89	B	예상 결과값	오류	D	D	C	C	B	B	A	A	오류
70~79	C	실제 결과값	오류	D	D	C	C	B	B	A	A	오류
0~69	D											

<보기>

- Equivalence Partition
- Cause-Effect Graph
- Base Path Test
- Boundary Value Analysis
- Error Guess
- Loop Test
- Condition Test
- Comparison Test
- Data Flow Test

답: **Boundary Value Analysis(경계값 분석)**

[해설] 동등 분할 기법 (Equivalence Partitioning), 조건 검사(Condition Test)

63. 다음 중 블랙 박스 테스트 기법에 해당하는 것을 모두 골라 기호(㉠~㉡)로 쓰시오. [22-1](#)

- ㉠ Base Path Testing ㉡ Condition Testing
- ㉢ Boundary Value Analysis ㉣ Equivalence Partitioning
- ㉤ Data Flow Testing ㉥ Cause-Effect Graph
- ㉦ Branch Coverage Testing ㉧ Statement Coverage Testing
- ㉨ Boundary Division Analysis

답: ㉢, ㉣, ㉥

[해설] 블랙 박스 테스트 기법

1. 동등 분할 기법(Equivalence Partitioning)
2. 경계값 분석 기법(Boundary Value Analysis)
3. 오류 예측 기법(Error Guessing)
4. 원인 결과 그래프 기법(Cause Effect Graph)

64. 테스트 기법 중 그래프를 활용하여 입력 데이터 간의 관계와 출력에 영향을 미치는 상황을 체계적으로 분석한 다음 효용성이 높은 테스트 케이스를 선정하여 검사하는 기법을 <보기>에서 찾아 쓰시오. [21-3](#)  **클릭!**

<보기>

- Equivalence Partition • Boundary Value Analysis • Condition Test
- Cause-Effect Graph • Error Guess • Comparison Test
- Base Path Test • Loop Test • Data Flow Test

답: **Cause-Effect Graph**

65. 테스트 기법 중 다음과 같이 '평가 점수표'를 미리 정해 놓은 후 각 영역에 해당하는 입력값을 넣고, 예상되는 출력값이 나오는지 실제 값과 비교하는 명세 기반 테스트 기법을 <보기>에서 찾아 쓰시오. [23-3](#)

<평가 점수표>

평가점수	성적등급
90~100	A
80~89	B
70~79	C
0~69	D

<케이스>

테스트 케이스	1	2	3	4
점수범위	0~69	70~79	80~89	90~100
입력값	60	75	82	96
예상 결과값	D	C	B	A
실제 결과값	D	C	B	A

<보기>

- Equivalence Partition • Boundary Value Analysis • Equivalence Value
- Cause-Effect Graph • Error Guess • Comparison Test
- Base Path Test • Loop Test • Data Flow Test

답: **Equivalence Partition**

[해설]

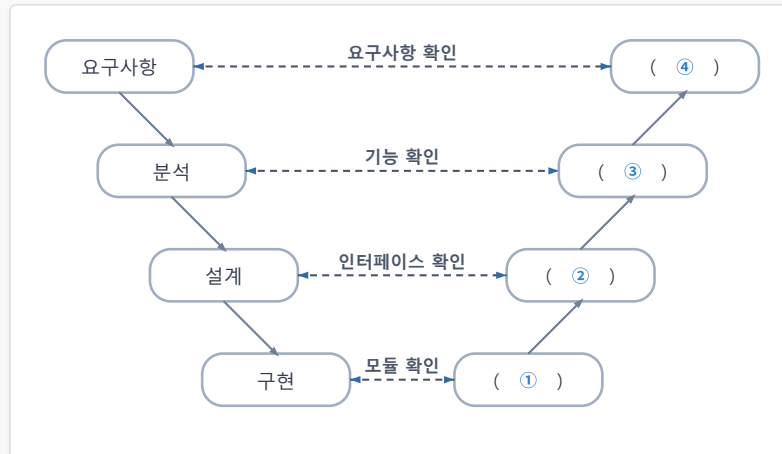
동치 분할 검사(Equivalence Partitioning Testing)

명세 기반 소프트웨어 테스트 기법: 동등 분할(동치 분할) 기법 분석

- 개념 및 원리 분석: 동등 분할 기법(Equivalence Partitioning)은 프로그램의 입력 조건들을 유효한 값과 유효하지 않은 값의 여러 영역(구간)으로 분류하고, 각 영역을 대표하는 대표값(결정값)을 선정하여 테스트 케이스를 설계하는 명세 기반 테스트 기법입니다.

- **문제 예시 분석:** 제시된 테이블에서는 점수 구간을 0~69, 70~79, 80~89, 90~100의 4개 클래스(동치 영역)로 균등하게 분할하였습니다.
 - 구간별로 한 개씩의 대표 입력값(60, 75, 82, 96)을 무작위 추출하여 입력 테스트를 수행하고 있으므로 전형적인 **Equivalence Partition(동등 분할)** 기법에 해당합니다.
- **오답 방지 팁:**
 - Boundary Value Analysis는 각 경계값(예: 69, 70, 79, 80 등) 주위의 값을 테스트하는 '경계값 분석 기법'이므로 오답입니다.
 - 보기 중 Equivalence Value는 혼동을 주기 위해 만들어진 잘못된 용어입니다.

66. 개발 단계에 따른 애플리케이션 테스트에 대한 다음 V-모델에서 괄호(①~④)에 들어갈 알맞은 테스트를 쓰시오. [22-1](#)



- 답: ① **단위 테스트(Unit Test)** ② **통합 테스트(Integration Test)**
 ③ **시스템 테스트(System Test)** ④ **인수 테스트(Acceptance Test)**

67. 애플리케이션 테스트에 대한 다음 설명에서 각 지문(①, ②)에 해당하는 용어를 <보기>에서 찾아 기호(㉠~㉣)로 쓰시오. [21-1](#)

- ① 코딩 직후 소프트웨어 설계의 최소 단위인 모듈이나 컴포넌트에 초점을 맞춰 수행하는 테스트로, 모듈 테스트라고도 부른다. 사용자의 요구사항을 기반으로 한 기능성 테스트를 최우선으로 인터페이스, 외부적 I/O, 자료구조, 독립적 기초 경로, 오류 처리 경로, 경계 조건 등을 검사한다.
- ② 모듈들을 결합하여 하나의 시스템으로 완성시키는 과정에서의 테스트를 의미하며, 모듈 간 또는 컴포넌트 간의 인터페이스가 정상적으로 실행되는지 검사한다.

<보기>

- | | | | |
|-----------|----------|----------|----------|
| ㉠ 시스템 테스트 | ㉡ 인수 테스트 | ㉢ 알파 테스트 | ㉣ 단위 테스트 |
| ㉤ 통합 테스트 | ㉥ 회귀 테스트 | | |

답: ① ㉣ ② ㉤

68. 애플리케이션 테스트에 대한 다음 설명에서 괄호(①, ②)에 들어갈 알맞은 테스트를 쓰시오. [22-2](#)

- 인수 테스트는 개발한 소프트웨어가 사용자의 요구사항을 충족하는지에 중점을 두고 테스트하는 방법이다.
- (①): 선정된 최종 사용자가 여러 명의 사용자 앞에서 행하는 테스트 기법으로, 실제 업무를 가지고 사용자가 직접 테스트한다.

- (②) : 개발자의 장소에서 사용자가 개발자 앞에서 행하는 테스트 기법으로, 테스트는 통제된 환경에서 행해지며, 오류와 사용상의 문제점을 사용자와 개발자가 함께 확인하면서 기록한다.

답: ① 베타 테스트(Beta Test) ② 알파 테스트(Alpha Test)

69. 통합 테스트에 관련된 다음 설명에서 괄호에 들어갈 알맞은 용어를 쓰시오. 21-2

통합 테스트는 단위 테스트가 끝난 모듈을 통합하는 과정에서 발생하는 오류 및 결함을 찾는 테스트 기법으로, 하위 모듈에서 상위 모듈 방향으로 통합하는 상향식 통합 테스트와 상위 모듈에서 하위 모듈 방향으로 통합하는 하향식 통합 테스트가 있다. 상향식 통합 테스트는 미완성이거나 문제가 있는 상위 모듈을 대체할 수 있는 테스트 드라이버가, 하향식 통합 테스트는 미완성이거나 문제가 있는 하위 모듈을 대체할 수 있는 테스트 ()이(가) 있어야 원활한 테스트가 가능하다.

답: 스텝(Stub)

70. 애플리케이션 테스트에 관한 다음 설명에서 괄호(①, ②)에 들어갈 알맞은 용어를 쓰시오. 21-3

- (①)는 소프트웨어의 하위 모듈에서 상위 모듈 방향으로 통합하면서 테스트하는 기법이다.
- 하나의 주요 제어 모듈과 관련된 종속 모듈의 그룹인 클러스터(Cluster)가 필요하다.
- 데이터의 입출력을 확인하기 위해 더미 모듈인 (②)를 생성한다.

답: ① 상향식 통합 테스트
② 드라이버(Driver) 또는 테스트 드라이버(Test Driver)

71. 다음 설명에 해당하는 테스트를 <보기>에서 찾아 쓰시오. 22-2

- 통합 테스트로 인해 변경된 모듈이나 컴포넌트에 새로운 오류가 있는지 확인하는 테스트이다.
- 이미 테스트된 프로그램의 테스트를 반복하는 것이다.
- 수정한 모듈이나 컴포넌트가 다른 부분에 영향을 미치는지, 오류가 생기지 않았는지 테스트하여 새로운 오류가 발생하지 않음을 보증하기 위해 반복 테스트한다.

<보기>

- | | | | |
|---------------|--------------|-------------|--------------|
| • Integration | • Big Bang | • System | • Acceptance |
| • Unit | • Regression | • White Box | • Black Box |

답: Regression(회귀)

[해설]

- ① 통합 테스트(Integration Test): 이름에서 의미하는 바와 같이 각각의 시스템들이 서로 어떻게 상호작용하고 제대로 작동하는지 테스트하는 것
- ② 빅뱅(Big Bang) 테스트: 시스템을 구성하는 모듈을 따로 구현하고 전체 시스템을 단번에 묶어 시험하는 방법, 모듈의 인터페이스와 결합을 테스트
- ③ 인수(Acceptance) 테스트: 정보 시스템의 검사 중 하나로서, 해당 시스템이 실제 운영 환경에서 사용될 준비가 되었는지 최종적으로 확인하는 테스트
- ④ 유닛(Unit) 테스트: 전체 코드 중 작은 부분을 테스트하는 것
- ⑤ Blackbox(블랙박스) 테스트: 내부 설계 구조나 작동원리를 모르는 상태에서 동작을 검사하는 테스트방식
- ⑥ Whitebox(화이트박스) 테스트: 화이트박스 테스트는 소프트웨어, 모듈 등의 소스코드를 오픈시킨 상태에서 논리적인 모든 경로를 테스트하여 테스트 케이스를 설계하는 방법

72. 특정한 몇몇 테스트 케이스의 입력 값들에 대해서만 기대하는 결과를 제공하는 오라클로, 전수 테스트가 불가능한 경우 사용하고, 경계값 및 구간별 예상값 결과 작성시 사용하는 오라클을 쓰시오. 20-4

답: 샘플링 오라클(Sampling Oracle)

73. 애플리케이션 성능이란 사용자가 요구한 기능을 최소한의 자원을 사용하여 최대한 많은 기능을 신속하게 처리하는 정도를 나타낸다. 애플리케이션 성능 측정의 지표에 대한 다음 설명에서 괄호(①~③)에 들어갈 알맞은 용어를 쓰시오. [20-1](#)

(①)	일정 시간 내에 애플리케이션이 처리하는 일의 양을 의미한다.
(②)	애플리케이션에 요청을 전달한 시간부터 응답이 도착할 때까지 걸린 시간을 의미한다.
(③)	애플리케이션에 작업을 의뢰한 시간부터 처리가 완료될 때까지 걸린 시간을 의미한다.
자원 활용률	애플리케이션이 의뢰한 작업을 처리하는 동안의 CPU, 메모리, 네트워크 등의 자원 사용률을 의미한다.

답: ① 처리량(Throughput)
② 응답 시간(Response Time)
③ 경과 시간(Turn Around Time)

74. 소스 코드 품질 분석 도구에 대한 다음 설명에서 괄호(①, ②)에 해당하는 용어를 <보기>에서 찾아 기호(㉠~㉤)로 쓰시오. [22-1](#)

소스 코드 품질 분석 도구는 소스 코드의 코딩 스타일, 코드에 설정된 코딩 표준, 코드의 복잡도, 코드에 존재하는 메모리 누수 현상, 스레드 결함 등을 발견하기 위해 사용하는 분석 도구이다.

- (①) 도구는 작성한 소스 코드를 실행하지 않고 코딩 표준이나 코딩 스타일, 결함 등을 확인하는 코드 분석 도구이다.
- (②) 도구는 소스 코드를 직접 실행하여 프로그램의 동작이나 반응을 추적하고 보고하는 분석 도구로, 프로그램 모니터링 기능이나 스냅샷 생성 기능들을 포함하고 있다.

<보기>

㉠ Static Analysis	㉡ Running Analysis	㉢ Test Execution	㉣ Performance
㉤ Dynamic Analysis	㉥ Test Control	㉦ Test Harness	㉧ Test Monitoring

답: ① ㉠ (Static Analysis) ② ㉤ (Dynamic Analysis)

[해설] Static Analysis(정적 분석), Dynamic Analysis(동적 분석)

75. 데이터를 제어하는 DCL의 하나인 GRANT의 기능에 대해 간략히 서술하시오. [21-3](#)

답: GRANT는 데이터베이스 관리자가 데이터베이스 사용자에게 권한을 부여하는 데 사용하는 명령이다.

[해설] DCL (Data Control Language, 데이터 제어어)

76. 데이터를 제어하는 DCL의 하나인 ROLLBACK에 대해 간략히 서술하시오. [20-2](#)

답: ROLLBACK은 트랜잭션이 실패한 경우 작업을 취소하고 이전 상태로 되돌리기 위한 명령이다.

77. 데이터베이스 보안에서 가용성(Availability)에 대해 간략히 서술하시오. [20-4](#)

답: 가용성은 인가받은 사용자는 시스템 내의 정보와 자원을 언제라도 사용할 수 있다는 보안 요건이다.

78. 보안 위협의 하나인 SQL Injection에 대해 간략히 서술하시오. [20-2](#)

답: **SQL Injection**은 웹 응용 프로그램에 SQL 구문을 삽입하여 내부 데이터베이스(DB) 서버의 데이터를 유출 및 변조하고 관리자 인증을 우회하는 공격 기법이다.

79. 암호화 알고리즘에 대한 다음 설명에서 괄호(①, ②)에 들어갈 알맞은 용어를 쓰시오. 22-2

- 암호화 알고리즘은 비밀번호, 주민번호, 은행계좌와 같은 중요 정보를 보호하기 위해 평문을 암호화된 문장으로 만드는 절차 또는 방법을 의미한다.
- 스위스의 라이(Lai)와 메시(Massey)는 1990년 PES를 발표하고, 이후 이를 개선한 IPES를 발표하였다. IPES는 128비트의 Key를 사용하여 64비트 블록을 암호화하는 알고리즘이며 현재는 (①)라고 불린다.
- (②)은 국가 안전 보장국(NSA)에서 개발한 암호화 알고리즘으로, 클리퍼 칩(Clipper Chip)이라는 IC 칩에 내장되어 있다. 80비트의 Key를 사용하여 64비트 블록을 암호화하며, 주로 전화기와 같은 음성 통신 장비에 삽입되어 음성 데이터를 암호화한다.

답: ① **IDEA(International Data Encryption Algorithm)**

② **Skipjack(스킵잭)**

80. 보안 프로토콜에 대한 다음 설명에 해당하는 용어를 영문 약어로 쓰시오. 22-1 001로

무선랜 보안에 사용된 WEP 방식을 보완한 데이터 보안 프로토콜로, 임시 키 무결성 프로토콜이라고도 한다. WEP의 취약성을 보완하기 위해 암호 알고리즘의 입력 키 길이를 128비트로 늘리고 패킷당 키 할당, 키값 재설정 등 키 관리 방식을 개선하였다.

답: **TKIP**

[해설] 임시 키 무결성 프로토콜(Temporal Key Integrity Protocol)

81. 다음에서 설명하고 있는 암호화 알고리즘을 쓰시오. 21-3

1974년 IBM이 개발하고 1975년 NBS에 의해 미국의 국가 표준으로 발표된 암호화 알고리즘으로, 블록 크기는 64비트, 키 길이는 56비트이며, 16회의 라운드를 수행한다. 컴퓨터 기술이 발달함에 따라 해독이 쉬워지면서 미국의 국가 표준이 2001년 AES로 대체되었다.

답: **DES(Data Encryption Standard)**

[해설] 데이터 암호화 표준 (Data Encryption Standard)

82. 보안 및 암호화와 관련된 다음 설명에 해당하는 용어를 쓰시오. 21-2, 24-2

- 2001년 미국 표준 기술 연구소(NIST)에서 발표한 대칭키 암호화 알고리즘이다.
- DES의 한계를 느낀 NIST에서 공모한 후 발표하였다.
- 블록 크기는 128비트이며, 키 길이에 따라 128, 192, 256으로 분류된다.

답: **AES(Advanced Encryption Standard)**

[해설] 고급 암호화 표준 (Advanced Encryption Standard)

● 양방향 알고리즘의 종류

알고리즘	특징
SEED	• 1999년 한국인터넷진흥원(KISA)에서 개발한 블록 암호화 알고리즘 • 블록 크기는 128비트이며, 키 길이에 따라 128, 256으로 분류됨
ARIA(Academy, Research Institute, Agency)	2004년 국가정보원과 산학연협회가 개발한 블록 암호화 알고리즘
DES(Data Encryption Standard)	• 1975년 미국 NBS에서 발표한 개인키 암호화 알고리즘 • 블록 크기는 64비트, 키 길이는 56비트이며 16회의 라운드를 수행함

	• DES를 3번 적용하여 보안을 더욱 강화한 3DES(Triple DES)도 있음
AES(Advanced Encryption Standard)	• 2001년 미국 표준 기술 연구소(NIST)에서 발표한 개인키 암호화 알고리즘 • DES의 한계를 느낀 NIST에서 공모한 후 발표 • 블록 크기는 128비트이며, 키 길이에 따라 AES-128, AES-192, AES-256으로 분류됨
RSA(Rivest Shamir Adleman)	• 1978년 MIT의 라이베스트(Rivest), 샤미르(Shamir), 애들먼(Adelman)에 의해 제안된 공개키 암호화 알고리즘 • 큰 숫자를 소인수분해 하기 어렵다는 것에 기반하여 만들어짐
ECC(Elliptic Curve Cryptography)	• 1985년 RSA 암호 방식의 대안으로 제안되었음. • 이산대수 문제를 타원곡선으로 옮겨 기밀성과 효율성을 높인 암호화 알고리즘.
IDEA(International Data Encryption Algorithm)	• 스위스의 라이(Lai)와 메시(Messey)가 1990년에 개발한 PES를 개선한 알고리즘 • 블록 크기는 64비트이고, 키 길이는 128비트임

83. 1991년 R.rivest가 MD4를 개선한 암호화 알고리즘으로, 각각의 512 비트짜리 입력 메시지 블록에 대해 차례로 동작한다. 각 512 비트 입력 메시지 블록을 처리하고 나면 128 비트 스테이트(state)의 값이 변하는 암호화 알고리즘을 쓰시오. [20-1](#)

답: **MD5(Message Digest algorithm 5)**

84. 보안 위협에 관한 다음 설명에서 괄호에 공통으로 들어갈 알맞은 용어를 쓰시오. [21-3](#)

() 스푸핑은 로컬 네트워크(LAN)에서 사용하는 () 프로토콜의 취약점을 이용한 공격 기법으로, 자신의 물리적 주소(MAC)를 변조하여 다른 PC에게 도달해야 하는 데이터 패킷을 가로채거나 방해한다.

답: **ARP(Address Resolution Protocol)**

[해설] 주소 결정 프로토콜(Address Resolution Protocol)

85. 다음은 네트워크 공격에 대한 패킷 로그를 표현한 것이다. 아래의 패킷 로그와 같이 공격자가 패킷의 출발지 주소(Address) 또는 포트(Port)를 임의로 변경하여 송신측 IP 주소 또는 포트를 동일하게 함으로써 송신 IP 주소가 자신이므로 자신에게 응답을 수행하게 된다. 이처럼 자신에 대해 무한히 응답하는 패킷을 계속 전송하여 컴퓨터의 실행 속도를 느리게 하거나 동작을 마비 시켜 서비스 거부 상태에 빠지도록 하는 네트워크 공격 유형이 무엇인지 쓰시오. [20-1](#)

```
source : 192.168.1.200
destination : 192.168.1.200
protocol : 6
src port : 21845
dst port : 21845
```

답: **LAND Attack(Local Area Network Denial Attack)**

[해설] LAND Attack 또는 Local Area Network Denial Attack(랜드 공격)

86. 다음 설명에서 괄호(①, ②)에 들어갈 알맞은 용어를 쓰시오. [22-3](#)

- (①)은 컴퓨터 보안에 있어서, 인간 상호 작용의 깊은 신뢰를 바탕으로 사람들을 속여 정상 보안 절차를 깨뜨리기 위한 비기술적 시스템 침입 수단을 의미한다.
- (②)는 특정 목적을 가지고 데이터를 수집하였으나, 이후 활용되지 않고 저장만 되어있는 대량의 데이터를 의미한다. 미래에 사용될 가능성을 고려하여 저장 공간에서 삭제되지 않고 보관되어 있으나, 이는 저장 공간의 낭비뿐만 아니라 보안 위협을 초래할 수도 있다.

답: ① 사회 공학(Social Engineering) ② 다크 데이터(Dark Data)

87. 네트워크 및 인터넷 보안에 관련된 다음 설명에서 괄호에 공통으로 들어갈 알맞은 용어를 쓰시오. 21-1

- ()은 '세션을 가로채다'라는 의미로, 정상적인 연결을 RST 패킷을 통해 종료시킨 후 재연결 시 희생자가 아닌 공격자에게 연결하는 공격 기법이다.
- TCP ()은 공격자가 TCP 3-Way-Handshake 과정에 끼어들으로써 서버와 상호 간의 동기화된 시퀀스 번호를 갖고 인가되지 않은 시스템의 기능을 이용하거나 중요한 정보에 접근할 수 있게 된다.

답: 세션 하이재킹(Session Hijacking)

88. 보안 위협에 대한 다음 설명에 해당하는 용어를 <보기>에서 찾아 쓰시오. 22-1, 26-1

목표 조직이 자주 방문하는 웹 사이트를 사전에 감염시켜 목표 조직의 일원이 웹 사이트에 방문했을 때 악성 코드에 감염되게 한다. 이후에는 감염된 PC를 기반으로 조직의 중요 시스템에 접근하거나 불능으로 만드는 등의 영향력을 행사하는 웹 기반 공격이다.

<보기>

- | | | |
|-----------------|---------------------|--------------------|
| • Pharming | • Drive-by Download | • Watering Hole |
| • Business SCAM | • Phishing | • Cyber Kill Chain |
| • Ransomware | • Sniffing | |

답: Watering Hole

89. AAA 서버에 관한 다음 설명에서 각 번호(①~③)에 들어갈 알맞은 용어를 <보기>에서 찾아 쓰시오. 21-3

AAA 서버는 사용자의 컴퓨터 자원 접근 처리와 서비스 제공에 있어서의 다음 3가지 기능을 제공하는 서버이다.

- ① - 접근하는 사용자의 신원을 검증하는 기능
- ② - 신원이 검증된 사용자에게 특징된 권한과 서비스를 허용하는 기능
- ③ - 사용자가 어떤 종류의 서비스를 이용했고, 얼마만큼의 자원을 사용했는지 기록 및 보관하는 기능

<보기>

- | | | | |
|---------------|------------------|-------------|-----------------|
| • Application | • Authentication | • Avalanche | • Authorization |
| • Accounting | • Ascii | | |

답: ① Authentication ② Authorization ③ Accounting

[해설] Authentication (인증): 사용자의 신원 확인

Authorization (인가): 접근 권한 부여

Accounting (감사): 활동 기록 및 추적

90. 네트워크 보안에 대한 다음 설명에서 괄호에 공통으로 들어갈 알맞은 용어를 영문 약어로 쓰시오. 22-2

- ()은 인터넷 등 통신 사업자의 공중 네트워크와 암호화 기술을 이용하여 사용자가 마치 자신의 전용 회선을 사용하는 것처럼 해주게 하는 보안 솔루션이다.
- 암호화된 규격을 통해 인터넷망을 전용선의 시설망을 구축한 것처럼 이용하므로 비용 부담을 줄일 수 있다.
- ()을 사용하면 두 장치 및 네트워크 사이에 암호화된 보안 터널이 생성되며, 터널에 사용되는 프로토콜에 따라 SSL ()과 IPSec ()으로 불린다.

답: VPN

[해설] VPN(Virtual Private Network, 가상 사설망)

91. 다음 설명에 해당하는 알맞은 용어를 쓰시오. 22-3

다양한 장비에서 발생하는 로그 및 보안 이벤트를 통합하여 관리하는 보안 솔루션으로, 방화벽, IDS, IPS, 웹 방화벽, VPN 등에서 발생한 로그 및 보안 이벤트를 통합하여 관리함으로써 비용 및 자원을 절약할 수 있는 특징이 있다. 또한 보안 솔루션 간의 상호 연동을 통해 종합적인 보안 관리 체계를 수립할 수 있다.

답: SIEM(Security Information & Event Management)

[해설] 보안 정보 및 이벤트 관리(Security Information and Event Management)

92. 헝가리안 표기법(Hungarian Notation)에 대해 간략히 서술하시오. 20-3

답: 헝가리안 표기법은 변수명 작성시 변수의 자료형을 알 수 있도록 자료형을 의미하는 문자를 포함하여 작성하는 방법이다.

93. 스니핑(Sniffing)은 사전적 의미로 '코를 킁킁 거리다, 냄새를 맡다'이다. 네트워크 보안에서 스니핑에 대한 개념을 간략히 한 문장(1 문장)으로 쓰시오. 20-4

답: 스니핑은 네트워크의 중간에서 남의 패킷 정보를 도청하는 해킹 유형의 하나로 수동적 공격에 해당한다.

94. C++에서 생성자(Constructor)에 대해 간략히 서술하시오. 20-3

답: 생성자는 객체 변수 생성에 사용되는 메소드로, 객체 변수를 생성하면서 초기화를 수행한다.

95. 다음 설명에 해당하는 운영체제(OS)를 쓰시오. 20-4

- 1960년대 AT&T 벨(Bell) 연구소가 MIT, General Electric사와 함께 공동 개발한 운영체제이다.
- 시분할 시스템(Time Sharing System)을 위해 설계된 대화식 운영체제이다.
- 대부분 C 언어로 작성되어 있어 이식성이 높으며 장치, 프로세스 간의 호환성이 높다.
- 트리 구조의 파일 시스템을 갖는다.

답: UNIX(유닉스)

96. 다음은 Python의 리스트 객체에 속한 메소드들에 대한 설명이다. 각 괄호(①~③)에 해당하는 메소드의 이름을 <보기>에서 찾아 쓰시오. 22-1

Python에서는 여러 요소들을 한 개의 이름으로 처리할 때 리스트(List)를 사용하며, 각 요소에는 정수, 실수, 문자열 등 다양한 자료형을 섞어 저장할 수 있다. 또한 리스트는 메소드를 활용하여 요소를 추가 및 삭제할 수 있을 뿐만 아니라 정렬하거나 다른 리스트와 병합하는 등의 다양한 작업을 손쉽게 수행할 수 있다.

- (①): 기존 리스트에 인수의 요소들을 추가하여 확장하는 메소드로, 여러 값을 한 번에 추가할 수 있다.
- (②): 리스트에서 맨 마지막 또는 인수의 값에 해당하는 위치의 요소를 삭제한 후 반환한다.
- (③): 리스트에 저장된 각 요소들의 순서를 역순으로 뒤집어 저장하는 메소드이다.

<보기>

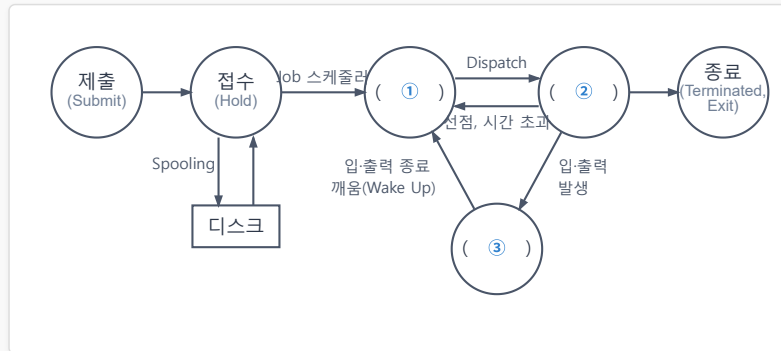
- | | | | | | |
|------------|----------|-------------|-----------|-----------|----------|
| • pop() | • push() | • reverse() | • index() | • write() | • sort() |
| • extend() | • copy() | | | | |

답: ① **extend()** ② **pop()** ③ **reverse()**

97. 리눅스의 커널 위에서 동작하며, 자바와 코틀린으로 애플리케이션을 작성하는 운영체제로, 휴대용 장치에서 주로 사용되는 이 운영체제의 이름을 쓰시오. [20-2](#)

답: **안드로이드(Android)**

98. 다음은 프로세스 상태 전이도이다. 괄호(①~③)에 들어갈 알맞은 상태를 쓰시오. [20-4](#)



답: ① **준비(Ready)** ② **실행(Run)** ③ **대기(Wait)**

99. HRN 비선점형 스케줄링의 우선순위를 구하는 계산식을 쓰시오. [20-1](#)

답: **(대기 시간 + 서비스 시간) / 서비스 시간**

100. 스케줄링에 대한 다음 설명에서 괄호(①~③)에 들어갈 알맞은 용어를 쓰시오. [22-3, 25-2](#)

- (①)는 준비상태 큐에서 기다리고 있는 프로세스들 중에서 실행 시간이 가장 짧은 프로세스에게 먼저 CPU를 할당하는 기법이다. 가장 적은 평균 대기 시간을 제공하는 최적 알고리즘이지만, 실행 시간이 긴 프로세스는 실행 시간이 짧은 프로세스에게 할당 순위가 밀려 무한 연기 상태가 발생할 수 있다.
- (②)는 시분할 시스템을 위해 고안된 방식으로, 준비상태 큐에 먼저 들어온 프로세스가 먼저 CPU를 할당받지만 각 프로세스는 시간 할당량 동안만 실행한 후 실행이 완료되지 않으면 다음 프로세스에게 CPU를 넘겨주고 준비상태 큐의 가장 뒤로 배치된다. 할당되는 시간이 작을 경우 문맥 교환 및 오버헤드가 자주 발생되어 요청된 작업을 신속히 처리할 수 없다.
- (③)는 현재 실행중인 프로세스의 남은 시간과 준비상태 큐에 새로 도착한 프로세스의 실행 시간을 비교하여 가장 짧은 실행 시간을 요구하는 프로세스에게 CPU를 할당하는 기법으로, 시분할 시스템에 유용하다. 준비상태 큐에 있는 각 프로세스의 실행 시간을 추적하여 보유하고 있어야 하므로 오버헤드가 증가한다.

답: ① **SJF(Shortest Job First)** ② **RR(Round Robin)**
③ **SRT(Shortest Remaining Time)**

[해설]

• 스케줄링 기법

비선점 (Non-preemptive) 스케줄링	• 이미 할당된 CPU를 다른 프로세스가 강제로 빼앗아 사용할 수 없는 스케줄링 기법 • 종류: FCFS(FIFO), SJF, HRN, 기한부, 우선순위 등
---------------------------------	---

선점 (Preemptive) 스케줄링	• 하나의 프로세스가 CPU를 할당받아 실행하고 있을 때 우선순위가 높은 다른 프로세스가 CPU를 강제로 빼앗아 사용할 수 있는 스케줄링 기법 • 종류: 선점 우선순위, SRT, Round Robin, 다단계 큐(MQ), 다단계 피드백 큐(MFQ) 등
-----------------------------------	---

• 비선점 스케줄링의 종류

FCFS (First-Come First-Served) = FIFO (First-In First-Out)	준비상태 큐에 도착한 순서에 따라 차례로 CPU를 할당하는 기법
SJF (Shortest Job First)	준비상태 큐에서 기다리고 있는 프로세스들 중에서 실행 시간이 가장 짧은 프로세스 에 게 먼저 CPU를 할당하는 기법
HRN (Highest Response-ratio Next)	• 실행 시간이 긴 프로세스에 불리한 SJF 기법을 보완하기 위한 것으로, 대기 시간과 서비스(실행) 시간을 이용하는 기법 • 우선순위를 계산하여 그 숫자가 가장 높은 것부터 낮은 순으로 우선순위가 부여됨
기한부 (Deadline)	• 프로세스에게 일정한 시간을 주어 그 시간 안에 프로세스를 완료하도록 하는 기법 • 프로세스가 제한된 시간 안에 완료되지 않을 경우 제거되거나 처음부터 다시 실행해야 함
우선순위 (Priority)	• 준비상태 큐에서 기다리는 각 프로세스마다 우선순위를 부여하여 그 중 가장 높은 프로세스에게 먼저 CPU를 할당하는 기법 • 우선순위가 동일할 경우 FCFS 기법으로 CPU를 할당함

• 선점 스케줄링의 종류

선점 우선순위	준비상태 큐의 프로세스들 중에서 우선순위가 가장 높은 프로세스에게 먼저 CPU를 할당하는 기법
SRT (Shortest Remaining Time)	• 비선점 스케줄링인 SJF 기법을 선점 형태로 변경한 기법으로, 선점 SJF 기법이라고도 함 • 현재 실행중인 프로세스의 남은 시간과 준비상태 큐에 새로 도착한 프로세스의 실행 시간을 비교하여 가장 짧은 실행 시간을 요구하는 프로세스에게 CPU를 할당 함
RR (Round Robin)	• 시분할 시스템(Time Sharing System)을 위해 고안된 방식으로, FCFS 알고리즘을 선점 (Preemptive) 형태로 변형한 기법 • FCFS 기법과 같이 준비상태 큐에 먼저 들어온 프로세스가 먼저 CPU를 할당받지만 각 프로세스는 시간 할당량(Time Slice, Quantum) 동안만 실행한 후 실행이 완료되지 않으면 다음 프로세스에게 CPU를 넘겨주고 준비상태 큐의 가장 뒤로 배치됨
다단계 큐 (MQ , Multi-level Queue)	• 프로세스들을 특정 그룹으로 분류할 수 있을 경우 그룹에 따라 각기 다른 준비상태 큐를 사용하는 기법 • 준비상태 큐를 상위 단계(작업 큐)와 하위 단계(사용자 큐)로 분할하여 운영함 • 각 큐는 독자적인 스케줄링 알고리즘을 가지고 있으며, 형제 큐 사이에는 절대적인 우선순위가 존재함 • 우선순위가 낮은 큐의 프로세스는 실행 중이더라도 우선순위가 높은 큐에 프로세스가 도착하면 CPU를 빼앗김(선점당함)
다단계 피드백 큐 (MFQ , Multi-level Feedback Queue)	• 특정 그룹의 프로세스가 하위 큐로 이동하지 못하는 다단계 큐 기법을 준비상태 큐 사이를 이동할 수 있도록 개선한 기법 • 각 큐마다 시간 할당량을 부여하며, 부여된 시간 할당량 동안 완료하지 못한 프로세스는 다음 단계의 큐로 이동함(단, 마지막 단계 큐는 FCFS 기법 사용)

101. 리눅스 또는 유닉스에서 'a.txt' 파일에 대해 다음 <처리 조건>과 같이 권한을 부여하고자 한다. <처리 조건>을 준수하여 적합한 명령문을 작성하시오. [20-2, 23-3](#)

<처리 조건>

- 사용자에게 읽기, 쓰기, 실행 권한을 부여한다.
- 그룹에게 읽기, 실행 권한을 부여한다.
- 기타 사용자에게 실행 권한만 부여한다.
- 한 줄로 작성하고, 8진법 숫자를 이용한 명령문을 이용한다.

답: **chmod 751 a.txt**

[해설]

사용자는 읽기, 쓰기, 실행 권한이 모두 있으므로 **rwX**

그룹은 읽기, 실행 권한만 있으므로 **r-X**

기타 사용자는 실행 권한만 있으므로 **--X**가 됩니다.

이를 8진수로 변환하는 과정은 다음과 같습니다.

```

rwX  r-X  --X
  ↓      (·는 0, 나머지는 1로 바꾸어 준다)
111  101  001
  ↓      (3자리 2진수를 8진수로 변환한다. 111 = 7, 101 = 5, 001 = 1)
7    5    1
  ↓      (chmod 명령문을 완성한다)
chmod 751 a.txt

```

102. 인터넷에 대한 다음 설명에서 괄호(①, ②)에 들어갈 알맞은 숫자를 쓰시오. 21-1

1. IPv6는 (①) 비트의 주소를 가지며, 인증성, 기밀성, 데이터 무결성의 지원으로 보안 문제를 해결할 수 있고, 주소의 확장성, 융통성, 연동성이 뛰어나다.
2. IPv4는 32 비트의 주소를 가지며 (②) 비트씩 4부분, 총 32비트로 구성되어 있다. IPv4는 네트워크 부분의 길이에 따라 A 클래스에서 E 클래스까지 총 5단계로 구성되어 있다.

답: ① **128** ② **8**

103. 192.168.1.0/24 네트워크를 FLSM 방식을 이용하여 3개의 Subnet으로 나누었을 때, 두 번째 네트워크의 브로드캐스트 IP 주소를 10진수 방식으로 쓰시오. 22-3

답: **192.168.1.127**

[해설]

- 192.168.1.0/24에서 '/24'는 서브넷 마스크를 의미합니다. 192.168.1.0를 2진수로 표현하면 11000000 10101000 00000001 00000000

즉 서브넷 마스크는 1의 개수가 24개라는 것으로 이를 2진수로 표현하면 11111111 11111111 11111111 00000000입니다. 10진수로 바꾸면 255 255 255 0

- 서브넷 마스크를 Subnet으로 나눌 때는 서브넷 마스크가 0인 부분, 즉 마지막 8비트를 이용하면 됩니다.
- Subnet으로 나눌 때 "3개의 Subnet으로 나눈다"는 것처럼 네트워크가 기준일 때는 왼쪽을 기준으로 나눌 네트워크 수에 필요한 비트를 할당하고 나머지 비트로 호스트를 구성하면 됩니다.
- 3개의 Subnet으로 구성하러 했으니 8비트 중 3을 표현하는데 필요한 2비트($2^2=4$)를 제외하고 나머지 6비트를 호스트로 구성하면 됩니다.
- 네트워크 개수 = 4개($=2^2$ (필요한 비트 수))
- 호스트 개수 = 256 / 네트워크 개수 = 256 / 4 = 64개($=2^6$ (남은 비트 수))
- 문제에서 FLSM(Fixed Length Subnet Mask), 즉 고정된 크기로 주소를 할당하러 했으므로 다음 표와 같이 64개의 호스트를 갖는 4개의 네트워크로 나눌 수 있으나 문제에서 3개의 Subnet을 구성하러 하였으므로 4번 네트워크는 사용되지 않습니다.
- 네트워크별로 첫 번째 주소는 네트워크 주소이고, 마지막 주소는 브로드캐스트 주소입니다.

네트워크	네트워크 주소	브로드캐스트 주소	Network IP	가용한 주소	broadcast IP
1	192.168.1.0	192.168.1.63	0	1 ~ 62	63
2	192.168.1.64	192.168.1.127	64	65 ~ 126	127
3	192.168.1.128	192.168.1.191	128	129 ~ 190	191
4	192.168.1.192	192.168.1.255	192	193 ~ 254	255

∴ 3개의 Subnet으로 나누어진 위의 네트워크에서 두 번째 네트워크의 브로드캐스트 주소는 **192.168.1.127**입니다.

104. 서브네팅(Subnetting)에 대한 다음 설명에서 괄호(①, ②)에 들어갈 알맞은 숫자를 쓰시오. 22-2

현재 IP 주소가 192.168.1.132이고, 서브넷 마스크가 255.255.255.192일 때, 네트워크 주소는 192.168.1.(①)이고, 해당 네트워크에서 네트워크 주소와 브로드캐스트 주소를 제외한 사용 가능한 호스트의 수는 (②)개이다.

답: ① **128** ② **62**

[해설]

• IP 주소는 네트워크 부분의 길이에 따라 다음과 같이 A 클래스에서 E 클래스까지 총 5단계로 구성되어 있으며, 각 클래스는 IP 주소의 앞자리로 구분할 수 있습니다.

A Class	0~127로 시작
B Class	128~191로 시작
C Class	192~223으로 시작
D Class	224~239로 시작
E Class	공용되지 않음

- 192.168.1.132는 C 클래스에 속한 주소로, C 클래스의 기본 서브넷 마스크는 255.255.255.0입니다. 이를 2진수로 표현하면 11111111 11111111 11111111 00000000으로, 1의 개수가 24개입니다.
- 문제에 주어진 서브넷 마스크 255.255.255.192를 2진수로 표현하면 11111111 11111111 11111111 11000000으로, C 클래스의 기본 서브넷 마스크보다 1의 개수가 2개, 즉 2비트가 많습니다. 이 2비트를 이용해 네트워크의 개수와 네트워크 안에 포함된 호스트의 개수를 계산합니다.
- 네트워크 개수 = $4^{(추가된 비트 수)}$
- 호스트 개수 = $256 / \text{네트워크 개수} = 256 / 4 = 64$
- 다음 표와 같이 64개의 호스트를 갖는 4개의 네트워크로 나눌 수 있습니다.
- 네트워크별로 첫 번째 주소는 네트워크 주소이고, 마지막 주소는 브로드캐스트 주소입니다.

네트워크	네트워크 주소	브로드캐스트 주소	Network IP	가용한 주소	broadcast IP
1	192.168.1.0	192.168.1.63	0	1 ~ 62	63
2	192.168.1.64	192.168.1.127	64	65 ~ 126	127
3	192.168.1.128	192.168.1.191	128	129 ~ 190	191
4	192.168.1.192	192.168.1.255	192	193 ~ 254	255

- 192.168.1.132는 세 번째 네트워크에 포함되어 있으며, 세 번째 네트워크의 네트워크 주소는 **192.168.1.128**입니다.
- 호스트의 수는 네트워크마다 64개의 호스트를 가지므로 64개이지만, 문제에서 네트워크 주소와 브로드캐스트 주소를 제외한다고 하였으므로 사용 가능 호스트의 수는 **62**개입니다.

105. 다음 IP 주소와 서브넷 마스크에 대한 내용에서 괄호(①, ②)에 들어갈 알맞은 숫자를 쓰시오. 25-2

호스트의 IP 주소가 223.13.234.132이고, 서브넷 마스크가 255.255.255.192일 때, 호스트가 속한 네트워크 주소는 223.13.234.(①)이고, 해당 네트워크에서 사용할 수 있는 호스트 수는 (②)개이다.

답: ① **128** ② **62**

[해설]

• IP 주소는 네트워크 부분의 길이에 따라 다음과 같이 A 클래스에서 E 클래스까지 총 5단계로 구성되어 있으며, 각 클래스는 IP 주소의 앞자리로 구분할 수 있습니다.

A Class	0~127로 시작
B Class	128~191로 시작
C Class	192~223으로 시작
D Class	224~239로 시작
E Class	공용되지 않음

- 223.13.234.132는 첫 번째 옥텟이 223이므로 C 클래스 영역에 해당하며, C 클래스의 서브넷 마스크 기본값은 255.255.255.0(2진수 유효 비트 24개)입니다.
- 문제에서 주어진 마스크 값 255.255.255.192를 이진수로 나열하면 11111111 11111111 11111111 11000000이 됩니다. 디폴트 마스크보다 마지막 단락(4옥텟)에서 호스트 비트 2비트가 서브넷 비트로 변환된 상태입니다.
- 분할된 총 서브넷(네트워크) 수 = $2^2 = 4$ 개
- 각 세그먼트당 할당되는 호스트 총 개수 = $256 / 4 = 64$ 개
- 이에 따라 아래 구조와 같이 64개 단위 크기를 갖는 총 4개의 격리된 서브넷 서프 대역 구조가 형성됩니다.

네트워크	네트워크 주소	브로드캐스트 주소
1	223.13.234.0	223.13.234.63
2	223.13.234.64	223.13.234.127
3	223.13.234.128	223.13.234.191
4	223.13.234.192	223.13.234.255

Network IP	가용한 주소 범위	broadcast IP
0	1 ~ 62	63
64	65 ~ 126	127
128	129 ~ 190	191
192	193 ~ 254	255

- 문제의 목적 호스트 IP인 223.13.234.132는 수치 비교 시 세 번째 네트워크 스코프 대역(128 ~ 191) 사이에 속해 있습니다. 따라서 이 호스트의 네트워크 시작 주소인 대표 ID는 **223.13.234.128**이 됩니다.
- 해당 네트워크 블록 내의 전체 할당량은 64개이지만, 시스템 예약으로 고정 배정되는 '네트워크 주소(128)'와 '브로드캐스트 주소(191)' 총 2개는 일반 클라이언트 및 노드에 직접 할당할 수 없습니다. 따라서 이를 제외한 실제 사용 가능 호스트 수는 총 **62**개입니다.

Ip:	223	13	234	132
sm:	255	255	255	192
Ip (IP 주소):	11011111	00001101	11101010	10000100
sm (서브넷 마스크):	11111111	11111111	11111111	11000000
and(시작주소):	11011111	00001101	11101010	10000000
and(마지막주소):	11011111	00001101	11101010	10111111

시작주소: 223.13.234.128

마지막주소: 223.13.234.191

사용가능 주소: 223.13.234.128 ~ 223.13.234.191 사이에 있는 값만 사용가능 따라서 $190 - 128 = 62$

105. 네트워크 및 인터넷과 관련된 다음 설명에 해당하는 용어를 영문(Full name 또는 약어)으로 쓰시오.

20-4

- 현재 사용하고 있는 IP 주소 체계인 IPv4의 주소 부족 문제를 해결하기 위해 개발되었다.
- 16비트씩 8부분, 총 128비트로 구성되어 있다.
- 각 부분을 16진수로 표현하고, 콜론(:)으로 구분한다.

- 인증성, 기밀성, 데이터 무결성의 지원으로 보안 문제를 해결할 수 있다.
- 주소의 확장성, 융통성, 연동성이 뛰어나며, 실시간 흐름 제어로 향상된 멀티미디어 기능을 지원한다.

답: **IPv6(Internet Protocol version 6)**

106. OSI 7 Layer에 대한 다음 설명에서 각 번호(①~③)에 들어갈 알맞은 계층(Layer)을 쓰시오. 21-3

OSI 7 Layer는 다른 시스템 간의 원활한 통신을 위해 ISO(국제표준화기구)에서 제안한 통신 규약(Protocol)이다.

- ① - 물리적으로 연결된 두 개의 인접한 개방 시스템들 간에 신뢰성 있고 효율적인 정보 전송을 할 수 있도록 연결 설정, 데이터 전송, 오류 제어 등의 기능을 수행한다.
- ② - 개방 시스템들 간의 네트워크 연결을 관리하며, 경로 제어, 패킷 교환, 트래픽 제어 등의 기능을 수행한다.
- ③ - 서로 다른 데이터 표현 형태를 갖는 시스템 간의 상호 접속을 위해 필요한 계층으로, 코드 변환, 데이터 암호화, 데이터 압축, 구문 검색 등의 기능을 수행한다.

답: ① **데이터 링크 계층(Data Link Layer)**
 ② **네트워크 계층(망 계층/Network Layer)**
 ③ **표현 계층(Presentation Layer)**

107. 다음이 설명하는 OSI 참조 모델의 계층을 쓰시오. 20-1

- 다양한 전송매체를 통해 비트 스트림을 전송한다.
- 전송에 필요한 두 장치 간의 실제 접속과 절단 등 기계적, 전기적, 기능적, 절차적 특성에 대한 규칙을 정의한다.

답: **물리 계층(Physical Layer)**

108. 다음 네트워크 관련 설명에서 괄호에 들어갈 알맞은 용어를 쓰시오. 20-3

심리학자 톰 마릴은 컴퓨터가 메시지를 전달하고, 메시지가 제대로 도착했는지 확인하며, 도착하지 않았을 경우 메시지를 재전송하는 일련의 방법을 '기술적 은어'를 뜻하는 ()이라는 용어로 정의하였다.

답: **프로토콜(Protocol)**

109. 프로토콜은 서로 다른 기기들 간의 데이터 교환을 원활하게 수행할 수 있도록 표준화시켜 놓은 통신 규약이다. 프로토콜의 기본 요소 3가지를 쓰시오. 20-1

답: **구문(Syntax), 의미(Semantics), 시간(Timing)**

110. 인터넷에 대한 다음 설명에서 괄호(①~③)에 들어갈 알맞은 용어를 <보기>에서 찾아 쓰시오. 22-2

인터넷이란 TCP/IP 프로토콜을 기반으로 하여 전 세계 수많은 컴퓨터와 네트워크들이 연결된 광범위한 컴퓨터 통신망이다.

- (①) : 월드 와이드 웹(WWW)에서 HTML 문서를 송수신 하기 위한 표준 프로토콜로, GET과 POST 메소드를 통해 메시지를 주고 받는다.
- (②) : 다른 문서나 그림으로 이동할 수 있는 연결을 가지고 있는 텍스트를 의미한다.
- (③) : 인터넷의 표준 문서인 하이퍼텍스트 문서를 만들기 위해 사용하는 언어이다.

<보기>

- | | | | | | |
|--------|--------|--------|-------------|---------|--------|
| • UDDI | • XML | • WSDL | • Hypertext | • DHTML | • HTML |
| • SOAP | • HTTP | | | | |

답: ① HTTP ② Hypertext ③ HTML

111. 네트워크에 관련된 다음 설명에서 괄호(①, ②)에 들어갈 알맞은 용어를 쓰시오. [21-2, 23-1, 24-2](#)

- (①) : 연결형 통신에서 주로 사용되는 방식으로, 출발지와 목적지의 전송 경로를 미리 연결하여 논리적으로 고정된 후 통신하는 방식
- (②) : 비연결형 통신에서 주로 사용되는 방식으로, 사전에 접속 절차를 수행하지 않고 헤더에 출발지에서 목적지까지의 경로 지정을 위한 충분한 정보를 붙여서 개별적으로 전달하는 방식

<보기>

• 회선 교환 방식 • 데이터그램 방식 • 가상 회선 방식 • 메시지 교환 방식

답: ① 가상 회선(Virtual Circuit) ② 데이터그램(Datagram)

112. 네트워크 및 인터넷과 관련된 다음 설명에 해당하는 용어를 쓰시오. [21-1](#)

인터넷 환경에서의 호스트 상호 간 통신에서 연결된 네트워크 접속 장치의 물리적 주소인 MAC 주소를 이용하여 IP 주소를 찾는 인터넷 계층의 프로토콜로, 역순 주소 결정 프로토콜이라 불린다.

답: RARP(Reverse Address Resolution Protocol)

[해설] 역순 주소 결정 프로토콜 (Reverse Address Resolution Protocol)

113. 다음 네트워크 관련 설명에서 괄호에 들어갈 알맞은 용어를 영문(Full name 또는 약어)으로 쓰시오. [20-3, 23-1](#)

IP(Internet Protocol)의 주요 구성원 중 하나로, OSI 계층 모델의 네트워크 계층에 속한다. 네트워크 컴퓨터의 운영체제에서 오류 메시지를 수신하거나, 전송 경로를 변경하는 등 오류 처리를 위한 제어 메시지를 주로 취급한다. 관련된 도구로 traceroute, ping이 있으며, ping of death와 같은 네트워크 공격 기법에 활용되기도 한다.

()는 TCP/IP 기반의 인터넷 통신 서비스에서 인터넷 프로토콜(IP)과 조합하여 통신 중에 발생하는 오류의 처리와 전송 경로의 변경 등을 위한 제어 메시지를 취급하는 무연결 전송용 프로토콜로, OSI 기본 참조 모델의 네트워크 계층에 속한다.

답: ICMP(Internet Control Message Protocol)

114. 다음 설명에서 괄호에 들어갈 알맞은 용어를 쓰시오. [22-3](#)

()는 한 번의 로그인으로 개인이 가입한 모든 사이트를 이용할 수 있게 해주는 시스템을 말한다. 개인의 경우, 사이트에 접속하기 위하여 아이디와 패스워드는 물론 이름, 전화번호 등 개인정보를 각 사이트마다 일일이 기록해야 하던 것이 한 번의 작업으로 끝나므로 불편함이 해소되며, 기업에서는 회원에 대한 통합관리가 가능해 마케팅을 극대화시킬 수 있다는 장점이 있다.

답: SSO(Single Sign On)

115. 네트워크 및 인터넷과 관련된 다음 설명에 해당하는 용어를 쓰시오. [21-2](#)

- 재난 및 군사 현장과 같이 별도의 고정된 유선망을 구축할 수 없는 장소에서 모바일 호스트(Mobile Host)만을 이용하여 구성한 네트워크이다.
- 망을 구성한 후 단기간 사용되는 경우나 유선망을 구성하기 어려운 경우에 적합하다.
- 멀티 홉 라우팅 기능을 지원한다.

답: 애드 혹 네트워크(Ad-hoc Network)

116. 네트워크에 대한 다음 설명에 해당하는 용어를 쓰시오. [20-4, 23-3](#)

- 우리말로 번역하면 '네트워크 주소 변환'이라는 의미의 영문 3글자 약사이다.
- 1개의 정식 IP 주소에 다량의 가상 사설 IP 주소를 할당 및 연결하는 방식이다.
- 1개의 IP 주소를 사용해서 외부에 접속할 수 있는 노드는 어느 시점에서 1개만으로 제한되는 문제가 있으나, 이 때에는 IP 마스커레이드(Masquerade)를 이용하면 된다.

답: **NAT(Network Address Translation)**

[해설]

네트워크 주소 변환 (NAT: Network Address Translation) 기술 분석

- **주요 개념:** 내부 네트워크에서 사용하는 **사설 IP 주소(Private IP)**를 외부 인터넷망에서 통신이 가능한 **공인 IP 주소(Public IP)**로 상호 변환해 주는 주소 매핑 기술입니다.
- **도입 목적 및 효과:** 기관 내부적으로 한정된 공인 IP 자원을 효율적으로 절약할 수 있으며, 외부 인터넷망에서는 내부 사설망의 실제 주소 구조를 파악할 수 없기 때문에 라우팅 보안성을 한층 강화하는 핵심 차단벽 역할을 수행합니다.
- **IP 마스커레이드(Masquerade):** 지문에 언급된 기법으로, 여러 개의 사설 IP가 1개의 공인 IP를 공유할 때 발생하는 동시 접속 한계를 극복하기 위해, 포트 번호(Port Number)를 각각 다르게 매핑하여 식별하는 **N:1(NAPT) 변환 방식**을 뜻합니다.

117. 다음 설명에서 가리키는 용어를 쓰시오. [20-2](#)

가. 정의

웹상에 존재하는 데이터를 개별 URI(Uniform Resource Identifier)로 식별하고, 각 URI에 링크 정보를 부여함으로써 상호 연결된 웹을 지향하는 모형이다. 링크 기능이 강조된 시맨틱웹의 모형에 속한다고 볼 수 있으며 텀 비너스리의 W3C를 중심으로 발전하고 있다.

나. 주요 기능

1. 공개된 데이터를 이용하면 내가 원하는 데이터가 이미 존재하는지, 어디에 존재하는지 알 수 있다.
2. URI로 구별되는 데이터 리소스의 자유로운 접근 및 이용이 가능하므로 큰 노력 없이 데이터의 매쉬업이 가능하다.
3. 내가 만든 데이터가 아니라도 URI를 이용하여 링크만 해주면 이용할 수 있다.

다. 4대 원칙

1. 통합 자원 식별자(URI)를 사용한다.
2. URI는 HTTP 프로토콜을 통해 접근할 수 있어야 한다.
3. RDF나 스파클 같은 표준을 사용한다.
4. 풍부한 링크 정보가 있어야 한다.

라. Linked Data와 Open Data를 결합한 용어이다.

답: **LOD(개방형 링크드 데이터)**

118. 경로 제어 프로토콜(Routing Protocol)에 대한 다음 설명에서 괄호(①~④)에 들어갈 알맞은 용어를 <보기>에서 찾아 쓰시오. [22-2](#) [223로](#)

- 경로 제어 프로토콜은 크게 자율 시스템 내부의 라우팅에 사용되는 (①)와 자율 시스템 간의 라우팅에 사용되는 (②)로 구분할 수 있다.
- (①)는 소규모 동종 자율 시스템에서 효율적인 RIP와 대규모 자율 시스템에서 많이 사용되는 (③)로 나누어진다.
 - (③)는 링크 상태(Link State)를 실시간으로 반영하여 최단 경로로 라우팅을 지원하는 특징이 있다.

• (④)는 (②)의 단점을 보완하여 만들어진 라우팅 프로토콜로, 처음 연결될 때는 전체 라우팅 테이블을 교환하고, 이후에는 변화된 정보만을 교환한다.

<보기>

- | | | | | | |
|-------|-------|-------|--------|-------|-------|
| • BGP | • AS | • HOP | • OSPF | • NAT | • ISP |
| • EGP | • IGP | | | | |

답: ① **IGP** ② **EGP** ③ **OSPF** ④ **BGP**

[해설]

IGP(Interior Gateway Protocol): 내부 라우팅 프로토콜
EGP(Exterior Gateway Protocol): 외부 라우팅 프로토콜
OSPF(Open Shortest Path First): 최단 경로 우선 프로토콜
BGP(Border Gateway Protocol): 경계 경로 프로토콜

119. 피투피(P2P) 네트워크를 이용하여 온라인 금융 거래 정보를 온라인 네트워크 참여자(Peer)의 디지털 장비에 분산 저장하는 기술을 쓰시오. [20-4](#)

답: **블록체인(Blockchain)**

120. 정보 보호에 대한 다음 설명에 해당하는 용어를 영문 약어로 쓰시오. [22-1, 26-1](#)  **001로**

정보 자산을 안전하게 보호하기 위한 보호 절차와 대책으로, 정보보호 관리 체계라고 한다. 조직에 맞는 정보보호 정책을 수립하고, 위험에 상시 대응하는 여러 보안 대책을 통합 관리한다. 공공 부문과 민간 기업 부문에서 이것을 평가하고 인증하는 사업을 한국인터넷진흥원(KISA)에서 운영중이다.

답: **ISMS**

[해설] ISMS(Information Security Management System): 정보보호 관리체계

121. 보안 및 보안 위협에 대한 다음 설명에서 괄호(①, ②)에 들어갈 알맞은 용어를 <보기>에서 찾아 쓰시오. [22-3](#)

- (①)은 칩 설계회사인 ARM(Advanced RISC Machine)에서 개발한 기술로, 하나의 프로세서(Processor) 내에 일반 애플리케이션을 처리하는 일반 구역(Normal World)과 보안이 필요한 애플리케이션을 처리하는 보안 구역(Secure World)으로 분할하여 관리하는 하드웨어 기반의 보안 기술이다.
- (②)은 네티즌들이 사이트에 접속할 때 주소를 잘못 입력하거나 철자를 빠뜨리는 실수를 이용하기 위해 이와 유사한 유명 도메인을 미리 등록하는 것으로 URL 하이재킹(Hijacking)이라고도 한다. 유명 사이트들의 도메인을 입력할 때 발생할 수 있는 온갖 도메인 이름을 미리 선점해 놓고 이용자가 모르는 사이에 광고 사이트로 이동하게 만든다.

<보기>

- | | | | |
|-----------------|--------------|--------------------|------------|
| • Pharming | • Tvishing | • Trustzone | • APT |
| • Typosquatting | • Hacktivism | • Watering Holidiv | • Smurfing |
| • Ransomware | • CSRF | | |

답: ① **Trustzone** ② **Typosquatting**

[해설] 트러스트존(TrustZone), 타이포스쿼팅(Typosquatting)

122. 다음 설명의 RAID에 해당하는 레벨(Level)을 답란의 괄호 안에 숫자로 쓰시오. [22-1](#)

여러 개의 하드디스크로 디스크 배열을 구성하고, 파일을 구성하는 데이터 블록들을 서로 다른 디스크에 분산하여 저장할 경우 저장된 블록들을 여러 디스크에서 동시에 읽고 쓸 수 있으므로 디스크의 속도가 매우 향상되는데, 이 기술을 RAID라고 한다. 레이드 방식 중 패리티가 없는 스트라이핑된 2개 이상의 디스크를 병렬로 연결하여 구성하는 이 방식은 디스크의 개수가 증가할수록 입출력 속도 및 저장 용량이 배로 증가하지만, 하나의 디스크만 손상되어도 전체 데이터가 유실되는 문제가 발생한다.

답: Level (0)

[해설] 리눅스에서의 레이드(RAID)는 여러 개의 하드 드라이브를 묶어서 하나의 논리적인 드라이브로 인식하게 하는 기술

RAID 0: RAID 0은 스트라이핑(Stripping) 기술을 사용하여 데이터를 여러 드라이브에 분산. 이로 인해 입출력 성능이 향상되지만, 하나의 드라이브가 손상되면 전체 데이터가 손실될 수 있으므로 데이터 복구 및 안정성이 없다.

- 2개 디스크 존재시 번갈아가면서 데이터 저장, 각 디스크가 2TB라면 총 가용 메모리는 4TB

RAID 1: RAID 1은 미러링(Mirroring) 기술을 사용하여 데이터를 여러 드라이브에 중복 저장한다. 한 드라이브가 손상되어도 나머지 드라이브에 데이터가 유지되므로 높은 안정성을 제공한다. 하지만 저장 공간의 절반이 중복되므로 저장 용량은 실제 드라이브의 절반이 된다.

- 2개 디스크 존재 시 하나의 디스크에 데이터가 저장되고 나머지 디스크에는 미러링되는 내용이 저장. 각 디스크가 2TB라면 총 가용 메모리는 2TB

RAID 5: RAID 5는 데이터와 패리티 정보를 여러 드라이브에 분산 저장하여 안정성을 제공하는 레벨. 패리티는 손상된 드라이브에서 데이터를 복구하는 데 사용된다. 최소 3개의 드라이브가 필요하며, 하나의 드라이브가 손상되어도 데이터를 복구할 수 있다.

- 패리티 : 장애가 발생한 뒤에 데이터를 복원하기 위해서 사용되는 부호

RAID 6: RAID 6은 RAID 5와 유사하지만 패리티 정보를 두 개의 드라이브에 저장하여 두 개의 드라이브가 동시에 손상되어도 데이터를 복구할 수 있다. 최소 4개의 드라이브가 필요하다.

RAID 10 (RAID 1+0): RAID 10은 RAID 1과 RAID 0을 결합한 방식으로, 미러링과 스트라이핑을 모두 사용하여 안정성과 성능을 모두 제공합니다. 최소 4개의 드라이브가 필요하며, 드라이브의 절반이 데이터 저장용이고 나머지 절반은 미러링에 사용된다.

123. 분산 컴퓨팅에 대한 다음 설명에 해당하는 용어를 쓰시오. 20-4

- 오픈 소스 기반 분산 컴퓨팅 플랫폼이다.
- 분산 저장된 데이터들은 클러스터 환경에서 병렬 처리된다.
- 일반 PC급 컴퓨터들로 가상화된 대형 스토리지를 형성하고 그 안에 보관된 거대한 데이터 세트를 병렬로 처리할 수 있도록 개발되었다.
- 더그 커팅과 마이크 캐퍼렐라가 개발했으며, 구글의 맵리듀스(MapReduce) 엔진을 사용하고 있다.

답: 하둡(Hadoop)

124. 데이터 마이닝(Data Mining)의 개념을 간략히 서술하시오. 20-1

답: 데이터 마이닝은 대량의 데이터를 분석하여 데이터에 내재된 변수 사이의 상호관계를 규명하여 일정한 패턴을 찾아내는 기법이다.

125. 데이터베이스 회복(Recovery) 기법에 관련된 다음 설명에서 괄호에 공통적으로 들어갈 가장 알맞은 용어를 쓰시오. 20-4

()은 트랜잭션이 데이터를 변경하면 트랜잭션이 부분 완료되기 전이라도 즉시 실제 DB에 그 내용을 반영하는 기법으로, 장애가 발생하여 회복 작업할 경우를 대비하여 갱신된 내용들을 로그(Log)에 보관시킨다. ()에서 회복 작업을 수행할 경우 Redo와 Undo 모두 수행이 가능하다.

답: 즉시 갱신 기법(Immediate Update)

126. 데이터베이스의 병행제어(Concurrency Control) 기법 중 하나로, 접근한 데이터에 대한 연산을 모두 마칠 때까지 추가적인 접근을 제한함으로써 상호 배타적으로 접근하여 작업을 수행하도록 하는 기법을 쓰시오. [21-2](#)

답: **로킹(Locking)**

127. 릴리즈 노트는 개발 과정에서 정리된 릴리즈 정보를 소프트웨어의 최종 사용자인 고객과 공유하기 위한 문서이다. 릴리즈 노트는 정확하고 완전한 정보를 기반으로 개발팀에서 직접 현재 시제로 작성해야 한다. 릴리즈 노트 작성 시 릴리즈 노트 이름, 소프트웨어 이름, 릴리즈 버전, 릴리즈 날짜, 릴리즈 노트 날짜, 릴리즈 노트 버전 등이 포함된 항목을 쓰시오. [20-1](#)

답: **머릿말(Header)**

128. 제시된 보기 중 형상 관리 도구에 해당하는 것을 3가지 고르시오. [22-3](#)

<보기>

• OLAP	• CVS	• Ant	• Maven	• Git	• Jenkins
• Spring	• SVN				

답: **CVS, Git, SVN**

[해설] CVS(Concurrent Version System), Git, SVN(Subversion)

129. 소프트웨어 개발에서의 작업 중 형상 통제에 대해 간략히 서술하시오. [20-3](#)

답: **형상 통제는 식별된 형상 항목에 대한 변경 요구를 검토하여 현재의 기준선이 잘 반영될 수 있도록 조정하는 작업이다.**

130. 다음 설명에서 괄호에 들어갈 알맞은 용어를 쓰시오. [20-2](#)

소프트웨어 ()는 소프트웨어 개발 단계의 각 과정에서 만들어지는 프로그램, 프로그램을 설명하는 문서, 데이터 등을 관리하는 것을 말한다. 소프트웨어의 개발 과정에서 만들어지는 여러 버전들의 변경 사항을 관리하는 일련의 활동이며 이를 지원하는 도구로 Git, SVN 등이 있다.

답: **형상관리(SCM)**

[해설] 형상 관리(SCM: Software Configuration Management)

131. 다음 설명에 해당하는 UNIX 명령어를 <보기>에서 찾아 쓰시오. [25-3](#)

- (①): 현재 작업중인 디렉터리 경로를 화면에 표시한다.
- (②): 현재 디렉터리의 파일 목록을 표시한다.
- (③): 디렉터리의 위치를 변경한다.
- (④): 파일을 복사한다.

<보기>

• ls • fork • ps • cp • cat • pwd • chmod • cd

답: ① **pwd** ② **ls** ③ **cd** ④ **cp**

[해설]

• Windows 기본 명령어

명령어	기능
DIR	현재 디렉터리의 파일 목록을 표시함 옵션 /P /W /O /S /A
COPY	파일을 복사함 예) copy abc.txt gilbut → abc.txt 파일을 gilbut 디렉터리로 복사
DEL	파일을 삭제함 예) del abc.txt → abc.txt 파일을 삭제
TYPE	파일의 내용을 표시함 예) type abc.txt → abc.txt 파일의 내용을 표시
REN	파일의 이름을 변경함 예) ren abc.txt 123.txt → abc.txt 파일의 이름을 123.txt로 변경
MD	디렉터리를 생성함 예) md gilbut → gilbut 디렉터를 생성함
CD	동일한 드라이브에서 디렉터리의 위치를 변경함 예) cd gilbut → 디렉터리의 위치를 gilbut으로 변경함
CLS	화면의 내용을 지움 예) cls → 화면에 표시되어 있는 모든 내용을 지움
ATTRIB	파일의 속성을 변경함 종류 RASH + 설정 - 해제 예) attrib +r abc.txt → abc.txt의 파일 속성을 읽기 전용으로 변경함
FIND	파일에서 문자열을 찾음 예) find "123" abc.txt → abc.txt에서 "123"이 포함된 문자열을 찾음
CHKDSK	디스크 상태를 점검함 예) chkdsk → 현재 드라이브의 상태를 점검함
FORMAT	디스크 표면을 트랙과 섹터로 나누어 초기화함 예) format c: → c 드라이브를 초기화함
MOVE	파일을 이동함 예) move abc.txt gilbut → abc.txt 파일을 gilbut 디렉터리로 이동함

• UNIX/LINUX 기본 명령어

명령어	기능
cat	파일 내용을 화면에 표시함 예) cat abc.txt → abc.txt 파일 내용을 화면에 표시
cd	디렉터리의 위치를 변경함 예) cd gilbut → gilbut 디렉터리로 이동
chmod	파일의 보호 모드를 설정하여 파일의 사용 허가를 지정함 예) chmod u=rwx abc.txt → user(u)에게 abc.txt 파일의 읽기(r), 쓰기(w), 실행(x) 권한을 부여(=)함
chown	파일 소유자와 그룹을 변경함 예) chown member1 abc.txt → abc.txt 파일의 소유자를 member1로 변경함
cp	파일을 복사함 예) cp abc.txt gilbut/abc2.txt → abc.txt 파일을 gilbut 디렉터리에 abc2.txt로 이름을 변경하여 복사
rm	파일을 삭제함 예) rm abc.txt → abc.txt 파일을 삭제
find	파일을 찾음 예) find abc.txt → abc.txt 파일을 찾음

명령어	기능
fsck	파일 시스템을 검사하고 보수함 예) fsck /dev/sda1 → /dev/sda1에 기록된 모든 파일 시스템을 검사하고 보수함
kill	PID(프로세스 고유 번호)를 이용하여 프로세스를 종료함 예) kill 1234 → PID가 1234인 프로세스를 종료함
fork	새로운 프로세스를 생성함 예) fork() → 새로운 프로세스를 생성함
killall	프로세스의 이름을 이용하여 프로세스를 종료함 예) killall gilbut → 프로세스 이름이 gilbut인 모든 프로세스를 종료함
ls	현재 디렉터리의 파일 목록을 표시함
mkdir	디렉터를 생성함 예) mkdir gilbut → gilbut 디렉터를 생성함
rmdir	디렉터를 삭제함 예) rmdir gilbut → gilbut 디렉터를 삭제함
mv	파일을 이동함 예) mv abc.txt gilbut/abc2.txt → abc.txt 파일을 gilbut 디렉터리에 abc2.txt로 이름을 변경하여 이동함
ps	현재 실행중인 프로세스를 표시함
pwd	현재 작업중인 디렉터리 경로를 화면에 표시함
top	시스템의 프로세스와 메모리 사용 현황을 표시함
who	현재 시스템에 접속해 있는 사용자를 표시함
tar	파일을 압축하거나 압축을 해제함. 예) tar cvf abc.tar * → 현재 디렉터리의 모든 파일을 'abc.tar'라는 이름으로 압축하며, 처리 과정을 자세하게 표시함. 예) tar xvf abc.tar → 압축된 'abc.tar' 파일을 현재 디렉터리에 해제하며, 처리 과정을 자세하게 표시함.
head	• 파일 내용의 앞부분을 출력함. • 옵션을 지정하지 않을 경우 기본값은 10행.
tail	• 파일 내용의 마지막 부분을 출력함. • 옵션을 지정하지 않을 경우 기본값은 10행.

132. 화이트박스 테스트의 검증 기준에 대한 다음 설명에 해당하는 용어를 <보기>에서 찾아 쓰시오. 23-2, 25-3

테스트 케이스를 소스 코드의 조건문에 포함된 개별 조건식의 결과가 True인 경우와 False인 경우가 한 번 이상 수행 되도록 설계한다.

<보기>

- | | |
|-----------|--------------|
| • 문장 커버리지 | • 분기 커버리지 |
| • 조건 커버리지 | • 분기/조건 커버리지 |

답: 조건 커버리지

[해설]

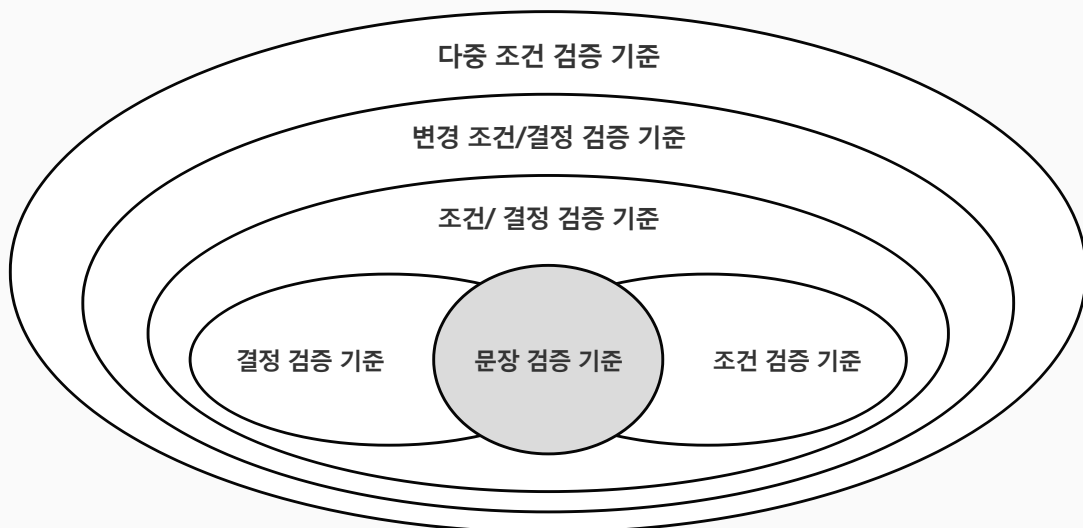
Whitebox(화이트박스) 테스트

- 화이트박스 테스트는 모듈의 원시 코드를 오픈시킨 상태에서 원시 코드의 논리적인 모든 경로를 테스트하여 테스트 케이스를 설계하는 방법임

- 모듈 안의 작동을 직접 관찰함
- 원시 코드(모듈)의 모든 문장을 한 번 이상 실행함으로써 수행됨

화이트박스 테스트의 종류	기능 / 설명
기초 경로 검사 (Base Path Testing)	• 테스트 케이스 설계자가 절차적 설계의 논리적 복잡성을 측정할 수 있게 해주는 테스트 기법 • 대표적인 화이트박스 테스트 기법임
제어 구조 검사 (Control Structure Testing)	• 조건 검사(Condition Testing) : 프로그램 모듈 내에 있는 논리적 조건을 테스트하는 테스트 케이스 설계 기법 • 루프 검사(Loop Testing) : 프로그램의 반복(Loop) 구조에 초점을 맞춰 실시하는 테스트 케이스 설계 기법 • 데이터 흐름 검사(Data Flow Testing) : 프로그램에서 변수의 정의와 변수 사용의 위치에 초점을 맞춰 실시하는 테스트 케이스 설계 기법

화이트박스 테스트 검증 기준	상세 설명 / 기준
문장 검증 기준 (Statement Coverage)	소스 코드의 모든 구문이 한 번 이상 수행되도록 테스트 케이스를 설계함.
결정 검증 기준 (Decision Coverage)	• 소스 코드의 모든 조건문에 대해 조건식의 결과가 True인 경우와 False인 경우가 한 번 이상 수행되도록 테스트 케이스를 설계함. • 분기 검증 기준(Branch Coverage)이라고도 함.
조건 검증 기준 (Condition Coverage)	소스 코드의 조건문에 포함된 개별 조건식의 결과가 True 인 경우와 False 인 경우가 한 번 이상 수행되도록 테스트 케이스를 설계함.
조건/결정 검증 기준 (Condition/Decision Coverage)	결정 검증 기준과 조건 검증 기준을 모두 만족하는 설계로, 조건문이 True 인 경우와 False 인 경우에 따라 조건 검증 기준의 입력 데이터를 구분하는 테스트 케이스를 설계함.
변경 조건/결정 검증 기준 (Modified Condition/Decision Coverage)	조건/결정 검증 기준을 향상시킨 검증 기준으로, 각 개별 조건식이 다른 개별 조건식의 영향을 받지 않고 전체 조건식의 결과에 독립적으로 영향을 주도록 테스트 케이스를 설계함.
다중 조건 검증 기준 (Multiple Condition Coverage)	소스 코드의 조건문에 포함된 모든 개별 조건식의 모든 조합을 고려하도록 테스트 케이스를 설계함.



Blackbox(블랙박스) 테스트

- 블랙박스 테스트는 소프트웨어가 수행할 특정 기능을 알기 위해서 **각 기능이 완전히 작동되는 것을 입증하는 테스트로**, 기능 테스트라고도 함
- 사용자의 요구사항 명세를 보면서 테스트함
- 주로 구현된 기능을 테스트함
- 소프트웨어 인터페이스를 통해 실시됨 [▶ 64로](#)

블랙박스 테스트의 종류	기능 / 설명
동치 분할 검사 (Equivalence Partitioning Testing, 동치 클래스 분해)	프로그램의 입력 조건에 타당한 입력 자료와 타당하지 않은 입력 자료의 개수를 균등하게 하여 테스트 케이스를 정하고, 해당 입력 자료에 맞는 결과가 출력되는지 확인하는 기법 (동등 분할 기법이라고도 함)
경계값 분석 (Boundary Value Analysis)	입력 조건의 중간값보다 경계값에서 오류가 발생할 확률이 높다는 점을 이용하여 입력 조건의 경계값을 테스트 케이스로 선정하여 검사하는 기법
원인-효과 그래프 검사 (Cause-Effect Graphing Testing)	입력 데이터 간의 관계와 출력에 영향을 미치는 상황을 체계적으로 분석한 다음 효용성이 높은 테스트 케이스를 선정하여 검사하는 기법
오류 예측 검사 (Error Guessing)	과거의 경험이나 확인자의 감각으로 테스트하는 기법
비교 검사 (Comparison Testing)	여러 버전의 프로그램에 동일한 테스트 자료를 제공하여 동일한 결과가 출력되는지 테스트하는 기법

133. 다음 설명에서 괄호(①~⑤)에 들어갈 알맞은 용어를 <보기>에서 찾아 기호(㉠~㉥)로 쓰시오. 25-3

- (①) 코드는 데이터 전송 시 오류를 검출하고 수정하는 코드로, 데이터 전송 과정에서 발생한 오류를 검출해 재전송 요구 없이 수신측에서 스스로 수정하는 (②) 방식에 속한다.
- 이에 반해 오류 발생 시 송신측에 재전송을 요구하는 방식은 (③)이며, 대표적인 종류에는 (④) 검사와 (⑤) 검사가 있다.
- (①) 코드 : 전송 데이터에 n비트의 검사 비트를 추가해 오류를 검출하고, 수정한다.
- (④) 검사 : 전송 데이터의 한 블록 끝에 1비트의 검사 비트 추가하여 오류를 검출한다.
- (⑤) 검사 : 송신측과 수신측이 동일한 특정 다항식을 사용하여 오류를 검출한다.

<보기>

- | | | | |
|----------|-------|-------|-----------|
| ㉠ CRC | ㉡ FEC | ㉢ BEC | ㉣ NAK |
| ㉤ Parity | ㉥ MD5 | ㉦ BCD | ㉧ Hamming |

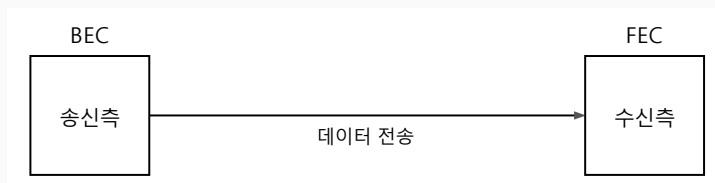
답:

①: ㉧ (Hamming) ②: ㉡ (FEC) ③: ㉢ (BEC) ④: ㉤ (Parity) ⑤: ㉠ (CRC)

[해설]

✓ 후진 오류 정정 (Backward Error Correction, **BEC**)

- 오류가 발생하면 역방향 채널을 통해 송신측에 재전송을 요구하는 방식입니다.
- Parity, CRC 등을 사용하여 오류를 검출하고, 오류 정정은 자동 반복 요청(ARQ)으로 수행합니다.



※ 자동 반복 요청(**ARQ**: Automatic Repeat reQuest)

- 정지-대기(Stop and Wait) ARQ
- 연속(Continuous) ARQ
 - ↳ Go-Back-N ARQ
 - ↳ 선택적 재전송(Selective Repeat) ARQ
- 적응적(Adaptive) ARQ

✓ 전진 오류 정정 (Forward Error Correction, **FEC**)

- 데이터 전송 과정에서 발생한 오류를 검출하여 재전송 없이 수신 측이 스스로 수정하는 방식입니다.
- 대표적으로 **Hamming Code(해밍 코드)**를 사용하여 오류를 검출하고 수정합니다.

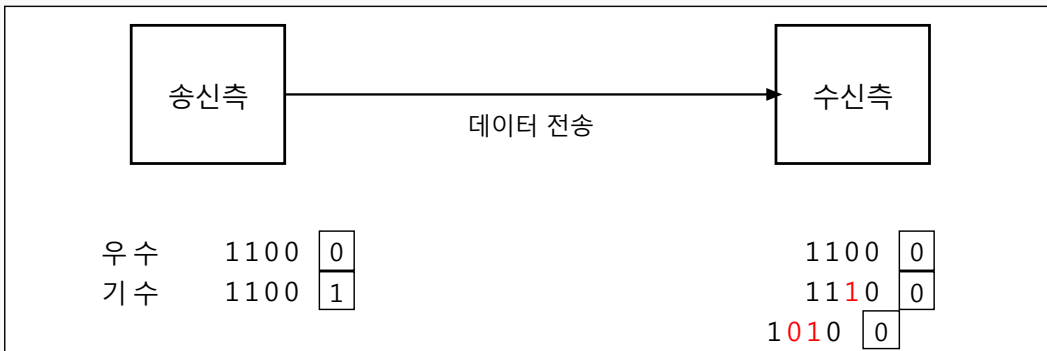
해밍 코드(Hamming Code)

- 수신 측에서 오류가 발생한 비트를 검출한 후 직접 수정하는 전진 오류 수정 방식(FEC)으로 자기 정정 부호라고도 함.
- 하나의 데이터 단위에 패리티 비트를 추가하여 오류를 검출하여 교정이 가능한 코드로, 2bit의 오류를 검출할 수 있으며 1bit의 오류를 교정함.
- 정보 비트 외에 잉여 비트가 많이 필요함.
- 전송 비트 중 1, 2, 4, 8, 16, 32, 64, 2^n 번째를 오류 검출을 위한 패리티 비트로 사용함.

패리티 검사(Parity Check)

- 데이터 블록에 1비트의 검사 비트인 패리티 비트(Parity Bit)를 추가하여 오류를 검출함.
- 가장 간단한 방식이지만, 2개의 비트에 오류가 동시에 발생하면 검출이 불가능함.
- 오류를 검출만 할 수 있고, 수정은 하지 못함.
- 홀수/짝수 수직 패리티 체크와 홀수/짝수 수평 패리티 체크가 있음.

- 우수(짝수)
- 기수(홀수)



순환 중복 (잉여) 검사 (CRC; Cyclic Redundancy Check)

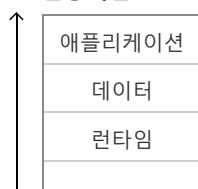
- 프레임 단위로 오류 검출을 위한 **다항식 코드(FCS)**를 사용하여 오류를 검출하는 방식임.
- 전송 과정에서 오류가 발생하면 송신 측에 재전송을 요구하는 **후진 오류 수정 방식(BEC)**에 속함.
- 동기식 전송에 사용되는 에러 검출 기법으로 데이터가 **프레임 단위로 전송될 때** 사용됨.
- **HDLC 프레임의 FCS**(프레임 검사 순서 필드)를 만드는 방법으로 사용됨.
- **집단 오류를 검출할 수 있고**, 검출률이 높으므로 가장 많이 사용됨.

134. 클라우드에 대한 다음 설명에서 괄호(①~③)에 들어갈 알맞은 용어를 내용 중에서 찾아 쓰시오. 23-3

클라우드 컴퓨팅은 각종 컴퓨팅 자원을 중앙 컴퓨터에 두고 인터넷 기능을 가진 단말기로 언제 어디서나 인터넷을 통해 컴퓨터 작업을 수행할 수 있는 환경을 의미한다. 중앙 컴퓨터는 복수의 데이터 센터를 가상화 기술로 통합한 대형 데이터 센터로, 각종 소프트웨어, 데이터, 보안 솔루션 기능 등 컴퓨팅 자원을 보유하고 있다. 사용자는 키보드와 모니터, 마우스를 갖추고 통신 포트만 연결하면 업무 수행이 가능하다.

즉 클라우드 컴퓨팅은 인터넷으로 가상화된 IT 리소스를 서비스로 제공하는 것을 의미하며, 클라우드 컴퓨팅에서 가상화하여 서비스로 제공하는 대상에 따라 IaaS, PaaS, SaaS로 구분되어 진다.

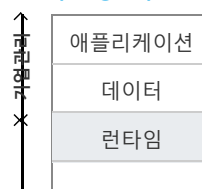
전통적인 IT



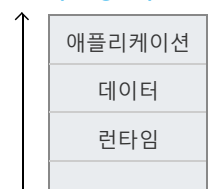
(①)

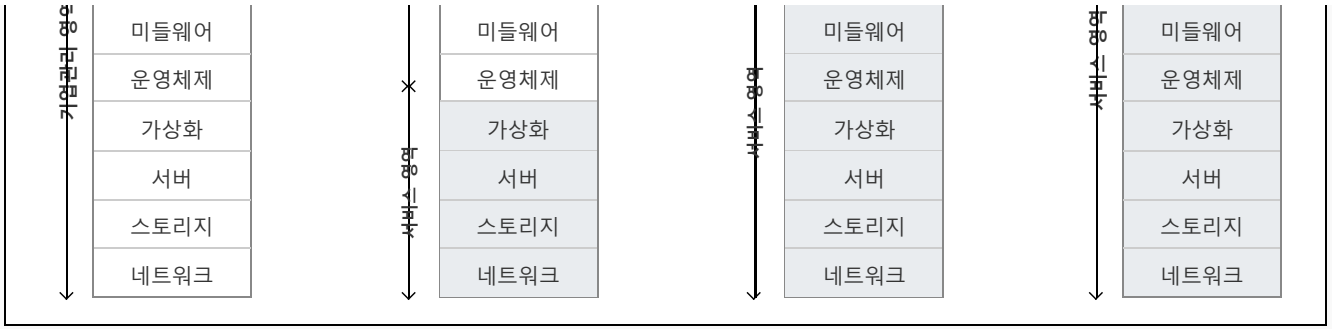


(②)



(③)





답: ① IaaS ② PaaS ③ SaaS

[해설]

- IaaS (Infrastructure as a Service): 서버, 스토리지, 네트워크 등의 하드웨어 인프라스트럭처를 가상화하여 사용자에게 제공하는 서비스
- PaaS (Platform as a Service): 런타임, 미들웨어, OS와 같은 소프트웨어 작성을 위한 플랫폼을 가상화하여 사용자에게 제공하는 서비스
- SaaS (Software as a Service): 소프트웨어의 여러 기능 중에서 사용자가 필요로 하는 기능만을 제공하는 서비스

135. 관계 연산자에 대한 다음 설명에서 각 번호(①~④)의 연산자를 의미하는 기호를 <보기>에서 찾아 쓰시오. 23-3, 25-3

연산자	특징
① Join	• 공통 속성을 중심으로 두 개의 릴레이션을 하나로 합쳐서 새로운 릴레이션을 만드는 연산이다. • Join의 결과는 Cartesian Product(교차곱)를 수행한 다음 Select를 수행한 것과 같다.
② Project	• 주어진 릴레이션에서 속성 리스트(Attribute List)에 제시된 속성 값만을 추출하여 새로운 릴레이션을 만드는 연산이다. • 연산 결과에 중복이 발생하면 중복이 제거된다. • 릴레이션의 열에 해당하는 속성을 추출하는 것이므로 수직 연산자라고도 한다.
③ Select	• 릴레이션에 존재하는 튜플 중에서 선택 조건을 만족하는 튜플의 부분집합을 구하여 새로운 릴레이션을 만드는 연산이다. • 릴레이션의 행에 해당하는 튜플(Tuple)을 구하는 것이므로 수평 연산자라고도 한다.
④ Division	• $X \supset Y$인 두 개의 릴레이션 $R(X)$와 $S(Y)$가 있을 때, R의 속성이 S의 속성값을 모두 가진 튜플에서 S가 가진 속성을 제외한 속성만을 구하는 연산이다.

<보기>

• σ • $\bowtie$ • $\div$ • $\pm$ • ∞ • π

답:

① $\bowtie$ ② π ③ σ ④ $\div$

[해설]

※ 순수 관계 연산자

Select σ (시그마)
Project π (파이)
Join $\bowtie$ (보타이)
Division $\div$

※ 일반 집합 연산자

합집합(UNION) $\cup$
교집합(INTERSECTION) $\cap$
차집합(DIFFERENCE) $-$
교차곱(CARTESIAN PRODUCT) $\times$

136. 다음 설명에 해당하는 용어를 <보기>에서 찾아 쓰시오. [23-3, 25-3](#)

- 인터넷 애플리케이션에서 사용자 인증에 사용되는 표준 인증 방법으로, 공개 API(OpenAPI)로 구현되었다.
- 인터넷 사용자가 웹사이트나 애플리케이션에 비밀번호를 제공하지 않고 자신에게 접근 권한을 부여하여 사용할 수 있다.
- 2010년 IETF에서 1.0이 공식 표준안으로 발표되었다.

<보기>

- | | | | |
|----------|--------|---------|--------|
| • OpenID | • IDEA | • OAuth | • SSPI |
| • SASL | • PEAP | • OIDC | • JAAS |

답: **OAuth**

[해설]

보안 솔루션

- 보안 솔루션이란 접근 통제, 침입 차단 및 탐지 등을 수행하여 외부로부터의 불법적인 침입을 막는 기술 및 시스템을 말함
- 주요 보안 솔루션
 - 방화벽
 - ① 방화벽은 기업이나 조직 내부의 네트워크와 인터넷 간에 전송되는 정보를 선별하여 수용·거부·수정하는 기능을 가진 침입 차단 시스템
 - ② 내부 네트워크에서 외부로 나가는 패킷은 그대로 통과시키고, 외부에서 내부 네트워크로 들어오는 패킷은 내용을 엄밀히 체크하여 인증된 패킷만 통과시키는 구조임
 - 침입 탐지 시스템(IDS; Intrusion Detection System)
 - ① 침입 탐지 시스템은 컴퓨터 시스템의 비정상적인 사용, 오용, 남용 등을 실시간으로 탐지하는 시스템
 - ② 오용 탐지(Misuse Detection) : 미리 입력해 둔 공격 패턴이 감지되면 이를 알려줌
 - ③ 이상 탐지(Anomaly Detection) : 평균적인 시스템의 상태를 기준으로 비정상적인 행위나 자원의 사용이 감지되면 이를 알려줌
 - 침입 방지 시스템(IPS; Intrusion Prevention System)
 - ① 침입 방지 시스템은 비정상적인 트래픽을 능동적으로 차단하고 격리하는 등의 방어 조치를 취하는 보안 솔루션
 - ② 침입 방지 시스템은 방화벽과 침입 탐지 시스템을 결합한 것
 - ③ 침입 탐지 기능으로 패킷을 하나씩 검사한 후 비정상적인 패킷이 탐지되면 방화벽 기능으로 해당 패킷을 차단함
 - 데이터 유출 방지(DLP; Data Leakage/Loss Prevention)
 - ① 데이터 유출 방지는 내부 정보의 외부 유출을 방지하는 보안 솔루션
 - ② 사내 직원이 사용하는 PC와 네트워크상의 모든 정보를 검색하고 메일, 메신저, 웹하드, 네트워크 프린터 등의 사용자 행위를 탐지·통제해 외부로의 유출을 사전에 막음
 - 웹 방화벽(Web Firewall)
 - ① 웹 방화벽은 일반 방화벽이 탐지하지 못하는 SQL 삽입 공격, Cross-Site Scripting(XSS) 등의 웹 기반 공격을 방어할 목적으로 만들어진 웹 서버에 특화된 방화벽
 - ② 웹 관련 공격을 감시하고 공격이 웹 서버에 도달하기 전에 이를 차단해 줌
 - VPN(Virtual Private Network, 가상 사설 통신망)
 - ① VPN은 인터넷 등 통신 사업자의 공중 네트워크와 암호화 기술을 이용하여 사용자가 마치 자신의 전용 회선을 사용하는 것처럼 해주는 보안 솔루션
 - ② SSL VPN: PC에 VPN Client 프로그램을 설치하여 VPN 서버에 접속하는 방식으로, 암호화를 위해 SSL 프로토콜을 사용함
 - ③ IPSec VPN: VPN 서버가 설치된 각각의 네트워크를 서로 연결하는 방식으로, 암호화를 위해 IPSec 프로토콜을 사용함
 - NAC(Network Access Control)

- ① NAC은 네트워크에 접속하는 내부 PC의 MAC 주소를 IP 관리 시스템에 등록한 후 일관된 보안 관리 기능을 제공하는 보안 솔루션
- ② 내부 PC의 소프트웨어 사용 현황을 관리하여 불법적인 소프트웨어 설치를 방지함
- SIEM(Security Information and Event Management)
 - ① SIEM은 다양한 장비에서 발생하는 로그 및 보안 이벤트를 통합하여 관리하는 빅 데이터 기반의 보안 솔루션
 - ② 방화벽, IDS, IPS, 웹 방화벽, VPN 등에서 발생한 로그 및 보안 이벤트를 통합하여 관리함으로써 비용 및 자원을 절약할 수 있음
 - ③ 장기간의 로그 및 보안 이벤트를 수집 및 검색할 수 있는 빅데이터 기반의 통합 로그 수집 시스템
SIM(Security Information Management) + SEM(Security Event Management)
- SSH(Secure SHell, 시큐어 셸) 23-1
 - ① SSH는 다른 컴퓨터에 로그인, 원격 명령 실행, 파일 복사 등을 수행할 수 있도록 다양한 기능을 지원하는 프로토콜 또는 이를 이용한 응용 프로그램.
 - ② 데이터 암호화와 강력한 인증 방법으로 보안성이 낮은 네트워크에서도 안전하게 통신할 수 있음.
 - ③ 키(key)를 통한 인증 방법을 사용하려면 사전에 클라이언트의 공개키를 서버에 등록해야 함.
 - ④ 기본적으로는 22번 포트를 사용 함.
- 템퍼 프루핑(Tamper Proofing)
 - ① 템퍼 프루핑은 소프트웨어의 위·변조가 발생할 경우 소프트웨어를 오작동하게 만들어 악용을 방지하는 기술.
 - ② 템퍼 프루핑은 해시 함수(Hash Function), 핑거 프린트(Fingerprint), 워터마킹(Watermark) 등의 보안 요소를 생성하여 소프트웨어에 삽입하고, 실행 코드를 난독화하며, 실행 시 원본 비교 및 데이터 확인을 수행함으로써 소프트웨어를 보호함.
- OAuth(Open Authorization, 공개 인증)
 - ① 인터넷 애플리케이션에서 사용자 인증에 사용되는 공개 API(OpenAPI)로 구현된 표준 인증 방법.
 - ② 인터넷 사용자들이 비밀번호를 제공하지 않고 다른 웹사이트 상의 자신들의 정보에 대해 웹사이트나 애플리케이션의 접근 권한을 부여할 수 있음.
 - ③ 2010년 IETF에서 1.0이 공식 표준안으로 발표되었음. (※ 이미지에 'ETF'로 적혀 있으나 오타로 보입니다.)

137. 다음 설명에 해당하는 알맞은 용어를 영문 약어로 쓰시오. 25-3

- 사용자가 패스워드를 요청할 때마다 암호 알고리즘과 해시 함수를 이용하여 새로운 일회용 패스워드를 생성하며, 한 번 사용된 패스워드는 폐기되어 재사용할 수 없다.
- 주로 은행 및 금융 거래 시 사용자 본인 인증을 강화하기 위해 널리 사용된다.

답: **OTP**

138. 접근 통제(Access Control)에 대한 다음 설명에서 괄호(①~③)에 들어갈 알맞은 용어를 <보기>에서 찾아 쓰시오. 23-3, 25-3

(①)	• 주체와 객체의 등급을 비교하여 접근 권한을 부여하는 방식이다. • 시스템이 접근통제 권한을 지정한다. • 데이터베이스 객체별로 보안 등급을 부여할 수 있다. • 사용자별로 인가 등급을 부여할 수 있다.
(②)	• 사용자의 역할에 따라 접근 권한을 부여하는 방식이다. • 중앙관리자가 접근통제 권한을 지정한다. • 임의 접근통제와 강제 접근통제의 단점을 보완하였다. • 다중 프로그래밍 환경에 최적화된 방식이다.
(③)	• 데이터에 접근하는 사용자의 신원에 따라 접근 권한을 부여하는 방식이다. • 데이터 소유자가 접근통제 권한을 지정하고 제어한다.

- 객체를 생성한 사용자가 생성된 객체에 대한 모든 권한을 부여받고, 부여된 권한을 다른 사용자에게 허가할 수도 있다.

<보기>

• DAC

• MAC

• RBAC

답: ① **MAC** ② **RBAC** ③ **DAC**

[해설]

접근 통제(Access Control) 3대 보안 모델 핵심 요약

- **MAC (강제 접근 통제 - Mandatory Access Control):**
시스템이 결정 주체가 되며, 주체의 **보안 등급**과 객체의 **보안 권한(레이블)**을 비교하여 강제적으로 권한을 제어하는 관리 방식입니다.
- **RBAC (역할 기반 접근 통제 - Role-Based Access Control):**
사용자의 신원이 아닌 조직 내에서의 **직책 및 직무 역할(Role)**에 따라 권한을 부여하며, 중앙 관리자가 권한의 지정을 통제하여 자원 관리가 용이합니다.
- **DAC (임의 접근 통제 - Discretionary Access Control):**
객체의 **소유자(Owner)**가 **임의로** 판단하여 다른 사용자에게 접근 권한을 분배하고 제어하는 사용자 중심의 접근 제어 방식입니다.

139. 릴레이션을 구성하는 용어들에 대한 다음 설명에서 괄호(①~③)에 들어갈 알맞은 답을 <보기>에서 찾아 쓰시오. [23-1, 25-3](#)

- (①) : 릴레이션을 구성하는 각각의 행을 의미하며, 파일 구조에서는 레코드에 해당함
- (②) : 데이터 개체를 구성하고 있는 속성들에 데이터 타입이 정의되어 구체적인 데이터 값을 가진 것으로, 실제 값을 가진 튜플을 의미함
- (③) : 튜플의 개수를 의미함

<보기>

- | | | | |
|-------|---------|------------|-------------|
| • 도메인 | • 차수 | • 속성 | • 튜플 |
| • 디그리 | • 카디널리티 | • 릴레이션 스키마 | • 릴레이션 인스턴스 |

답: ① **튜플** ② **릴레이션 인스턴스** ③ **카디널리티**

[해설]

관계형 데이터베이스 구조 핵심 핵심 용어 정리

- **튜플 (Tuple):**
릴레이션의 가로 행(Row)을 의미하며, 파일 시스템의 **레코드(Record)**와 동일한 개념입니다.
- **릴레이션 인스턴스 (Relation Instance):**
릴레이션 스키마에 정의된 구조를 따라 **실제 값들로 채워진 튜플들의 집합**을 의미합니다. 어느 한 시점의 데이터 상태를 뜻합니다.
- **카디널리티 (Cardinality):**
릴레이션에 포함된 **튜플(행)의 총 개수**를 의미하며, 기수라고도 부릅니다. (시간에 따라 동적으로 변함)
- **참고 (오답 선지 요약):** 속성(Attribute)은 세로 열(Column) / 차수(Degree) 또는 디그리는 속성의 개수 / 도메인(Domain)은 한 속성이 가질 수 있는 원자값들의 범위입니다.

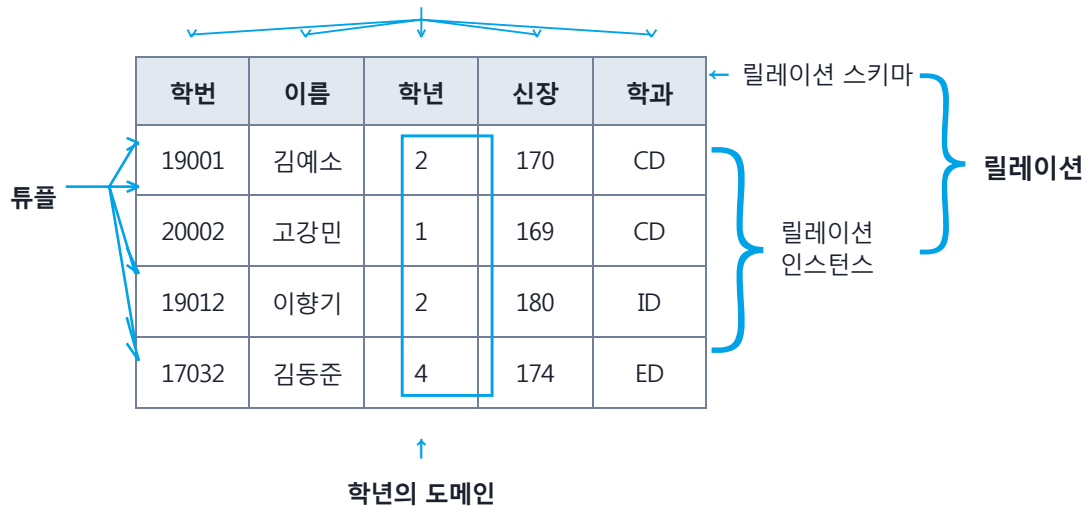
속성의 수

= 디그리

= 차수

<학생> 릴레이션

속성



140. 다음 설명에서 괄호에 들어갈 알맞은 디자인 패턴을 아래에서 찾아 쓰시오. [23-1](#), [25-2](#)

생성 패턴	구조 패턴	행위 패턴
Abstract Factory Builder Factory Method Prototype Singleton	Adapter Bridge Composite Decorator Facade Proxy	Chain of Responsibility Command Interpreter Iterator Mediator Memento Observer Strategy Template Method

() 은/는 복잡한 시스템을 개발하기 쉽도록 클래스나 객체들을 조합하는 패턴에 속하며, **대리자**라는 이름으로도 불린다. 내부에서는 객체 간의 복잡한 관계를 단순하게 정리해 주고, 외부에서는 객체의 세부적인 내용을 숨기는 역할을 한다.

답: **Proxy**

[해설]

GoF(Gang of Four) 디자인 패턴 중 구조 패턴(Structural Pattern) 분석

- 지문에서 가장 핵심이 되는 키워드는 '대리자'입니다. 영어 단어 뜻 자체로 대리인·대리자를 의미하는 디자인 패턴은 Proxy 패턴입니다.
- Proxy 패턴은 특정 객체에 대한 접근을 제어하거나 기능을 가로채기 위해, 실제 객체를 대신하는 **대리 객체 (Proxy Object)**를 인터페이스 형태로 앞단에 두어 클라이언트와 통신하게 만듭니다.
- 이를 통해 실제 주체 객체의 세부적인 내용(생성 비용이 크거나 보안이 필요한 로직 등)을 외부로부터 숨기고 보호하는 역할을 효과적으로 수행할 수 있습니다.
- 문제 보기 분류표에서도 확인할 수 있듯이, 클래스와 객체들을 조합하여 더 큰 구조를 만드는 **구조 패턴**에 명확히 속합니다.

핵심 키워드	패턴 메커니즘 특징	추천 정답
대리자 (Proxy)	실제 객체 접근 전 제어권 대행, 세부 숨김	Proxy
창구·정면 (Facade)	복잡한 하위 서브시스템들에 대한 통합 인터페이스 제공	Facade

※ 디자인패턴은 모듈 간의 관계 및 인터페이스를 설계할 때 참조할 수 있는 전형적인 해결 방식 또는 예제를 의미함. **GoF의 디자인패턴**은 생성패턴, 구조패턴, 행위패턴으로 구분됨.

- **생성 패턴**(Creational Pattern) : 클래스나 객체의 생성과 참조 과정을 정의하는 패턴.
- **구조 패턴**(Structural Pattern) : 구조가 복잡한 시스템을 개발하기 쉽도록 클래스나 객체들을 조합하여 더 큰 구조로 만드는 패턴.
- **행위 패턴**(Behavioral Pattern) : 클래스나 객체들이 서로 상호작용하는 방법이나 책임 분배 방법을 정의하는 패턴.

📌 45로

생성 패턴(Creational Pattern)	구조 패턴(Structural Pattern)	행위 패턴(Behavioral Pattern)
추상 팩토리(Abstract Factory)	어댑터(Adapter)	책임 연쇄(Chain of Responsibility)
빌더(Builder)	브리지(Bridge)	커맨드(Command)
팩토리 메소드(Factory Method)	컴포지트(Composite)	인터프리터(Interpreter)
프로토타입(Prototype)	데코레이터(Decorator)	반복자(Iterator)
싱글톤(Singleton)	퍼사드(Facade)	중재자(Mediator)
	플라이웨이트(Flyweight)	메멘토(Memento)
	프록시(Proxy)	옵서버(Observer)
	📌 227로	상태(State)
		전략(Strategy)
		템플릿 메소드(Template Method)
		방문자(Visitor)

- **추상 팩토리(Abstract Factory):** 구체적인 클래스에 의존하지 않고, 인터페이스를 통해 서로 연관·의존하는 객체들의 그룹으로 생성하여 추상적으로 표현함.
- **빌더(Builder):** 작게 분리된 인스턴스를 건축 하듯이 조합하여 객체를 생성함. 객체의 생성 과정과 표현 방법을 분리하고 있어, 동일한 객체 생성에서도 서로 다른 결과를 만들어 낼 수 있음.
- **팩토리 메소드(Factory Method):** [📌 43로](#) 가상 생성자(Virtual Constructor) 패턴이라고도 함. 객체 생성을 서브 클래스에서 처리하도록 분리하여 캡슐화 한 패턴으로 상위 클래스에서 인터페이스만 정의하고 실제 생성은 서브 클래스가 담당함.
- **프로토타입(Prototype):** 원본 객체를 복제하는 방법으로 객체를 생성하는 패턴으로 일반적인 방법으로 객체를 생성하며, 비용이 큰 경우 주로 이용함.
- **싱글톤(Singleton):** 하나의 객체를 생성하면 생성된 객체를 어디서든 참조할 수 있지만, 여러 프로세스가 동시에 참조할 수는 없고, 클래스 내에서 인스턴스가 하나뿐임을 보장하며, 불필요한 메모리 낭비를 최소화 할 수 있음.
- **어댑터(Adapter):** 호환성이 없는 클래스들의 인터페이스를 다른 클래스가 이용할 수 있도록 변환해주는 패턴으로, 기존의 클래스를 이용하고 싶지만 인터페이스가 일치하지 않을 때 이용함.
- **브리지(Bridge):** 구현부에서 추상층을 분리하여, 서로가 독립적으로 확장할 수 있도록 구성한 패턴으로, 기능과 구현을 두 개의 별도 클래스로 구현함.
- **컴포지트(Composite):** 여러 객체를 가진 복합 객체와 단일 객체를 구분 없이 다루고자 할 때 사용하는 패턴으로, 객체들을 트리 구조로 구성하여 디렉터리 안에 디렉터리가 있듯이 복합 객체 안에 복합 객체가 포함되는 구조를 구현할 수 있음.
- **데코레이터(Decorator):** 객체 간의 결합을 통해 능동적으로 기능들을 확장할 수 있는 패턴으로, 임의의 객체에 부가적인 기능을 추가하기 위해 다른 객체들을 덧붙이는 방식으로 구현함.
- **퍼사드(Facade):** 복잡한 서브 시스템 외부에 상위 인터페이스를 정의하여 서브 시스템의 기능을 간편하게 사용할 수 있도록 하는 패턴으로, 서브 시스템 사이의 결합도를 낮추어 시스템 관리를 원활하게 해줌.
- **플라이웨이트(Flyweight):** 하나의 무거운 객체를 새로 생성하는 대신, 가능한 한 공유해서 사용함으로써 메모리를 절약하는 패턴으로, 대량의 유사 객체를 생성하거나 변경할 때 유용하게 사용됨.
- **프록시(Proxy):** 접근이 까다로운 객체와 실제 객체 사이에 **대리자(Proxy)**를 두어, 주체 객체에 대한 접근을 제어하거나 기능을 가로채는 패턴으로, 내부에서는 객체 간의 복잡한 관계를 단순하게 정리해 주고 외부에서는 객체의 세부적인 내용을 숨기는 역할을 함.
- **책임 연쇄(Chain of Responsibility):** 정적으로 결정된 핸들러가 아니라, 런타임에 동적으로 처리 객체들의 체인을 따라 요청을 전달하는 패턴으로, 요청을 처리할 수 있는 객체가 여러 개 존재할 때 요청을 보내는 객체와 처리하는 객체 간의 결합도를 낮출 수 있음.

- **커맨드(Command):** 하나의 요청을 객체의 형태로 캡슐화하여 요청에 필요한 정보를 저장하거나 로그를 남길 수 있는 패턴으로, 요청을 큐에 저장하거나 로깅, 취소(Undo) 연산 등을 지원할 수 있음.
- **인터프리터(Interpreter):** 특정 언어의 문법 규칙을 나타내는 클래스 라이브러리를 정의하여 문장을 해석하고 실행하는 패턴으로, 문법의 규칙을 클래스로 표현하므로 문법이 변경되거나 확장될 때 유용함.
- **반복자(Iterator):** 자료 구조와 같이 접근하기 복잡한 복합 객체의 내부 표현 방법을 노출하지 않고, 순차적인 접근이 가능하게 하는 패턴으로, 객체 내부 구조를 은닉하면서도 컬렉션 내의 모든 항목에 일관된 방식으로 접근할 수 있음. [168로](#)
- **중재자(Mediator):** 수많은 객체들 간의 복잡한 상호작용(Interaction)을 캡슐화하여 하나의 객체로 정의하는 패턴으로, 객체 사이의 의존성을 줄여 결합도를 느슨하게(Loose Coupling) 만들어 줌.
- **메멘토(Memento):** 객체의 내부 상태를 캡슐화하여 필요할 때 객체를 이전 상태로 되돌릴 수 있는 기능을 제공하는 패턴으로, 작업 취소(Undo) 기능이나 복구 지점을 구현할 때 주로 활용됨.
- **옵서버(Observer):** 한 객체의 상태가 변화하면 객체에 상호작용하는 회사나 관찰자(Observer)들에게 상태 변화를 통지하고 자동으로 상태를 갱신해주는 패턴으로, 주로 분산 이벤트 처리 시스템을 구현할 때 이용함.
- **상태(State):** 객체의 상태에 따라 동일한 동작을 다르게 처리해야 할 때 사용하는 패턴으로, 객체 상태를 캡슐화하고 이를 참조하는 방식으로 상태 변환에 따른 행위 변경을 클래스화하여 상태 지향적인 제어를 가능하게 함.
- **전략(Strategy):** 동일한 목적을 가진 알고리즘들을 개별적으로 캡슐화하여 필요할 때 동적으로 상호 교환하여 사용할 수 있도록 하는 패턴으로, 클라이언트에 영향을 주지 않고 알고리즘을 자유롭게 변경할 수 있음.
- **템플릿 메소드(Template Method):** 상위 클래스에서 골격이 되는 전체적인 알고리즘 구조를 정의하고, 구체적인 단계는 서브 클래스에서 재정의하여 구현하도록 하는 패턴으로, 알고리즘의 구조를 변경하지 않고 특정 단계를 변형할 수 있음.
- **방문자(Visitor):** 데이터 구조(객체 구조)와 처리를 분리하여, 구조를 변경하지 않고도 새로운 연산(행위)을 기존 객체에 동적으로 추가할 수 있도록 하는 패턴으로, 객체의 구조는 안정적이지만 연산이 자주 추가될 때 유용함.

141. 다음 설명에 해당하는 용어를 <보기>에서 찾아 기호(㉠~㉤)로 쓰시오. [25-2](#)

- 데이터베이스를 구성하는 가장 작은 논리적 단위로, 릴레이션에서 열(Column)에 해당한다.
- 파일 구조상의 데이터 항목 또는 데이터 필드에 해당된다.
- 개체의 특성을 기술한다.

<보기>

- | | | | |
|----------|----------------|---------------|-------------|
| ㉠ Domain | ㉡ Tuple | ㉢ Attribute | ㉣ Integrity |
| ㉤ Degree | ㉥ Relationship | ㉦ Cardinality | ㉧ Entity |

답: ㉢

142. 서비스 거부(DoS) 공격 기법 중 TCP가 신뢰성 있는 전송을 위해 사용하는 3-way-handshake 과정을 의도적으로 중단시킴으로써 공격 대상지인 서버가 대기 상태에 놓여 정상적인 서비스를 수행하지 못하게 하는 공격 방법을 가리키는 용어를 쓰시오. [25-2](#)

답: SYN Flooding

[해설]

TCP 3-Way Handshake 취약점을 이용한 네트워크 공격 분석

- **SYN Flooding**은 정상적인 연결 생성 과정에서 서버가 Client로부터 SYN 패킷을 받은 후 응답(YN+ACK)을 보내고, 마지막 ACK를 기다리는 대기 상태(Half-Open)의 자원 제한을 악용하는 대표적인 **DoS(서비스 거부)** 공격입니다.
- 공격자는 가상의 IP 주소로 다량의 SYN 패킷을 보내 서버의 연결 대기 큐(Queue)를 꽉 채우며, 이로 인해 일반 사용자가 서버에 접속하지 못하게 만듭니다.
- **세션 하이재킹(Session Hijacking)**
상호 인증 과정을 거친후 접속해 있는 서버와 서로 접속되어 클라이언트 사이의 세션 정보를 가로챈 후 인증 정보 없이도 가로챈 세션을 이용해 공격자가 원래의 클라이언트인 것처럼 위장하여 서버의 자원이나 데이터를 무단으로 사용하는 것으로, TCP 3-Way-Handshake 과정에 끼어들어서 클라이언트와 서버 간의 동기화된 시퀀스 번호를 가로채 서버에 무단으로 접근함.

공격 기법	핵심 원리 (TCP 연결 관점)	주요 공격 목적
SYN Flooding	3-Way Handshake 과정을 의도적으로 중단/지연시켜 서버를 대기 상태로 만들	서버 자원 고갈 (서비스 마비)
Session Hijacking	연결 확립 후, 클라이언트와 서버 간의 동기화된 시퀀스 번호를 가로채어 사이에 끼어들	인증 우회 및 데이터/자원 무단 사용

◎ 서비스 거부(DoS; Denial of Service) 공격

서비스 거부 공격이란 표적이 되는 서버의 자원을 고갈시킬 목적으로 다수의 공격자 또는 시스템에서 대량의 데이터를 한 곳의 서버에 집중적으로 전송함으로써, 표적이되는 서버의 정상적인 기능을 방해하는 것

주요 서비스 거부 공격의 유형

1. Ping of Death(죽음의 핑)

- Ping of Death는 Ping 명령을 전송할 때 패킷의 크기를 인터넷 프로토콜 허용 범위 이상으로 전송하여 공격 대상의 네트워크를 마비시키는 서비스 거부 공격 방법
- 공격에 사용되는 큰 패킷은 수백 개의 패킷으로 분할되어 전송되는데, 공격 대상은 분할된 대량의 패킷을 수신함으로써 분할되어 전송된 패킷을 재조립 해야 하는 부담과 분할되어 전송된 각각의 패킷들의 ICMP Ping 메시지에 대한 응답을 처리하느라 시스템이 다운되게됨

2. SMURFING(스머핑)

- SMURFING은 IP나 ICMP의 특성을 악용하여 엄청난 양의 데이터를 한 사이트에 집중적으로 보냄으로써 네트워크를 불능 상태로 만드는 공격 방법
- 공격자는 송신 주소를 공격 대상지의 IP 주소로 위장하고 해당 네트워크 라우터의 브로드캐스트 주소를 수신지로 하여 패킷을 전송하면, 라우터의 브로드캐스트 주소로 수신된 패킷은 해당 네트워크 내의 모든 컴퓨터로 전송됨
- 해당 네트워크 내의 모든 컴퓨터는 수신된 패킷에 대한 응답 메시지를 송신 주소인 공격 대상지로 집중적으로 전송하게 되는데, 이로 인해 공격 대상지는 네트워크 과부하로 인해 정상적인 서비스를 수행할 수 없게 됨
- SMURFING 공격을 무력화하는 방법 중 하나는 각 네트워크 라우터에서 브로드캐스트 주소를 사용할 수 없게 미리 설정해 놓는 것

3. SYN Flooding

- TCP(Transmission Control Protocol)는 신뢰성 있는 전송을 위해 3-way-handshake를 거친 후에 데이터를 전송하게 되는데, SYN Flooding은 공격자가 가상의 클라이언트로 위장하여 3-way-handshake 과정을 의도적으로 중단시킴으로써 공격 대상지인 서버가 대기 상태에 놓여 정상적인 서비스를 수행하지 못하게 하는 공격 방법
- SYN Flooding에 대비하기 위해 수신지의 'SYN' 수신 대기 시간을 줄이거나 침입 차단 시스템을 활용함

4. TearDrop

- 데이터의 송·수신 과정에서 패킷의 크기가 커 여러 개로 분할되어 전송될 때 분할 순서를 알 수 있도록 Fragment Offset 값을 함께 전송하는데, TearDrop은 이 Offset 값을 변경시켜 수신 측에서 패킷을 재조립할 때 오류로 인한 과부하를 발생시킴으로써 시스템이 다운되도록 하는 공격 방법
- TearDrop에 대비하기 위해 Fragment Offset이 잘못된 경우 해당 패킷을 폐기하도록 설정함

5. LAND Attack(Local Area Network Denial Attack)

- LAND Attack은 패킷을 전송할 때 송신 IP 주소와 수신 IP 주소를 모두 공격 대상의 IP 주소로 하여 공격 대상에게 전송하는 것으로, 이 패킷을 받은 공격 대상은 송신 IP 주소가 자신이므로 자신에게 응답을 수행하게 되는데, 이러한 패킷이 계속해서 전송될 경우 자신에 대해 무한히 응답하게 하는 공격
- LAND Attack에 대비하기 위해 송신 IP 주소와 수신 IP 주소의 적절성을 검사함

6. DDoS 공격(Distributed Denial of Service, 분산 서비스 거부) 공격

- DDoS 공격은 여러 곳에 분산된 공격 지점에서 한 곳의 서버에 대해 분산 서비스 공격을 수행하는 것
- 네트워크에서 취약점이 있는 호스트들을 탐색한 후 이들 호스트들에 분산 서비스 공격용 툴을 설치하 에이전트(Agent)로 만든 후 DDoS 공격에 이용함
- 분산 서비스 공격용 툴: 에이전트(Agent)의 역할을 수행하도록 설계된 프로그램으로 데몬(Daemon)이라고 부르며, 다음과 같은 종류가 있음

종류	내용
Trin00	가장 초기 형태의 데몬으로, 주로 UDP Flooding 공격을 수행함

종류	내용
TFN(Tribe Flood Network)	UDP Flooding 뿐만 아니라 TCP SYN Flood 공격, ICMP 응답 요청, 스머핑 공격 등을 수행함
TFN2K	TFN의 확장판
Stacheldraht	- 이전 툴들의 기능을 유지하면서, 공격자, 마스터, 에이전트가 쉽게 노출되지 않도록 암호화된 통신을 수행함 - 툴이 자동으로 업데이트됨

143. 준비상태 큐에 각 프로세스의 도착 시간과 실행 시간이 다음과 같을 때 RR(Round Robin)로 스케줄링 할 때 평균 대기 시간을 쓰시오. (단, Time Slice는 4초이다.) [25-2](#)

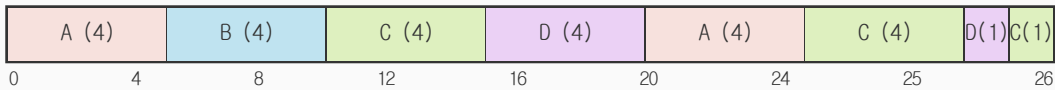
프로세스	도착 시간	실행 시간
A	0	8
B	1	4
C	2	9
D	3	5

답: 11.75초 (또는 47/4)

[해설]

선점형 스케줄링 RR(Round Robin)의 정밀 분석 및 대기 시간 계산

1) Gantt Chart (프로세스 변환 과정):



2) 시간대별 준비상태 큐(Ready Queue) 흐름 및 CPU 할당:

- 0초 ~ 4초: 0초에 A 도착 후 즉시 실행. 실행 중 1초에 B, 2초에 C, 3초에 D가 순서대로 큐에 유입됨. [큐 상태: B, C, D] -> 4초 시점에 남은 실행 시간이 4초인 A가 큐 뒤로 이동. [큐 최종 상태: B, C, D, A]
- 4초 ~ 8초: B가 4초 동안 실행 후 종료(완료). [큐 최종 상태: C, D, A]
- 8초 ~ 12초: C가 4초 동안 실행 (남은 시간 5초). 종료 후 큐 뒤로 이동. [큐 최종 상태: D, A, C]
- 12초 ~ 16초: D가 4초 동안 실행 (남은 시간 1초). 종료 후 큐 뒤로 이동. [큐 최종 상태: A, C, D]
- 16초 ~ 20초: A가 남은 4초를 모두 실행하고 종료(완료). [큐 최종 상태: C, D]
- 20초 ~ 24초: C가 4초 동안 실행 (남은 시간 1초). 종료 후 큐 뒤로 이동. [큐 최종 상태: D, C]
- 24초 ~ 25초: D가 남은 1초를 실행하고 종료(완료). [큐 최종 상태: C]
- 25초 ~ 26초: C가 남은 1초를 실행하고 최종 종료(완료).

3) 프로세스별 대기 시간 계산 (대기 시간 = 종료 시간 - 도착 시간 - 실행 시간):

- * A: 20 (종료) - 0 (도착) - 8 (실행) = 12초
- * B: 8 (종료) - 1 (도착) - 4 (실행) = 3초
- * C: 26 (종료) - 2 (도착) - 9 (실행) = 15초
- * D: 25 (종료) - 3 (도착) - 5 (실행) = 17초

4) 평균 대기 시간 계산:

평균 대기 시간 = $(12 + 3 + 15 + 17) / 4 = 47 / 4 = 11.75$ 초

※ 주의: 기존 자격증 기출 문제의 대기 시간 공식 적용 시 정교하게 계산된 합산 값 분수를 취합니다.

145. 다음 설명에 해당 하는 네트워크 보안 관련 용어를 쓰시오. 25-1

- 상호 인증 과정을 거친후 접속해 있는 서버와 서로 접속되어 클라이언트 사이의 세션 정보를 가로채는 공격 기법이다.
- 접속을 위한 인증 정보 없이도 가로챈 세션을 이용해 공격자가 원래의 클라이언트인 것처럼 위장하여 서버의 자원이나 데이터를 무단으로 사용한다.
- TCP 3-Way-Handshake 과정에 끼어들어서 클라이언트와 서버 간의 동기화된 시퀀스 번호를 가로채 서버에 무단으로 접근하는 공격이 대표적이다.

답: 세션 하이재킹 (Session Hijacking)

[해설]

세션 하이재킹(Session Hijacking)의 특징 분석

- **세션 가로채기:** 사용자가 웹 서버나 시스템에 로그인하여 정상적으로 '세션(Session)'이 성립된 후, 공격자가 이 세션 ID나 연결 정보를 가로채어 해당 세션을 빼앗는 기법입니다.
- **우회 접속:** 이미 상호 인증(Authentication)을 완료한 정상적인 세션을 탈취하기 때문에, 공격자는 별도의 아이디나 비밀번호 입력 과정 없이도 시스템 권한을 획득할 수 있습니다.
- **TCP 세션 하이재킹:** 대표적인 방식으로 TCP 3-Way Handshake 과정에서 서버와 클라이언트가 주고받는 시퀀스 번호(Sequence Number)를 예측 및 위조하여 연결을 가로채고, 클라이언트를 강제로 리셋(RST)시킨 뒤 자신이 정상 서버/클라이언트인 것처럼 속여 통신을 이어 나갑니다.

146. 다음은 무결성 제약 조건에 대한 설명이다. 괄호(①~③)에 들어갈 알맞은 용어를 <보기>에서 찾아 쓰시오. 25-1

구분	(①) 무결성 제약 조건	(②) 무결성 제약 조건	(③) 무결성 제약 조건
제약 대상	속성	튜플	속성, 튜플
NULL 값		기본키	외래키
릴레이션 내 제약 조건 개수	속성의 개수와 동일	1개	0~여러 개

<보기>

- | | | | |
|-------|----------|-------|------|
| • 데이터 | • 사용자 정의 | • 도메인 | • 키 |
| • 속성 | • 개체 | • 범위 | • 참조 |

답: ① 도메인, ② 개체, ③ 참조

[해설]

관계형 데이터베이스의 핵심 무결성 제약 조건 분석

- **① 도메인 무결성(Domain Integrity):** 릴레이션에서 튜플의 속성값들이 정의된 도메인에 속한 값이어야 한다는 지정 조건입니다. 제약의 대상이 개별 '속성'이며, 각 속성마다 제약 조건이 설정되므로 속성의 개수와 제약 조건의 개수가 동일합니다.
- **② 개체 무결성(Entity Integrity):** 기본키(Primary Key)를 구성하는 어떤 속성도 NULL 값이나 중복값을 가질 수 없음을 정의하는 조건입니다. 한 릴레이션에 기본키는 오직 1개만 존재하므로 제약 조건의 개수도 1개입니다.

- ③ 참조 무결성(Referential Integrity): 외래키(Foreign Key) 값은 참조하는 릴레이션의 기본키 값과 동일하거나 NULL이어야 한다는 조건입니다. 참조 관계에 따라 외래키는 릴레이션 내에 0개 또는 여러 개가 존재할 수 있습니다.
- 사용자 정의 무결성(User-Defined Integrity): 속성 값들이 사용자가 정의한 제약조건에 만족해야 한다는 규정.
- NULL 무결성: 릴레이션의 특정 속성 값이 NULL이 될 수 없도록 하는 규정.
- 고유(Unique) 무결성: 릴레이션의 특정 속성에 대해 각 튜플이 갖는 속성값들이 서로 달라야 한다는 규정.
- 키(Key) 무결성: 하나의 릴레이션에는 적어도 하나의 키가 존재해야 한다는 규정.
- 관계(Relationship) 무결성: 릴레이션에 어느 한 튜플의 삽입 가능 여부 또는 한 릴레이션과 다른 릴레이션의 튜플들 사이의 관계에 대한 적절성 여부를 지정한 규정.

147. 다음 설명에 해당하는 알맞은 용어를 **영문 약어**로 쓰시오. 25-1

- 다항식 코드를 사용하여 오류를 검출하는 방식이다.
- 동기식 전송에서 주로 사용되며, HDLC 프레임의 FCS(프레임 검사 순서 필드)에 사용된다.
- 집단 오류를 검출할 수 있고, 검출률이 높으므로 가장 많이 사용한다.

답: CRC

[해설]

순환 중복 검사(CRC, Cyclic Redundancy Check)의 핵심 특징 분석

- 다항식 코드 이용: 데이터 전송 과정에서 발생하는 오류를 검출하기 위해 미리 지정된 **생성 다항식(Generator Polynomial)**을 기반으로 나눗셈 연산을 수행하는 방식입니다.
- FCS 필드 활용: 데이터 프레임의 후미에 부착되는 ****FCS(Frame Check Sequence, 프레임 검사 순서 필드)****를 생성하는 데 사용되며, 데이터 링크 계층의 대표적인 프로토콜인 HDLC나 이더넷(Ethernet) 등에서 널리 쓰입니다.
- 높은 신뢰성: 연속적으로 오류가 발생하는 **집단 오류(Burst Error)**에 대한 검출 능력이 매우 뛰어나며, 구조가 단순하면서도 검출률이 거의 100%에 가까워 동기식 데이터 통신에서 가장 표준적으로 채택되는 오류 검출 기법입니다.

148. 다음 설명에 해당하는 용어를 <보기>에서 찾아 쓰시오. 25-1

사용자에게 컴퓨터가 바이러스에 감염되었다고 속이거나 보안 위험이 있다고 거짓 경고 메시지를 보내어 불안감을 조성한 후 문제 해결을 명목으로 불필요한 악성 소프트웨어 구매를 유도하는 악성 소프트웨어이다.

<보기>

- | | | |
|---------|--------|---------|
| • 랜섬웨어 | • 애드웨어 | • 스파이웨어 |
| • 스퀘어웨어 | • 키로거 | • 봇넷 |

답: 스퀘어웨어

[해설]

스퀘어웨어(Scareware)의 개념 및 특징

- 공포심 유발(Scare): 이용자에게 가짜 시스템 경고, 바이러스 감염 팝업 창을 띄워 심리적 불안감이나 공포심을 조장하는 것이 가장 큰 특징입니다.
- 금전적 이득 목적: 허위로 위협 요소를 치료해 주겠다는 명목 하에 가짜 유료 백신이나 보안 소프트웨어 프로그램을 결제하도록 기만하고 유도하는 악성 코드 유형입니다.

※ 오답 보기 분석:

- 랜섬웨어: 파일을 암호화한 뒤 인질로 삼아 금전을 요구하는 형태
- 애드웨어: 시스템에 원치 않는 광고를 강제로 노출시키는 프로그램
- 키로거: 사용자의 키보드 입력 내용을 무단으로 탈취 및 기록하는 악성 코드

종류	내용
애드웨어 (Adware)	소프트웨어 자체에 광고를 포함하여 이를 보는 대가로 무료로 사용하는 소프트웨어
트랙웨어 (Trackware)	적절한 사용자 동의 없이 사용자 정보를 수집하는 프로그램으로 스파이웨어(Spyware)라고도 불림
스미싱 (Smishing)	문자 메시지(SMS)를 이용해 사용자의 개인 신용 정보를 빼내는 수법.
스피어 피싱 (Spear Phishing)	사회 공학의 한 기법으로, 특정 대상을 선정한 후 그 대상에게 일반적인 이메일로 위장한 메일을 지속적으로 발송하여, 발송 메일의 본문 링크나 첨부된 파일을 클릭하도록 유도해 사용자의 개인 정보를 탈취하는 공격.
APT (Advanced Persistent Threats, 지능형 지속 위협)	다양한 IT 기술과 방식들을 이용해 조직적으로 특정 기업이나 조직 네트워크에 침투해 활동 거점을 마련한 뒤 때를 기다리면서 보안을 무력화시키고 정보를 수집한 다음 외부로 빼돌리는 형태의 공격.
Rootkit  236로	- 시스템에 침입한 후 침입 사실을 숨긴 채 백도어, 트로이목마를 설치하고, 원격 접근, 내부 사용 흔적 삭제, 관리자 권한 획득 등 주로 불법적인 해킹에 사용되는 기능들을 제공하는 프로그램들의 모음. - 컴퓨터의 운영체제에서 실행 파일과 실행 중인 프로세스를 숨김으로써 운영체제 검사 및 백신 프로그램의 탐지를 피할 수 있음. ※ "Root"와 "Kit"의 합성어, 컴퓨터의 관리자 계정(Root)의 권한을 획득 할 수 있도록 하는 도구 모음(kit)을 의미함.
무작위 대입 공격 (Brute Force Attack)	암호화된 문서의 암호키를 찾아내기 위해 적용 가능한 모든 값을 대입하여 공격하는 방식.
큐싱 (Qshing)	QR코드(Quick Response Code)를 통해 악성 앱의 다운로드를 유도하거나 악성 프로그램을 설치하도록 하는 금융사기 기법의 하나로, QR코드와 개인정보 및 금융정보를 낚는다(Fishing)는 의미의 합성 신조어.
EDR (Endpoint Detection & Response)	엔드포인트 영역에 대한 지속적인 모니터링을 통해 행위 기반 위협 탐지 및 분석, 대응 기능을 제공하는 솔루션. ※ 엔드포인트 : 요청을 보내는 시작점인 사용자의 PC, 스마트폰이나 서비스의 종착점인 서버 등을 의미함
TMS (Threat Management System)	전사적 IT인프라에 대한 위협정보들을 수집·분석·경보·관리하는 정보보호 통합관리 시스템으로, 실시간으로 공신력 있는 대외 정보보호기관의 위협정보들을 수집·분석하여 정보보호관리자에게 제공함.
크리덴셜 스템핑 (Credential Stuffing)	무차별 대입 공격의 일종으로 공격자가 미리 확보해 놓은 로그인 자격 증명을 다른 계정에 무작위로 대입해 사용자 계정을 탈취하는 공격 방식.
POODLE (Padding Oracle On Downgraded Legacy Encryption) 공격	TLS 연결을 SSL 3.0으로 낮춰 SSL 3.0의 취약점을 이용하여 암호문을 해독하는 공격 기법.
DDE (Dynamic Data Exchange)	마이크로소프트 오피스와 애플리케이션 사이에서 데이터를 전달하는 프로토콜로, 엑셀 등 오피스 프로그램에서 DDE가 활성화될 시 일반 문서로 위장한 악성 코드가 전파될 수 있음.

149. 현재 IP 주소가 192.168.35.10이고, 서브넷 마스크가 255.255.252.0인 컴퓨터에서 브로드캐스팅으로 다른 IP로 정보를 전달하려고 한다. 이때 수신할 수 있는 알맞은 IP 주소를 <보기>에서 골라 모두 작성하시

오. 25-1

<보기>

- ㉠ 192.168.32.1 ㉡ 192.168.33.50 ㉢ 192.168.34.126
㉣ 192.168.35.100 ㉤ 192.168.35.200

답: ㉠, ㉡, ㉣, ㉤, ㉤

[해설]

네트워크 ID 영역 분리를 통한 가용 IP 주소 범위 산출 과정

- ip 192.168.35.10
11000000 10101000 00100011 00001010
- sm 255.255.252.0
11111111 11111111 11111100 00000000 (0은 사용가능, 1을 사용 불가능)
- and 연산 둘다 1이면 1,
시작주소: 11000000 10101000 00100000 00000000
마지막주소는 뒤에 10개를 모두 1로
마지막주소: 11000000 10101000 00100011 11111111
 - 기본 IP 3옥텟: 001000 11 (35)
 - 서브넷 마스크 : 111111 00 (252)
 - -----
 - 네트워크 시작 : 001000 00 (32) → 192.168.32.0 (네트워크 대표 주소)
 - 브로드캐스트 : 001000 11 (35) → 192.168.35.255 (호스트 비트가 전부 1인 상태)
- 수신 가능한 호스트 IP 범위 도출:
동일 서브넷 내부에서 브로드캐스트 패킷을 정상 수신할 수 있는 유효 호스트 주소 대역은 192.168.32.1부터 192.168.35.254까지입니다.

Network 주소	가용한 주소	Broadcast 주소
192.168.32.0	192.168.32.1~192.168.35.254	192.168.35.255

150. 프로토콜에 대한 관한 다음 설명에서 괄호(①, ②)에 들어갈 알맞은 용어를 쓰시오. 25-1

(①)는 호스트의 IP 주소를 호스트와 연결된 네트워크 접속 장치의 물리적 주소(MAC Address)로 변환하는 프로토콜이고, (②)는 (①)와 반대로 물리적 주소를 IP 주소로 변환하는 프로토콜이다.

답: ① ARP ② RARP

[해설]

네트워크 계층과 데이터 링크 계층 간의 주소 매핑 프로토콜

- ARP (Address Resolution Protocol):
네트워크 상에서 목적지 컴퓨터의 IP 주소는 알고 있으나 물리적인 네트워크 카드 주소(MAC 주소)를 모를 때, 이를 통신이 가능하도록 매칭해주는 주소 해석 프로토콜입니다.
- RARP (Reverse Address Resolution Protocol):
ARP의 역과정을 수행합니다. 로컬 디스크가 없는 워크스테이션(무디스크 PC) 등이 처음 부팅될 때 자신의 MAC 주소를 기반으로 게이트웨이로부터 IP 주소를 할당받기 위해 사용됩니다.

※ MAC(Media Access Control): 제조사에서 네트워크 인터페이스 카드에 부여한 고유 식별자.

◇ TCP/IP(Transmission Control Protocol/Internet Protocol)

인터넷에 연결된 서로 다른 기종의 컴퓨터들 간에 데이터를 주고받을 수 있도록 하는 표준 프로토콜.

- TCP/IP는 TCP 프로토콜과 IP 프로토콜이 결합된 것을 의미함.

프로토콜	특징
TCP (Transmission Control Protocol)	• OSI 7계층의 전송 계층에 해당함. • 신뢰성 있는 연결형 서비스를 제공함. • 패킷의 다중화, 순서 제어, 오류 제어, 흐름 제어 기능을 제공함.

프로토콜	특징
IP (Internet Protocol)	- OSI 7계층의 네트워크 계층에 해당함. - 데이터그램을 기반으로 하는 비연결형 서비스를 제공함. - 패킷의 분해/조립, 주소 지정, 경로 선택 기능을 제공함.

※ 연결형(접속) 통신 : 송·수신 측 간을 논리적으로 연결한 후 데이터를 전송하는 방식, 가상 회선 방식

※ 비연결형(비접속) 통신 : 송·수신 측 간에 논리적 연결 없이 데이터를 전송하는 방식, 데이터그램 방식

● TCP/IP의 구조

OSI 7계층	TCP/IP 계층	기능 및 프로토콜 종류
응용 계층 표현 계층 세션 계층	응용 계층	- 응용 프로그램 간의 데이터 송·수신 제공 - TELNET, FTP, SMTP, SNMP, DNS, HTTP 등
전송 계층	전송 계층	- 호스트들 간의 신뢰성 있는 통신 제공 - TCP, UDP, RTP, DCCP
네트워크 계층	인터넷 계층	- 데이터 전송을 위한 주소 지정, 경로 설정을 제공함 - IP, ICMP, IGMP, ARP, RARP, IPSEC
데이터 링크 계층 물리 계층	네트워크 액세스 계층	- 실제 데이터(프레임)를 송·수신하는 역할 - Ethernet, IEEE 802, HDLC, X.25, RS-232C, ARQ 등

151. 다음 설명에 해당하는 디자인 패턴을 <보기>에서 찾아 쓰시오. [25-1](#)

- 호환성이 없는 클래스들의 인터페이스를 다른 클래스가 이용할 수 있도록 변환해주는 패턴이다.
- 기존의 클래스를 이용하고 싶지만 인터페이스가 일치하지 않을 때 이용한다.

<보기>

생성 패턴

- Abstract Factory
- Builder
- Factory Method
- Prototype
- Singleton

구조 패턴

- Adapter
- Bridge
- Composite
- Decorator
- Proxy

행위 패턴

- Command
- Interpreter
- Iterator
- Mediator
- Observer

답: **Adapter**

[해설]

어댑터(Adapter)

152. 다음에 제시된 코드를 이용하여 순서도의 번호(①~⑥)에 들어갈 알맞은 코드를 작성하고, 문장 커버리지로 구성할 테스트 케이스를 작성하시오. [25-1](#)

<코드>

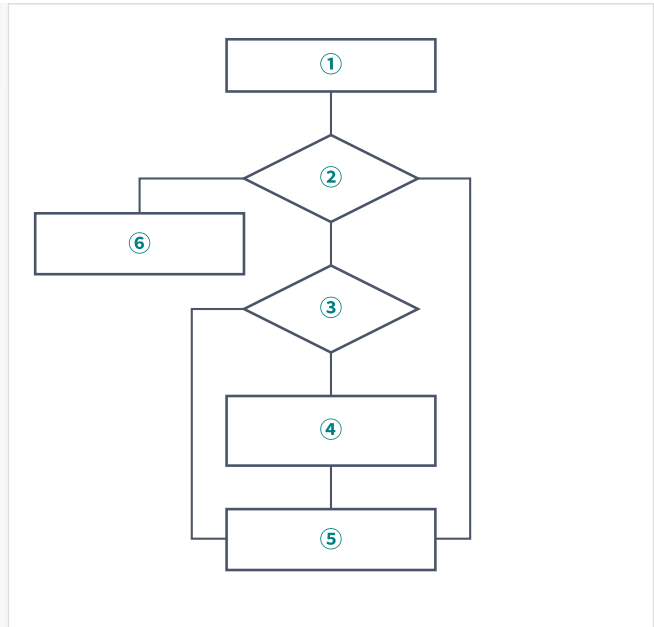
```
int Main(int b[], int m, int x) {
    int a = 0;
    while (a < m || b[a] < x) {
        if (b[a] < 0)
            b[a] = -b[a];
        a++;
    }
}
```

<순서도>

```

    return 1;
}

```



답:

- ① : `int a = 0;`
- ② : `a < m || b[a] < x`
- ③ : `b[a] < 0`
- ④ : `b[a] = -b[a];`
- ⑤ : `a++;`
- ⑥ : `return 1;`
- 문장 커버리지 기준 테스트 케이스 : ① → ② → (③) → (④) → (⑤) → (②) → (⑥)

[해설]

1. 코드와 순서도 매칭

- ①은 반복문(while) 진입 전 초기화 문장인 `int a = 0;`에 해당합니다.
- ②는 조건 판단 마름모로, 루프의 조건식인 `a < m || b[a] < x`가 들어갑니다. 거짓(False)일 때 루프를 빠져나와 ⑥(`return 1;`)으로 진행합니다.
- ③은 내부의 if 조건문인 `b[a] < 0`입니다. 참일 때 참일 때 실행되는 문장인 ④(`b[a] = -b[a];`)를 거친 후 ⑤(`a++;`)로 가고, 거짓일 때는 곧바로 ⑤로 우회합니다.

2. 문장 커버리지 (Statement Coverage)

- 프로그램 소스 코드의 모든 문장(과정 노드)이 최소한 한 번은 실행되도록 경로를 구성해야 합니다.
- 모든 실행문(①, ④, ⑤, ⑥)을 전부 통과하기 위해서는 `if (b[a] < 0)` 조건이 반드시 참이 되는 경로를 포함하여 루프를 돌고 탈출해야 하므로, 올바른 실행 경로는 ① → ② → ③ → ④ → ⑤ → ② → ⑥이 됩니다.

153. 데이터베이스와 관련된 용어들에 대한 다음 설명에서 괄호(①~④)에 들어갈 알맞은 답을 <보기>에서 찾아 기호(㉠~㉣)로 쓰시오. 25-1

- (①) : 릴레이션에서 속성의 개수
- (②) : 릴레이션에서 튜플의 개수
- (③) : 다른 릴레이션의 기본키를 참조하는 속성 또는 속성들의 집합
- (④) : 하나의 애트리뷰트가 취할 수 있는 같은 타입의 원자값들의 집합

<보기>

- ㉠ Domain
- ㉡ Attribute
- ㉢ Degree
- ㉣ Candidate

답: ① Ⓢ ② Ⓢ ③ Ⓢ ④ Ⓢ

[해설]

관계형 데이터베이스 구조 및 핵심 용어 정의

- ① 차수 (Degree):
릴레이션을 구성하는 속성(Attribute, 열)의 전체 개수를 의미합니다.
- ② 카디널리티 (Cardinality):
릴레이션에 입력된 튜플(Tuple, 행)의 전체 개수를 의미하며, 기수라고도 합니다.
- ③ 외래키 (Foreign Key):
한 릴레이션의 속성 중에서 다른 릴레이션의 기본키(Primary Key)를 참조하여 릴레이션 간의 관계를 표현하는 키입니다.
- ④ 도메인 (Domain):
릴레이션에서 하나의 애트리뷰트(속성)가 취할 수 있는 같은 타입의 원자값들의 법적 범위 또는 집합을 의미합니다. (예: 성별 속성의 도메인은 '남', '여')

154. 3개의 페이지를 수용할 수 있는 주기억장치가 있으며, 초기에는 모두 비어 있다고 가정한다. 다음의 순서로 페이지 참조가 발생할 때, LRU 페이지 교체 알고리즘을 사용할 경우 몇 번의 페이지 부재(Page Fault)가 발생하는지 쓰시오. [24-3](#)

페이지 참조 순서 : 7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1

답: 12회

[해설]

LRU(Least Recently Used) 페이지 교체 알고리즘

- 개념: 새로운 페이지를 메모리에 적재할 때, 가장 오랫동안 참조되지 않은(가장 예전에 사용된) 페이지를 교체 대상으로 선택하는 기법입니다.
- 특징: 국부성(Locality)의 원리에 기반하여 최근에 사용된 페이지는 가까운 미래에 다시 사용될 가능성이 높다는 가정하에 동작합니다. (아래 a: 참조 페이지, b:페이지 프레임, c: 부재발생)

a	7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
	7	7	7	2	2	2	2	4	4	4	0	0	0	1	1	1	1	1	1	1
b		0	0	0	0	0	0	0	0	3	3	3	3	3	3	0	0	0	0	0
			1	1	1	3	3	3	2	2	2	2	2	2	2	2	2	7	7	7
c	●	●	●	●		●		●	●	●	●			●		●		●		

■ 총 Page Fault(부재) 횟수는 12회

용어	정의
페이지 부재 (Page Fault)	프로그램이 실행 중 주기억장치(RAM)에 없는 페이지를 참조하려고 할 때 발생하는 현상으로, 이를 해결하기 위해 보조기억장치에서 해당 페이지를 가져와야 하므로 오버헤드가 발생함.
페이지 적중 (Page Hit)	참조하려는 페이지가 이미 주기억장치(RAM)에 로드되어 있어 디스크 접근 없이 즉시 해당 데이터를 읽거나 쓸 수 있는 상태.

155. 다음 설명에 해당하는 알맞은 용어를 쓰시오. [24-3](#)

- IP나 ICMP의 특성을 악용하여 엄청난 양의 데이터를 한 사이트에 집중적으로 보냄으로써 네트워크를 불능 상태로 만드는 공격 방법이다.
- 공격자는 송신 주소를 공격 대상지의 IP 주소로 위장하고 해당 네트워크 라우터의 브로드캐스트 주소를 수신지로 하여 패킷을 전송하면, 라우터의 브로드캐스트 주소로 수신된 패킷은 해당 네트워크 내의 모든 컴퓨터로 전송된다.
- 해당 네트워크 내의 모든 컴퓨터는 수신된 패킷에 대한 응답 메시지를 송신 주소인 공격 대상지로 집중적으로 전송하게 되는데, 이로 인해 공격 대상지는 네트워크 과부하로 인해 정상적인 서비스를 수행할 수 없게 된다.

답: 스머핑 (Smurfing 또는 Smurf Attack)

[해설]

스머핑(Smurfing)의 개념 및 특징

- **IP 위장(Spoofing):** 출발지 IP 주소를 공격 대상자의 IP로 속여 패킷을 보냅니다.
- **증폭 공격:** 브로드캐스트 주소로 보낸 요청이 다수의 호스트를 거쳐 공격 대상자에게 반사-집중되게 만드는 디도스(DDoS) 공격의 일종입니다.

※ 유사 네트워크 공격 유형 분석:

- **핑 오브 데스(Ping of Death):** 규격 이상의 거대한 패킷을 전송하여 시스템을 다운시키는 공격
- **SYN 플러딩(SYN Flooding):** TCP 동기화 요청(SYN)만 계속 보내고 응답(ACK)을 주지 않아 서버의 대기 큐를 고갈시키는 공격

공격 명칭	핵심 메커니즘
스머핑 (Smurfing)	출발지 IP 위장 + ICMP Echo Request + 라우터 브로드캐스트를 이용한 트래픽 반사 및 증폭
핑 오브 데스 (Ping of Death)	ICMP 패킷을 단편화(Fragmentation)하여 매우 크게 만들어 재조립 과정에서 과부하 유발
디도스 (DDoS)	감염된 다수의 좀비 PC(봇넷)를 동원하여 특정 대상 시스템에 대량의 트래픽을 집중시킴

156. 다음 설명에 해당하는 알맞은 용어를 영문 약어 3글자로 쓰시오. 24-3

- 인터넷 등 통신 사업자의 공중 네트워크와 암호화 기술을 이용하여 사용자가 마치 자신의 전용 회선을 사용하는 것처럼 해주는 보안 솔루션이다.
- 암호화된 규격을 통해 인터넷망을 전용선의 사설망을 구축한 것처럼 이용하므로 비용 부담을 줄일 뿐만 아니라 원격지의 지사, 영업소, 이동 근무자가 지역적인 제한 없이 업무를 수행할 수 있다.
- 종류에는 IPSec, SSL 등이 있다.

답: VPN

[해설]

VPN(Virtual Private Network, 가상 사설망)의 개념

- **가상선 구축:** 공용 네트워크(인터넷) 위에 터널링과 암호화 기술을 적용해 독점적인 사설 전용망처럼 안전하게 통신할 수 있게 합니다.
- **비용 절감 및 유연성:** 고가의 물리적 전용선을 설치하지 않고도 원격지에서 안전한 내부망 접속을 지원합니다.

프로토콜 종류	주요 특징 및 계층
IPSec VPN	네트워크 계층(3계층)에서 암호화 및 인증을 수행하며, 주로 본사-지사 간 고정된 연결에 사용

프로토콜 종류	주요 특징 및 계층
SSL/TLS VPN	전송/응용 계층에서 작동하며, 웹 브라우저 기반으로 별도 프로그램 없이 원격 접속이 용이함

157. 소프트웨어 공학의 디자인 패턴(Design Pattern)에 대한 다음 설명에서 괄호에 공통으로 들어갈 알맞은 답을 쓰시오. 24-3

- () 패턴은 클래스나 객체들이 서로 상호작용하는 방법이나 책임 분배 방법을 정의하는 패턴이다.
- () 패턴은 하나의 객체로 수행할 수 없는 작업을 여러 객체로 분배하면서 결합도를 최소화 할 수 있도록 도와준다.
- () 패턴에는 책임 연쇄(Chain of Responsibility), 반복자(Iterator), 옵서버(Observer) 패턴 등이 있다.

답: 행위 (또는 행위 패턴, Behavioral)

[해설]

GoF(Gang of Four) 디자인 패턴의 3대 분류

- 생성 패턴(Creational): 객체의 생성 과정에 관여하며, 객체의 생성과 참조 방식을 캡슐화하는 패턴
- 구조 패턴(Structural): 클래스나 객체들을 조합하여 더 큰 구조를 구성하는 패턴
- 행위 패턴(Behavioral): 객체들이 서로 상호작용하는 방법이나 알고리즘, 책임 분배를 정의하는 패턴

패턴 유형	핵심 역할	대표적인 패턴 종류
생성 패턴	객체 생성 효율화 및 추상화	Singleton, Factory Method, Abstract Factory, Builder, Prototype
구조 패턴	객체/클래스 결합 및 구조화	Adapter, Bridge, Composite, Decorator, Facade, Flyweight, Proxy
행위 패턴	객체 간 알고리즘 및 상호작용 정의	Observer, Strategy, Iterator, Command, Template Method, State

158. 화이트박스 테스트의 검증 기준에 대한 다음 설명에 해당하는 용어를 <보기>에서 찾아 기호(㉠~㉣)로 쓰시오. 24-3

- ① 소스 코드의 모든 구문이 한 번 이상 수행되도록 테스트 케이스를 설계한다.
- ② 소스 코드의 모든 조건문에 대해 조건식의 결과가 True인 경우와 False인 경우가 한 번 이상 수행되도록 테스트 케이스를 설계한다.
- ③ 소스 코드의 조건문에 포함된 개별 조건식의 결과가 True인 경우와 False인 경우가 한 번 이상 수행되도록 테스트 케이스를 설계한다.

<보기>						
㉠ 선택	㉡ 경로	㉢ 조건	㉣ 분기	㉤ 함수	㉥ 문장	㉦ 루프

답: ① ㉤ ② ㉢ ③ ㉣

[해설]

화이트박스 테스트 커버리지(Coverage) 검증 기준

- 문장 검증(Statement Coverage): 프로그램의 모든 실행 가능한 명령문(구문)을 적어도 한 번은 실행하도록 요구합니다.
- 분기 검증(Branch/Decision Coverage): 각 조건문의 전체 결정 결과가 전체 참(True)과 전체 거짓(False)을 최소한 한 번씩 가지도록 제어 흐름을 테스트합니다.
- 조건 검증(Condition Coverage): 조건문 내부의 개별 논리식(개별 조건식)이 각각 참(True)과 거짓(False)이 되도록 독립적으로 테스트합니다.

검증 기준	핵심 측정 목표
문장(구문) 커버리지	코드 내부의 모든 '라인/문장' 실행 여부 검증

검증 기준	핵심 측정 목표
분기(결정) 커버리지	조건문(if문 전체)의 결과가 True / False가 되는 경로 검증
조건 커버리지	조건문 안에 속한 '개별 논리 인자들' 각각이 True / False가 되는지 검증

159. 다음에 제시된 URL 구조에서 <보기>의 순서대로 URL에 해당하는 번호(1~5)를 쓰시오. [24-3](#)

foo://example.com:8042/over/there?name=ferret#nose

└─1─┐ ┌───2───┐ ┌──3──┐ ┌──4──┐ └─5─┘

<보기>

- ① query : 서버에 전달할 추가 데이터
- ② path : 서버 내의 특정 자원을 가리키는 경로
- ③ scheme : 리소스에 접근하는 방법이나 프로토콜
- ④ authority : 사용자 정보, 호스트명, 포트 번호
- ⑤ fragment : 특정 문서 내의 위치

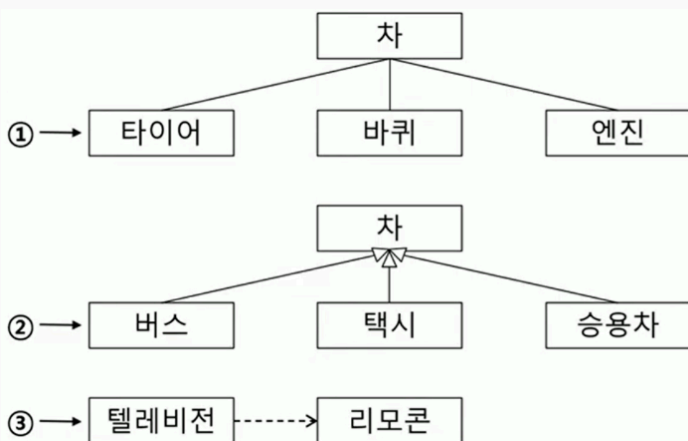
답: ① 4 ② 3 ③ 1 ④ 2 ⑤ 5

[해설]

URI/URL 표준 구조 분석 (RFC 3986 기준)

- **Scheme (1번 - foo):** 리소스 자원에 어떤 프로토콜/수단으로 연결할지 선언 (예: http, https, ftp 등)
- **Authority (2번 - example.com:8042):** 인터넷 상의 호스트 서버 식별자 및 포트 번호 포함
- **Path (3번 - /over/there):** 서버 내에서 해당 리소스가 위치한 디렉터리 경로 경로
- **Query (4번 - ?name=ferret):** 서버 측 프로그램으로 파라미터를 넘기기 위한 질의 문자열 (? 기호로 시작)
- **Fragment (5번 - #nose):** 웹 페이지나 문서 내부의 특정 책갈피/앵커 위치를 지정 (# 기호로 시작)

160. 다음 UML 모델에서 각 번호(①~③)에 해당하는 관계를 <보기>에서 찾아 기호(㉠~㉥)로 쓰시오. [24-3](#) [001로](#)



<보기>

- ㉠ 의존 ㉡ 연관 ㉢ 일반화 ㉣ 집합 ㉤ 포함 ㉥ 실체화

답: ① ㉡ ② ㉢ ③ ㉠

[해설]

UML(Unified Modeling Language) 관계성 요약

- **연관 관계(Association, ㉡)** - 연관 관계는 2개 이상의 사물이 서로 관련된 있는 관계
 - 사물 사이를 실선으로 연결하여 표현함
 - 방향성은 화살표로 표현함

- 양방향 관계의 경우 화살표를 생략하고 실선으로만 연결함
- 다중도를 선 위에 표기함

- **포함 관계(Composition):** '전체-부분' 관계 중 영구적이고 강한 결합을 뜻합니다. 전체 객체(차)가 소멸하면 부분 객체(엔진, 타이어)도 함께 소멸하는 특징을 가집니다. (실선 끝에 채워진 마름모로 표시)
- **일반화 관계(Generalization, ⊃):** 객체지향의 상속 관계를 뜻합니다. 구체적인 하위 클래스(버스, 택시)에서 상위 클래스(차) 방향으로 속이 빈 삼각형 화살표 실선으로 연결합니다.
- **의존 관계(Dependency, ⊖):** 한 클래스가 다른 클래스의 메서드를 호출하거나 매개변수로 사용하는 등 일시적인 참조가 일어날 때 쓰는 관계입니다. (점선 화살표로 표시)

161. 키(Key)에 대한 다음 설명에서 각 번호(①~④)에 해당하는 키를 <보기>에서 찾아 기호(㉠~㉥)로 쓰시오. 24-3

- ① 다른 릴레이션의 기본키를 참조하는 속성 또는 속성들의 집합을 의미함
- ② 릴레이션을 구성하는 속성들 중에서 튜플을 유일하게 식별하기 위해 사용되는 속성들의 부분집합으로, 유일성과 최소성을 모두 만족시켜야 함
- ③ 후보키가 둘 이상일 때 기본키를 제외한 나머지 후보키를 의미함
- ④ 한 릴레이션 내에 있는 속성들의 집합으로 구성된 키로, 유일성은 만족하지만, 최소성은 만족하지 못함

<보기>

- ㉠ 슈퍼키 ㉡ 외래키 ㉢ 대체키 ㉣ 후보키 ㉤ 기본키 ㉥ 필드키

답: ① ㉡ ② ㉤ ③ ㉢ ④ ㉠

[해설]

관계형 데이터베이스(RDB) 키의 특성 정의

- **유일성(Uniqueness):** 하나의 키 값으로 릴레이션 내 유직한 하나의 튜플을 구별해낼 수 있는 성질
- **최소성(Minimality):** 키를 구성하는 속성들 중 꼭 필요한 최소한의 속성으로만 구성되어야 하는 성질

키 종류	유일성 여부	최소성 여부	설명 및 특징
슈퍼키 (Super Key)	○	X	튜플 식별은 가능하나 불필요한 속성이 섞여 있는 키
후보키 (Candidate Key)	○	○	기본키가 될 자격을 갖춘 정제된 식별 키
대체키 (Alternate Key)	○	○	후보키 중 대표(기본키)로 선출되지 못하고 탈락한 나머지 키
외래키 (Foreign Key)	참조 조건에 따름		다른 테이블의 기본키를 참조하여 테이블 간 관계를 형성하는 키

162. 다음 무결성 제약 조건에 대한 설명에서 괄호에 들어갈 알맞은 용어를 쓰시오. 24-3

() 무결성은 기본 테이블의 기본키를 구성하는 어떤 속성도 Null 값이나 중복값을 가질 수 없다는 규정이다.

답: 개체 (또는 개체 무결성)

[해설]

주요 무결성 제약 조건 3대 유형

- **개체 무결성(Entity Integrity):** 테이블의 기본키는 절대 비어있을 수 없고(Not Null), 유일해야 합니다(Unique).
- **참조 무결성(Referential Integrity):** 외래키 값은 참조하는 테이블의 기본키 값과 일치하거나 Null이어야 합니다. 즉, 존재하지 않는 대상의 참조를 방지합니다.

- **도메인 무결성(Domain Integrity):** 특정 속성의 값이 지정된 범위나 타입 정의(Domain)에 만족하는 값이어야 함을 제한합니다.

163. 네트워크 및 인터넷과 관련된 다음 설명에 해당하는 용어를 <보기>에서 찾아 기호(㉠~㉤)로 쓰시오. [24-3](#)

- 재난 및 군사 현장과 같이 별도의 고정된 유선망을 구축할 수 없는 장소에서 모바일 호스트(Mobile Host)만을 이용하여 구성된 네트워크이다.
- 망을 구성한 후 단기간 사용되는 경우나 유선망을 구성하기 어려운 경우에 적합하다.
- 멀티 홉 라우팅 기능을 지원한다.

<보기>

- | | |
|---------------------------|--------------------------|
| ㉠ Mesh Network | ㉡ Peer-to-Peer Network |
| ㉢ Virtual Private Network | ㉣ Ad-hoc Network |
| ㉤ Sensor Network | ㉥ Infrastructure Network |

답: ㉣

[해설]

애드혹 네트워크(Ad-hoc Network)의 개념

- **자율적 네트워크:** AP(Access Point) 같은 중앙 집중형 고정 기지국 인프라 없이, 노드(단말기)들이 자율적으로 직접 서로를 연결하는 무선 네트워크입니다.
- **멀티홉 동적 라우팅:** 물리적 거리가 멀 경우 중간 호스트들이 패킷을 릴레이처럼 전달해 주며 동적으로 네트워크 토폴로지가 유지됩니다. 긴급 구조 현장에 최적화되어 있습니다.

※ 보기 내 주요 기술 용어 분석:

- **Mesh Network:** 그물망 구조로 기기들이 다대다 연결되어 커버리지를 확장하는 무선 네트워크
- **Sensor Network:** 센서 노드들이 계측한 환경 정보를 싱크 노드로 수집 및 중계하는 정보망

164. 다음 설명에 해당하는 용어를 쓰시오. [24-2](#)

- 시스템의 성능을 향상하고 개발 및 운영의 편의성 등을 높이기 위해 정규화된 데이터 모델을 의도적으로 통합, 중복, 분리하여 정규화 원칙을 위배하는 행위이다.
- 이를 수행하면 시스템의 성능이 향상되고 관리 효율성은 증가하지만 데이터의 일관성 및 정합성이 저하될 수 있다.
- 과도한 수행은 오히려 성능을 저하시킬 수 있다.

답: **반정규화, Denormalization, 비정규화, 역정규화**

[해설]

반정규화 (Denormalization)

정규화된 데이터베이스 디자인에서 조인(Join) 연산의 부하를 줄이고 단순 조회 질의의 반응 속도를 고속화하기 위해, 의도적으로 설계 규칙을 깨뜨리고 엔티티나 속성을 병합/중복시키는 성능 향상 튜닝 기법입니다. 데이터의 삽입/수정 시 무결성 오버헤드가 동반되므로 신중하게 트레이드 오프를 조율해야 합니다.

165. 다음 설명에 해당하는 프로토콜을 쓰시오. [24-2](#)

- 네트워크 계층에서 IP 패킷 단위의 데이터 변조 방지 및 은닉 기능을 제공하는 프로토콜이다.
- 주요 구성 요소에는 AH(Authentication Header), ESP(Encapsulating Security Payload), SA(Security Association), IKE(Internet Key Exchange)가 있다.

- 주요 기능에는 암호화, 무결성, 인증, 재전송 방지가 있다.

답: **IPsec (IP Security)**

[해설]

IPsec (IP Security)

IP 계층(네트워크 계층)에서 보안성을 확보하기 위해 전송되는 패킷의 변조 방지, 은닉, 기밀성 및 무결성을 제공하는 프로토콜 제품군입니다. 주요 구성 요소로 인증을 담당하는 AH, 암호화와 인증을 모두 제공하는 ESP, 연결 설정을 위한 SA, 키 교환을 수행하는 IKE 등이 결합하여 안전한 종단 간(End-to-End) 통신이나 VPN 구현에 활용됩니다.

166. 모듈에 대한 다음 설명에 해당하는 응집도(Cohesion)를 <보기>에서 찾아 기호(㉠~㉤)로 쓰시오. 24-2

모듈 내 하나의 활동으로부터 나온 출력 데이터를 그 다음 활동의 입력 데이터로 사용할 경우의 응집도

<보기>

- ㉠ 기능적 응집도 ㉡ 순차적 응집도 ㉢ 교환적 응집도 ㉣ 절차적 응집도
㉤ 시간적 응집도 ㉥ 논리적 응집도 ㉦ 유연적 응집도

답: ㉡

[해설]

모듈의 응집도(Cohesion) 단계 분석  [클릭!](#)

순차적 응집도(Sequential Cohesion)는 모듈 내부의 한 활동에서 도출된 출력 결과물(Output)이 바로 다음 활동의 입력값(Input)으로 직접 활용되는 밀접한 형태의 응집도 구조를 의미합니다.

167. 준비상태 큐에 각 프로세스의 도착 시간과 실행 시간이 다음과 같을 때 SRT(Shortest Remaining Time)로 스케줄링 할 때 평균 대기 시간을 쓰시오. 24-2

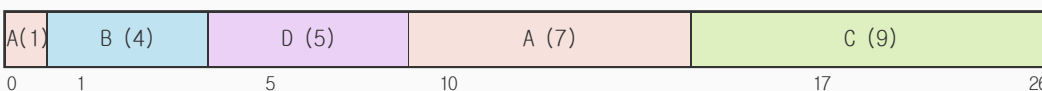
프로세스	도착 시간	실행 시간
A	0	8
B	1	4
C	2	9
D	3	5

답: **6.5**

[해설]

선점형 스케줄링 SRT(Shortest Remaining Time)의 정밀 분석 및 대기 시간 계산

1) Gantt Chart (프로세스 변환 과정):



2) 시간대별 준비상태 큐(Ready Queue) 흐름 및 CPU 할당:

- **0초 ~ 1초:** A가 도착하여 즉시 실행을 시작함. [큐 상태: 빈 상태] -> 1초 시점에 B가 도착하면서 선점 조건 비교가 발생함.
- **1초 ~ 5초:** 1초 시점에 A의 남은 시간은 7초, 새로 도착한 B의 실행 시간은 4초임. B가 더 짧으므로 **CPU를 선점함**. 실행 도중 2초에 C(9초), 3초에 D(5초)가 도착하여 대기함. [큐 상태: D(5), A(7), C(9)] -> 5초에 B는 실행을 완료하고 최종 종료됨.

- **5초 ~ 10초:** 5초 시점에 큐에 대기 중인 프로세스 중 남은 시간이 가장 짧은 D(5초)가 선택되어 실행됨. [큐 상태: A(7), C(9)] -> 10초에 D는 실행을 완료하고 최종 종료됨.
- **10초 ~ 17초:** 10초 시점에 남은 프로세스 A(7초)와 C(9초) 중 더 짧은 **A가 CPU를 할당**받아 남은 작업을 모두 수행함. [큐 상태: C(9)] -> 17초에 A는 실행을 완료하고 최종 종료됨.
- **17초 ~ 26초:** 마지막으로 큐에 남아있던 C(9초)가 복귀하여 남은 작업을 모두 완료하고 26초에 최종 종료됨.

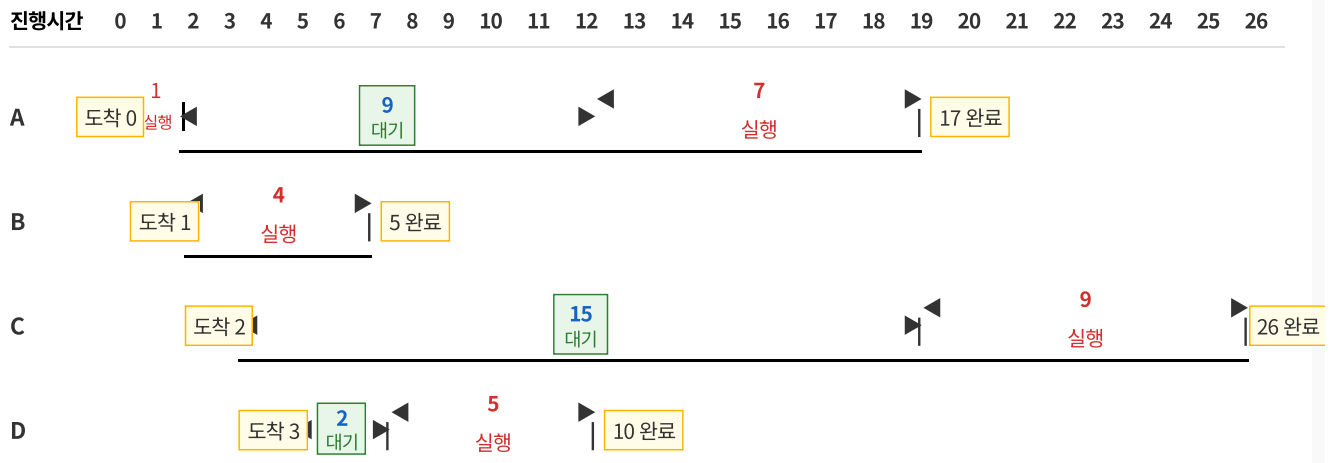
3) 프로세스별 대기 시간 계산 (대기 시간 = 종료 시간 - 도착 시간 - 실행 시간):

- **A:** 17 (종료) - 0 (도착) - 8 (실행) = **9초** (1초~10초 동안 대기)
- **B:** 5 (종료) - 1 (도착) - 4 (실행) = **0초** (대기 없음)
- **C:** 26 (종료) - 2 (도착) - 9 (실행) = **15초** (2초~17초 동안 대기)
- **D:** 10 (종료) - 3 (도착) - 5 (실행) = **2초** (3초~5초 동안 대기)

4) 평균 대기 시간 계산:

평균 대기 시간 = $(9 + 0 + 15 + 2) / 4 = 26 / 4 = 6.5$ 초

※ 주의: SRT는 잔여 실행 시간이 가장 짧은 프로세스에게 수시로 CPU를 넘겨주는 선점형 방식이므로, 간트 차트 스냅샷별 남은 시간 트래킹이 핵심입니다.



• 평균 대기 시간 : $(9 + 0 + 15 + 2) / 4 = 26 / 4 = 6.5$

• 평균 반환 시간 : $(17 + 4 + 24 + 7) / 4 = 52 / 4 = 13$

결과 **6.5초**

168. 다음 설명에 해당하는 디자인 패턴을 <보기>에서 찾아 쓰시오. 24-2

- 자료 구조와 같이 접근이 잦은 객체에 대해 동일한 인터페이스를 사용하도록 하는 패턴이다.
- 내부 표현 방법의 노출 없이 순차적인 접근이 가능하다.

<보기> 클릭!

생성 패턴	구조 패턴	행위 패턴
Abstract Factory	Adapter	Command
Builder	Bridge	Interpreter
Factory Method	Composite	Iterator
Prototype	Decorator	Mediator
Singleton	Proxy	Observer

답: **Iterator**

[해설]

행위 패턴 - 이터레이터 패턴(Iterator Pattern)

컬렉션(리스트, 배열, 트리 등)의 구현 방법을 노출시키지 않으면서도, 내부에 저장된 요소들에 순차적으로 접근할 수 있는 표준화된 통일 인터페이스를 제공하는 디자인 패턴입니다.

지문에 제시된 "내부 표현 방법의 노출 없이"와 "순차적인 접근이 가능하다"라는 설명은 객체 지향 디자인 패턴 중 GoF(Gang of Four) 행위 패턴에 속하는 **Iterator** 패턴의 핵심 정의입니다.

169. 모듈에 대한 다음 설명에 해당하는 결합도(Coupling)를 <보기>에서 찾아 기호(㉠~㉥)로 쓰시오. 24-2

- 어떤 모듈이 다른 모듈 내부의 논리적인 흐름을 제어하기 위해 제어 신호나 제어 요소를 전달하는 결합도이다.
- 하위 모듈에서 상위 모듈로 제어 신호가 이동하여 하위 모듈이 상위 모듈에게 처리 명령을 내리는 권리 전도 현상이 발생하게 된다.

<보기>  클릭!

- ㉠ 자료 결합도 ㉡ 스탬프 결합도 ㉢ 제어 결합도
㉣ 공통 결합도 ㉤ 내용 결합도 ㉥ 외부 결합도

답: ㉢

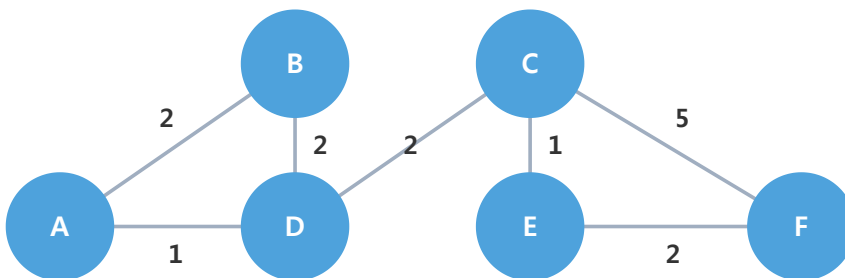
[해설]

모듈 간 결합도 - 제어 결합도(Control Coupling)

모듈 간 통신 시 단순 연산용 데이터 값만 송수신하는 것이 아니라, "어떻게 처리하라는 명령이나 로직 제어 플래그 신호(Flag, Switch 등)"를 매개변수로 던져 상대 모듈의 로직 흐름을 좌지우지 제어할 때 기록되는 높은 수준의 결합도입니다.

특히 지문에 서술된 바와 같이 하위 단위 시스템에서 거꾸로 상위 컨트롤러에게 동작 지시 신호를 보내어 통제권을 뒤흔드는 **권리 전도(Inversion of Authority)** 문제를 유발하므로 결합도 평가 지표 상에서 좋지 않은 설계 유형으로 관리됩니다.

170. 다음 그림에서 ㉠~㉦는 라우터이고, 각 링크 상의 숫자는 가중치 값을 나타낼 때 RIP(Routing Information Protocol)을 이용한 A에서 F까지의 최적 경로를 쓰시오. 24-2



답: A → D → C → F

[해설]

RIP(Routing Information Protocol)의 메트릭 산정 기준 분석

기출문제를 풀 때 가장 주의해야 할 함정 유형입니다. 문제 도면에 각 링크마다 가중치 숫자(2, 1, 5 등)가 기재되어 있어 마치 다익스트라(Dijkstra) 알고리즘이나 OSPF 프로토콜처럼 최단 링크 비용 합산으로 계산하도록 유도하고 있습니다.

하지만 문제 조건에서 명시한 거리 벡터 라우팅 프로토콜인 **RIP의 유일한 메트릭(Metric) 지표는 오직 홉 카운트(Hop Count, 거쳐 가는 라우터의 개수)**입니다. 링크에 명시된 가중치 숫자는 전부 무시합니다.

출발지 **A**에서 목적지 **F**까지 이동할 때, 거쳐 가는 라우터 수가 가장 최소가 되는 유일한 최적 홉 카운트 경로는

- **A → D → C → F 경로: 홉 카운트 = 3회**
- **A → B → D → C → F 경로: 홉 카운트 = 4회**
- **A → D → C → E → F 경로: 홉 카운트 = 4회**

171. 다음 설명에 해당하는 라우팅 프로토콜을 쓰시오. [24-1](#)  **클릭!**

- RIP의 단점을 해결하여 새로운 기능을 지원하는 인터넷 프로토콜이다.
- 최단 경로 탐색에 Dijkstra 알고리즘을 사용한다.
- 대규모 네트워크에서 많이 사용된다.
- 링크 상태를 실시간으로 반영하여 최단 경로로 라우팅을 지원한다.

답: **OSPF**

[해설]

링크 상태 라우팅 프로토콜 - OSPF (Open Shortest Path First)

- **링크 상태(Link State) 알고리즘:** 거리 벡터 방식인 RIP와 달리 네트워크 토폴로지 변경 사항을 실시간으로 링크 상태 광고(LSA)를 통해 전파하고 전 지도의 맵을 구성합니다.
- **다익스트라(Dijkstra) 알고리즘 활용:** 모든 라우터가 최단 경로 트리(Shortest Path Tree)를 독자적으로 계산하여 경로 비용(Cost) 메트릭이 최소인 최적의 루트를 산출합니다.
- **대규모 네트워크 적합성:** 홉 카운트 제약(최대 15홉)이 있는 RIP와 달리 Area 단위를 나누어 대규모망을 체계적이고 효율적으로 관리할 수 있습니다.

172. 다음 설명에 해당하는 라우팅 프로토콜(Routing Protocol)을 쓰시오. [20-3](#)

- RIP의 단점을 해결하여 새로운 기능을 지원하는 인터넷 프로토콜이다.
- 인터넷 망에서 이용자가 최단 경로를 선정할 수 있도록 라우팅 정보에 노드 간의 거리 정보, 링크 상태 정보를 실시간으로 반영하여 최단 경로로 라우팅을 지원한다.
- 대규모 네트워크에서 많이 사용된다.
- 최단 경로 탐색에 Dijkstra 알고리즘을 사용한다.
- 라우팅 정보에 변화가 생길 경우 변화된 정보만 네트워크 내의 모든 라우터에 알린다.
- 링크 스테이트 라우팅 알고리즘을 사용하며, 하나의 자율 시스템(AS)에서 동작하면서 내부 라우팅 프로토콜의 그룹에 도달한다.

답: **OSPF**

173. 다음의 정규화(Normalization) 과정은 어떤 단계의 정규화 과정인지 <보기>에서 찾아 쓰시오. 24-1

<주문>

주문번호	고객번호	주소
A345	100	서울
D347	200	부산
A210	300	광주
B230	200	부산

<주문>

주문번호	고객번호
A345	100
D347	200
A210	300
B230	200

<고객>

고객번호	주소
100	서울
200	부산
300	광주

<보기>

- 제1정규형
- 제2정규형
- 제3정규형
- 보이스/코드 정규형
- 제4정규형
- 제5정규형

답: 제3정규형

[해설]

이행적 함수 종속성(Transitive Functional Dependency) 제거 분석

- 기본키 및 함수 종속성 파악: 변경 전 <주문> 테이블의 기본키는 주문번호입니다. 주문번호를 알면 고객번호를 알 수 있고(주문번호 → 고객번호), 고객번호를 알면 그 고객의 주소를 알 수 있습니다(고객번호 → 주소).
- 이행적 함수 종속 관계 식별: 결과적으로 주문번호 → 고객번호이고 고객번호 → 주소이므로, 주 식별자인 주문번호를 통해 주소 속성이 종속되는 주문번호 → 주소라는 이행적 함수 종속성이 성립합니다.
- 테이블 분리(정규화) 과정: 이러한 이행적 함수 종속 관계를 만족하는 속성들을 별도의 릴레이션으로 분리하여 기본키에 의해서만 속성들이 결정되도록 쪼개는 단계를 제3정규형(3NF)이라고 합니다.
 - <주문> 테이블: 주문번호 → 고객번호 (이행 종속 제거)
 - <고객> 테이블: 고객번호 → 주소 (독립적인 관계 정립)

<주문목록>

제품번호	제품명	재고수량	주문번호	고객번호	주소	주문수량
1001	모니터	2000	A345	100	서울	150
			D347	200	부산	300
1007	마우스	9000	A210	300	광주	600
			A345	100	서울	400
			B230	200	부산	700
1201	키보드	2100	D347	200	부산	300



<제품>

제품번호	제품명	재고수량
1001	모니터	2000
1007	마우스	9000
1201	키보드	2100

<제품주문>

주문번호	제품번호	고객번호	주소	주문수량
A345	1001	100	서울	150
D347	1001	200	부산	300
A210	1007	300	광주	600
A345	1007	100	서울	400
B230	1007	200	부산	700
D347	1201	200	부산	300

정규화 과정

비정규 릴레이션

↓ 도메인이 원자값

1NF

↓ 부분적 함수 종속 제거

2NF

↓ 이행적 함수 종속 제거

3NF

↓ 결정자이면서 후보키가 아닌 것 제거

BCNF

↓ 다치 종속 제거

4NF

↓ 조인 종속성 이용

5NF

174. 다음에 제시된 응집도(Cohesion)를 높은 순에서 낮은 순으로 나열하시오. 24-1  클릭!

- ㉠ 기능적 응집도(Functional Cohesion)
- ㉡ 교환적 응집도(Communication Cohesion)
- ㉢ 우연적 응집도(Coincidental Cohesion)
- ㉣ 시간적 응집도(Temporal Cohesion)

답: ㉠ → ㉡ → ㉣ → ㉢

[해설]

모듈 내부 요소의 연관 지표 - 응집도(Cohesion) 단계 분석

소프트웨어 공학에서 모듈의 독립성을 높이기 위해서는 "결합도는 낮게(Weak), 응집도는 높게(Strong)" 구성해야 합니다.

[Strong] 기능적(Functional) > 순차적(Sequential) > 교환적(Communication) > 절차적(Procedural) > 시간적(Temporal) > 논리적(Logical) > 우연적(Coincidental) [Weak]

- ㉠ 기능적 응집도 (모듈 내부의 모든 요소가 하나의 단일 기능을 수행)
- ㉡ 교환적/통신적 응집도 (동일한 입출력 자료를 공유하여 서로 다른 기능을 수행)
- ㉢ 시간적 응집도 (특정 시점에 연동되어 쓰이는 기능들을 한 모듈 내에서 처리)
- ㉣ 우연적 응집도 (모듈 내부 요소들이 서로 아무런 연관성 없이 구성됨)

175. 다음 설명에 해당하는 디자인 패턴을 <보기>에서 찾아 쓰시오. 24-1

- 구체적인 클래스에 의존하지 않고, 인터페이스를 통해 서로 연관의존하는 객체들의 그룹으로 생성하여 추상적으로 표현한다.
- 키트(Kit) 패턴이라고도 부른다.
- 연관된 서브 클래스를 묶어 한 번에 교체하는 것이 가능하다.

<보기>  클릭!

생성 패턴	구조 패턴	행위 패턴
Abstract Factory	Adapter	Command
Builder	Bridge	Interpreter
Factory Method	Composite	Iterator
Prototype	Decorator	Mediator
Singleton	Proxy	Observer

답: Abstract Factory

[해설]

생성 패턴 - 추상 팩토리 패턴(Abstract Factory Pattern)

구체적인 팩토리 서브 클래스를 명시하지 않고도 연관-의존적인 복수 객체들의 그룹(패밀리)을 생성하기 위한 총괄 인터페이스를 제공하는 디자인 패턴입니다.

176. 3개의 페이지를 수용할 수 있는 주기억장치가 있으며, 초기에는 모두 비어 있다고 가정한다. 다음의 순서로 페이지 참조가 발생할 때, LRU와 LFU 페이지 교체 알고리즘을 사용하면 각각 페이지 결함(Page Fault)의 발생 횟수를 쓰시오. 24-1

알고리즘	페이지 참조 순서
LRU	1, 2, 3, 1, 2, 4, 5, 1

LFU	1, 2, 3, 1, 2, 4, 1, 2, 3, 4
-----	------------------------------

답: LRU: 6회, LFU: 6회

[해설]

페이지 교체 알고리즘별 가상메모리 할당 시뮬레이션 (초기 프레임 수 = 3)

1. LRU (Least Recently Used) 알고리즘 분석

가장 오랫동안 참조되지 않은 페이지를 대상(Victim Page)으로 선정하여 교체하는 알고리즘입니다.

참조 스트림	1	2	3	1	2	4	5	1
프레임 1	1	1	1	1	1	4	4	1
프레임 2	-	2	2	2	2	2	5	5
프레임 3	-	-	3	3	3	3	3	1
결과 (F/H)	●	●	●			●	●	●

- 참조 4 시점: 프레임 내부 [1, 2, 3] 중 최근 사용 순서는 2 → 1 → 3 순이므로, 가장 오랫동안 쓰이지 않은 **3이 제거**되고 4가 진입합니다. (Fault)
- 참조 5 시점: 프레임 내부 [1, 2, 4] 중 최근 사용 순서는 4 → 2 → 1 순이므로, 가장 오래된 **1이 제거**되고 5가 진입합니다. (Fault)
- 참조 1 시점: 프레임 내부 [4, 2, 5] 중 최근 사용 순서는 5 → 4 → 2 순이므로, 가장 오래된 **2가 제거**되고 1이 진입합니다. (Fault)
- LRU 총 페이지 결함(Page Fault) 횟수 = **6회**

2. LFU (Least Frequently Used) 알고리즘 분석

과거 참조 횟수(빈도수)가 가장 적은 페이지를 교체 대상으로 선정하는 알고리즘입니다. (빈도수가 같을 경우 일반적으로 가장 오래 있었던 페이지인 FIFO 규칙을 적용합니다.)

참조 스트림	1	2	3	1	2	4	1	2	3	4
프레임 1	1	1	1	1	1	1	1	1	1	4
프레임 2	-	2	2	2	2	2	2	2	2	2
프레임 3	-	-	3	3	3	4	4	4	3	3
결과 (F/H)	●	●	●			●			●	●

- 참조 4 시점 (직전 누적 카운트: [1: 2회, 2: 2회, 3: 1회]): 빈도수가 가장 낮았던 **3이 제거**되고 4가 프레임에 적재됩니다. (Fault)
- 참조 3 시점 (직전 누적 카운트: [1: 3회, 2: 3회, 4: 1회]): 빈도수가 1회로 가장 낮은 **4가 제거**되고 3이 복귀합니다. (Fault)
- 참조 4 시점 (직전 누적 카운트: [1: 3회, 2: 3회, 3: 1회]): 빈도수가 1회로 가장 낮은 **3이 다시 제거**되고 4가 재진입합니다. (Fault)
- LFU 총 페이지 결함(Page Fault) 횟수 = **6회**

177. 다음은 조인(Join)에 대한 설명이다. 괄호(①~③)에 들어갈 알맞은 조인의 종류를 <보기>에서 찾아 쓰시오. [24-1](#)

- (①)은 조인에 참여하는 두 릴레이션의 속성 값을 비교하여 조건을 만족하는 튜플만 반환하는 조인이다.
- (②)은 (①)에서 = 연산자를 사용한 조인으로, 일반적으로 조인이라고 하면 (②)을 의미한다.
- (②)의 결과 릴레이션의 차수는 첫 번째 릴레이션과 두 번째 릴레이션의 차수를 합한 것이다.
- (③)은 (②)의 결과 릴레이션에서 중복된 속성을 제거하여 수행하는 연산, 즉 (②)에서 중복 속성 중 하나가 제거된 것이다.
- (③)의 핵심은 두 릴레이션의 공통된 속성을 매개체로 하여 두 릴레이션의 정보를 '관계'로 묶어 내는 것이다.

<보기>

자연 조인	외부 조인	셀프 조인	세타 조인
동등 조인	교차 조인	자연 조인	자동 조인

답: ① 세타 조인 ② 동등 조인 ③ 자연 조인

[해설]

관계해석/관계대수 조인 연산자의 계층 구조

- ① **세타 조인(Theta Join)**: 비교 연산자(<, >, =, ≠, ≤, ≥)를 일반화하여 속성 간의 비교 조건을 만족하는 조건 조인 구조입니다.
- ② **동등 조인(Equi Join)**: 세타 조인의 한 종류로서 오직 **비교 연산자가 '='(Equal)인 조인**을 가리키며, 조건에 참여한 중복 컬럼이 결과 릴레이션에 그대로 나열되므로 차수(Degree)가 양측의 합이 됩니다.
- ③ **자연 조인(Natural Join)**: 동등 조인의 결과물에서 속성 이름이 같은 **중복 컬럼을 자동으로 제거**하여 결합해 주는 가장 표준적이고 유기적인 조인 형태입니다.

EQUI JOIN(동등 조인)

- JOIN에 참여하는 두 릴레이션의 속성 값을 비교하여 조건을 만족하는 튜플만 반환하는 조인을 THETA JOIN(세타 조인)이라고 하며, 조인에 사용되는 조건에는 =, ≠, <, ≤, >, ≥가 있음.
- EQUI JOIN(동등 조인)은 조인에 사용되는 조건 중 =(equal) 비교에 의해 같은 값을 가지는 행을 연결하여 결과를 생성하는 INNER JOIN 방법임.
- EQUI JOIN의 결과 표시되는 속성(차수)은 두 릴레이션의 속성을 더한 것과 같음.
- EQUI JOIN에서 JOIN 조건이 '='일 때 동일한 속성이 두 번 나타나게 되는데, 이 중 중복된 속성을 제거하여 같은 속성을 한 번만 표기하는 방법을 NATURAL JOIN(자연 조인)이라고 함.
- EQUI JOIN에서 연결 고리가 되는 공통 속성을 JOIN 속성이라고 함.

- WHERE절을 이용한 EQUI JOIN의 표기 형식

```
SELECT [테이블명1.]속성명, [테이블명2.]속성명, ...
FROM 테이블명1, 테이블명2, ...
WHERE 테이블명1.속성명 = 테이블명2.속성명;
```

- NATURAL JOIN절을 이용한 EQUI JOIN의 표기 형식

```
SELECT [테이블명1.]속성명, [테이블명2.]속성명, ...
FROM 테이블명1 NATURAL JOIN 테이블명2;
```

- JOIN ~ USING절을 이용한 EQUI JOIN의 표기 형식

```
SELECT [테이블명1.]속성명, [테이블명2.]속성명, ...
FROM 테이블명1 JOIN 테이블명2 USING(속성명);
```

178. 다음 설명에 해당하는 커버리지(Coverage)를 <보기>에서 찾아 쓰시오. [24-1](#)

- 개별 조건식이 다른 개별 조건식의 영향을 받지 않고 전체 조건식의 결과에 독립적으로 영향을 주는 구조적 테스트 케이스이다.
- 해당 개별 조건식이 전체 조건식의 결과에 영향을 주는 조건 조합을 찾아 커버리지를 테스트하는 방법이다.
- 프로그램에 있는 모든 결정 포인트 내의 전체 조건식이 적어도 한 번은 참과 거짓을 만족해야 한다.
- 프로그램에 있는 결정 포인트 내의 모든 개별 조건식이 적어도 한 번은 참과 거짓을 만족해야 한다.

<보기>

· All Path	· Multiple Condition	· MC/DC	· Condition/Decision
· Condition	· Decision	· Statement	

답: **MC/DC**

[해설]

변경 조건/결정 커버리지 (MC/DC: Modified Condition/Decision Coverage)

- **핵심 정의**: 각 개별 조건식이 다른 개별 조건식의 영향을 받지 않고(**독립적으로**) 전체 결과에 참/거짓 변화를 유발할 수 있음을 검증하는 강력한 화이트박스 테스트 기준입니다.
- **주요 특징**: 복합 조건식에서 발생할 수 있는 방대한 조건 조합(다중 조건 커버리지)의 단점인 테스트 케이스 개수

를 N + 1개 수준으로 최적화하면서도 높은 결함 발견율을 보장하므로, 항공기나 철도 등 높은 안전성이 요구되는 임베디드 소프트웨어 검증에 필수적으로 활용됩니다.

문장 검증 기준(Statement Coverage)

- 소스 코드의 모든 구문이 한 번 이상 수행되도록 테스트 케이스를 설계함.

결정 검증 기준(Decision Coverage)

- 소스 코드의 모든 조건문에 대해 조건식의 결과가 True인 경우와 False인 경우가 한 번 이상 수행되도록 테스트 케이스를 설계함.
- 분기 검증 기준(Branch Coverage)이라고도 함.

조건 검증 기준(Condition Coverage)

- 소스 코드의 조건문에 포함된 개별 조건식의 결과가 True인 경우와 False인 경우가 한 번 이상 수행되도록 테스트 케이스를 설계함.

179. 보안 위협에 대한 다음 설명에 해당하는 용어를 <보기>에서 찾아 쓰시오. 24-1

- 시스템에 침입한 후 침입 사실을 숨긴 채 백도어, 트로이목마를 설치하고, 원격 접근, 내부 사용 흔적 삭제, 관리자 권한 획득 등 주로 불법적인 해킹에 사용되는 기능들을 제공하는 프로그램들의 모음이다.
- 자신 또는 다른 소프트웨어의 존재를 감춰줌과 동시에 허가되지 않은 컴퓨터나 소프트웨어의 영역에 접근할 수 있게 하는 용도로 설계되었다.
- 이 프로그램이 설치되면 자신이 뚫고 들어온 모든 경로를 바꾸어 놓고, 명령어들을 은폐해 놓기 때문에 해커가 시스템을 원격에서 해킹하고 있어도 이 프로그램이 설치되어 있는 사실조차 감지하기 어렵다.
- 공격자가 보안 관리자나 보안 시스템의 탐지를 피하면서 시스템을 제어하기 위해 설치하는 악성 프로그램으로, 운영체제의 합법적인 명령어를 해킹하여 모아놓았다.
- 운영체제에서 실행 파일과 실행 중인 프로세스를 숨김으로써 운영체제 검사 및 백신 프로그램의 탐지를 피할 수 있다.

<보기>  클릭!

· Worm	· Logic Bomb	· Spyware	· Honeypot
· Bug Bounty	· Rootkit	· Bootkit	· Ransomware

답: **Rootkit**

[해설]

루트킷 (Rootkit)

- 핵심 정의:** 시스템에 침입한 사실을 숨긴 채 백도어 설치, 원격 접근, 로그 삭제, 프로세스 은폐 등 관리자 권한(Root)을 획득하여 불법적인 행위를 유지하기 위해 사용하는 악성 프로그램들의 모음(Kit)입니다.
- 주요 특징:** 커널 및 운영체제의 합법적인 명령어와 시스템 호출을 가로채거나 변조하여 동작하므로 백신 프로그램이나 일반 보안 관리 도구로 탐지하기가 매우 까다롭습니다.

웜 (Worm)

- 독자적으로 실행되며 네트워크를 통해 자신을 복제하고 전파하여 대역폭을 고갈시키거나 시스템에 손상을 주는 악성코드입니다.

논리 폭탄 (Logic Bomb)

- 특정 날짜나 시간, 혹은 특정 조건이 만족되었을 때 자동으로 악성 동작(데이터 삭제, 시스템 파괴 등)을 수행하도록 설계된 코드입니다.

랜섬웨어 (Ransomware)

- 사용자의 파일을 암호화하여 접근하지 못하도록 한 뒤, 이를 복구해 주는 대가로 금전(몸값)을 요구하는 악성 소프트웨어입니다.

180. 다음 설명에 해당하는 용어를 <보기>에서 찾아 쓰시오. 24-1

다양한 IT 기술과 방식들을 이용해 조직적으로 특정 기업이나 조직 네트워크에 침투해 활동 거점을 마련한 뒤 때를 기다리면서 보안을 무력화시키고 정보를 수집한 다음 외부로 빼돌리는 형태의 공격으로, 일반적으로 공격은 침투, 검색, 수집, 유출의 4단계로 실행된다.

- **침투(Infiltration)** : 목표로 하는 시스템을 악성코드로 감염시켜 네트워크에 침투한다.
- **검색(Exploration)** : 시스템에 대한 정보를 수집하고 기밀 데이터를 검색한다.
- **수집(Collection)** : 보호되지 않은 시스템의 데이터를 수집하고, 시스템 운영을 방해하는 악성코드를 설치한다.
- **유출(Exfiltration)** : 수집한 데이터를 외부로 유출한다.

<보기>

· MITM	· ATM	· XDR	· APT
· Key Logger Attack	· 사회공학 기법	· TearDrop	· SMURFING

답: **APT**

[해설]

지능형 지속 위협 (APT: Advanced Persistent Threat)

- **핵심 정의**: 특정 조직이나 기업을 표적으로 삼고 다양한 IT 기술과 해킹 기법을 동원하여, 장기간에 걸쳐 **지속적이고 지능적으로** 침투·잠복하며 정보를 수집하고 유출하는 정밀 타격형 공격 방식입니다.
- **주요 특징**: 단발성 공격과 달리 탐지를 피하기 위해 오랜 기간 숨어 지내며 시스템의 취약점을 분석하고 내부 거점을 확대해 나가는 것이 특징입니다.

중간자 공격 (MITM: Man In The Middle)

- 통신하는 두 사용자 사이에 끼어들어 송수신 데이터를 도청하거나 변조하는 가로채기 형태의 공격 기법입니다.

티어드롭 (TearDrop)

- IP 데이터그램의 분할(Fragmentation) 과정에서 오프셋(Offset) 값을 조작하여 수신 측이 패킷을 재조합할 때 오류나 과부하를 일으키게 하는 DoS 공격 기법입니다.

스머핑 (SMURFING)

- 출발지 IP를 피해자의 IP로 위조(Spoofing)한 후, 네트워크 내 전체 호스트에게 브로드캐스트로 ICMP Echo Request를 보내 대량의 응답 패킷이 피해자에게 집중되도록 하는 디도스(DDoS) 공격 기법입니다.

181. 다음 설명에 해당하는 프로토콜을 쓰시오. 23-3

자료를 일정한 크기로 정하여 순서대로 전송하는 자료의 전송방식으로, 셀이라 부르는 53Byte의 고정 길이 패킷을 이용하여 처리가 단순하고 고속망에 적합하다. 또한 연속적으로 셀을 보낼 때 다중화를 하지 않고 셀 단위로 동기가 이루어지지만 경우에 따라 동기식 시간 분할 다중화를 사용하기도 한다. CBR, VBR의 처리가 가능하며, B-ISDN과 결합하여 서비스를 제공하기도 한다.

답: **ATM (Asynchronous Transfer Mode / 비동기 전송 모드)**

[해설]

비동기 전송 모드 (ATM: Asynchronous Transfer Mode) 기술 핵심 요약

- **핵심 키워드**: "53바이트 고정 길이 셀(Cell)", "B-ISDN의 핵심 전송 기술", "CBR, VBR 서비스 지원"
- **셀(Cell) 구조 분석**: ATM 패킷은 오직 **53바이트(헤더 5바이트 + 사용자 데이터 48바이트)**의 고정된 크기만 사용합니다. 가변 길이 패킷에 비해 오버헤드가 적고 하드웨어적인 고속 스위칭 처리가 가능합니다.

- 서비스 클래스 기능: 데이터의 특성에 따라 실시간성 보장이 필요한 **CBR(Constant Bit Rate)**이나 가변적 대역폭을 쓰는 **VBR(Variable Bit Rate)** 등의 품질(QoS) 메커니즘을 완벽하게 세분화하여 처리합니다.

182. 다음은 오류가 발생하는 JAVA 프로그램이다. 프로그램을 분석하여 오류가 발생하는 라인을 쓰시오. [23-3](#)

```
1 class Person {
2     private String name;
3     public Person(String val) {
4         name = val;
5     }
6     public static String get() {
7         return name;
8     }
9     public void print() {
10        System.out.println(name);
11    }
12 }
13 public class Test {
14     public static void main(String[] args) {
15         Person obj = new Person("Kim");
16         obj.print();
17     }
18 }
```

답: 7

[해설]

7 return name; // 컴파일 에러 발생 위치

Java의 static(정적) 멤버와 인스턴스 멤버의 메모리 로드 및 접근 제한 분석

- 2번 라인의 변수 name은 static 키워드가 붙지 않은 **인스턴스 변수(비정적 멤버 변수)**입니다. 인스턴스 변수는 객체가 메모리에 실제 생성(new 키워드로 할당)되어야만 사용할 수 있는 변수입니다.
- 6번 라인의 메서드 get()은 static 키워드가 붙은 **정적 메서드(클래스 메서드)**입니다. static이 붙은 멤버는 프로그램이 시작되어 클래스가 로드되는 시점에 **메모리의 Method Area(Static Area)에 자동으로 생성**되므로 객체 생성 없이도 사용이 가능합니다.
- 따라서 7번 라인의 return name;을 실행하는 시점에는 static 메서드인 get()은 이미 메모리에 올라와 존재하지만, 인스턴스 변수인 name은 아직 객체가 생성되지 않아 **메모리에 존재하지 않을 수 있습니다**.
- Java 규칙상 **정적(static) 메서드 내부에서는 인스턴스(비정적) 변수나 인스턴스 메서드를 직접 참조하거나 호출할 수 없습니다**. 접근 단계에서 Non-static field 'name' cannot be referenced from a static context 컴파일 에러를 발생시키므로 오류 라인은 7번입니다.

구분	정적 멤버 (static)	인스턴스 멤버 (non-static)
해당 코드 예시	public static String get()	private String name;
메모리 할당 시점	클래스가 로드될 때 (최초 1회)	new로 객체를 생성할 때마다
상호 참조 관계	인스턴스 멤버 접근 불가능 (Error)	정적(static) 멤버 언제든지 접근 가능

183. 다음 설명에 해당하는 프로토콜을 쓰시오. [23-3](#)

- 거리 벡터 라우팅 프로토콜이라고도 불리며, 최단 경로 탐색에 Bellman-Ford 알고리즘이 사용된다.

- 소규모 동종의 네트워크 내에서는 효율적이거나, 최대 홉(Hop) 수가 제한되므로 대규모 네트워크에서는 사용할 수 없다.
- 일정 시간 동안 라우팅 정보가 갱신되지 않으면 해당 경로를 이상 상태로 간주한다.

답: **RIP (Routing Information Protocol)**

[해설]

RIP (Routing Information Protocol) 경로 선택 정보 프로토콜 특징

- **라우팅 메커니즘**: 대표적인 **거리 벡터(Distance Vector)** 기반의 내부 라우팅 프로토콜(IGP)로, 최적 경로 계산을 위해 **벨만-포드(Bellman-Ford)** 알고리즘을 사용합니다.
- **홉 수(Hop Count)의 한계**: 최대 홉 수를 **15**로 제한하며, **16** 홉은 도달 불가능(Unreachable)한 경로로 간주합니다. 이 제약 때문에 규모가 큰 대규모 네트워크(엔터프라이즈급망)에는 적용이 불가능합니다.
- **주기적 업데이트 패닉 방지**: 매 30초마다 인접 라우터들과 라우팅 테이블 전체를 동기화하며, 일정 시간(기본 180초) 동안 특정 경로의 광고가 수신되지 않으면 홀드다운(Holddown) 상태를 거쳐 해당 경로를 이상 상태 및 유효하지 않은 경로로 처리합니다.

184. 무결성에 대한 다음 설명에서 괄호에 들어갈 알맞은 답을 쓰시오. 23-3

무결성이란 데이터베이스에 저장된 데이터 값과 그것이 표현하는 현실 세계의 실제 값이 일치하는 정확성을 의미한다. 무결성 제약 조건은 데이터베이스에 들어 있는 데이터의 정확성을 보장하기 위해 부정확한 자료가 데이터베이스 내에 저장되는 것을 방지하기 위한 제약 조건을 말한다.

<회원>

이름	주민번호	주소
kim	800212-2*****	서울
choi	820911-1*****	경기
kang	750815-1*****	인천

<결제>

결제번호	주문 상품	회원
1	rx-123	choi
2	dp-01	park
3	qiv-433	kang

위의 두 테이블에서는 <결제> 테이블의 '회원' 속성이 <회원> 테이블의 '이름' 속성을 참고하는 외래키이므로 () 무결성 제약 조건이 준수되어야 한다.

답: **참조**

[해설]

데이터베이스 무결성 제약 조건: 참조 무결성(Referential Integrity) 분석

- **개념 정의: 참조 무결성 제약 조건**은 외래키 값은 원래 테이블의 기본키 값과 동일하거나, 혹은 NULL 값이어야 한다는 규정입니다. 즉, 테이블 간의 참조 관계가 항상 유효함을 보장하는 조건입니다.
- **문제 데이터 연계 분석**: 지문에서 명시했듯이 <결제> 테이블의 '회원' 속성이 <회원> 테이블의 '이름' 속성을 참조하는 **외래키(Foreign Key)** 역할을 하고 있으므로, 괄호에 들어갈 알맞은 제약 조건은 '**참조**'입니다.
- **참고 (실제 데이터의 오류 예시)**:
 - <결제> 테이블의 회원 속성값으로 park가 들어가 있지만 참조 대상인 <회원> 테이블의 이름 속성에는 park라는 회원이 존재하지 않습니다.
 - 따라서 실제 데이터베이스 관점에서는 이 구조가 **참조 무결성 제약 조건을 위배**하고 있는 상태입니다.
- **핵심 요약**:
 - 기본키(PK), NULL 불가를 묻는다면 → **개체 무결성**
 - 외래키(FK), 타 테이블 참조를 묻는다면 → **참조 무결성**

185. 소프트웨어 데이터의 비정상적인 수정이 감지되면 소프트웨어를 오작동하게 만들어 악용을 방지하는 기술이다. 해시 함수, 핑거 프린트, 워터마킹 등의 보안 요소를 생성하여 소프트웨어에 삽입하고, 실행코드를 난독화하며, 실행 시 원본 비교 및 데이터 확인을 수행함으로써 소프트웨어를 보호하는 이 기술을 가리키는 용어를 쓰시오. 23-2

답: **템퍼 프루핑 (Tamper Proofing / 역변조 방지)**

[해설]

템퍼 프루핑 (Tamper Proofing) 보안 기술 분석

- 프로그램이나 데이터의 비정상적인 위·변조(Tampering) 행위가 감지되었을 때, 소프트웨어 스스로 작동을 멈추거나 의도적인 오작동(장애)을 유발하여 크래킹 및 무단 악용을 원천 차단하는 능동형 보안 방어 기술입니다.
- 지문에 제시된 것처럼 무결성을 검증하기 위한 해시 함수, 저작권 추적을 위한 핑거프린팅 및 워터마킹, 리버스 엔지니어링을 어렵게 만드는 코드 난독화(Obfuscation) 기술 등이 유기적으로 결합되어 시스템의 견고함을 높입니다.

186. 디자인 패턴에 대한 다음 설명에서 괄호(①, ②)에 들어갈 알맞은 패턴을 <보기>에서 찾아 쓰시오. 23-2

- (①): 하나의 객체를 생성하면 생성된 객체를 어디서든 참조할 수 있지만, 여러 프로세스가 동시에 참조할 수 없는 패턴으로, 불필요한 메모리 낭비를 최소화 할 수 있음
- (②): 각 클래스들의 데이터 구조에서 처리 기능을 분리하여 별도로 구성함으로써, 클래스를 수정하지 않고도 새로운 연산의 추가가 가능함

<보기>

생성 패턴	구조 패턴	행위 패턴
추상 팩토리(Abstract Factory) 프로토타입(Prototype) 싱글톤(Singleton)	어댑터(Adapter) 브리지(Bridge) 프록시(Proxy)	인터프리터(Interpreter) 중재자(Mediator) 옵서버(Observer) 방문자(Visitor)

답: ① **싱글톤(Singleton)** ② **방문자(Visitor)**

[해설]

- ① **싱글톤(Singleton)**: 전역 변수를 사용하지 않고 객체를 하나만 생성하도록 하며, 생성된 객체를 어디에서든지 참조할 수 있도록 하는 디자인 패턴입니다. 시스템 내에서 단 하나의 인스턴스만 보장하므로 불필요한 메모리 낭비를 최소화할 수 있습니다.
- ② **방문자(Visitor)**: 데이터 구조와 처리를 분리하는 패턴입니다. 데이터 구조를 변경하지 않으면서 새로운 연산(기능)을 추가할 수 있도록 해주므로, 클래스를 수정하지 않고도 동적으로 기능을 추가하거나 변경하기 용이합니다.

187. 다음 설명에서 괄호(①~⑤)에 들어갈 알맞은 용어를 <보기>에서 찾아 쓰시오. 23-2

전송 오류의 발생에는 감쇠, 지연 왜곡, 잡음 등 다양한 원인이 있으며, 이러한 오류를 검출하고 수정하는 것으로 알려진 대표적인 방식이 (①) 코드 방식이다.

(①) 코드 방식은 하나의 데이터 단위에 (④) 비트를 추가하여 오류를 검출하여 교정이 가능한 코드로, 2bit의 오류를 검출할 수 있으며 1bit의 오류를 교정한다. 데이터 비트 외에 잉여 비트가 많이 필요하다는 단점이 있다.

(①) 코드 방식은 수신측에서 오류를 정정하는 (②)에 해당한다. (②)는 데이터 전송 과정에서 오류가 발생하면 송신측에 재전송을 요구하는 (③)와는 달리 재전송 요구 없이 스스로 수정하기 때문에 연속적인 데이터 전송이 가능하다.

(③)는 (④) 검사, (⑤) 등을 통해 오류를 검출하고 ARQ(Automatic Repeat reQuest)로 오류를 제어한다.

(④) 검사는 오류 검사를 위해 데이터 비트 외에 1bit의 체크 비트를 추가하는 것으로 1bit의 오류만 검출할 수 있다. 1의 개수에 따라 짝수(④)와 홀수(④)로 나뉜.

(⑤)는 다항식 코드를 사용하여 오류를 검출하는 방식이다. 동기식 전송에서 주로 사용되며, HDLC 프레임의 FCS(프레임 검사 순서 필드)에 사용되는 방식이다. 집단 오류를 검출할 수 있고, 검출률이 높으므로 가장 많이 사용한다.

<보기>

- NAK • CRC • FEC • BCD
- Parity • Hamming • MD5 • BEC

답: ① Hamming ② FEC ③ BEC ④ Parity ⑤ CRC

[해설]

① Hamming / ④ Parity

해밍(Hamming) 코드는 패리티(Parity) 비트를 추가하여 1비트의 오류를 스스로 정정할 수 있는 전송 오류 제어 방식입니다.

② FEC (전진 오류 정정) / ③ BEC (후진 오류 정정)

수신측에서 스스로 오류를 정정하는 방식을 **FEC**(Forward Error Correction)라고 하며, 오류 검출 후 송신측에 재전송(ARQ)을 요구하는 방식을 **BEC**(Backward Error Correction)라고 합니다.

⑤ CRC (순환 중복 검사)

다항식 코드를 사용하여 집단 오류를 검출하는 방식으로, HDLC 프레임의 FCS 필드 등에 널리 쓰이는 신뢰성이 높은 오류 검출 방식입니다.

188. HDLC(High-level Data Link Control)에 대한 다음 설명에서 괄호(①~⑤)에 들어갈 알맞은 용어를 <보기>에서 찾아 쓰시오. 26-1, 23-2

HDLC는 비트(Bit) 위주의 프로토콜로, 각 프레임에 데이터 흐름을 제어하고 오류를 검출할 수 있는 비트 열을 삽입하여 전송한다. 포인트 투 포인트(Point-to-Point) 및 멀티 포인트(Multi-Point), 루프(Loop) 등 다양한 데이터 링크 형태에 동일하게 적용이 가능하다는 특징이 있다.

HDLC의 프레임 구조는 헤더, 텍스트, 트레일러로 구분되며, 헤더는 다시 플래그, 주소부, 제어부로 구분할 수 있는데, 제어부에는 프레임의 종류를 식별하기 위해 사용한다. 제어부의 첫 번째, 두 번째 비트를 사용하여 (①) 프레임, (②) 프레임, (③) 프레임으로 구분한다.

(①) 프레임은 I 프레임으로 불리며, 제어부가 '0'으로 시작하는 프레임으로, 사용자 데이터를 전달하거나 피기백킹(Piggybacking) 기법을 통해 데이터에 대한 확인 응답을 보낼 때 사용된다.

(②) 프레임은 S 프레임으로 불리며, 제어부가 '10'으로 시작하는 프레임으로, 오류 제어와 흐름 제어를 위해 사용된다.

(③) 프레임은 U 프레임으로 불리며, 제어부가 '11'로 시작하는 프레임으로, 링크의 동작 모드 설정과 관리를 한다.

(③) 프레임에서 설정할 수 있는 동작 모드는 표준 응답 모드, (④), (⑤)의 세 가지로 구분된다.

표준 응답 모드는 반이중 통신을 하는 포인트 투 포인트(Point-to-Point) 또는 멀티 포인트(Multi-Point) 불균형 링크 구성에 사용되며, 종국은 주국의 허가(Poll)가 있을 때에만 송신하는 특징이 있다.

(④)는 포인트 투 포인트(Point-to-Point) 균형 링크에서 사용되며, 혼합국끼리 허가 없이 언제나 전송할 수 있다.

(⑤)는 전이중 통신을 하는 포인트 투 포인트(Point-to-Point) 불균형 링크 구성에 사용되며, 종국은 주국의 허가(Poll) 없이도 송신이 가능하지만 링크 설정이나 오류 복구 등의 제어 기능은 주국만 가능하다.

<보기>

- | | | | |
|-------------|-------|-------|-------|
| • 비동기 응답 모드 | • 주소부 | • 제어부 | • ARQ |
| • 정보 | • 비번호 | • 감독 | • 플래그 |
| • 비동기 균형 모드 | | | |

답: ① 정보 ② 감독 ③ 비번호 ④ 비동기 균형 모드 ⑤ 비동기 응답 모드

[해설]

HDLC 프레임 종류

- **정보**(Information) 프레임: 제어부가 '0'으로 시작하며, 사용자 데이터 및 피기백킹 확인 응답 전송용입니다.
- **감독**(Supervisory) 프레임: 제어부가 '10'으로 시작하며, 흐름 제어 및 오류 제어(ACK, NAK 등)를 위해 사용됩니다.
- **비번호**(Unnumbered) 프레임: 제어부가 '11'로 시작하며, 링크의 동작 모드 설정과 관리를 수행합니다.

HDLC 데이터 링크 동작 모드 (④, ⑤)

- **NRM** (표준 응답 모드): 주국의 허가(Poll)가 있을 때만 종국이 송신 가능 (불균형 링크)
- **ABM** (비동기 균형 모드): 주국/종국 구분 없이 혼합국끼리 허가 없이 언제든지 송신 가능 (균형 링크)
- **ARM** (비동기 응답 모드): 종국이 주국의 허가 없이도 송신이 가능하나, 링크 제어 권한은 주국에게만 있음 (불균형 링크)

189. 다음 <보기>에 나열된 암호화 알고리즘을 대칭키 암호화 알고리즘과 비대칭키 암호화 알고리즘으로 구분하시오. [23-2](#)

<보기>

• RSA • DES • ARIA • ECC • SEED • AES

답: 대칭키 암호화 알고리즘: **DES, ARIA, SEED, AES**
비대칭키 암호화 알고리즘: **RSA, ECC**

[해설]

- **대칭키(Symmetric Key) 암호화 알고리즘**: 암호화할 때 사용하는 키와 복호화할 때 사용하는 키가 **동일한** 방식입니다. 계산 속도가 빠르지만 키 분배 및 관리에 어려움이 있습니다.

- **DES**: IBM사에서 개발한 블록 암호 표준(56비트 키 사용)
- **AES**: DES의 취약점을 보완하기 위해 NIST에서 채택한 고급 블록 암호 표준
- **SEED / ARIA**: 대한민국 한국인터넷진흥원(KISA) 및 국가정보원에서 개발한 국산 블록 암호 표준

- **비대칭키(Asymmetric Key / 공개키) 암호화 알고리즘**: 암호화에는 **공개키(Public Key)**, 복호화에는 **개인키(Private Key)**를 사용하는 타인 간 상이한 키 쌍 방식입니다.

- **RSA**: 소인수분해의 수학적 난해함에 기반을 둔 가장 대표적인 공개키 알고리즘
- **ECC**: 타원곡선 이산대수 문제에 기반하여 RSA보다 짧은 키 길이로도 대등한 보안 강도를 제공하는 알고리즘

190. 암호화 알고리즘에 대한 다음 설명에서 괄호에 들어갈 알맞은 용어를 쓰시오. [23-2](#)

()는 임의의 길이의 입력 데이터나 메시지를 고정된 길이의 값이나 키로 변환하는 알고리즘으로, 복호화가 거의 불가능한 일방향 함수이다. 무결성 검증을 위해 사용될 뿐만 아니라 정보보호의 다양한 분야에서 활용되며, 종류에는 SHA 시리즈, MD5, N-NASH, SNEFRU 등이 있다.

답: **해시 (Hash)**

[해설]

일방향 해시 함수 (One-Way Hash Function)

- 입력 값의 길이에 상관없이 언제나 **고정된 고유의 길이(정해진 비트 수)**를 가진 결과물(해시시각/다이제스트)을 출력합니다.
- **일방향성(One-wayness)**: 결과값을 가지고 원래의 입력 데이터나 평문을 역산하여 유추하는 **복호화가 원천적으로 불가능**한 특성을 지닙니다.
- **눈사태 효과(Avalanche Effect)**: 입력 데이터가 단 1비트만 달라져도 완전히 다른 예측 불가능한 해시값이 출력되므로 데이터의 **무결성(변조 여부) 검증**에 강력하게 활용됩니다.
- **알고리즘 종류**: 지문에 제시된 **MD5, SHA 시리즈**(SHA-1, SHA-256/384/512 등), 국산 알고리즘인 HAS-160 등이 있습니다.

191. 애플리케이션 테스트에 관한 다음 설명에서 괄호(①, ②)에 들어갈 알맞은 용어를 쓰시오. 26-1, 23-2

하향식 통합 테스트는 프로그램의 상위 모듈에서 하위 모듈 방향으로 통합하면서 테스트하는 기법이다. 깊이가 우선 통합법이나 넓이 우선 통합법을 사용하며, 주요 제어 모듈의 종속 모듈들을 (①)으로 대체한다는 특징이 있다.

상향식 통합 테스트는 프로그램의 하위 모듈에서 상위 모듈 방향으로 통합하면서 테스트하는 기법이다. 하위 모듈들을 클러스터(Cluster)로 결합하며, 상위 모듈에서 데이터의 입·출력을 확인하기 위해 더미 모듈인 (②)을 작성한다는 특징이 있다.

답: ① **스텝 (Stub)** ② **드라이버 (Driver)**

[해설]

통합 테스트(Integration Test) 기법: 하향식 vs 상향식

- **하향식 통합 테스트 (Top-Down)**: 구조도 상위 메인 제어 모듈에서 시작해 하향 구조로 테스트해 나갑니다. 아직 개발되지 않은 하위 모듈들의 빈자리를 채우기 위해 **임시 가짜 모듈인 스텝(Stub)**을 배치하며, 깊이가 우선(Depth-First) 또는 넓이 우선(Breadth-First) 방식으로 점진적 변환 결합을 수행합니다.
- **상향식 통합 테스트 (Bottom-Up)**: 최하위 컴포넌트나 모듈 단위들을 먼저 결합(클러스터 구성)하며 상위 방향으로 전개합니다. 하위 모듈들의 제어와 데이터 입출력을 테스트해 줄 상위 더미 컨트롤러가 필요하므로 **테스트 드라이버(Driver)**를 임시로 작성하여 조작합니다.

192. 다음 설명에 해당하는 알맞은 용어를 쓰시오. 23-1

- 데이터링크 계층의 프로토콜 중 하나로, 터널링 프로토콜인 PPTP와 VPN의 구현에 사용하는 L2F의 기술적 장점들을 결합하여 만든 프로토콜이다.
- 자체적으로 암호화 및 인증 기능을 제공하지 않아 다른 보안 프로토콜과 함께 사용되는 경우가 많다.

답: **L2TP (Layer 2 Tunneling Protocol)**

[해설]

L2TP (Layer 2 Tunneling Protocol) 핵심 요약

- **개념**: Microsoft 사의 PPTP(Point-to-Point Tunneling Protocol)와 Cisco 사의 L2F(Layer 2 Forwarding) 기술의 장점을 결합하여 IETF에서 표준화한 데이터 링크 계층(2계층) 터널링 프로토콜입니다.
- **보안 특징**: L2TP 자체적으로는 패킷의 기밀성(암호화)이나 데이터 인증을 제공하지 않기 때문에, 실제 보안이 중요한 VPN 환경에서는 네트워크 계층의 보안 프로토콜인 **IPsec(IP Security)**과 결합하여 '**L2TP/IPsec**' 형태로 주로 사용됩니다.

193. 다음 설명에 해당하는 알맞은 용어를 영문 3글자로 쓰시오. 23-1

- 다른 컴퓨터에 로그인, 원격 명령 실행, 파일 복사 등을 수행할 수 있도록 다양한 기능을 지원하는 프로토콜 또는 이를 이용한 응용 프로그램이다.
- 데이터 암호화와 강력한 인증 방법으로 보안성이 낮은 네트워크에서도 안전하게 통신할 수 있다.
- 키(key)를 통한 인증 방법을 사용하려면 사전에 클라이언트의 공개키를 서버에 등록해야 한다.
- 기본적으로는 22번 포트를 사용한다.

답: **SSH (Secure Shell)**

[해설]

SSH (Secure Shell) 핵심 요약

- **개념:** 기존의 암호화되지 않은 전송 방식(Telnet, Rlogin 등)의 보안 취약점을 해결하기 위해 개발된 **원격 제어 및 파일 전송용 암호화 네트워크 프로토콜**입니다.
- **보안 특징:** 패킷을 완전히 암호화하여 스니핑(Sniffing) 등 대청취 공격을 방어합니다. 공개키 기반 인증(비대칭키)과 세션 데이터 암호화(대칭키)를 혼합하여 사용하며, 클라이언트의 **공개키(Public Key)**를 서버의 **특정 경로 (~/.ssh/authorized_keys)에 미리 등록해 두고 인증을 수행**합니다.
- **포트:** 전송 계층 프로토콜로 TCP를 사용하며, 기본 지정 포트는 **22번**입니다.

194. 멀웨어(Malware)에 대한 다음 설명에서 괄호(①~③)에 들어갈 알맞은 용어를 <보기>에서 찾아 쓰시오. 23-1

- (①): 윈도우나 응용 프로그램의 취약점 또는 E-mail 등을 통해 전파되며, (③)과(와) 같이 자기복제가 가능하며 네트워크를 통해 스스로 전파가 가능하다.
- (②): 정상적인 응용 프로그램에 포함되어 실행되는 악성코드로, 정상적인 응용 프로그램으로 위장하고 있다가 활성화되면 공격자는 이를 이용하여 사용자의 컴퓨터를 조종할 수 있게 된다.
- (③): 정상 파일을 감염시키며, 자기복제가 가능하다. 파일을 통해 감염되며 네트워크를 통해 스스로 전파되지는 못한다.

<보기> •웜 •바이러스 •트로이 목마

답: ① **웜** ② **트로이 목마** ③ **바이러스**

[해설]

주요 악성코드(Malware) 유형별 핵심 특징 비교

- ① **웜 (Worm):** 독자적으로 실행 가능한 악성 프로그램입니다. 다른 파일을 감염시키지 않는 대신, 네트워크 취약점 등을 이용해 **스스로 감염 및 증식을 반복하며 대량의 트래픽을 유발**하는 것이 핵심 특징입니다.
- ② **트로이 목마 (Trojan Horse):** 유용한 프로그램인 것처럼 **정상적인 소프트웨어로 위장**하여 사용자의 설치를 유도합니다. 웜이나 바이러스와 달리 자기 복제 능력이 없으나, 내부에 백도어 등을 포함하여 원격 제어나 정보 유출을 일삼습니다.
- ③ **바이러스 (Virus):** 반드시 **정상적인 파일이나 프로그램(숙주)에 기생**하여 전파되는 악성코드입니다. 숙주 파일이 실행될 때 함께 활성화되어 스스로를 복제하고 파일을 파괴하지만, 네트워크를 통해 스스로 전파되는 능력은 없습니다.

195. 스키마에 대한 다음 설명에서 괄호(①~③)에 들어갈 알맞은 답을 <보기>에서 찾아 쓰시오. 23-1

- (①) 스키마는 데이터베이스의 전체적인 논리적 구조로, 모든 응용 프로그램이나 사용자들이 필요로 하는 데이터를 종합한 조직 전체의 데이터베이스이다.

- (②) 스키마는 실제로 저장될 레코드의 형식, 저장 데이터 항목의 표현 방법, 내부 레코드의 물리적 순서 등을 나타낸다.
- (③) 스키마는 사용자나 응용 프로그래머가 각 개인의 입장에서 필요로 하는 데이터베이스의 논리적 구조를 정의한 것이다.

<보기> • 외부 • 내부 • 개념

답: ① 개념 ② 내부 ③ 외부

[해설]

데이터베이스 스키마 3단계 구조(3-Level Schema Architecture) 핵심 요약

- ① 개념 스키마 (Conceptual Schema): 데이터베이스의 전체적인 논리적 구조를 정의한 것으로, 개성·관계·제약조건을 포함하며 조직 전체의 관점에서 하나만 존재합니다. 접근 권한, 보안 정책 및 무결성 규칙에 대한 명세도 이 단계에서 정의됩니다.
- ② 내부 스키마 (Internal Schema): 데이터베이스의 물리적 저장 구조를 정의한 것입니다. 데이터가 물리적 디스크 장치에 실제로 저장되는 방식, 인덱스 구조, 필드 크기, 데이터 압축 및 암호화 기법 등을 다룹니다.
- ③ 외부 스키마 (External Schema): 개별 사용자나 응용 프로그래머의 관점에서 바라보는 논리적 구조로, 서브 스키마 (Sub Schema)라고도 부릅니다. 동일한 개념 스키마를 기반으로 하더라도 접근 목적에 따라 서로 다른 형태의 여러 외부 스키마가 존재할 수 있습니다.

196. 애플리케이션 테스트에 대한 다음 설명에서 괄호(①, ②)에 공통으로 들어갈 알맞은 답을 쓰시오. 21-1

- (①)은 입력 조건의 중간값보다 경계값에서 오류가 발생할 확률이 높다는 점을 이용한 검사 기법이고, (②)는 입력 조건이 유효한 경우와 그렇지 않은 경우의 입력 자료의 개수를 균등하게 정하는 검사 기법이다.
- 예를 들어 $0 \leq x \leq 10$ 과 같은 조건이 있을 때, (①)은 -1, 0, 10, 11을 입력값으로, (②)는 0이상 10 이하의 수 n개와 0미만 10초과의 수 n개를 입력값으로 정한다.

답: ① 경계값 분석 (Boundary Value Analysis)

② 동치 분할 검사, 동치 클래스 분해 (Equivalence Partitioning Testing) (각 문항별로 하나만 쓰면됨)

[해설]

블랙박스 테스트(Black-box Test)의 주요 명세 기반 기법

- ① 경계값 분석 (Boundary Value Analysis): 입력 조건의 유효 범위 경계 주변(최솟값, 경계 바로 직전/직후, 최댓값 등)에서 개발자의 논리적 오류가 많이 발생한다는 점에 착안하여 테스트 케이스를 설계하는 기법입니다. 주어진 예시에서 0 과 10 이 경계이므로, 바깥쪽 한 칸과 안쪽 경계인 -1, 0, 10, 11을 선택하여 검증합니다.
- ② 동치 분할 / 동등 분할 (Equivalence Partitioning): 프로그램의 입력 데이터 영역을 유효값(Valid) 그룹과 무효값(Invalid) 그룹으로 균등하게 분할(클래스화)하고, 각 영역을 대표하는 대표값을 선정하여 테스트 케이스를 도출하는 기법입니다. 예시처럼 유효 구간(0~10)에서 n개, 무효 구간(0 미만, 10 초과)에서 각각 대표값을 균등하게 뽑아 테스트 신뢰성을 확보합니다.

197. 데이터 모델의 구성 요소에 대한 다음 설명에서 괄호(①~③)에 들어갈 알맞은 답을 <보기>에서 찾아 기호로 쓰시오. 21-1  클릭!

- (①)는 모듈 간의 인터페이스로 데이터, 지역 변수 등을 직접 참조하거나 수정할 때의 결합도이다.
- (②)는 모듈 간의 인터페이스로 배열이나 레코드 등의 자료 구조가 전달될 때의 결합도이다.

- (③)는 모듈 간의 인터페이스로 파라미터가 아닌 모듈 밖에 선언된 전역 변수를 사용하여 전역 변수를 갱신하는 방식으로 상호작용하는 때의 결합도이다.

<보기>

- | | | |
|----------|-----------|----------|
| ㉠ 자료 결합도 | ㉡ 스템프 결합도 | ㉢ 제어 결합도 |
| ㉣ 공통 결합도 | ㉤ 내용 결합도 | ㉥ 외부 결합도 |

답: ① ㉤ ② ㉡ ③ ㉣

198. 소프트웨어 공학의 디자인 패턴(Design Pattern)에 대한 다음 설명에서 괄호에 들어갈 알맞은 답을 쓰시오. 20-4

- 디자인 패턴은 1995년 GoF(Gang of Four)라고 불리는 에릭 감마(Erich Gamma), 리차드 헬름(Richard Helm), 랄프 존슨(Ralph Johnson), 존 블리시디스(John Vissides)가 처음으로 구체화 및 체계화하였다.
- 디자인 패턴은 수많은 디자인 패턴들 중 가장 일반적인 사례에 적용될 수 있는 패턴들을 분류하여 정리함으로써, 지금까지도 소프트웨어 공학이나 현업에서 가장 많이 사용되는 디자인 패턴이다.
- 디자인 패턴은 총 23가지이며, 생성, 구조, ()의 3가지로 분류한다.

답: 행위 (Behavioral)

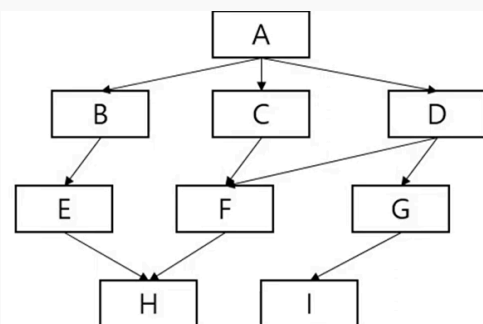
[해설]

GoF(Gang of Four) 디자인 패턴의 3대 분류

에릭 감마를 비롯한 4명의 전문가(GoF)가 정리한 23개의 디자인 패턴은 그 목적에 따라 다음과 같이 3가지 카테고리로 분류됩니다.

- 생성 패턴 (Creational Patterns) - 5개: 객체의 생성 과정에 관여하는 패턴으로, 객체의 생성과 참조 과정을 캡슐화하여 시스템의 유연성을 높입니다. (예: Singleton, Factory Method, Abstract Factory 등)
- 구조 패턴 (Structural Patterns) - 7개: 클래스나 객체들을 조합하여 더 큰 구조를 만드는 패턴입니다. 서로 다른 인터페이스를 가진 객체들을 묶어 하나의 큰 구조를 형성합니다. (예: Adapter, Decorator, Facade 등)
- 행위 패턴 (Behavioral Patterns) - 11개: 클래스나 객체들이 서로 상호작용하는 방법과 책임을 분산하는 방법을 정의하는 패턴입니다. 하나의 객체로 수행할 수 없는 작업을 여러 객체로 분배하면서 결합도를 낮춥니다. (예: Observer, Strategy, Command 등)

199. 다음의 모듈 관계를 표현한 시스템 구조도를 참고하여 팬인(Fan-In)이 2 이상인 모듈을 모두 쓰시오. 20-1 001로



답: F, H

SQL 27문제

[SQL 명령문의 기본 형식과 조건] 001로

1. 다음과 같이 테이블을 정의하고 튜플을 삽입하였을 때 각 번호(①, ②)의 SQL문을 실행한 결과를 쓰시오.
22-3

```
CREATE TABLE 부서 (
    부서코드 INT PRIMARY KEY,
    부서명 VARCHAR(20)
);

CREATE TABLE 직원 (
    직원코드 INT PRIMARY KEY,
    부서코드 INT,
    직원명 VARCHAR(20),
    FOREIGN KEY(부서코드) REFERENCES 부서(부서코드)
    ON DELETE CASCADE
);

INSERT INTO 부서 VALUES(10, '영업부');
INSERT INTO 부서 VALUES(20, '기획부');
INSERT INTO 부서 VALUES(30, '개발부');

INSERT INTO 직원 VALUES(1001, 10, '이진수');
INSERT INTO 직원 VALUES(1002, 10, '곽연경');
INSERT INTO 직원 VALUES(1003, 20, '김선길');
INSERT INTO 직원 VALUES(1004, 20, '최민수');
INSERT INTO 직원 VALUES(1005, 20, '이용갑');
INSERT INTO 직원 VALUES(1006, 30, '박종일');
INSERT INTO 직원 VALUES(1007, 30, '박미경');
```

① SELECT DISTINCT COUNT(부서코드) FROM 직원 WHERE 부서코드 = 20;

② DELETE FROM 부서 WHERE 부서코드 = 20;
SELECT DISTINCT COUNT(부서코드) FROM 직원;

답: ① 3 ② 4

[해설]

```
CREATE TABLE 부서 (
    부서코드 INT PRIMARY KEY,
    부서명 VARCHAR(20)
);

CREATE TABLE 직원 (
    직원코드 INT PRIMARY KEY,
    부서코드 INT,
    직원명 VARCHAR(20),
    FOREIGN KEY(부서코드) REFERENCES 부서
(부서코드)

    ON DELETE CASCADE
);
```

<부서> 테이블을 생성한다.

'부서코드' 속성은 정수형이며, 기본키로 정의한다.

'부서명' 속성은 가변길이 문자 20자이다.

<직원> 테이블을 생성한다.

'직원코드' 속성은 정수형이며, 기본키로 정의한다.

'부서코드' 속성은 정수형이다.

'직원명' 속성은 가변길이 문자 20자이다.

'부서코드' 속성은 <부서> 테이블의 '부서코드' 속성을 참조하는 외래키이다.

<부서> 테이블에서 튜플이 삭제되면 관련된 모든 튜플이 함께 삭제된다.

① INSERT INTO 부서 VALUES(10, '영업부');

② INSERT INTO 부서 VALUES(20, '기획부');

- ③ INSERT INTO 부서 VALUES (30, '개발부');
- ④ INSERT INTO 직원 VALUES (1001, 10, '이진수');
- ⑤ INSERT INTO 직원 VALUES (1002, 10, '곽연경');
- ⑥ INSERT INTO 직원 VALUES (1003, 20, '김선길');
- ⑦ INSERT INTO 직원 VALUES (1004, 20, '최민수');
- ⑧ INSERT INTO 직원 VALUES (1005, 20, '이용갑');
- ⑨ INSERT INTO 직원 VALUES (1006, 30, '박종일');
- ⑩ INSERT INTO 직원 VALUES (1007, 30, '박미경');

- ①~③번 SQL문이 수행된 후 <부서> 테이블은 다음과 같습니다.

부서코드	부서명
10	영업부
20	기획부
30	개발부

- ④~⑩번 SQL문이 수행된 후 <직원> 테이블은 다음과 같습니다.

직원코드	부서코드	직원명
1001	10	이진수
1002	10	곽연경
1003	20	김선길
1004	20	최민수
1005	20	이용갑
1006	30	박종일
1007	30	박미경

① SQL문

SELECT DISTINCT COUNT(부서코드) FROM 직원 WHERE 부서코드 = 20;	'부서코드'의 개수를 표시하되, 표시된 개수 중 중복된 값은 한 번만 표시한다. <직원> 테이블에서 검색한다. '부서코드'가 20인 튜플만을 대상으로 한다.
--	---

- 문제의 SQL문은 DISTINCT가 '부서코드'에 적용되는 것이 아니라 'COUNT(부서코드)'에 적용됨에 유의해야 합니다.
- **WHERE 부서코드 = 20** : '부서코드'가 20인 자료만을 검색합니다.

부서코드
20
20
20

- **SELECT DISTINCT COUNT(부서코드)** : 'COUNT(부서코드)'의 결과인 3에는 중복된 값이 없으므로 3이 그대로 표시됩니다.

COUNT(부서코드)
3

② SQL문

```
DELETE FROM 부서
WHERE 부서코드 = 20;
SELECT DISTINCT COUNT(부서코드)
FROM 직원;
```

<부서> 테이블에서 튜플을 삭제한다.
'부서코드'가 20인 튜플만을 대상으로 한다.
'부서코드'의 개수를 표시하되, 표시된 개수 중 중복된 값은 한 번만 표시한다.
<직원> 테이블에서 검색한다.

- **DELETE FROM 부서 WHERE 부서코드 = 20;** : <직원> 테이블의 '부서코드'는 <부서> 테이블의 '부서코드'를 참조하고, <부서> 테이블의 '부서코드'가 삭제되면 이를 참조하는 <직원> 테이블의 모든 튜플도 같이 삭제되도록 정의되었으므로, DELETE문 수행 후의 <부서>와 <직원> 테이블은 다음과 같습니다.

<부서>

부서코드	부서명
10	영업부
30	개발부

<직원>

직원코드	부서코드	직원명
1001	10	이진수
1002	10	곽민경
1006	30	박종일
1007	30	박미경

- **SELECT DISTINCT COUNT(부서코드) FROM 직원;** : <직원> 테이블에 대한 'COUNT(부서코드)'의 결과인 4에는 중복된 값이 없으므로 4가 그대로 표시됩니다.

COUNT(부서코드)
4

2. 학생(STUDENT) 테이블에 전기과 학생이 50명, 전산과 학생이 100명, 전자과 학생이 50명 있다고 할 때, 다음 SQL문 ①, ②, ③의 실행 결과로 표시되는 튜플의 수를 쓰시오. (단, DEPT 필드는 학과를 의미한다.)
[20-1, 22-3, 26-1](#)

- ① **SELECT DEPT FROM STUDENT;**
- ② **SELECT DISTINCT DEPT FROM STUDENT;**
- ③ **SELECT COUNT(DISTINCT DEPT) FROM STUDENT WHERE DEPT = '전산과';**

답: ① **200** ② **3** ③ **1**

[해설]

SELECT DEPT 'DEPT'를 표시한다.
FROM STUDENT; <STUDENT> 테이블을 대상으로 검색한다.

- <STUDENT> 테이블에서 'DEPT'를 검색합니다.
- 총 200개의 튜플이 들어 있고 검색 조건이 없으므로 튜플의 수는 **200**입니다.

SELECT DISTINCT DEPT 'DEPT'를 표시하되, 같은 'DEPT' 속성의 값은 한 번만 표시한다.
FROM STUDENT; <STUDENT> 테이블을 대상으로 검색한다.

- <STUDENT> 테이블에서 'DEPT'를 검색하는 데 중복된 결과는 처음의 한 개만 검색에 포함시킵니다.
- 전기과 50개 튜플의 'DEPT' 속성의 값이 같으므로 1개, 전산과 100개 튜플의 'DEPT' 속성의 값이 같으므로 1개, 전자과 50개 튜플의 'DEPT' 속성의 값이 같으므로 1개를 검색에 포함시키므로 **3**개의 튜플이 검색된다.

SELECT COUNT(DISTINCT DEPT) 'DEPT'의 개수를 표시하되, 같은 'DEPT' 속성의 값은 한 번만 계산한다.
FROM STUDENT <STUDENT> 테이블을 대상으로 검색한다.
WHERE DEPT = '전산과'; 'DEPT'가 "전산과"인 자료만을 대상으로 검색한다.

- <STUDENT> 테이블에서 'DEPT' 속성의 값이 '전산과'인 튜플에 대해 중복을 제거하고 개수는 1이다.

3. 다음은 <제품>(제품명, 단가, 제조사) 테이블을 대상으로 "H" 제조사에서 생산한 제품들의 '단가'보다 높은 '단가'를 가진 제품의 정보를 조회하는 <SQL문>이다. 괄호에 알맞은 답을 적어 <SQL문>을 완성하시오. [22-2](#)

```
SELECT 제품명, 단가, 제조사
FROM 제품
WHERE 단가 > (      ) (SELECT 단가 FROM 제품 WHERE 제조사 = 'H');
```

답: **ALL**

[해설]

② SELECT 제품명, 단가, 제조사
FROM 제품
WHERE 단가 > **ALL** (

① SELECT 단가
FROM 제품
WHERE 제조사 = 'H'
);

'제품명', '단가', '제조사' 속성을 표시한다.
<제품> 테이블에서 검색한다.
'단가'가 하위 질의로 검색된 모든(ALL) 단가보다 큰 자료만을 대상으로 한다.
'단가'를 표시한다.
<제품> 테이블에서 검색한다.
제조사가 "H"인 자료만을 대상으로 한다.

- 문제의 질의문은 하위 질의(Subquery)가 있는 질의문입니다.
- 먼저 WHERE 조건에 지정된 하위 질의의 SELECT문을 해석한 다음 그 결과를 본 질의의 조건에 있는 '단가' 속성과 비교합니다.
- <제품> 테이블에 다음과 같은 자료가 들어있다고 가정하여 설명합니다.

<제품>

제품명	단가	제조사
냉장고	200	H
TV	150	H
세탁기	300	H
건조기	250	A
핸드폰	400	B
컴퓨터	500	C

① <제품> 테이블에서 '제조사' 속성의 값이 "H"인 튜플의 '단가' 속성의 값을 검색합니다.

단가
200
150
300

② <제품> 테이블에서 '단가' 속성의 값이 ①번에서 검색된 모든(ALL) 단가(150, 200, 300)보다 큰 자료(즉, 최대값인 300보다 큰 자료)를 대상으로 '제품명', '단가', '제조사'를 표시합니다.

제품명	단가	제조사
핸드폰	400	B

제품명	단가	제조사
컴퓨터	500	C

4. 다음 <TABLE>을 참조하여 <SQL문>을 실행했을 때 출력되는 결과를 쓰시오. <TABLE>에 표시된 'NULL'은 값이 없음을 의미한다.) [22-2](#)

<TABLE>

INDEX	COL1	COL2
1	2	NULL
2	4	6
3	3	5
4	6	3
5	NULL	3

<SQL문>

```
SELECT COUNT(COL2)
FROM TABLE
WHERE COL1 IN (2, 3)
OR COL2 IN (3, 5);
```

답: 3

[해설]

```
SELECT COUNT(COL2)
FROM TABLE
WHERE COL1 IN (2, 3)
OR COL2 IN (3, 5);
```

'COL2'의 개수를 표시한다.
<TABLE>에서 검색한다.
'COL1'이 2 또는 3이거나,
'COL2'가 3 또는 5인 튜플만을 대상으로 한다.

• 질의문의 조건을 만족하는 튜플은 다음과 같습니다.

INDEX	COL1	COL2
1	2	NULL
2	4	6
3	3	5
4	6	3
5	NULL	3

• 조건에 맞는 'COL2' 속성만 추출하면 다음과 같습니다.

COL2
NULL
5
3
3

• COUNT(COL2)는 'COL2' 필드의 개수를 계산하지만 NULL 값은 제외하므로 3입니다.

5. 다음은 <EMPLOYEE> 릴레이션에 대해 <관계 대수식>을 수행했을 때 출력되는 <결과>이다. <결과>의 각 괄호(①~⑤)에 들어갈 알맞은 답을 쓰시오. [22-2, 25-2](#)

<관계 대수식>

$\pi_{TTL}(\text{EMPLOYEE})$

<EMPLOYEE>

INDEX	AGE	TTL
1	48	부장
2	25	대리
3	41	과장
4	36	차장

 $\Rightarrow$

<결과>

(①)
(②)
(③)
(④)
(⑤)

답: ① TTL
 ② 부장
 ③ 대리
 ④ 과장
 ⑤ 차장

[해설]

문제의 <관계 대수식>에서 사용된 π 는 주어진 릴레이션에서 속성 리스트(Attribute List)에 제시된 속성 값만을 추출하여 새로운 릴레이션을 만드는 **PROJECT** 연산이므로, <EMPLOYEE> 릴레이션에서 'TTL' 속성이 추출되어 속성명인 'TTL'부터 모든 속성 값이 <결과>로 나타납니다.

Select는 릴레이션에 존재하는 튜플 중에서 선택 조건을 만족하는 튜플의 부분집합을 구하여 새로운 릴레이션을 만드는 연산.

- 연산자의 기호는 그리스 문자 시그마(σ)를 사용.

 $\sigma_{Avg \geq 90}$ (성적)

<성적>

Name	Kor	Eng	Mat	Tot	Avg
고회식	100	90	100	290	<u>96.6</u>
김은소	80	80	100	260	86.6
최미경	100	70	80	250	83.3
김준용	90	100	90	280	<u>93.3</u>
윤정희	85	95	90	270	<u>90.0</u>

Name	Kor	Eng	Mat	Tot	Avg
고회식	100	90	100	290	96.6
김준용	90	100	90	280	93.3
윤정희	85	95	90	270	90.0

Project는 주어진 릴레이션에서 속성 리스트(Attribute List)에 제시된 속성 값만을 추출하여 새로운 릴레이션을 만드는 연산. (결과에 중복이 발생하면 중복이 제거됨) 연산자의 기호는 그리스 문자 파이(π)를 사용.

 $\pi_{Name, Avg}$ (성적)

Name	Avg
고회식	96.6
김은소	86.6
최미경	83.3
김준용	93.3
윤정희	90.0

Join은 공통 속성을 중심으로 두 개의 릴레이션을 하나로 합쳐서 새로운 릴레이션을 만드는 연산.

- 연산자의 기호는 $\bowtie$ 를 사용.

성적 $\bowtie_{No=No}$ 학적부성적 $\bowtie_{No}$ 학적부

<성적>

<학적부>

No	Name	Kor
9801	고회식	100
9802	김은소	80
9803	최미경	100
9804	김준용	90
9805	윤정희	85

No	Addr
9801	망원동
9802	서교동
9803	성산동
9804	합정동
9805	공덕동

성적.No	Name	Kor	학적부.No	Addr
9801	고회식	100	9801	망원동
9802	김은소	80	9802	서교동
9803	최미경	100	9803	성산동
9804	김준용	90	9804	합정동
9805	윤정희	85	9805	공덕동

No	Name	Kor	Addr
9801	고회식	100	망원동
9802	김은소	80	서교동
9803	최미경	100	성산동
9804	김준용	90	합정동
9805	윤정희	85	공덕동

Division은 $X \supset Y$ 인 두 개의 릴레이션 $R(X)$ 와 $S(Y)$ 가 있을 때, R 의 속성이 S 의 속성값을 모두 가진 튜플에서 S 가 가진 속성을 제외한 속성만을 구하는 연산.

- 연산자의 기호는 $\div$ 를 사용.

구입자[구입품코드 $\div$ 생산품코드]생산품1

<구입자>

번호	이름	구입품코드
1	고회식	A
1	고회식	B
2	김준용	A
2	김준용	B
2	김준용	C
4	윤정희	C

<생산품1>

생산품코드	번호	이름
A	1	고회식
B	2	김준용

6. SQL과 관련된 다음 설명에서 괄호(①, ②)에 들어갈 알맞은 답을 쓰시오. 21-2

UPDATE문은 테이블에 있는 튜플의 내용을 갱신할 때 사용하는 명령문으로, DML에 해당한다. 다른 DML로는 INSERT, DELETE가 있으며, 각각 새로운 튜플을 삽입하거나 삭제할 때 사용한다.

<학부생> 테이블

학부	학과번호	입학생수	담당관
정경대학	110	300	김해울
공과대학	310	250	이성관
인문대학	120	400	김해울
정경대학	120	300	김성수
인문대학	420	180	이율해

다음은 <학부생> 테이블에서 '입학생수'가 300 이상인 튜플의 '학과번호'를 999로 갱신하는 SQL문이다.

(①) 학부생 (②) 학과번호 = 999 WHERE 입학생수 >= 300;

답: ① UPDATE ② SET

[해설]

```
UPDATE 학부생
SET 학과번호 = 999
WHERE 입학생수 >= 300;
```

'학부생' 테이블을 갱신하라.
'학과번호'를 999로 갱신하라.
'입학생수'가 300이상인 튜플만을 대상으로 하라.

• SQL 실행 결과

학부	학과번호	입학생수	담당관
정경대학	999	300	김해울
공과대학	310	250	이성관
인문대학	999	400	김해울
정경대학	999	300	김성수
인문대학	420	180	이윤해

7. <EMP_TBL> 테이블을 참고하여 <SQL문>의 실행 결과를 쓰시오. [21-1](#), [24-1](#)

<EMP_TBL>

EMPNO	SAL
100	1500
200	3000
300	2000

<처리 조건>

```
SELECT COUNT(*) FROM EMP_TBL
WHERE EMPNO > 100 AND SAL >= 3000 OR EMPNO = 200;
```

답: 1

[해설]

1. 연산자 우선순위 확인:

SQL의 WHERE 절에서 AND 연산은 OR 연산보다 우선순위가 높습니다.

따라서 조건식은 (EMPNO > 100 AND SAL >= 3000) OR (EMPNO = 200)입니다.

2. 조건 필터링 (첫 번째 파트: AND):

- 100행: EMPNO(100) > 100은 False이므로 제외.

- 200행: EMPNO(200) > 100 (True) AND SAL(3000) >= 3000 (True) → 만족(True).

- 300행: EMPNO(300) > 100 (True) AND SAL(2000) >= 3000 (False) → 제외.

3. 조건 필터링 (두 번째 파트: OR):

- EMPNO = 200인 행을 찾으면 200번 행이 해당합니다.

- 결과적으로 200번 행 하나만 최종 조건에 만족하게 됩니다. (합집합 개념)

4. 결과: COUNT(*) 함수는 조건에 맞는 행의 개수를 구하는 것으로 만족하는 행은 200번 행 1개다.

8. <학생> 테이블에서 '이름'이 "민수"인 튜플을 삭제하고자 한다. 다음 <처리 조건>을 참고하여 SQL문을 작성하시오. [20-3](#), [23-1](#)

<처리 조건>

- 최소한의 코드로 작성될 수 있도록 SQL문을 구성한다.
- 명령문 마지막의 세미콜론(;)은 생략이 가능하다.

- 인용 부호가 필요한 경우 작은따옴표(')를 사용한다.

답: **DELETE FROM 학생 WHERE 이름 = '민수';**

[해설]

```
DELETE
FROM 학생
WHERE 이름 = '민수';
```

삭제하라.
<학생> 테이블을 대상으로 하라.
'이름'이 "민수"인 자료만을 대상으로 한다.

9. 다음 <속성 정의서>를 참고하여 <학생> 테이블에 대해 20자의 가변 길이를 가진 '주소' 속성을 추가하는 <SQL문>을 완성하시오. (단, SQL문은 ISO/IEC 9075 표준을 기반으로 작성하시오.) [20-3](#)

<속성 정의서>

속성명	데이터타입	제약조건	테이블명
학번	CHAR(10)	UNIQUE	학생
이름	VARCHAR(8)	NOT NULL	학생
주민번호	CHAR(13)		학생
학과	VARCHAR(16)	FOREIGN KEY	학생
학년	INT		학생

<SQL문>

(①) TABLE 학생 (②) 주소 VARCHAR (20);

답: ① **ALTER** ② **ADD**

[해설]

```
ALTER TABLE 학생
ADD 주소 VARCHAR (20);
```

수정할 테이블의 이름은 <학생>이다.
가변 길이의 문자 20자리인 '주소' 속성을 추가한다.

10. 다음 <학생> 테이블을 참고하여 <처리 조건>에서 요구하는 SQL문을 작성하시오. [20-2](#)

<학생>

학번 (varchar)	이름 (varchar)	학년 (number)	수강과목 (varchar)	점수 (number)	연락처 (varchar)
20E0232	김인영	3	세무행정	4.5	010-5412-4544
19D0024	이성화	2	토목개론	3	010-1548-4796
20E0135	성유수	4	실용법학	3.5	010-9945-7411
20E0511	우인혁	1	데이터론	2	010-3451-4972

<처리 조건>

- 3, 4학년의 학번, 이름을 조회한다.
- IN 예약어를 사용해야 한다.
- 속성명 아래의 괄호는 속성의 자료형을 의미한다.

답: **SELECT 학번, 이름 FROM 학생 WHERE 학년 IN (3, 4);**

[해설]

```
SELECT 학번, 이름
FROM 학생
```

'학번', '이름'을 표시한다.
<학생> 테이블에서 검색한다.

WHERE 학년 IN (3, 4);

'학년'의 값이 3 또는 4인 자료만을 대상으로 한다.

<결과>

학번	이름
20E0232	김인영
20E0135	성유수

11. 다음 <student> 테이블을 참고하여 'name' 속성으로 'idx_name'이라는 인덱스를 생성하는 SQL문을 작성하시오. [20-2](#)

<student>

stid	name	score	deptid
2001	brown	85	PE01
2002	white	45	EF03
2003	black	67	UW11

답: **CREATE INDEX idx_name ON student(name);**

[해설]

```
CREATE INDEX idx_name  
ON student(name);
```

'idx_name'이라는 이름의 인덱스를 생성한다.
<student> 테이블의 'name' 속성을 사용한다.

[정렬]

12. 다음은 <성적> 테이블에서 이름(name)과 점수(score)를 조회하되, 점수를 기준으로 내림차순 정렬하여 조회하는 <SQL문>이다. 괄호(①~③)에 알맞은 답을 적어 <SQL문>을 완성하시오. [22-1](#)

<성적>

name	class	score
정기찬	A	85
이영호	C	74
황정형	C	95
김지수	A	90
최은영	B	82

<SQL문>

```
SELECT name, score  
FROM 성적  
( ① ) BY ( ② ) ( ③ );
```

답: ① **ORDER** ② **score** ③ **DESC**

[해설]

```
SELECT name, score  
FROM 성적  
ORDER BY score DESC;
```

'name'과 'score'를 표시한다.
<성적> 테이블에서 검색한다.
'score'를 기준으로 내림차순 정렬한다. (ASC: 오름차순)

[그룹]

13. 다음은 <회원> 테이블에서 '이름'이 "이"로 시작하는 회원들을 '가입일' 순으로 내림차순 정렬하는 <SQL문>이다. 괄호(①, ②)에 들어갈 알맞은 답을 쓰시오. [21-2](#)

<회원> 테이블

회원번호	이름	성별	가입일
1001	이진성	남	2021-06-23
1002	조이랑	여	2021-06-24
1003	최민수	남	2021-06-28
1004	김차희	여	2021-07-03
1005	이미경	여	2021-07-10

<SQL문>

```
SELECT * FROM 회원 WHERE 이름 LIKE '( ① )' ORDER BY 가입일 ( ② );
```

답: ① **이%** ② **DESC**

[해설]

```
SELECT *  
FROM 회원  
WHERE 이름 LIKE '이%'  
ORDER BY 가입일 DESC;
```

모든 속성을 표시한다.
<회원> 테이블에서 검색한다.
'이름'이 '이'로 시작하는 튜플만을 대상으로 한다.
'가입일'을 기준으로 내림차순 정렬한다.

• SQL 실행 결과

회원번호	이름	성별	가입일
1005	이미경	여	2021-07-10
1001	이진성	남	2021-06-23

14. 다음 질의 내용에 대한 SQL문을 완성하십시오. [20-4](#)

질의	학생 테이블에서 학과별 튜플의 개수를 검색하십시오. (단, 아래의 실행 결과가 되도록 한다.)
----	---

<학생>

학번	이름	학년	학과	주소
20160011	김영란	2	전기	서울
19210113	이재우	3	컴퓨터	대구
21168007	함소진	1	전자	부산
19168002	우혜정	3	전자	광주
18120073	김진수	4	컴퓨터	울산

<실행결과>

학과	학과별튜플수
전기	1
컴퓨터	2
전자	2

<처리 조건>

- WHERE 조건절은 사용할 수 없다.
- GROUP BY는 반드시 포함한다.
- 집계함수(Aggregation Function)를 적용한다.
- 학과별튜플수 컬럼이름 출력에 Alias(AS)를 활용한다.
- 문장 끝의 세미콜론(;)은 생략해도 무방하다.
- 인용부호 사용이 필요한 경우 단일 따옴표(' ')를 사용한다.

답: **SELECT 학과, COUNT(*) AS 학과별튜플수 FROM 학생 GROUP BY 학과;**

[해설]

```
SELECT 학과, COUNT(*) AS 학과별튜플수
FROM 학생
GROUP BY 학과;
```

'학과'와 개수를 표시하되, 개수의 필드명을 '학과별튜플수'로 표시한다.

<학생> 테이블을 대상으로 검색한다.

'학과'를 기준으로 그룹을 지정한다.

15. 다음 <성적> 테이블에서 과목별 점수의 평균이 90점 이상인 '과목이름', '최소점수', '최대점수'를 검색하고자 한다. <처리조건>을 참고하여 적합한 SQL문을 작성하시오. [20-3](#), [23-1](#)

<성적>

학번	과목번호	과목이름	학점	점수
a2001	101	컴퓨터구조	6	95
a2002	101	컴퓨터구조	6	84
a2003	302	데이터베이스	5	89
a2004	201	인공지능	5	92
a2005	302	데이터베이스	5	100
a2006	302	데이터베이스	5	88
a2007	201	인공지능	5	93

<결과>

과목이름	최소점수	최대점수
데이터베이스	88	100
인공지능	92	93

<처리조건>

- 최소한의 코드로 작성될 수 있도록 SQL문을 구성한다.
- WHERE문은 사용하지 않는다.
- GROUP BY와 HAVING을 이용한다.
- 집계함수(Aggregation Function)를 사용하여 명령문을 구성한다.
- '최소점수', '최대점수'는 별칭(Alias)을 위한 AS문을 이용한다.
- 명령문 마지막의 세미콜론(;)은 생략이 가능하다.
- 인용 부호가 필요한 경우 작은따옴표(' ')를 사용한다.

답:

```
SELECT 과목이름, MIN(점수) AS 최소점수, MAX(점수) AS 최대점수 FROM 성적
GROUP BY 과목이름 HAVING AVG(점수) >= 90;
```

[해설]

GROUP BY 집계 기능 및 HAVING 조건 조건식 도출 과정 분석

- **그룹화 기준 도출 (GROUP BY):** 과목별 연산을 수행하고 결과 테이블 헤더에 과목이름이 독립 속성으로 출력되므로, GROUP BY 과목이름 구문을 사용하여 과목명 단위로 데이터를 묶어야 합니다.
- **그룹 조건 필터링 (HAVING):** 과목별 점수의 **평균이 90점 이상**인 그룹만 추출해야 합니다. 처리 조건에서 WHERE 절 사용을 전면 금지하고 그룹 집계 조건을 명시했으므로, 그룹 대상 필터링 전용인 HAVING AVG(점수) >= 90을 사용합니다.

- 컴퓨터구조 그룹 평균: $(95 + 84) / 2 = 89.5$ 점 → [탈락]
- 데이터베이스 그룹 평균: $(89 + 100 + 88) / 3 = 92.33$ 점 → [합격]
- 인공지능 그룹 평균: $(92 + 93) / 2 = 92.5$ 점 → [합격]

• **출력 속성 및 별칭 정의 (SELECT ... AS):** 최종 결과 항목에 부합하도록 집계 함수 최소값(MIN)과 최대값(MAX)을 구성하고, 조건 지침에 맞춰 명시적으로 별칭 지정 예약어인 AS 최소점수, AS 최대점수를 선언해 줍니다.

[조인]

16. <A> 테이블과 테이블을 참고하여 <SQL문>의 실행 결과를 쓰시오. [21-3](#), [25-3](#)

<A>	
NAME	RULE
Smith	S%
Allen	%T%
Scott	

<SQL문>

```
SELECT COUNT(*) CNT FROM A CROSS JOIN B WHERE A.NAME LIKE B.RULE;
```

답: 4

[해설]

```
SELECT COUNT(*) CNT
FROM A CROSS JOIN B
WHERE A.NAME LIKE B.RULE
```

질의문은 각 절을 분리하여 이해하면 쉽습니다.

- **SELECT COUNT(*) CNT** : 튜플의 개수를 표시하되, 필드명은 'CNT'로 표시합니다. (※ 'SELECT COUNT(*) AS CNT'에서 AS가 생략된 형태입니다.)
- **FROM A CROSS JOIN B** : <A>와 를 교차 조인(CROSS JOIN)한 결과를 대상으로 검색합니다.

A.NAME	B.RULE
Smith	S%
Smith	%T%
Allen	S%
Allen	%T%
Scott	S%
Scott	%T%

- **WHERE A.NAME LIKE B.RULE** : <A> 테이블의 'NAME' 필드 값이 테이블의 'RULE' 필드에 저장된 문자열 패턴과 일치하는 튜플만을 대상으로 합니다.

※ 테이블의 'RULE' 필드에 저장된 값은 'S%'와 '%T%'와 같이 문자 패턴인 '%' 기호가 포함되어 있으므로, 조건문의 LIKE 연산자와 결합되면 다음과 같이 적용됩니다.

- **A.NAME LIKE 'S%'** : 'A.NAME'이 "S"로 시작하는 레코드를 검색합니다. (해당 행: Smith / Scott)
- **A.NAME LIKE '%T%'** : 'A.NAME'이 "T"를 포함하는 레코드를 검색합니다. (해당 행: Smith / Scott) ※ 소대문자를 가리지 않는 설정 또는 패턴 일치로 대문자 T 포함 여부 매칭

A.NAME	B.RULE
Smith	S%
Smith	%T%
Allen	S%
Allen	%T%
Scott	S%
Scott	%T%

- CROSS JOIN된 결과에서 조건을 만족하는 튜플은 총 4개(Smith-S%, Smith-%T%, Scott-S%, Scott-%T%)입니다. 그러므로 검색된 튜플의 개수는 4가 됩니다.

NAME	RULE
Smith	S%
Smith	%T%
Scott	S%
Scott	%T%

- 만약 테이블 형태로 표현하라고 하면 아래와 같이 작성한다.

CNT
4

17. 다음 <사원> 테이블과 <동아리> 테이블을 조인(Join)한 <결과>를 확인하여 <SQL문>의 괄호(①, ②)에 들어갈 알맞은 답을 쓰시오. [21-2](#)

<사원>			<동아리>		<결과>		
코드	이름	부서	코드	동아리명	코드	이름	동아리명
1601	김명해	인사	1601	테니스	1601	김명해	테니스
1602	이진성	경영지원	1731	탁구	1602	이진성	NULL
1731	박영광	개발	2001	볼링	1731	박영광	탁구
2001	이수진	-			2001	이수진	볼링

<SQL문>

SELECT a.코드, 이름, 동아리명 FROM 사원 a LEFT JOIN 동아리 b (①) a.코드 = b.(②);

답: ① ON ② 코드

[해설]

- ① SELECT a.코드, 이름, 동아리명
- ② FROM 사원 a LEFT JOIN 동아리 b
- ③ ON a.코드 = b.코드;

- ① a가 가리키는 <사원> 테이블의 '코드'와 '이름', <동아리> 테이블의 '동아리명'을 표시합니다.
- ② · LEFT JOIN이므로 좌측의 <사원> 테이블이 기준이 됩니다. <사원> 테이블에 있는 모든 튜플은 결과에 표시하고, 우측의 <동아리> 테이블에서는 관련이 있는 튜플(코드가 매칭되는 튜플)만 가져와 표시합니다. 관련이 없

는 경우(이진성, 1602) 동아리명 속성값은 빈칸(NULL)으로 표시됩니다.

· <사원>, <동아리> 테이블의 별칭(Alias)으로 각각 <a>, 를 지정합니다.

③ <사원> 테이블의 '코드'와 <동아리> 테이블의 '코드'를 기준으로 서로 JOIN합니다.

18. 다음 <R>과 <S> 테이블에 대해 관계대수 연산 ' $R \div S$ '를 실행했을 때 출력되는 결과를 테이블로 작성하시오. [25-3](#)

<R>		<S>
A	B	B
a1	b1	b1
a2	b2	b3
a1	b3	

답:

A
a1

[해설]

관계대수 디비전(Division, $\div$) 연산의 원리

- **차수 정의:** 테이블 $R(X, Y)$ 와 테이블 $S(Y)$ 가 있을 때, $R \div S$ 는 테이블 S 의 모든 속성 값(Y)을 전부 가지고 있는 R 의 속성 값(X)들을 추출하는 연산입니다.
- **속성 매칭:** 나누는 수인 테이블 <S>의 B 속성 값은 현재 {b1, b3}입니다.
- **행 추적 비교:**
 - 값 a1은 속성 B 값으로 b1과 b3을 모두 가지고 있으므로 결과에 포함됩니다.
 - 값 a2는 속성 B 값으로 b2만 가지고 있어 조건({b1, b3})을 충족하지 못하므로 탈락합니다.

따라서 속성 B를 제외한 속성 A만 남기어 중복을 제거한 최종 릴레이션의 결과 필드는 [a1] 단 하나만 도출됩니다.

<사원>		<직원>	
사번	이름	사번	이름
1	고회식	2	김준용
2	김준용	3	윤정희

$\pi_{\text{이름}}(\text{사원}) \cup \pi_{\text{이름}}(\text{직원})$ $\pi_{\text{이름}}(\text{사원}) \cap \pi_{\text{이름}}(\text{직원})$ $\pi_{\text{이름}}(\text{사원}) - \pi_{\text{이름}}(\text{직원})$ $\pi_{\text{이름}}(\text{사원}) \times \pi_{\text{이름}}(\text{직원})$

이름
고회식
김준용
윤정희

이름
김준용

이름
고회식

사원.이름	직원.이름
고회식	김준용
고회식	윤정희
김준용	김준용
김준용	윤정희

19. 다음 <A> 테이블을 참조하여 <SQL문>을 실행했을 때 출력되는 결과를 쓰시오. (<A> 테이블에 표시된 'NULL'은 값이 없음을 의미한다.) 25-3

<A>

COL1	COL2
2	NULL
3	6
2	3
NULL	3
4	5

<SQL문>

```
SELECT COUNT(COL2) FROM A
WHERE COL1 IN (2, 3) OR COL2 IN (3, 5);
```

답: 4

[해설]

WHERE 조건절 만족 여부 및 COUNT 함수의 NULL 처리 메커니즘 분석

- WHERE 조건식 분석 (COL1 IN (2, 3) OR COL2 IN (3, 5)):
테이블 A의 총 5개 행 중에서 조건(COL1이 2 또는 3이거나, 혹은 COL2가 3 또는 5인 행)을 필터링합니다.
 - 1행 (2, NULL): COL1이 2이므로 참(선택)
 - 2행 (3, 6): COL1이 3이므로 참(선택)
 - 3행 (2, 3): COL1이 2이고 COL2가 3이므로 참(선택)
 - 4행 (NULL, 3): COL2가 3이므로 참(선택)
 - 5행 (4, 5): COL2가 5이므로 참(선택)결과적으로 모든 행(1~5행 전체)이 조건절을 통과하여 최종 집계 대상이 됩니다.
- COUNT(COL2) 집계 함수 계산:
COUNT(*)과 달리 특정 컬럼을 지정한 COUNT(COL2) 함수는 해당 범위 내 데이터 중 NULL 값을 제외하고 개수를 카운트합니다.
 - 선택된 5개 행의 COL2 값들은 각각 [NULL, 6, 3, 3, 5]입니다. 여기서 첫 번째 행의 NULL 값을 제외한 실질적인 데이터 개수를 세면 총 3개(6, 3, 5)가 됩니다. (※ 3은 중복 데이터이지만 DISTINCT 조건이 없으므로 중복을 포함하여 카운트합니다.)

결과 4

20. 다음 <emp>와 <sal> 테이블을 참조하여 <SQL문>을 실행했을 때 출력되는 결과를 쓰시오. 25-1

<emp>

id	name
1002	홍길동
1004	강감찬
1006	김유신
1008	이순신

<sal>

id	incentive
1002	300
1004	400
1008	1000
1009	500

<SQL문>

```
SELECT name, incentive
FROM emp, sal
WHERE emp.id = sal.id AND incentive >= 500;
```

답:

name	incentive
이순신	1000

[해설]

관계형 데이터베이스 내부 조인(Inner Join) 및 WHERE 조건 조건 필터링 분석

- **조인 조건 분석 (WHERE emp.id = sal.id):** 두 테이블 간에 공통으로 존재하는 id 속성값을 매칭하여 연결(내부 조인)합니다.
 - 매칭되는 ID는 **1002**(홍길동 - 300), **1004**(강감찬 - 400), **1008**(이순신 - 1000)입니다.
 - emp의 1006(김유신)과 sal의 1009(500)는 서로 일치하는 ID가 상대 테이블에 없으므로 조인 결과에서 제외됩니다.
- **필터링 조건 분석 (AND incentive >= 500):** 조인된 결과 데이터 중에서 incentive의 값이 500 이상인 행(Row)만 최종 추출합니다.
 - 홍길동(300)과 강감찬(400)은 500 미만이므로 탈락합니다.
 - 이순신(1000)은 500 이상이므로 최종 조건을 만족합니다.
- **최종 출력 컬럼 (SELECT name, incentive):** 선택된 행에서 명시된 속성인 name과 incentive 컬럼만 속성명(헤더)과 함께 격자 형태로 출력됩니다.

21. 다음에 제시된 <학생>과 <학부> 테이블에 대한 릴레이션 스키마를 참고하여 각 SQL문의 요구사항에 맞도록 괄호(①~④)에 알맞은 답을 적어 SQL문을 완성하시오. [24-2](#)

릴레이션 스키마	<학생>				<학부>			
	학번(PK)	이름	나이	학과	학번(PK)	이름	주소	나이

<SQL 1> 문

[요구사항] 신입생 정보가 확인되어 <학부> 테이블에 신입생 정보를 추가한다.

[SQL] **INSERT INTO** 학부(학번, 이름, 주소, 나이) (①) (240912, '최재균', '서울', 20);

<SQL 2> 문

[요구사항] <SQL 1> 문에서 추가한 학생을 <학부> 테이블에서 검색한 후 검색된 자료를 <학생> 테이블에 추가한다.

[SQL] **INSERT INTO** 학생(학번, 이름, 나이, 학과) (②) 학번, 이름, 나이, '컴퓨터공학' **FROM** 학부 **WHERE** 이름 = '최재균';

<SQL 3> 문

[요구사항] <학생> 테이블의 모든 자료를 조회한다.

[SQL] **SELECT** * (③) 학생;

<SQL 4> 문

[요구사항] 학생이 휴학 신청해서 해당 학생의 '학과' 필드의 값을 "휴학"으로 변경한다.

[SQL] **UPDATE** 학생 (④) 학과 = '휴학' **WHERE** 학번 = 240912;

답: ① **VALUES** ② **SELECT** ③ **FROM** ④ **SET**

[해설]

SQL 기본 데이터 조작어(DML) 구문 작성 규칙 분석

- ① **INSERT INTO ... VALUES** 구문: 테이블에 직접적인 상수 데이터 값을 행(Row)으로 명시하여 삽입할 때는 VALUES 키워드를 사용합니다.
- ② **INSERT INTO ... SELECT** 대량 데이터 삽입 구문: 다른 테이블에서 조회(검색)한 결과 데이터를 그대로 삽입(Subquery가 포함된 INSERT)할 때는 VALUES가 생략되며 삽입할 데이터의 형식을 구성하는 SELECT가 들어갑니다.
- ③ **SELECT ... FROM** 기본 조회 구문: 테이블의 데이터를 기본적으로 질의(조회)할 때 대상 테이블을 지정하기 위해 검색 대상 리스트 뒤에 필수적으로 FROM 키워드가 와야 합니다.
- ④ **UPDATE ... SET** 수정 구문: 기존에 존재하는 테이블 내부 행의 값을 변경하고자 할 때 변경할 컬럼과 값을 지정해 주는 SET 키워드가 필요합니다. (UPDATE 테이블명 SET 컬럼 = 값 WHERE 조건)

22. 다음 <R1>과 <R2> 테이블을 참조하여 <SQL문>을 실행했을 때 출력되는 결과를 쓰시오.
(SQL을 실행하였을 때 출력되는 속성명과 값들을 모두 답안에 적으시오.) [24-1](#)

<R1>

A	B	C
1	a	x
2	b	y
3	c	t

<R2>

C	D	E
x	k	k
y	k	t
z	p	k

<SQL문>

```
SELECT B
FROM R1
WHERE C IN (SELECT C FROM R2 WHERE D = 'k');
```

답:

B
a
b

[해설]

- 1. 서브쿼리(Subquery) 먼저 실행:
SELECT C FROM R2 WHERE D='k' 구문이 먼저 실행됩니다.
<R2> 테이블에서 D의 값이 'k'인 행의 C 속성값을 찾으면 'x'와 'y'가 반환됩니다.
- 2. 메인쿼리(Main Query) 실행:
서브쿼리의 결과에 따라 메인 쿼리는 SELECT B FROM R1 WHERE C IN ('x', 'y')로 변환됩니다.
- 3. 최종 결과 도출:
<R1> 테이블에서 C의 속성값이 'x' 또는 'y'인 행을 필터링하면 첫 번째 행(1, a, x)과 두 번째 행(2, b, y)이 조건에 만족합니다. 이 중 B 속성만 출력해야 하므로 결과는 속성명 B 아래에 a, b가 차례로 출력됩니다.

23. 다음 <employee>와 <project> 테이블을 참조하여 <SQL문>을 실행했을 때 출력되는 결과를 쓰시오.
[24-3](#)

<employee>

no	first_name	last_name	project_id
1	John	Doe	10
2	Jim	Carry	20
3	Rachel	Redmond	10

<project>

project_id	name
10	Alpha
20	Beta
10	Gamma

<SQL문>

```
SELECT count(*)
FROM employee AS e JOIN project AS p ON e.project_id = p.project_id
WHERE p.name IN (
```

```
SELECT name FROM project p WHERE p.project_id IN (
    SELECT project_id FROM employee GROUP BY project_id HAVING count(*) < 2
);
```

답:

count(*)
1

[해설]

다중 서브쿼리(Subquery)와 그룹화 조건(HAVING), 조인(JOIN) 연산 수행 과정 분석

- 최하위 서브쿼리 분석 (HAVING count(*) < 2):
 - employee 테이블을 project_id별로 그룹화하면 ID 10번은 2명(John, Rachel), 20번은 1명(Jim)입니다.
 - 이 중 개수가 2 미만(< 2)인 그룹을 필터링하므로, 만족하는 project_id는 20이 됩니다.
- 중간 서브쿼리 분석 (WHERE p.project_id IN (20)):
 - project 테이블에서 project_id가 20인 행의 name 속성값 'Beta'가 됩니다.
- 메인 쿼리 WHERE 조건 및 조인 분석:
 - employee(e)와 project(p) 테이블을 project_id 기반으로 내부 조인(INNER JOIN) 합니다.
 - 조인된 전체 결과 행 중에서 메인 WHERE p.name IN ('Beta') 조건을 적용해 프로젝트명이 'Beta'인 행만 필터링 합니다.
 - 해당 조건을 만족하는 결합 데이터는 [no: 2, Jim, Carry, project_id: 20] + [project_id: 20, name: Beta] 단 1개의 행만 존재합니다.
- 최종 결과 출력 (SELECT count(*)):
 - 조건에 맞는 최종 행의 총개수를 세는 집계 함수이므로 결과 헤더 count(*) 아래에 값 1이 출력됩니다.

24. <R>과 <S> 테이블에 대해 <SQL문>을 실행하였을 때 나타나는 결과를 작성하시오. (SQL을 실행하였을 때 출력되는 속성명과 값들을 모두 답안에 적으시오.) [23-3](#)

<R>

A	B
1	a
2	b
3	c

<S>

A	B
1	a
2	c
4	d

<SQL문>

```
SELECT A FROM R
UNION
SELECT A FROM S
ORDER BY A DESC;
```

답:

A
4
3
2
1

[해설]

관계질의 집합 연산(UNION) 및 정렬(ORDER BY) 과정 분석

- 첫 번째 SELECT문 결과: SELECT A FROM R의 결과값은 테이블 R의 A 속성값인 1, 2, 3입니다.
- 두 번째 SELECT문 결과: SELECT A FROM S의 결과값은 테이블 S의 A 속성값인 1, 2, 4입니다.
- UNION 연산 수행 (합집합 및 중복 제거): UNION 연산자는 두 집합의 결과를 하나로 합치면서 **중복된 데이터는 자동으로 제거**합니다.

- {1, 2, 3} ∪ {1, 2, 4} = {1, 2, 3, 4}가 집합 결과로 도출됩니다. (중복값 1과 2가 한 번만 남음)
- 만약 중복을 포함하는 연산이었다면 UNION ALL을 사용했어야 합니다.
- **ORDER BY 절 정렬 조건 반영 (DESC):** 최종 결과 집합을 속성 A를 기준으로 **내림차순(Descending)** 정렬합니다. 따라서 큰 수부터 작은 수 순서대로 정렬되어 격자 데이터로 4, 3, 2, 1이 차례대로 출력됩니다.

25. 다음 <학생> 테이블에 (9816021, '한국산', 3, '경영학개론', '050-1234-1234')인 데이터를 삽입하고자 한다. <처리조건>을 참고하여 적합한 SQL문을 작성하시오. [23-2](#)

<학생>

학번	이름	학년	신청과목	연락처
9815932	김태산	3	경영정보시스템	050-5234-1894
9914511	박명록	2	경제학개론	050-1415-4986
0014652	이익명	1	국제경영	050-6841-6781
9916425	김혜리	2	재무관리	050-4811-1187
9815945	이지영	3	인적자원관리	050-9785-8845

<처리조건>

- 최소한의 코드로 작성될 수 있도록 SQL문을 구성한다.
- 명령문 마지막의 세미콜론(;)은 생략이 가능하다.
- 인용 부호가 필요한 경우 작은따옴표(')를 사용한다.

답:

```
INSERT INTO 학생 VALUES (9816021, '한국산', 3, '경영학개론', '050-1234-1234');
```

[해설]

- **SELECT(검색):** SELECT~ FROM~ WHERE~
- **INSERT(삽입):** INSERT INTO~ VALUES~
- **DELETE(삭제):** DELETE~ FROM~ WHERE~
- **UPDATE(변경):** UPDATE~ SET~ WHERE~

데이터 조작어(DML) - INSERT 문 조작 및 처리 조건 분석

- **INSERT INTO 문법 분석:** 테이블에 새로운 행을 삽입할 때 사용하는 기본 명령어 문법 구문은 다음과 같습니다.
 - 구문 1: INSERT INTO 테이블명 (속성명1, 속성명2, ...) VALUES (데이터1, 데이터2, ...);
 - 구문 2: INSERT INTO 테이블명 VALUES (데이터1, 데이터2, ...);
- **최소한의 코드 구성 조건 반영 (처리조건 1):** 테이블에 정의된 모든 속성 순서대로 값을 전체 삽입하는 경우, 속성명 열거 리스트(학번, 이름, 학년, 신청과목, 연락처) 부분을 완전히 과감하게 생략할 수 있습니다. 따라서 가장 코드가 짧은 구문 2 형식을 채택하여 작성해야 최적의 답안이 됩니다.
- **리터럴 데이터 형식 구분 (처리조건 3):** 숫자 데이터(9816021, 3)는 인용 부호 없이 그대로 기술하며, 문자열 데이터('한국산', '경영학개론', '050-1234-1234')는 반드시 지문 규정대로 작은따옴표(')로 묶어서 표현해야 안전하게 득점할 수 있습니다.

26. 다음 <처리조건>에 부합하는 <SQL문>이 완성되도록 괄호에 적합한 옵션을 쓰시오. [23-2](#)

<처리조건>

- <학생> 뷰를 제거한다.
- <학생> 뷰를 참조하는 모든 데이터도 연쇄적으로 제거한다.

<SQL문>

DROP VIEW 학생 (_____);

답: CASCADE

[해설]

데이터 정의어(DDL) - DROP 명령의 연쇄 삭제 옵션 분석

- **DROP 구문의 옵션 정의:** 스키마, 도메인, 테이블, 뷰, 인덱스 등 데이터베이스 개체를 삭제할 때 사용하는 DROP 문은 하위 참조 개체들을 어떻게 처리할지 결정하는 두 가지 핵심 옵션을 가집니다.
 - **CASCADE:** 제거할 개체를 참조하는 다른 모든 개체들을 **연쇄적으로 함께 제거**합니다.
 - **RESTRICT:** 다른 개체가 제거할 개체를 참조하고 있을 경우, **삭제 명령 자체를 취소(제한)**합니다.
- **처리 조건과의 매칭 (처리 조건 2):** 지문에서 "<학생> 뷰를 참조하는 모든 데이터도 연쇄적으로 제거한다"라는 명확한 기준을 제시했으므로, 괄호 안에 들어갈 키워드는 '연쇄'의 의미를 가진 **CASCADE**가 됩니다.
- **시험용 팁 (오타 주의):** 참고로 지문의 SQL문 예시 상에 오타(DROP VEW)가 있더라도, 자격증 시험 답안지에는 표준 SQL 문법 및 채점 기준에 맞게 **CASCADE** 키워드를 정확하게 스펠링 오류 없이 기재해야 정답으로 인정됩니다.

프로그래밍 - JAVA 8문제 001로

1. 다음 JAVA로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. 21-3

```
public class Test {  
    public static void main(String[] args) {  
        int w = 3, x = 4, y = 3, z = 5;  
        if ((w == 2 | w == y) & !(y > z) & (1 == x ^ y != z)) {  
            w = x + y;  
            if (7 == x ^ y != w)  
                System.out.println(w);  
            else  
                System.out.println(x);  
        }  
        else {  
            w = y + z;  
            if (7 == y ^ z != w)  
                System.out.println(w);  
            else  
                System.out.println(z);  
        }  
    }  
}
```

답: 7

[해설]

```
public class Test {  
    public static void main(String[] args) {  
        ① int w = 3, x = 4, y = 3, z = 5;  
        ② if((w == 2 | w == y) & !(y > z) & (1 == x ^ y != z)) {  
            ③ w = x + y;  
            ④ if(7 == x ^ y != w)  
                ⑤ System.out.println(w);  
            else  
                System.out.println(x);  
        }  
        ⑥ }  
    }  
}
```

```

else {
    w = y + z;
    if(7 == y ^ z != w)
        System.out.println(w);
    else
        System.out.println(z);
}
}

```

- 정수형 변수 w, x, y, z를 선언하고 각각 3, 4, 3, 5로 초기화한다.
- 조건이 참이면 3번부터 6번 이전까지의 문장을, 거짓이면 6번 아래 else의 다음 문장부터 7번 이전까지의 문장을 수행한다. 연산자 우선순위에 따라 다음의 순서로 조건의 참/거짓을 확인한다.

$(w = 2 \mid w = y) \ \& \ !(y > z) \ \& \ (1 == x \wedge y \neq z)$

1 2 3 4 5

 6 7

 8 9

- 1 : w의 값 3과 2는 같지 않으므로 거짓(0)이다.
- 2 : w의 값 3과 y의 값 3은 같으므로 참(1)이다.
- 3 : y의 값 3은 z의 값 5보다 크지 않으므로 거짓(0)이지만, 앞에 !(논리 not)가 있으므로 참(1)이다.
- 4 : 1과 x의 값 4는 같지 않으므로 거짓(0)이다.
- 5 : y의 값 3과 z의 값 5는 같지 않으므로 참(1)이다.

- 6 1 | 2 : ①의 결과 0과 ②의 결과 1을 | (비트 or) 연산하면 결과는 1이다.

```

0000(0)
| 0001(1)
-----
0001(1)

```

- 7 4 ^ 5 : ④의 결과 0과 ⑤의 결과 1을 ^ (비트 xor) 연산하면 결과는 1이다.

```

0000(0)
^ 0001(1)
-----
0001(1)

```

- 8 6 & 3 : ⑥의 결과 1과 ③의 결과 1을 & (비트 and) 연산하면 결과는 1이다.

```

0001(1)
& 0001(1)
-----
0001(1)

```

- 9 8 & 7 : ⑧의 결과 1과 ⑦의 결과 1을 & (비트 and) 연산하면 결과는 1이다.

∴ 최종 결과는 1이며, 1은 조건에서 참(True)을 의미하므로 3번으로 이동한다.

- 3 w에 x와 y의 합을 저장한다. (w = 7)

- 4 조건이 참이면 5번 문장을, 거짓이면 5번 아래 else 다음 문장을 수행한다. 연산자 우선순위에 따라 다음의 순서로 조건의 참/거짓을 확인한다.

$7 == x \wedge y \neq w$

1 2

 3

- 1 : 7과 x의 값 4는 같지 않으므로 결과는 거짓(0)이다.
- 2 : y의 값 3과 w의 값 7은 같지 않으므로 결과는 참(1)이다.

- ③ ① ^ ② : ①의 결과 0과 ②의 결과 1을 ^(비트 xor) 연산하면 결과는 1이다.

∴ 최종 결과는 10이며, 1은 조건에서 참을 의미하므로 ⑤번 문장을 수행한다.

- ⑤ w의 값 7을 출력하고 커서를 다음 줄의 처음으로 옮긴다. 모든 if문이 종료되었으므로 ⑦번으로 이동하여 프로그램을 종료한다.

결과 7

2. 다음 Java로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. 21-1

```
public class Test {  
    public static void main(String[] args) {  
        int j, i;  
        for (j = 0, i = 0; i <= 5; i++) {  
            j += i;  
            System.out.print(i);  
            if (i == 5) {  
                System.out.print("=");  
                System.out.print(j);  
            }  
            else  
                System.out.print("+");  
        }  
    }  
}
```

답: 0+1+2+3+4+5=15

[해설]

```
public class Test {  
    public static void main(String[] args) {  
        ① int j, i;  
        ② for (j = 0, i = 0; i <= 5; i++) {  
            ③ j += i;  
            ④ System.out.print(i);  
            ⑤ if (i == 5) {  
                ⑥ System.out.print("=");  
                ⑦ System.out.print(j);  
            }  
            else  
                ⑧ System.out.print("+");  
        }  
    }  
}
```

- ① 정수형 변수 j, i를 선언한다.
- ② 반복 변수 i가 0부터 1씩 증가하면서 5보다 작거나 같은 동안 ③~⑧번을 반복 수행한다. i가 0으로 초기화될 때 j도 0으로 초기화된다.

3. $j = j + i$ 와 동일하다. j 에 i 의 값을 누적시킨다.
4. i 의 값을 출력한다.
5. i 가 5와 같으면 6, 7 번을 수행하고, 아니면 8 번을 수행한다.
6. $=$ 을 출력한다.
7. j 의 값을 출력한다.
8. $+$ 를 출력한다.

※ 반복문 실행에 따른 변수들의 변화는 다음과 같다.

i	j	출력 누적 상태
0	0	0+
1	1	0+1+
2	3	0+1+2+
3	6	0+1+2+3+
4	10	0+1+2+3+4+
5	15	0+1+2+3+4+5=15
6	-	

결과 0+1+2+3+4+5=15

3. 다음은 변수 n 에 저장된 10진수를 2진수로 변환하여 출력하는 Java 프로그램이다. 프로그램을 분석하여 괄호(①, ②)에 들어갈 알맞은 답을 쓰시오. [20-4](#)

```
public class Test {
    public static void main(String[] args) {
        int a[] = new int[8];
        int i = 0;
        int n = 10;
        while( ( ① ) ) {
            a[i++] = ( ② );
            n /= 2;
        }
        for(i = 7; i >= 0; i--)
            System.out.print(a[i]);
    }
}
```

답: ① $n > 0$ ② $n \% 2$

[해설]

```
public class Test {
    public static void main(String[] args) {
        ① int a[] = new int[8];
        ② int i = 0;
        ③ int n = 10;
        ④ while(n > 0) {
            ⑤ a[i++] = n % 2;
            ⑥ n /= 2;
        }
        ⑦ for(i = 7; i >= 0; i--)
```

```

8      System.out.print(a[i]);
    }
}

```

① 8개의 요소를 갖는 정수형 배열 a를 선언한다.

배열 a 0 0 0 0 0 0 0 0

※ Java는 배열 선언 시 초기화를 하지 않아도 자동으로 0으로 초기화된다.

② 정수형 변수 i를 선언하고 0으로 초기화한다.

③ 정수형 변수 n을 선언하고 10으로 초기화한다.

④ n이 0보다 큰 동안 ⑤~⑥ 번을 반복 수행한다.

⑤ i++은 후치 증가 연산자이므로, a[i]에 n을 2로 나눈 나머지를 저장한 후, i의 값에 1을 더한다.

⑥ 'n = n / 2;'와 동일하다. n을 2로 나눈 값을 n에 저장한다.

※ 반복문 실행에 따른 변수들의 값의 변화는 다음과 같다.

i	n	n % 2	a[8]
0	10		
1	5	0	0 0 0 0 0 0 0 0
2	2	1	0 1 0 0 0 0 0 0
3	1	0	0 1 0 0 0 0 0 0
4	0	1	0 1 0 1 0 0 0 0

⑦ 반복 변수 i가 7에서 시작하여 1씩 감소하면서 0보다 크거나 같은 동안 ⑧ 번을 반복 수행한다.

⑧ a[i]의 값을 출력한다.

※ 반복문 실행에 따른 변수들의 값의 변화는 다음과 같다.

배열 a 0 1 0 1 0 0 0 0

i	출력
7	0
6	00
5	000
4	0000
3	00001
2	000010
1	0000101
0	00001010
-1	

4. 다음 Java로 구현된 프로그램을 분석하여 괄호(①, ②)에 들어갈 알맞은 답을 쓰시오. [20-4](#)

```

public class Test {
    public static void main(String []args) {
        int ary[][] = new int[( ① )][( ② )];
        int n = 1;
        for(int i = 0; i < 3; i++) {
            for(int j = 0; j < 5; j++) {

```

```

        ary[i][j] = j * 3 + i + 1;
        System.out.print(ary[i][j] + " ");
    }
    System.out.println();
}
}
}

```

답: ① 3 ② 5

[해설]

```

public class Test {
    public static void main(String []args) {
        ① int ary[][] = new int[3][5];
        ② int n = 1;
        ③ for(int i = 0; i < 3; i++) {
            ④ for(int j = 0; j < 5; j++) {
                ⑤ ary[i][j] = j * 3 + i + 1;
                ⑥ System.out.print(ary[i][j] + " ");
            }
            ⑦ System.out.println();
        }
    }
}

```

- ① 3행 5열의 요소를 갖는 정수형 2차원 배열 ary를 선언한다.
- ② 정수형 변수 n을 선언하고 1로 초기화한다.
- ③ 반복 변수 i가 0에서 시작하여 1씩 증가하면서 3보다 작은 동안 ④~⑦번을 반복 수행한다.
- ④ 반복 변수 j가 0에서 시작하여 1씩 증가하면서 5보다 작은 동안 ⑤~⑥번을 반복 수행한다.
- ⑤ ary[i][j]에 j*3+i+1을 연산한 값을 저장한다.
- ⑥ ary[i][j]의 값을 출력하고 공백을 한 칸 띄운다.
- ⑦ 커서를 다음 줄의 처음으로 옮긴다.

※ 반복문 실행에 따른 변수들의 값의 변화는 다음과 같다.

n	i	j	ary[3][5]	출력															
1	0	0	<table><tr><td>1</td><td>4</td><td>7</td><td>10</td><td>13</td></tr><tr><td></td><td></td><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td><td></td><td></td></tr></table>	1	4	7	10	13											1 4 7 10 13
		1		4	7	10	13												
		1																	
		2																	
3																			
4																			
5																			
	1	0	<table><tr><td>1</td><td>4</td><td>7</td><td>10</td><td>13</td></tr><tr><td>2</td><td>5</td><td>8</td><td>11</td><td>14</td></tr><tr><td></td><td></td><td></td><td></td><td></td></tr></table>	1	4	7	10	13	2	5	8	11	14						1 4 7 10 13 2 5 8 11 14
		1		4	7	10	13												
		2		5	8	11	14												
		1																	
		2																	
3																			
4																			

n	i	j	ary[3][5]	출력
		5		
		0		
		1		
	2	2	1 4 7 10 13	1 4 7 10 13
		3	2 5 8 11 14	2 5 8 11 14
		4	3 6 9 12 15	3 6 9 12 15
		5		
	3			

5. 다음 Java로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. [20-3](#)

```
public class Test {
    public static void main(String[] args) {
        int i = 0, c = 0;
        while (i < 10) {
            i++;
            c *= i;
        }
        System.out.println(c);
    }
}
```

답: 0

[해설]

```
public class Test {
    public static void main(String[] args) {
        ❶ int i = 0, c = 0;
        ❷ while (i < 10) {
            ❸ i++;
            ❹ c *= i;
        }
        ❺ System.out.println(c);
    }
}
```

- ❶ 정수형 변수 i와 c를 선언하고 각각 0으로 초기화한다.
- ❷ i가 10보다 작은 동안 ❸~❹번을 반복 수행한다.
- ❸ 'i = i + 1;'과 동일하다. i의 값에 1을 누적시킨다.
- ❹ 'c = c * i'와 동일하다. c * i의 값을 c에 저장한다.
- ❺ c의 값을 화면에 출력하고 커서를 다음 줄 처음으로 옮긴다.

※ 반복문 실행에 따른 변수들의 변화는 다음과 같다.

i	c
0	0
1	0
2	0
3	0

i	c
4	0
5	0
6	0
7	0
8	0
9	0
10	0

결과 0

6. 다음 Java로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. [20-3](#)

```
public class Test {
    public static void main(String[] args) {
        int a = 0, sum = 0;
        while (a < 10) {
            a++;
            if (a % 2 == 1)
                continue;
            sum += a;
        }
        System.out.println(sum);
    }
}
```

답: 30

[해설]

```
public class Test {
    public static void main(String[] args) {
        ❶ int a = 0, sum = 0;
        ❷ while (a < 10) {
            ❸ a++;
            ❹ if (a % 2 == 1)
                ❺ continue;
            ❻ sum += a;
        }
        ❼ System.out.println(sum);
    }
}
```

- ❶ 정수형 변수 a와 sum을 선언하고 각각 0으로 초기화한다.
- ❷ a가 10보다 작은 동안 ❸~❻번을 반복 수행한다.
- ❸ 'a = a + 1;'과 동일하다. a의 값에 1을 누적시킨다.
- ❹ a%2, 즉 a를 2로 나눈 나머지가 1이면 ❺번을 수행하고, 아니면 ❻번으로 이동한다.

- 5 while문의 시작점인 2번으로 제어를 이동시킨다. (아래의 누적합 sum 식을 건너뛰고 상위 조건식으로 제어 점프)
- 6 'sum = sum + a;'와 동일하다. sum에 a의 값을 누적시킨다.
- 7 sum의 값을 출력하고 커서를 다음 줄 처음으로 옮긴다.

※ 반복문 실행에 따른 변수들의 변화는 다음과 같다.

a	sum
0	0
1	
2	2
3	
4	6
5	
6	12
7	
8	20
9	
10	30

결과 30

7. 다음 JAVA로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. [22-3](#)

```
public class Test {
    public static void main(String[] args) {
        int result[] = new int[5];
        int arr[] = { 77, 32, 10, 99, 50 };
        for (int i = 0; i < 5; i++) {
            result[i] = 1;
            for (int j = 0; j < 5; j++) {
                if (arr[i] < arr[j])
                    result[i]++;
            }
            for (int k = 0; k < 5; k++)
                System.out.print(result[k]);
        }
    }
}
```

답: 24513

[해설]

문제의 코드는 배열 arr에 저장된 값들의 순위를 구하여 배열 result에 저장하는 알고리즘입니다.

```
public class Test {
    public static void main(String[] args) {
        1 int result[] = new int[5];
        2 int arr[] = { 77, 32, 10, 99, 50 };
        3 for(int i = 0; i < 5; i++) {
        4     result[i] = 1;
```

```

5      for(int j = 0; j < 5; j++) {
6          if (arr[i] < arr[j])
7              result[i]++;
8      }
9      for(int k = 0; k < 5; k++)
10         System.out.print(result[k]);
11 }

```

❶ 5개의 요소를 갖는 정수형 배열 result를 선언한다.

result 0 | 0 | 0 | 0 | 0

※ Java에서는 배열을 선언하고 초기화하지 않으면 배열의 모든 요소가 0으로 초기화됩니다.

❷ 5개의 요소를 갖는 정수형 배열 arr을 선언하고 초기화한다.

arr 77 | 32 | 10 | 99 | 50

❸ 반복 변수 i가 0부터 1씩 증가하면서 5보다 작은 동안 ❹~❷번을 반복 수행한다.

❹ 다른 점수들과 비교하기 전까지는 모든 점수의 석차는 1등이므로, result[i]에 1을 저장한다.

❺ 반복 변수 j가 0부터 1씩 증가하면서 5보다 작은 동안 ❻, ❼번을 반복 수행한다.

❻ 현재 점수(arr[i])가 비교 점수(arr[j])보다 작으면 석차를 1 증가시키기 위해 ❼번으로 이동하고, 아니면 반복문의 시작인 ❺번으로 이동한다.

❼ 'result[i] = result[i] + 1;'과 동일하다. i번째 점수의 석차를 1씩 증가시킨다.

※ 반복문 실행에 따른 변수들의 변화는 다음과 같다.

i	j	arr[i]	arr[j]	result [0] [1] [2] [3] [4]
0				1 0 0 0 0
	0	77	77	
	1		32	
	2		10	
	3		99	
	4		50	2 0 0 0 0
1				2 1 0 0 0
	0	32	77	2 2 0 0 0
	1		32	
	2		10	
	3		99	2 3 0 0 0
	4		50	2 4 0 0 0
2				2 4 1 0 0
	0	10	77	2 4 2 0 0
	1		32	2 4 3 0 0
	2		10	
	3		99	2 4 4 0 0
	4		50	2 4 5 0 0
3		99		2 4 5 1 0
	0		77	
	1		32	
	2		10	

i	j	arr[i]	arr[j]	result [0] [1] [2] [3] [4]
	3		99	
	4		50	
4		50		2 4 5 1 1
	0		77	2 4 5 1 2
	1		32	
	2		10	
	3		99	
	4		50	2 4 5 1 3
5				

⑧ 반복 변수 k가 0부터 1씩 증가하면서 5보다 작은 동안 ⑨번을 반복 수행한다.

⑨ result[k]의 값을 출력한다.

결과 24513

8. 다음 Java로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. [21-1](#)

```
public class Test {
    public static void main(String[] args) {
        int aa[][] = { {45, 50, 75},
                        {89} };
        System.out.println(aa[0].length);
        System.out.println(aa[1].length);
        System.out.println(aa[0][0]);
        System.out.println(aa[0][1]);
        System.out.println(aa[1][0]);
    }
}
```

답 :

3
1
45
50
89

[해설]

```
public class Test {
    public static void main(String[] args) {
        ① int aa[][] = { {45, 50, 75},
                        {89} };
        ② System.out.println(aa[0].length);
        ③ System.out.println(aa[1].length);
        ④ System.out.println(aa[0][0]);
        ⑤ System.out.println(aa[0][1]);
        ⑥ System.out.println(aa[1][0]);
    }
}
```

}

- ❶ 4개의 요소를 갖는 정수형 2차원 배열 aa를 선언한다.

배열 aa

aa[0][0]	aa[0][1]	aa[0][2]
45	50	75
89		
aa[1][0]		

- ❷ aa[0] 배열의 길이 3을 출력하고 커서를 다음 줄의 처음으로 옮긴다.

• **length** : length는 배열 클래스의 속성으로, 배열 요소의 개수가 저장되어 있다. aa[0] 배열은 2차원 배열 aa의 첫 번째 행을 가리키는 것이므로, aa[0].length는 첫 번째 행의 요소 수 **3**이다.

- ❸ aa[1] 배열의 길이 1을 출력하고 커서를 다음 줄의 처음으로 옮긴다.

• aa[1] 배열은 2차원 배열 aa의 두 번째 행을 가리키는 것이므로, aa[1].length는 두 번째 행의 요소 수 **1**이다.

- ❹ aa[0][0]의 값 **45**를 출력하고 커서를 다음 줄의 처음으로 옮긴다.

- ❺ aa[0][1]의 값 **50**을 출력하고 커서를 다음 줄의 처음으로 옮긴다.

- ❻ aa[1][0]의 값 **89**를 출력하고 커서를 다음 줄의 처음으로 옮긴다.

프로그래밍 - 제어 2문제 001로

1. 다음 C언어로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. 22-3

```
#include <stdio.h>
main() {
    int s, el = 0;
    for (int i = 6; i <= 30; i++) {
        s = 0;
        for (int j = 1; j <= i / 2; j++)
            if (i % j == 0)
                s = s + j;
        if (s == i)
            el++;
    }
    printf("%d", el);
}
```

답: 2

[해설]

어떤 정수의 약수 중 자신을 제외한 약수를 모두 합하면 자신과 같아지는 수가 있다. 예를 들어 6의 약수 1, 2, 3, 6 중 6을 제외한 1, 2, 3을 더하면 6이 되어 자신과 같아진다(완전수).

완전수: 자기 자신을 제외한 약수들의 합이 자기 자신이 되는 수

i	j						s						el
6	1	1	1	1			0	0	0	0			0
7	2	2	2	2			1	1	1	1			1
8	3	3	3	3			3		3	4			
9	4	4	4	4			6		7				

i	j					s					el
			5	5							

```
#include <stdio.h>
main() {
    ① int s, el = 0;
    ② for (int i = 6; i <= 30; i++) {
    ③     s = 0;
    ④     for (int j = 1; j <= i / 2;
j++)
    ⑤         if (i % j == 0)
    ⑥             s = s + j;
    ⑦     if (s == i)
    ⑧         el++;
    }
    ⑨ printf("%d", el);
}
```

- ① 정수형 변수 s, el을 선언하고, el을 0으로 초기화한다.
- ② 반복 변수 i가 6부터 1씩 증가하면서 30보다 작거나 같은 동안 ③~⑧번을 반복 수행한다.
- ③ s에 0을 저장한다.
- ④ 반복 변수 j가 1부터 1씩 증가하면서 i/2보다 작거나 같은 동안 ⑤, ⑥번을 반복 수행한다.
- ⑤ i를 j로 나눈 나머지가 0이면 ⑥번으로 이동하고, 아니면 반복문의 처음인 ④번으로 이동한다.
- ⑥ s에 j의 값을 누적시킨다. 구해진 약수를 더하는 과정이다.
- ⑦ s와 i의 값이 같으면 약수를 모두 더한 값과 자신이 같은 수를 찾은 것이므로, ⑧번으로 이동하고, 아니면 현재 반복문의 처음인 ②번으로 이동한다.
- ⑧ 'el = el + 1;'과 동일하다. 약수를 모두 더한 값과 자신이 같은 수의 개수를 누적시키는 과정이다.

• 6~30까지 수의 약수 중 자신을 제외한 값은 다음과 같다. (11, 13, 17, 19, 23, 29 소수는 제외)

6: 1 2 3	10: 1 2 5	12: 1 2 3 4 6
14: 1 2 7	15: 1 3 5	16: 1 2 4 8
18: 1 2 3 6 9	20: 1 2 4 5 10	21: 1 3 7
22: 1 2 11	24: 1 2 3 4 6 8 12	25: 1 5
26: 1 2 13	27: 1 3 9	28: 1 2 4 7 14
30: 1 2 3 5 6 10 15		

⑨ el의 값 2를 정수로 출력한다.

결과 2

2. 다음은 정수를 역순으로 출력하는 C언어 프로그램이다. 예를 들어 1234의 역순은 4321이다. 단, 1230처럼 0으로 끝나는 정수는 고려하지 않는다. 프로그램을 분석하여 괄호(①~③)에 들어갈 알맞은 연산자들을 쓰시오. [22-1](#)

```
#include <stdio.h>
int main() {
    int number = 1234;
    int div = 10, result = 0;

    while (number ( ① ) 0) {
        result = result * div;
        result = result + number ( ② ) div;
        number = number ( ③ ) div;
    }
    printf("%d", result);
}
```

답: ① != 또는 > ② % ③ /

[해설]

```
#include <stdio.h>
int main() {
  ❶ int number = 1234;
  ❷ int div = 10, result = 0;

  ❸ while (number != 0) {
    ❹ result = result * div;
    ❺ result = result + number %
div;
    ❻ number = number / div;
  }
  ❼ printf("%d", result);
}
```

- ❶ 정수형 변수 number를 선언하고 1234로 초기화한다.
- ❷ 정수형 변수 div와 result를 선언하고 각각 10과 0으로 초기화한다.
- ❸ number가 0이 아닐 동안(또는 0보다 클 동안) ❹~❻번을 반복 수행한다.
- ❹ 새로운 나머지를 더하기 전에 기존의 나머지가 저장된 result에 10을 곱해 자릿수를 올려 준다.
- ❺ 새로운 나머지(10으로 나눈 나머지)를 result에 더한다.
- ❻ 다음 나머지를 구하기 위해 number를 10으로 나눈다(몫 구하기).

• 반복문 실행에 따른 변수들의 변화는 다음과 같다.

number	div	result
1234	10	0
123		0
12		4
1		40
0		43
		430
		432
		4320
		4321

❼ result의 값 4321을 정수로 출력한다.

결과 4321

프로그래밍 - 포인터 5문제

1. 다음 C언어로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. [22-2](#)

```
#include <stdio.h>
int len(char* p);

int main( ) {
  char* p1 = "2022";
  char* p2 = "202207";
  int a = len(p1);
  int b = len(p2);
  printf("%d", a + b);
}

int len(char* p) {
  int r = 0;
  while (*p != '\0') {
    p++;
    r++;
  }
  return r;
}
```

[해설]

```
#include <stdio.h>

int len(char* p);          // len() 함수의 프로토타입 선언이다.

main( ) {
    1 char* p1 = "2022";
    2 char* p2 = "202207";
    3,10 int a = len(p1);
    11,18 int b = len(p2);
    19 printf("%d", a + b);
}

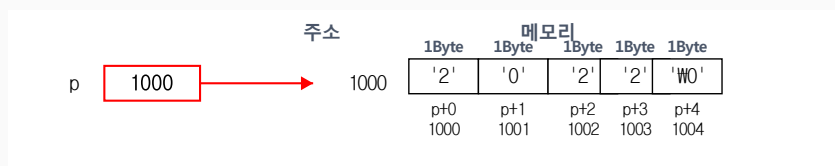
4,12 int len(char* p) {
    5,13 int r = 0;
    6,14 while (*p != '\0') {
    7,15     p++;
    8,16     r++;
    }
    9,17 return r;
}
```

모든 C 프로그램은 반드시 main() 함수에서 시작한다.

- ❶ 문자형 포인터 변수 p1을 선언하고 "2022"를 가리키는 주소로 초기화한다.
- ❷ 문자형 포인터 변수 p2를 선언하고 "202207"을 가리키는 주소로 초기화한다. 다음의 그림에서 p1과 p2가 할당된 공간의 주소는 임의로 정한 것이며, 이해를 돕기 위해 10진수로 표현했다.

주소	메모리						
p1 1000	'2'	'0'	'2'	'2'	'\0'		
p2 2000	'2'	'0'	'2'	'2'	'0'	'7'	'\0'

- ❸ 정수형 변수 a를 선언하고 p1의 값을 인수로 len() 함수를 호출한 후 돌려받은 값으로 초기화한다.
- ❹ 정수를 반환하는 len() 함수의 시작점이다. ❸번에서 전달받은 p1의 값을 문자형 포인터 변수 p가 받는다.



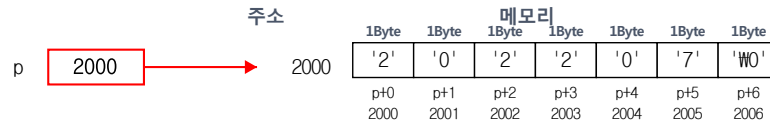
- ❺ 정수형 변수 r을 선언하고 0으로 초기화한다. r은 문자의 수를 카운트하기 위한 변수이다.
- ❻ p가 가리키는 곳의 값이 '\0'이 아닌 동안 ❶, ❷번을 반복 수행한다.
- ❼ 'p = p + 1;'과 동일하다. '\0'이 나올 때까지, 즉 문자열의 끝을 찾을 때까지 주소를 1씩 증가시킨다.
- ❽ 'r = r + 1;'과 동일하다. "2022"에 포함된 문자의 개수를 센다.

※ 반복문 실행에 따른 변수들의 변화는 다음과 같다.

p	r	*p
1000	0	'2'

p	r	*p
1001	1	'0'
1002	2	'2'
1003	3	'2'
1004	4	'\0'

- 9 함수를 호출했던 10번으로 r의 값, 즉 문자의 개수 4를 반환한다.
- 10 a에는 4가 저장된다.
- 11 정수형 변수 b를 선언하고 p2의 값을 인수로 len() 함수를 호출한 후 돌려받은 값으로 초기화한다.
- 12 정수를 반환하는 len() 함수의 시작점이다. 11번에서 전달받은 p2의 값을 p가 받는다.



- 13 정수형 변수 r을 선언하고 0으로 초기화한다.
- 14 p가 가리키는 곳의 값이 'W0'이 아닌 동안 15, 16번을 반복 수행한다.
- 15 'p = p + 1;'과 동일하다. 'W0'이 나올 때까지 문자열이 저장된 주소를 1씩 증가시킨다.
- 16 'r = r + 1;'과 동일하다. "202207"에 포함된 문자의 개수를 센다.

※ 반복문 실행에 따른 변수들의 변화는 다음과 같다.

p	r	*p
2000	0	'2'
2001	1	'0'
2002	2	'2'
2003	3	'2'
2004	4	'0'
2005	5	'7'
2006	6	'\0'

- 17 함수를 호출했던 18번으로 r의 값 6을 반환한다.
- 18 b에는 6이 저장된다.
- 19 4+6의 결과인 10을 정수로 출력한다.

결과 10

2. 다음 C언어로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. 22-2

```
#include <stdio.h>
int main( ) {
    int a[4] = { 0, 2, 4, 8 };
    int b[3];
    int* p;
    int sum = 0;
    for (int i = 1; i < 4; i++) {
        p = a + i;
```

```

        b[i - 1] = *p - a[i - 1];
        sum = sum + b[i - 1] + a[i];
    }
    printf("%d", sum);
}

```

답: 22

[해설]

```

#include <stdio.h>
int main() {
    1   int a[4] = { 0, 2, 4, 8 };
    2   int b[3];
    3   int* p;
    4   int sum = 0;
    5   for (int i = 1; i < 4; i++) {
    6       p = a + i;
    7       b[i - 1] = *p - a[i - 1];
    8       sum = sum + b[i - 1] + a[i];
    }
    9   printf("%d", sum);
}

```

- ❶ 4개의 요소를 갖는 정수형 배열 a를 선언하고 초기화한다.

a	[0]	[1]	[2]	[3]
	0	2	4	8

- ❷ 3개의 요소를 갖는 정수형 배열 b를 선언한다.

b	[0]	[1]	[2]

- ❸ 정수형 포인터 변수 p를 선언한다.

- ❹ 정수형 변수 sum을 선언하고 0으로 초기화한다.

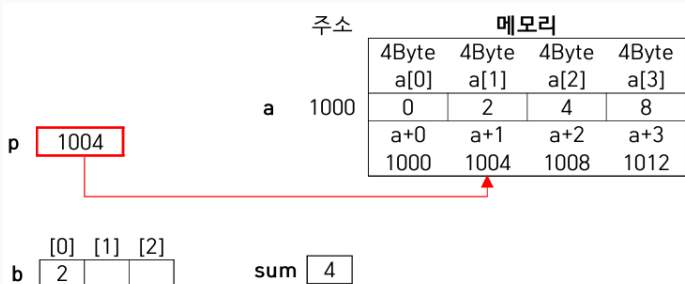
- ❺ 반복 변수 i가 1부터 1씩 증가하면서 4보다 작은 동안 ❹~❸번을 반복 수행한다.

첫 번째 반복 (i = 1)

- ❹ p에 a+1의 주소를 저장한다. p에 a 배열의 두 번째 요소인 a[1]의 주소를 저장한다.

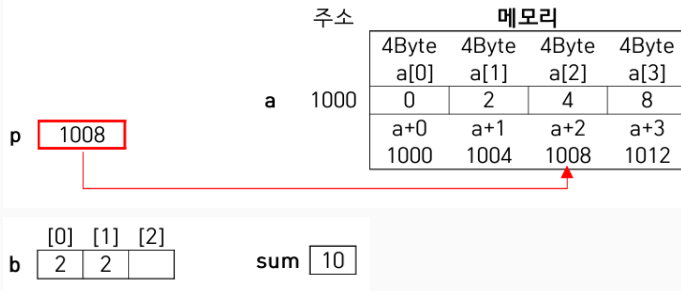
- ❺ b[0]에 p가 가리키는 곳의 값 2에서 a[0]의 값 0을 뺀 값 2를 저장한다.

- ❸ sum에 b[0]의 값 2와 a[1]의 값 2를 더한 값 4를 누적한다.



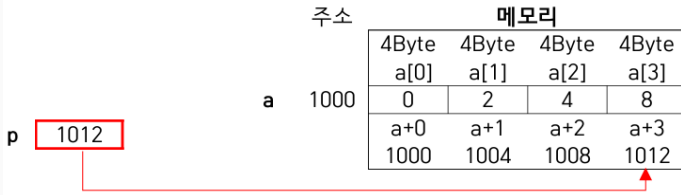
두 번째 반복 (i = 2)

- p에 a+2의 주소인 1008을 저장한다.
- b[1]에 p가 가리키는 곳의 값 4에서 a[1]의 값 2를 뺀 값인 2를 저장한다.
- sum에 b[1]의 값 2와 a[2]의 값 4를 더한 값 6을 누적한다.

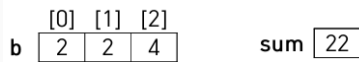


세 번째 반복 (i = 3)

- p에 a+3의 주소인 1012를 저장한다.



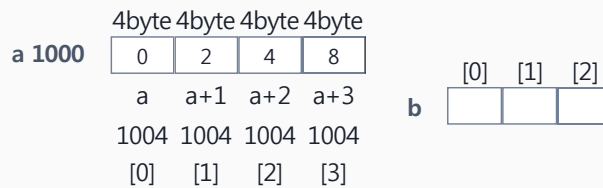
- b[2]에 p가 가리키는 곳의 값 8에서 a[2]의 값 4를 뺀 값인 4를 저장한다.
- sum에 b[2]의 값 4와 a[3]의 값 8을 더한 값 12를 누적한다.



- i가 4가 되면서 for문을 빠져나가 9 번으로 이동한다.
- 9 sum의 값 22를 정수로 출력한다.

결과 22

추가 설명



p	i	sum
		0

p	i	sum
1004	1	0
		4

p	i	sum
1004	1	0
1008	2	4
		10

p	i	sum
1004	1	0
1008	2	4
1012	3	10
		22

b[i - 1] = *p - a[i - 1];

sum = sum + b[i - 1] + a[i];

3. 다음 C 언어로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. 21-3

```
#include <stdio.h>
int main() {
    int* array[3];
    int a = 12, b = 24, c = 36;
    array[0] = &a;
    array[1] = &b;
    array[2] = &c;
    printf("%d", *array[1] + **array + 1);
}
```

답: 37

[해설]

```
#include <stdio.h>

int main() {
  1   int* array[3];
  2   int a = 12, b = 24, c = 36;
  3   array[0] = &a;
  4   array[1] = &b;
  5   array[2] = &c;
  6   printf("%d", *array[1] + **array + 1);
}
```

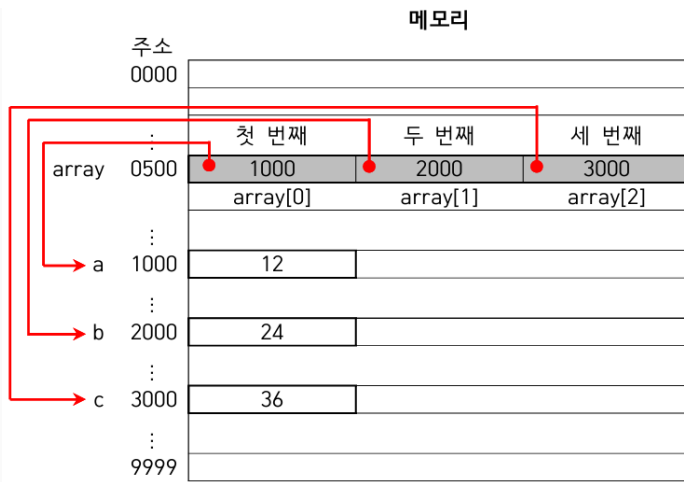
- ❶ 3개의 요소를 갖는 정수형 포인터 배열 array를 선언한다. 주소는 임의로 정한 것이다.

메모리			
주소	0000		
	⋮	첫 번째	두 번째
array	0500	array[0]	array[1]
	⋮		
	1000		
	⋮		
	2000		
	⋮		
	3000		
	⋮		
	9999		

- ❷ 정수형 변수 a, b, c에 각각 12, 24, 36을 저장한다.

메모리			
주소	0000		
	⋮	첫 번째	두 번째
array	0500	array[0]	array[1]
	⋮		
a	1000	12	
	⋮		
b	2000	24	
	⋮		
c	3000	36	
	⋮		
	9999		

- ❸~❺ array[0]에 a의 주소를, array[1]에 b의 주소를, array[2]에 c의 주소를 저장한다.



⑥ array[1]이 가리키는 곳의 값과 *array가 가리키는 곳의 값과 1을 더한 후 정수로 출력한다.

- ***array[1]** : array[1]에는 2000이 저장되어 있고 2000이 가리키는 곳의 값은 24이다.
- ****array**
 - array : 배열의 이름만 지정하면 배열의 첫 번째 요소의 주소인 &array[0], 즉 500을 의미한다.
 - *array : array는 500이고 500이 가리키는 곳의 값은 1000이다.
 - **array : *array는 1000이고 1000이 가리키는 곳의 값은 12이다.

∴ $24 + 12 + 1 = 37$

결과 **37**

4. 다음 C언어로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. [21-2](#)

```
#include <stdio.h>
int main() {
    int ary[3];
    int s = 0;
    *(ary + 0) = 1;
    ary[1] = *(ary + 0) + 2;
    ary[2] = *ary + 3;
    for (int i = 0; i < 3; i++) {
        s = s + ary[i];
    }
    printf("%d", s);
}
```

답: **8**

[해설]

```
#include <stdio.h>
int main() {
    ① int ary[3];
    ② int s = 0;
    ③ *(ary + 0) = 1;
    ④ ary[1] = *(ary + 0) + 2;
    ⑤ ary[2] = *ary + 3;
    ⑥ for (int i = 0; i < 3; i++) {
    ⑦     s = s + ary[i];
    }
```

```
8 printf("%d", s);
}
```

1 3개의 요소를 갖는 정수형 배열 ary를 선언한다.

메모리			
0000 0000	:	ary[0]	ary[1]
0015 FC40 → 0015 FC40	:		
:	:	ary	ary+1
FFFF FFFF	:	0015 FC40	0015 FC41
		0015 FC42	

2 정수형 변수 s를 선언하고 0으로 초기화한다.

3 ary+0이 가리키는 곳에 1을 저장한다. (ary[0] = 1 대입과 동일)

메모리			
0000 0000	:	ary[0]	ary[1]
0015 FC40 → 0015 FC40	:	1	
:	:	ary	ary+1
FFFF FFFF	:	0015 FC40	0015 FC41
		0015 FC42	

4 ary[1]에 ary+0이 가리키는 곳의 값 1에 2를 더한 값을 저장한다. (ary[1] = 3)

메모리			
0000 0000	:	ary[0]	ary[1]
0015 FC40 → 0015 FC40	:	1	3
:	:	ary+0	ary+1
FFFF FFFF	:	0015 FC40	0015 FC41
		0015 FC42	

5 ary[2]에 ary가 가리키는 곳의 값 1에 3을 더한 값을 저장한다. (ary[2] = 4)

메모리			
0000 0000	:	ary[0]	ary[1]
0015 FC40 → 0015 FC40	:	1	3
:	:	ary	ary+1
FFFF FFFF	:	0015 FC40	0015 FC41
		0015 FC42	

6 반복 변수 i가 0에서 시작하여 1씩 증가하면서 3보다 작은 동안 7번을 반복 수행한다.

7 s에 ary[i]의 값을 누적한다.

※ 반복문 실행에 따른 변수들의 변화는 다음과 같다.

i	ary[i]	s
0	1	1
1	3	4
2	4	8
3		

결과 8

5. 다음 C 언어로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. [20-4](#)

```
#include <stdio.h>
main() {
    char* p = "KOREA";
    printf("%s\n", p);
    printf("%s\n", p + 3);
    printf("%c\n", *p);
    printf("%c\n", *(p + 3));
    printf("%c\n", *p + 2);
}
```

```
}
```

답: KOREA
EA
K
E
M

[해설]

```
#include <stdio.h>

main() {
    1 char* p = "KOREA";
    2 printf("%s\n", p);
    3 printf("%s\n", p + 3);
    4 printf("%c\n", *p);
    5 printf("%c\n", *(p + 3));
    6 printf("%c\n", *p + 2);
}
```

- ① 문자형 포인터 변수 p를 선언하고, 문자열 "KOREA"가 저장된 곳의 주소를 저장한다.

메모리						
0000 0000						
p 0015 FC40 → 0015 FC40	1Byte	1Byte	1Byte	1Byte	1Byte	1Byte
	'K'	'O'	'R'	'E'	'A'	'\0'
	p	p+1	p+2	p+3	p+4	p+5
	0015 FC40	0015 FC41	0015 FC42	0015 FC43	0015 FC44	0015 FC45
FFFF FFFF						

※ 문자열을 저장하는 경우 문자열의 끝을 의미하는 널 문자('\0')가 추가로 저장되며, 출력시 널 문자는 표시되지 않습니다.

- ② p의 위치부터 문자열의 끝('\0')까지 모든 문자를 하나의 문자열로 출력하고 커서를 다음 줄의 처음으로 옮긴다. 결과: KOREA

KOREA

- ③ p+3의 위치부터 문자열의 끝('\0')까지 모든 문자를 하나의 문자열로 출력하고 커서를 다음 줄의 처음으로 옮긴다. 결과: EA

- ④ p가 가리키는 곳의 문자를 출력하고 커서를 다음 줄의 처음으로 옮긴다. 결과: K

- ⑤ p+3이 가리키는 곳의 문자를 출력하고 커서를 다음 줄의 처음으로 옮긴다. 결과: E

- ⑥ p가 가리키는 곳의 문자에 2를 더한 값을 출력하고 커서를 다음 줄의 처음으로 옮긴다.

※ "KOREA"라는 문자열이 메모리에 저장될 때 문자로 저장되는 것이 아니라 해당 문자의 아스키 코드 값이 저장됩니다. 즉 'K'는 'K'에 해당하는 아스키 코드 값인 75가 저장됩니다. 그러므로 p가 가리키는 곳의 값인 75에 2를 더한 77을 문자로 출력한다는 것은 알파벳 순서상 'K'의 다다음 문자 'M'을 출력한다. 결과: M

프로그래밍 - 구조체 5문제 001로

1. 다음 C언어로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. 22-2

```
#include <stdio.h>
struct A {
    int n;
    int g;
};
main( ) {
    struct A st[2];
    for (int i = 0; i < 2; i++) {
        st[i].n = i;
        st[i].g = i + 1;
    }
}
```

```

    }
    printf("%d", st[0].n + st[1].g);
}

```

답: 2

[해설]

```

#include <stdio.h>

struct A {
    int n;           // A의 멤버로 정수형 변수 n을 선언한다.
    int g;           // A의 멤버로 정수형 변수 g를 선언한다.
};

main() {
    ❶ struct A st[2];
    ❷ for (int i = 0; i < 2; i++) {
    ❸     st[i].n = i;
    ❹     st[i].g = i + 1;
    }
    ❺ printf("%d", st[0].n + st[1].g);
}

```

모든 C 프로그램은 반드시 main() 함수에서 시작한다.

❶ A 구조체 형태로 배열 st를 선언한다.

	int n	int g
st[0]		
st[1]		

❷ 반복 변수 i가 0부터 1씩 증가하면서 2보다 작은 동안 ❸, ❹번을 반복 수행한다.

❸ st[i].n에 i의 값을 저장한다.

❹ st[i].g에 i+1의 값을 저장한다.

※ 반복문 실행에 따른 변수들의 변화는 다음과 같다.

• 1회전 (i = 0)

	int n	int g
st[0]	0	1
st[1]		

• 2회전 (i = 1)

	int n	int g
st[0]	0	1
st[1]	1	2

❺ i가 2가 되면서 for문을 빠져나가 ❺번으로 이동한다.

❺ 0+2의 결과인 2를 정수로 출력한다.

2. 다음 C언어로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. 21-1

```

#include <stdio.h>
main() {
    struct insa {
        char name[10];
    };
}

```

```

    int age;
} a[] = { {"Kim", 28, "Lee", 38, "Park", 42, "Choi", 31 } };
struct insa* p;
p = a;
p++;
printf("%s\n", p->name);
printf("%d\n", p->age);
}

```

답:

Lee

38

[해설]

```

#include <stdio.h>
main() {
1   struct insa {
2       char name[10];
3       int age;
4   } a[] = { "Kim", 28, "Lee", 38, "Park", 42, "Choi", 31 };
5   struct insa* p;
6   p = a;
7   p++;
8   printf("%s\n", p->name);
9   printf("%d\n", p->age);
}

```

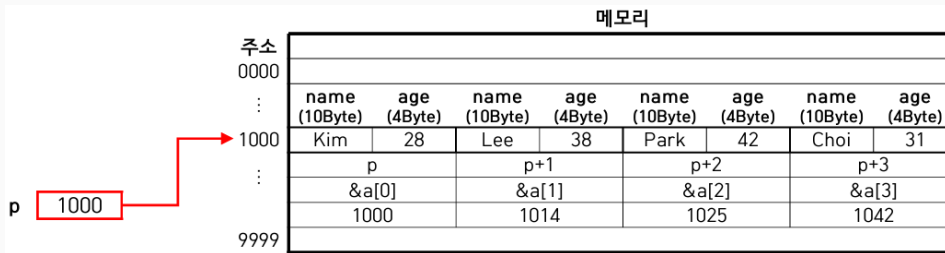
- 1 구조체 insa를 정의한다. 구조체의 멤버를 지정할 때는 [변수명].[멤버이름]으로 지정하지만, 포인터 변수를 이용해 구조체의 멤버를 지정할 때는 [변수명]->[멤버이름]으로 지정한다.
- 2 insa의 멤버로 10개의 요소를 갖는 문자형 배열 name을 선언한다.
- 3 insa의 멤버로 정수형 변수 age를 선언한다.
- 4 insa 구조체 형태로 a 배열을 선언하고 초기화한다. 배열의 크기를 지정하지 않았으므로 초기값으로 지정된 수만큼 배열의 요소가 만들어진다.

	char name[10]	int age
insa[0]	insa[0].name[0]~insa[0].name[9]	insa[0].age
insa[1]	insa[1].name[0]~insa[1].name[9]	insa[1].age
insa[2]	insa[2].name[0]~insa[2].name[9]	insa[2].age
insa[3]	insa[3].name[0]~insa[3].name[9]	insa[3].age

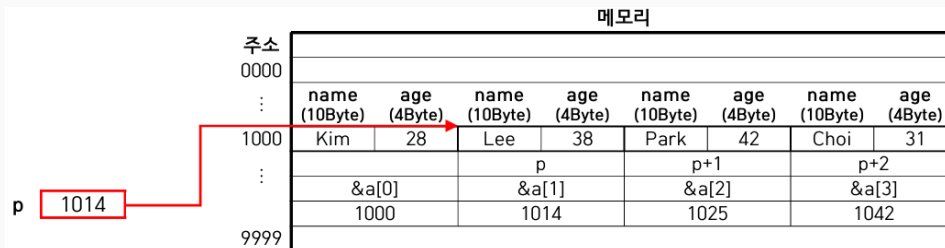
	char name[10]	int age
insa[0]	'K' 'i' 'm' '\0' :	28
insa[1]	'L' 'e' 'e' '\0' :	38
insa[2]	'P' 'a' 'r' 'k' '\0' :	42
insa[3]	'C' 'h' 'o' 'i' '\0' :	31

※ 문자열을 저장하는 경우 문자열의 끝을 의미하는 널 문자('\0')가 추가로 저장되며, 출력 시 널 문자는 표시되지 않습니다.

- 5 insa 구조체를 가리키는 포인터 변수 p를 선언한다.
- 6 p에 a를 지정한다. a는 배열명이므로 배열의 시작 위치인 a[0]의 주소가 p에 지정된다. 주소는 임의로 정한 것이다.



- 7 'p = p + 1;'과 동일하다. p의 값을 1 증가시킨다. 주소에 1을 더하는 것은 다음 자료를 가리키는 것을 의미하므로, p는 a[1]의 주소를 가리키게 된다.



- 8 p가 가리키는 곳의 멤버 name을 문자열 형태로 출력한다. 이어서 커서를 다음 줄의 처음으로 옮긴다.

결과 Lee

- 9 p가 가리키는 곳의 멤버 age를 정수형으로 출력한다. 이어서 커서를 다음 줄의 처음으로 옮긴다.

결과
Lee
38

3. 다음 C 언어로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. 21-3

```
#include <stdio.h>
struct jsu {
    char nae[12];
    int os, db, hab, hhab;
};

int main() {
    struct jsu st[3] = { {"데이터1", 95, 88}, {"데이터2", 84, 91}, {"데이터3", 86, 75} };
    struct jsu* p;
    p = &st[0];
    (p + 1)->hab = (p + 1)->os + (p + 2)->db;
    (p + 1)->hhab = (p + 1)->hab + p->os + p->db;
    printf("%d", (p + 1)->hab + (p + 1)->hhab);
}
```

답: 501

[해설]

```
#include <stdio.h>
(A) struct jsu {
    char nae[12];
    int os, db, hab, hhab;
};
```

```

int main() {
1   struct jsu st[3] = { {"데이터1", 95, 88}, {"데이터2", 84, 91}, {"데이터3", 86, 75} };
2   struct jsu* p;
3   p = &st[0];
4   (p + 1)->hab = (p + 1)->os + (p + 2)->db;
5   (p + 1)->hhab = (p + 1)->hab + p->os + p->db;
6   printf("%d", (p + 1)->hab + (p + 1)->hhab);
}

```

㉠ 구조체 jsu를 정의한다. 12개의 요소를 갖는 문자 배열 nae와 정수형 os, db, hab, hhab을 멤버로 갖는다.

```

struct jsu {
    char nae[12];
    int os, db, hab, hhab;
};

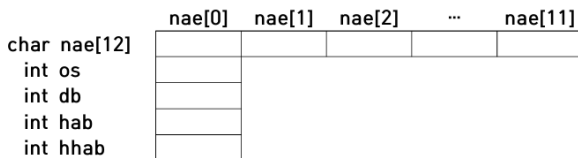
```

구조체 jsu를 정의한다. 구조체를 정의한다는 것은 int나 char 같은 자료형을 하나 만든다는 의미다. 구조체의 멤버를 지정할 때는 [변수명].[멤버이름]으로 지정하지만, 포인터 변수를 이용해 구조체의 멤버를 지정할 때는 [변수명]->[멤버이름]으로 지정한다.

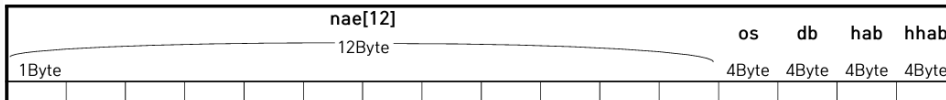
12개의 요소를 갖는 문자 배열 nae를 선언한다.

정수형 변수 os, db, hab, hhab를 선언한다.

㉠ 구조체 jsu의 구조



※ 위 구조체는 다음과 같이 메모리의 연속된 공간에 저장된 후 사용됩니다.



1 구조체 jsu 자료형으로 3개짜리 배열 st를 선언하고 초기화한다.

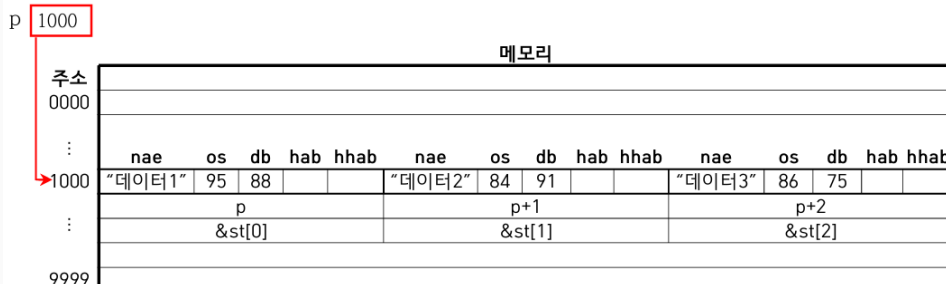
	char nae[12]	int os	int db	int hab	int hhab
st[0]	st[0].nae[0]~st[0].nae[11]	st[0].os	st[0].db	st[0].hab	st[0].hhab
st[1]	st[1].nae[0]~st[1].nae[11]	st[1].os	st[1].db	st[1].hab	st[1].hhab
st[2]	st[2].nae[0]~st[2].nae[11]	st[2].os	st[2].db	st[2].hab	st[2].hhab

	char nae[12]	int os	int db	int hab	int hhab
st[0]	‘데’ ‘이’ ‘터’ 1 ‘\0’	95	88		
st[1]	‘데’ ‘이’ ‘터’ 2 ‘\0’	84	91		
st[2]	‘데’ ‘이’ ‘터’ 3 ‘\0’	86	75		

※ 문자열을 저장하는 경우 문자열의 끝을 의미하는 널 문자('0')가 추가로 저장되며, 출력 시 널 문자는 표시되지 않습니다. 또한 영문 숫자는 1Byte, 한글은 2Byte를 차지합니다.

2 구조체 jsu의 포인터 변수 p를 선언한다.

3 p에 st 배열의 첫 번째 요소의 주소를 저장한다. 주소는 임의로 정한 것이다. (p = &st[0])



4 (p+1)->hab에 (p+1)->os 값(84)과 (p+2)->db 값(75)을 더한 후 저장한다. st[1].hab = 84 + 75 = 159가 저장된다.

메모리															
주소															
0000															
⋮															
1000	nae	os	db	hab	hhab	nae	os	db	hab	hhab	nae	os	db	hab	hhab
	“데이터1”	95	88			“데이터2”	84	91	159		“데이터3”	86	75		
⋮	p					p+1					p+2				
	&st[0]					&st[1]					&st[2]				
9999															

- 5 (p+1)->hhab에 (p+1)->hab 값(159)과 p->os 값(95) 및 p->db 값(88)을 모두 더한 후 저장한다. $st[1].hhab = 159 + 95 + 88 = 342$ 가 저장된다.

메모리															
주소															
0000															
⋮															
1000	nae	os	db	hab	hhab	nae	os	db	hab	hhab	nae	os	db	hab	hhab
	"데이터1"	95	88			"데이터2"	84	91	159	342	"데이터3"	86	75		
⋮	p					p+1					p+2				
	&st[0]					&st[1]					&st[2]				
9999															

- 6 (p+1)->hab 값(159)과 (p+1)->hhab 값(342)을 더한 후 정수로 출력한다.

결과 501

4. 다음은 C언어 코드이다. 출력 결과를 쓰시오. [수]

```
#include <stdio.h>
typedef struct {
    void (*s)(int a);
} Payment;
void card(int a) {
    printf("card:%d, ", a);
}
void cash(int a) {
    printf("cash: %d", a);
}
void set(Payment *p, void (*s)(int a)){ p->s = s; }
void execute(Payment *p, int a) { p->s(a); }
int main() {
    Payment c;

    set(&c, card);
    execute(&c, 10000);

    set(&c, cash);
    execute(&c, 5000);

    return 0;
}
```

답: card:10000, cash: 5000

[해설]

이 코드는 함수 포인터(Function Pointer)를 활용해서 상황에 따라 다른 함수를 동적으로 실행하는 방식을 보여주고 있습니다. 객체지향 프로그래밍의 다형성(Strategy Pattern)을 C 언어로 구현한 좋은 예시예요.

1. 첫 번째 실행 흐름 (카드 결제)

set(&c, card);를 통해 Payment 구조체 c 안에 있는 함수 포인터 s가 card 함수를 가리키도록 설정합니다.

execute(&c, 10000);을 실행하면 구조체 내부에 저장된 card(10000)이 호출됩니다.
card 함수 내부의 printf("card:%d, ", a);에 의해 card:10000, (뒤에 공백과 심표 포함)이 먼저 출력됩니다.
2. 두 번째 실행 흐름 (현금 결제)
set(&c, cash);를 통해 함수 포인터 s가 이번에는 cash 함수를 가리키도록 바꿉니다.
execute(&c, 5000);을 실행하면 변경된 cash(5000)이 호출됩니다.
cash 함수 내부의 printf("cash: %d", a);에 의해 cash: 5000이 이어 붙어서 출력됩니다.
따라서 최종적으로 줄바꿈 없이 card:10000, cash: 5000이 화면에 나타나게 됩니다.

5. 다음 C언어로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. 25-3

```
#include <stdio.h>
struct Test {
    int i;
    const char *g;
};

int main() {
    struct Test test[] = {{1, "AB"}, {2, "DC"}, {3, "EB"}};
    struct Test *p = &test[1];
    printf("%s", p->g + (p->i - 1));
    return 0;
}
```

답 : C

[해설]

구조체란?

- 배열 - 자료의 형과 크기가 동일한 변수의 모임
- 구조체 - 자료의 종류가 다른 변수의 모임

구조체 정의

```
struct sawon {
    char name[10];
    char jikwi[10];
    int pay;
};
```

구조체 변수 선언

```
struct sawon ansan;

struct sawon *seoul;

struct sawon ansan, *seoul;
```

```
struct sawon {
    char name[10];
    char jikwi[10];
    int pay;
} ansan, *seoul;
```

구조체 멤버 지정

```
ansan.name = "김한국";
ansan.jikwi = "대리";
ansan.pay = 4000000;
```

```
seoul->name = "홍길동";
seoul->jikwi = "과장";
seoul->pay = 5000000;
```

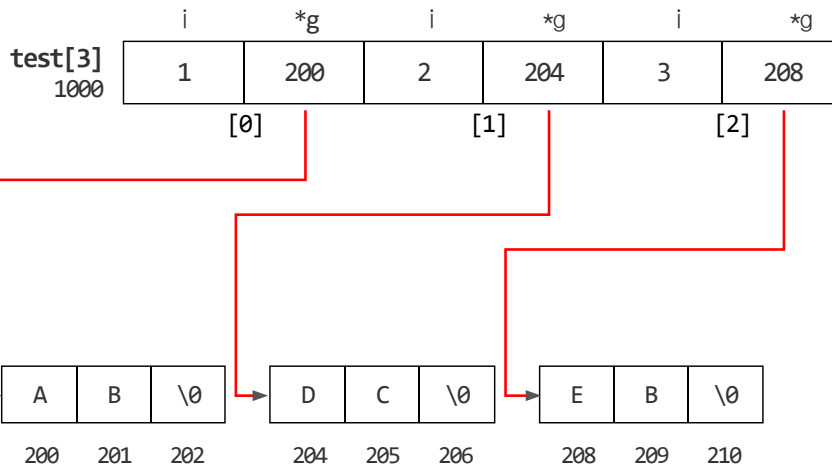
```
(*seoul).name = "홍길동";
(*ansan).jikwi = "과장";
(*ansan).pay = 5000000;
```

	ansan	*seoul
name[10]		
jikwi[10]		
pay		

구조체 Test

const로 선언되면 한 번 저장된 값을 변경할 수 없음.

i	*g



%s: 문자열의 시작 위치부터 문자열의 끝을 의미하는 널 문자(\0) 전까지를 출력함.
 (p->i - 1) 값은 1
 p->g + 1

프로그래밍 - 함수 46문제 001로

1. 다음 C언어로 구현된 프로그램을 분석하여 배열 <mines>의 각 칸에 들어갈 값을 쓰시오. 22-3

```
#include <stdio.h>
main( ) {
    int field[4][4] = { {0,1,0,1}, {0,0,0,1}, {1,1,1,0}, {0,1,1,1} };
    int mines[4][4] = { {0,0,0,0}, {0,0,0,0}, {0,0,0,0}, {0,0,0,0} };
    int w = 4, h = 4;
    for (int y = 0; y < h; y++) {
        for (int x = 0; x < w; x++) {
            if (field[y][x] == 0) continue;
            for (int j = y - 1; j <= y + 1; j++) {
                for (int i = x - 1; i <= x + 1; i++) {
                    if (chkover(w, h, j, i) == 1)
                        mines[j][i] += 1;
                }
            }
        }
    }
}

int chkover(int w, int h, int j, int i) {
    if (i >= 0 && i < w && j >= 0 && j < h) return 1;
    return 0;
}
```

배열 <field>				배열 <mines> (답)			
0	1	0	1	1	1	3	2
0	0	0	1	3	4	5	3
1	1	1	0	3	5	6	4
0	1	1	1	3	5	5	3

[해설]

[알고리즘의 이해]

문제의 코드는 배열 field의 요소가 0이 아닌, 즉 1인 경우, 배열 mines에서 해당 위치를 중심으로 3행 3열의 범위에 1을 더하는 프로그램입니다.

- w와 h는 배열의 행과 열의 길이가 저장된 변수입니다.
- y와 x는 배열 field의 행과 열 위치를 지정해 주는 변수입니다.
- j와 i는 배열 mines에서 1을 더할 범위의 행과 열 위치를 지정해 주는 변수입니다.
- **chkover()** 함수는 j와 i가 배열의 크기를 벗어나는지 검사하는 함수입니다. 벗어나다고 판단되면 0을 반환하여 해당 위치에는 1을 더하지 않도록 합니다.

※ 반복문 수행에 따른 배열 mines에 저장되는 값의 변화는 다음과 같습니다.

- 배열 field에서 1의 위치

배열 <field>

0	1	0	1
0	0	0	1
1	1	1	0
0	1	1	1

- 배열 field의 요소가 1인 위치에 대한 배열 mines의 값 변화

배열 <mines>

+1	+1	+1	
+1	+1	+1	

배열 <mines>

1	1	1+1	+1
1	1	1+1	+1

배열 <mines>

1	1	2+1	1+1
1	1	2+1	1+1
		+1	+1

배열 <mines>

	1	1	3	2
	1+1	1+1	3	2
	+1	+1	1	1
	+1	+1		

배열 <mines>

1	1	3	2
2+1	2+1	3+1	2
1+1	1+1	1+1	1
1+1	1+1	+1	

배열 <mines>

1	1	3	2
3	3+1	4+1	2+1
2	2+1	2+1	1+1
2	2+1	1+1	+1

배열 <mines>

1	1	3	2
3	4	5	3
2+1	3+1	3+1	2
2+1	3+1	2+1	1

배열 <mines>

1	1	3	2
3	4	5	3
3	4+1	4+1	2+1
3	4+1	3+1	1+1

배열 <mines>

1	1	3	2
3	4	5	3
3	5	5+1	3+1
3	5	4+1	2+1

배열 <mines>

1	1	3	2
3	4	5	3
3	5	6	4
3	5	5	3

모든 C 프로그램은 반드시 main() 함수에서 시작한다.

- ❶ 4행 4열의 2차원 배열 field를 선언하고 초기화한다.

배열 <field>

0	1	0	1
0	0	0	1
1	1	1	0
0	1	1	1

- ❷ 4행 4열의 2차원 배열 mines를 선언하고 초기화한다.

배열 <mines>

0	0	0	0
0	0	0	0
0	0	0	0
0	0	0	0

- ❸ 정수형 변수 w와 h를 선언하고, 각각 4로 초기화한다.
- ❹ 반복 변수 y가 0부터 1씩 증가하면서 h의 값 4보다 작은 동안 ❺~❽번을 반복 수행한다.
- ❺ 반복 변수 x가 0부터 1씩 증가하면서 w의 값 4보다 작은 동안 ❻~❿번을 반복 수행한다.
- ❻ field[y][x]의 값이 0이면 ❺번으로 제어를 넘겨 다음 반복을 수행하고, 0이 아니면 ❼번으로 이동한다.
- ❼ 반복 변수 j가 y-1부터 1씩 증가하면서 y+1보다 작거나 같은 동안 ❸~❿번을 반복 수행한다.
- ❽ 반복 변수 i가 x-1부터 1씩 증가하면서 x+1보다 작거나 같은 동안 ❹~❿번을 반복 수행한다.
- ❹ w, h, j, i의 값을 인수로 chkover() 함수를 호출한 결과가 1이면 ❿번으로 이동하고, 아니면 ❸번으로 제어를 넘긴다.
- ❿ 정수를 반환하는 chkover() 함수의 시작점이다. ❹번에서 전달받은 인수를 w, h, j, i가 받는다.
- ⓫ i와 j가 0보다 크거나 같고, w와 h의 값인 4보다 작으면 함수를 호출했던 ❸번으로 1을 반환하며 함수를 종료하고, 아니면 ❿번으로 이동한다.
- ⓫ 함수를 호출했던 ❸번으로 0을 반환하며 함수를 종료한다.
- ⓫ ❸번에서 1을 반환받았으면 ❿번으로, ❿번에서 0을 반환받았으면 ❸번으로 이동한다.
- ❿ 'mines[j][i] = mines[j][i] + 1'과 동일하다. mines[j][i]의 값을 1 누적시킨다.

2. 다음 JAVA로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. 22-3

```
public class Test {
    static int[] mkarr( ) {
        int[] tmpArr = new int[4];
        for (int i = 0; i < tmpArr.length; i++)
            tmpArr[i] = i;
        return tmpArr;
    }
    public static void main(String[] args) {
        int[] arr;
        arr = mkarr( );
        for (int i = 0; i < arr.length; i++)
            System.out.print(arr[i]);
    }
}
```

[해설]

```

public class Test {
    3    static int[] mkarr( ) {
    4        int[] tmpArr = new int[4];
    5        for (int i = 0; i < tmpArr.length; i++)
    6            tmpArr[i] = i;
    7        return tmpArr;
    }

    public static void main(String[] args) {
    1        int[] arr;
    28       arr = mkarr( );
    9        for (int i = 0; i < arr.length; i++)
    10           System.out.print(arr[i]);
    }
}

```

모든 Java 프로그램은 반드시 main() 함수에서 시작한다.

- ① 정수형 배열 arr을 선언한다. 배열의 요소가 생략되었으므로 배열의 위치를 저장하는 arr만이 메모리에 생성된다.

arr

※ 배열 arr에 4개의 요소(1, 2, 3, 4)를 임의로 초기화하여 선언하면 다음과 같습니다.

(다음 그림에서 arr에 저장된 주소는 임의로 정한 것이며, 이해를 돕기 위해 10진수로 표현했습니다.)



※ ①번과 같이 요소의 값과 개수를 생략하고 선언만 했다는 것은 arr이 위 그림의 2000과 같은 메모리를 가리키는 주소를 갖지 않은 채 선언만 되었다는 의미입니다. arr은 이후 ⑦번 과정에서 정수형 배열의 주소를 전달받아 위와 같이 일반적인 형태를 갖추게 됩니다.

- ② mkarr() 메소드를 호출한 후 돌려받은 값을 arr에 저장한다.
- ③ 정수형 배열을 반환하는 mkarr() 메소드의 시작점이다.
- ④ 4개의 요소를 갖는 정수형 배열 tmpArr을 선언한다.

	[0]	[1]	[2]	[3]
tmpArr	0	0	0	0

※ Java에서는 배열을 선언하고 초기화하지 않으면 배열의 모든 요소가 0으로 초기화됩니다.

- ⑤ 반복 변수 i가 0부터 1씩 증가하면서 배열 tmpArr의 길이 4보다 작은 동안 ⑥번을 반복 수행한다.

• length : 배열 클래스의 속성으로 배열 요소의 개수가 저장되어 있음

- ⑥ tmpArr[i]에 i의 값을 저장한다.

※ 반복문 실행에 따른 변수들의 변화는 다음과 같다.

i	tmpArr			
	[0]	[1]	[2]	[3]
0	0	0	0	0
1	0	1	0	0
2	0	1	2	0
3	0	1	2	3

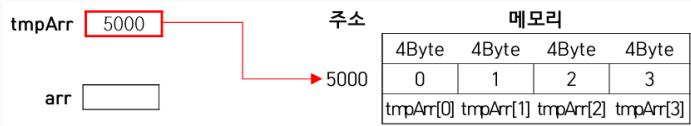
i	tmpArr [0] [1] [2] [3]
4	

⑦ 배열 tmpArr을 반환한다. 배열의 이름을 반환하면 배열의 시작 주소가 반환된다.

⑧ ⑦번에서 전달받은 배열의 시작 주소를 배열 arr에 저장한다.

※ 배열 tmpArr은 ③~⑥번 과정을 거치면서 다음과 같은 형태를 갖추게 됩니다.

(다음 그림에서 tmpArr이 저장된 주소는 임의로 정한 것이며, 이해를 돕기 위해 10진수로 표현했습니다.)



※ tmpArr이 가진 주소 5000이 ⑦~⑧번에서 return을 통해 arr에 반환되면서 arr은 tmpArr과 같은 주소를 갖는 배열이 됩니다.



⑨ 반복 변수 i가 0부터 1씩 증가하면서 배열 arr의 길이 4보다 작은 동안 ⑩번을 반복 수행한다.

⑩ arr[i]의 값을 출력한다.

※ 반복문 실행에 따른 변수들의 변화는 다음과 같다.

i	arr[i]	출력
0	0	0
1	1	01
2	2	012
3	3	0123
4		

결과 0123

3. 다음 C언어로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. 22-1

```
#include <stdio.h>
int isPrime(int number) {
    for (int i = 2; i < number; i++) {
        if (number % i == 0) return 0;
    }
    return 1;
}

int main() {
    int number = 13195;
    int max_div = 0;
    for (int i = 2; i < number; i++) {
        if (isPrime(i) == 1 && number % i == 0) max_div = i;
    }
    printf("%d", max_div);
}
```

답: 29

[해설]

```

#include <stdio.h>

5 int isPrime(int number) {
6     for (int i = 2; i < number; i++)
7         if (number % i == 0) return 0;
8     return 1;
9 }

int main() {
1 int number = 13195;
2 int max_div = 0;
3 for (int i = 2; i < number; i++)
4 9     if (isPrime(i) == 1 && number % i == 0) max_div = i;
10 printf("%d", max_div);
}

```

모든 C 프로그램은 반드시 main() 함수에서 시작한다.

- 1 정수형 변수 number를 선언하고 13195로 초기화한다.
- 2 정수형 변수 max_div를 선언하고 0으로 초기화한다.
- 3 반복 변수 i가 2부터 1씩 증가하면서 number보다 작은 동안 4번을 반복 수행한다.

첫 번째 반복

- 4 i의 값 2를 인수로 isPrime을 호출한 결과가 1이고 number를 i로 나눈 나머지가 0이면 max_div에 i의 값을 저장한다.
- 5 정수를 반환하는 isPrime() 함수의 시작점이다. 4번에서 전달받은 2를 number가 받는다.
- 6 반복 변수 i가 2부터 1씩 증가하면서 2보다 작은 동안 7번을 반복 수행한다. i의 값 2는 2보다 작지 않으므로 7번을 수행하지 않고 8번으로 이동한다.
- 8 1을 반환하면서 함수를 호출했던 9번으로 이동한다.
- 9 8번에서 돌려받은 값은 1이지만, number의 값 13195를 i의 값 2로 나눈 나머지는 1이므로 max_div = i를 수행하지 않고 3번으로 돌아가 i의 값을 1 증가시킨다.

두 번째 반복

- 4 i의 값 3을 인수로 isPrime을 호출한 결과가 1이고 number를 i로 나눈 나머지가 0이면 max_div에 i의 값을 저장한다.
- 5 4번에서 전달받은 3을 number가 받는다.
- 6 반복 변수 i가 2부터 1씩 증가하면서 3보다 작은 동안 7번을 반복 수행한다.
- 7 3을 i로 나눈 나머지가 0이면 0을 반환하면서 함수를 호출했던 9번으로 이동한다.

6~7번 반복문 실행에 따른 변수들의 변화는 다음과 같다.

number	i
3	2
	3

- 8 1을 반환하고 함수를 호출했던 9번으로 이동한다.
- 9 8번에서 돌려받은 값은 1이지만, number의 값 13195를 i의 값 3으로 나눈 나머지는 1이므로 max_div = i를 수행하지 않고 3번으로 돌아가 i의 값을 1 증가시킨다.

세 번째 반복

- 4 i의 값 4를 인수로 isPrime을 호출한 결과가 1이고 number를 i로 나눈 나머지가 0이면 max_div에 i의 값을 저장한다.
- 5 4번에서 전달받은 4를 number가 받는다.
- 6 반복 변수 i가 2부터 1씩 증가하면서 4보다 작은 동안 7번을 반복 수행한다.

⑦ 4를 i로 나눈 나머지가 0이면 0을 반환하면서 함수를 호출했던 ⑨번으로 이동한다.

⑥~⑦번 반복문 실행에 따른 변수들의 변화는 다음과 같다.

number	i
4	2

⑨ ⑧번에서 돌려받은 값이 0이고, number의 값 13195를 i의 값 4로 나눈 나머지는 3이므로 `max_div = i`를 수행하지 않고 ③번으로 돌아가 i의 값을 1 증가시킨다.

네 번째 반복

④ i의 값 5를 인수로 `isPrime`을 호출한 결과가 1이고 number를 i로 나눈 나머지가 0이면 `max_div`에 i의 값을 저장한다.

⑤ ④번에서 전달받은 5를 number가 받는다.

⑥ 반복 변수 i가 2부터 1씩 증가하면서 5보다 작은 동안 ⑦번을 반복 수행한다.

⑦ 5를 i로 나눈 나머지가 0이면 0을 반환하고 함수를 호출했던 ⑨번으로 이동한다.

⑥~⑦번 반복문 실행에 따른 변수들의 변화는 다음과 같다.

number	i
5	2
	3
	4
	5

⑧ 1을 반환하고 함수를 호출했던 ⑨번으로 이동한다.

⑨ ⑧번에서 돌려받은 값이 1이고, number의 값 13195를 i의 값 5로 나눈 나머지도 0이므로 `max_div`에 5를 저장한 후 ③번으로 돌아가 i의 값을 1 증가시킨다.

※ 위의 과정을 통해 다음 사항들을 알 수 있다.

- `isPrime()` 함수는 인수를 2에서 시작하여 전달받은 수보다 1 작을 때까지 나눴을 때 끝까지 나머지가 0이 아니면 1을 반환하는 것으로 보아 소수를 찾는 함수임을 알 수 있다.
- ⑨번에서 `isPrime()`가 1이라는 것은 i가 소수임을 의미하고, number를 i로 나눈 나머지가 0이라는 것은 i가 number의 약수라는 의미이므로, `max_div`에는 소수이자 number의 약수인 수가 저장된다.
- i의 값이 1씩 증가하면서 number보다 1 작을 때까지 위 과정을 수행하므로 number의 소수로 된 약수 중 가장 큰 소수에 해당하는 값이 `max_div`에 저장된다.
- $13195 = 5 \times 7 \times 13 \times 29$ 이며, 이 중 가장 큰 소수인 29가 최종적으로 `max_div`에 저장된다.
- 자세한 값의 변화는 다음 표를 통해 확인하자.

main() 함수			isPrime() 함수		
number	i	max_div	number	i	반환값
13195	2	0	2	2	1
⋮	3		3	2	1
⋮				3	
	4		4	2	0
	5	5	5	2	1
	⋮			3	
	⋮			4	
	⋮			5	
⋮	⋮	⋮	⋮	⋮	⋮
	29	29	29	2	1
				3	
				4	
				⋮	
				28	

main() 함수			isPrime() 함수		
number	i	max_div	number	i	반환값
				29	
⋮	⋮	⋮	⋮	⋮	⋮
	13194		13194	2	0
	13195				

⑩ max_div의 값 29를 정수로 출력한다.

결과 29

4. 다음 C언어로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. [21-2](#)

```
#include <stdio.h>
main() {
    int res = mp(2, 10);
    printf("%d", res);
}

int mp(int base, int exp) {
    int res = 1;
    for (int i = 0; i < exp; i++)
        res *= base;
    return res;
}
```

답: 1024

[해설]

```
#include <stdio.h>
main() {
    ①⑦ int res = mp(2, 10);
    ⑧ printf("%d", res);
}

② int mp(int base, int exp) {
    ③ int res = 1;
    ④ for (int i = 0; i < exp; i++)
    ⑤     res *= base;
    ⑥ return res;
}
```

- ① 정수형 변수 res를 선언하고, 2와 10을 인수로 mp() 함수를 호출한 후 돌려받은 값을 res에 저장한다.
- ② 정수를 반환하는 mp() 함수의 시작점이다. ①번에서 전달받은 2와 10을 base와 exp가 받는다.
- ③ 정수형 변수 res를 선언하고 1로 초기화한다.
- ④ 반복 변수 i가 0에서 시작하여 1씩 증가하면서 10보다 작은 동안 ⑤번을 반복 수행한다.
- ⑤ 'res = res * base;'와 동일하다. res와 base를 곱한 값을 res에 저장한다.

※ 반복문 실행에 따른 변수들의 값의 변화는 다음과 같다.

base	exp	i	res
2	10		1
		0	2
		1	4
		2	8
		3	16
		4	32
		5	64
		6	128
		7	256
		8	512
		9	1024
		10	

- ⑥ res의 값 1024를 mp()를 호출했던 ⑦번으로 반환한다.
- ⑦ ⑥번으로부터 반환받은 값 1024를 res에 저장한다.
- ⑧ res의 값을 출력한다.

결과 1024

5. 다음 Java로 구현된 프로그램을 분석하여 괄호에 들어갈 알맞은 예약어를 쓰시오. 21-2

```
public class Test {
    public static void main(String[] args) {
        System.out.print(Test.check(1));
    }
    (        ) String check(int num){
        return (num >= 0) ? "positive" : "negative";
    }
}
```

답: static

[해설]

static은 클래스 이름으로 메소드에 접근하기 위해 사용하는 예약어로, 메소드를 사용하기 위해서는 메소드가 포함된 클래스의 객체 변수를 선언한 후 [객체 변수].[메소드]의 방식으로 접근해야 하지만 static을 이용하면 객체 변수 없이 [클래스 이름].[메소드]의 방식으로 접근하는 것이 가능해집니다.

```
public class Test {
    public static void main(String[] args) {
        ①④ System.out.print(Test.check(1));
    }
    ② static String check(int num){
        ③ return (num >= 0) ? "positive" : "negative";
    }
}
```

- ① 1을 인수로 Test 클래스의 check() 메소드를 호출한 후 돌려받은 값을 출력한다.
- ② 문자열을 반환하는 check() 메소드의 시작점이다. ①번에서 전달받은 1을 num이 받는다.
- ③ num이 0보다 크거나 같으면 "positive"를 반환하고, 아니면 "negative"를 반환한다. num의 값 1은 0보다 크므로 "positive"를 ④번으로 반환한다.
- ④ ③번에서 돌려받은 positive를 출력한다.

6. 다음 C언어로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. [20-3](#)

```
#include <stdio.h>
int r1() {
    return 4;
}
int r10() {
    return (30 + r1());
}
int r100() {
    return (200 + r10());
}
int main() {
    printf("%d\n", r100());
    return 0;
}
```

답: 234

[해설]

```
#include <stdio.h>
int r1() {
    ④ return 4;
}
int r10() {
    ③⑤ return (30 + r1());
}
int r100() {
    ②⑥ return (200 + r10());
}
int main() {
    ①⑦ printf("%d\n", r100());
    ⑧ return 0;
}
```

4 반환

34(30 + 4) 반환

234(200 + 34) 반환

- ① 인수 없이 r100() 함수를 호출한 다음 반환받은 값을 화면에 출력한다. ②번으로 이동한다.
- ② r10() 함수를 호출하여 반환받은 값에 200을 더한 후 r100() 함수를 호출한 ①번으로 반환한다. r10() 함수를 호출하기 위해 ③번으로 이동한다.
- ③ r1() 함수를 호출하여 반환받은 값에 30을 더한 후 r10() 함수를 호출한 ②번으로 반환한다. r1() 함수를 호출하기 위해 ④번으로 이동한다.
- ④ 반환값 4를 가지고 r1() 함수를 호출했던 ⑤번으로 이동한다.
- ⑤ ④번에서 반환받은 값 4에 30을 더한 34를 가지고 r10() 함수를 호출했던 ⑥번으로 이동한다.
- ⑥ ⑤번에서 반환받은 값 34에 200을 더한 234를 가지고 r100() 함수를 호출했던 ⑦번으로 이동한다.
- ⑦ ⑥번에서 반환받은 값 234를 정수로 출력하고 커서를 다음 줄로 이동한다.
- ⑧ main() 함수에서의 'return 0'은 프로그램의 종료를 의미한다.

결과 234

7. 다음 Java로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. [20-1](#)

```
public class Test {
    static int[] arr( ) {
        int a[ ] = new int[4];
        int b = a.length;
        for(int i = 0; i < b; i++)
    }
```

```

        a[i] = i;
        return a;
    }
    public static void main(String[] args) {
        int a[] = arr();
        for(int i = 0; i < a.length; i++)
            System.out.print(a[i] + " ");
    }
}

```

답: 0 1 2 3

[해설]

모든 Java 프로그램은 반드시 main() 메소드부터 시작해야 한다.

```

public class Test {
    2    static int[] arr( ) {
    3        int a[ ] = new int[4];
    4        int b = a.length;
    5        for(int i = 0; i < b; i++)
    6            a[i] = i;
    7        return a;
    }

    public static void main(String[] args) {
    18    int a[] = arr();
    9        for(int i = 0; i < a.length; i++)
    10        System.out.print(a[i] + " ");
    }
}

```

- ① 정수형 배열 a를 선언하고, 인수 없이 arr() 메소드를 호출한 다음 돌려받은 값을 배열 a에 저장한다.
 - ② 메소드의 리턴값이 정수형 배열인 arr() 메소드의 시작점이다.
 - ③ 4개의 요소를 갖는 정수형 배열 a를 선언한다.
 - ④ 정수형 변수 b를 선언하고 배열 a의 길이인 4로 초기화한다.
 - **length**: 배열 클래스의 속성으로 배열 요소의 개수가 저장되어 있다. a 배열은 4개의 요소를 가지므로 a.length는 4를 가지고 있다.
 - ⑤ 반복 변수 i가 0에서 시작하여 1씩 증가하면서 b보다 작은 동안 ⑥번 문장을 반복 수행한다.
 - ⑥ a[i]에 반복 변수 i의 값을 저장한다.
- | | a[0] | a[1] | a[2] | a[3] |
|-----|------|------|------|------|
| a[] | 0 | 1 | 2 | 3 |
- ⑦ 배열 a에 저장된 값들을 호출한 곳(main() 메소드)으로 반환한다.

```

public static void main(String[] args) {
    18 int a[] = arr();
    9    for(int i = 0; i < a.length; i++)
    10    System.out.print(a[i] + " ");
}

```

- ⑧ arr() 메소드로부터 반환받은 값들을 main() 메소드의 배열 a에 저장한다.
- ⑨ 반복 변수 i가 0에서 시작하여 1씩 증가하면서 a 배열의 길이 4보다 작은 동안 ⑩번 문장을 반복 수행한다.
- ⑩ a[i]의 값과 공백 한 칸을 출력한다.

※ ②번에서 인수로 배열의 이름, 즉 배열의 시작 주소가 전달되었으므로 호출된 arr() 메소드에서의 값의 변화는 main() 메소드의 a 배열에도 영향을 줍니다. 그러므로 출력되는 a 배열의 값은 정렬이 수행된 후의 값이 출력됩니다.

결과 0 1 2 3

8. 다음 C언어로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. 20-1

```
#include <stdio.h>
void align(int a[ ]) {
    int temp;
    for (int i = 0; i < 4; i++)
        for (int j = 0; j < 4 - i; j++)
            if (a[j] > a[j+1]) {
                temp = a[j];
                a[j] = a[j+1];
                a[j+1] = temp;
            }
}

main( ) {
    int a[ ] = { 85, 75, 50, 100, 95 };
    align(a);
    for (int i = 0; i < 5; i++)
        printf("%d ", a[i]);
}
```

답: 50 75 85 95 100

[해설]

문제의 코드는 버블 정렬 알고리즘을 이용하여 배열 a에 저장된 값을 오름차순으로 정렬한 후 출력하는 프로그램이다. 버블 정렬 알고리즘은 첫 번째 자료와 두 번째 자료를, 두 번째 자료와 세 번째 자료를, 세 번째와 네 번째를, ... 이런 식으로 (마지막-1)번째 자료와 마지막 자료를 비교하여 교환하면서 자료를 정렬한다. 1회전을 수행하고 나면 가장 큰 자료가 맨 뒤로 이동하므로 2회전에서는 맨 끝에 있는 자료는 정렬에서 제외되고, 2회전을 수행하고 나면 끝에서 두 번째 자료까지는 정렬에서 제외된다. 이렇게 정렬을 1회전 수행할 때마다 정렬에서 제외되는 데이터가 하나씩 늘어난다.

```
main( ) {
    1 int a[ ] = { 85, 75, 50, 100, 95 };
    2 align(a);
    13 for (int i = 0; i < 5; i++)
    14     printf("%d ", a[i]);
}
```

① 배열을 선언할 때 사용할 개수를 정하지 않으면 초기값으로 지정된 수만큼 배열의 요소가 만들어진다.

	a[0]	a[1]	a[2]	a[3]	a[4]
배열 a	85	75	50	100	95

② a를 인수로 하여 함수 align()을 호출한다. 인수로 배열의 이름을 지정하면 배열의 시작 주소가 인수로 전달된다. 그러니까 align(a)는 align(&a[0])과 같은 의미다.

```
3 void align(int a[]) {
4     int temp;
5     for (int i = 0; i < 4; i++)
6         for (int j = 0; j < 4 - i; j++)
```

```

7         if (a[j] > a[j+1]) {
8             temp = a[j];
9             a[j] = a[j+1];
10            a[j+1] = temp;
11        }
12    }

```

③ 반환값이 없는 align() 함수의 시작점이다. ②에서 'align(a)'라고 했으므로 정수형 배열 a는 main() 함수의 a 배열의 시작 주소를 받는다.

	a[0]	a[1]	a[2]	a[3]	a[4]
배열 a	85	75	50	100	95

- ④ 정수형 변수 temp를 선언한다.
- ⑤ 반복 변수 i가 0에서 시작하여 1씩 증가하면서 4보다 작은 동안 ⑥번을 반복하여 수행한다.
- ⑥ 반복 변수 j가 0에서 시작하여 1씩 증가하면서 4-i보다 작은 동안 ⑦번을 반복하여 수행한다.
- ⑦ a[j]가 a[j+1]보다 크면 ⑧~⑩ 사이의 문장을 실행한다.

※ 반복문 실행에 따른 변수들의 변화는 다음과 같다.

i	j	a[j]	a[j+1]	temp	배열 a																				
0	0	85	75	85	<table><tr><td>a[0]</td><td>a[1]</td><td>a[2]</td><td>a[3]</td><td>a[4]</td></tr><tr><td>85</td><td>75</td><td>50</td><td>100</td><td>95</td></tr><tr><td>75</td><td>85</td><td>85</td><td>95</td><td>100</td></tr><tr><td></td><td>50</td><td></td><td></td><td></td></tr></table>	a[0]	a[1]	a[2]	a[3]	a[4]	85	75	50	100	95	75	85	85	95	100		50			
	a[0]	a[1]	a[2]	a[3]		a[4]																			
	85	75	50	100		95																			
	75	85	85	95		100																			
		50																							
	1	75	85	85																					
	2	85	50	100																					
3	50	85																							
4	85	100																							
	100	95																							
	95	100																							
1	0	75	50	75	<table><tr><td>a[0]</td><td>a[1]</td><td>a[2]</td><td>a[3]</td><td>a[4]</td></tr><tr><td>75</td><td>50</td><td>85</td><td>95</td><td>100</td></tr><tr><td>50</td><td>75</td><td></td><td></td><td></td></tr></table>	a[0]	a[1]	a[2]	a[3]	a[4]	75	50	85	95	100	50	75								
	a[0]	a[1]	a[2]	a[3]		a[4]																			
	75	50	85	95		100																			
	50	75																							
1	50	75																							
2	75	85																							
3	85	95																							
2	0	50	75		<table><tr><td>a[0]</td><td>a[1]</td><td>a[2]</td><td>a[3]</td><td>a[4]</td></tr><tr><td>50</td><td>75</td><td>85</td><td>95</td><td>100</td></tr></table>	a[0]	a[1]	a[2]	a[3]	a[4]	50	75	85	95	100										
	a[0]	a[1]	a[2]	a[3]		a[4]																			
	50	75	85	95		100																			
1	75	85																							
	2																								
3	0	50	75		<table><tr><td>a[0]</td><td>a[1]</td><td>a[2]</td><td>a[3]</td><td>a[4]</td></tr><tr><td>50</td><td>75</td><td>85</td><td>95</td><td>100</td></tr></table>	a[0]	a[1]	a[2]	a[3]	a[4]	50	75	85	95	100										
	a[0]	a[1]	a[2]	a[3]		a[4]																			
50	75	85	95	100																					
1																									
4																									

- ⑪ if문의 끝이다.
- ⑫ 함수를 마치고 align(a) 함수를 호출했던 main() 함수로 제어를 옮긴다. a배열의 주소를 받아서 처리했으므로 main() 함수에 있는 a 배열에 정렬의 결과가 그대로 반영되어 있다. 즉 정렬 결과를 반환할 필요가 없다.

```

main( ) {
    int a[] = { 85, 75, 50, 100, 95 };
    align(a);
13    for (int i = 0; i < 5; i++)
14        printf("%d ", a[i]);
}

```

- ⑬ 반복 변수 i가 0에서 시작하여 1씩 증가하면서 5보다 작은 동안 ⑭번을 반복 수행한 후 프로그램을 종료한다.
- ⑭ a[i]의 값을 정수로 출력한 후 한 칸을 띄운다.

결과 50 75 85 95 100

9. 다음 C언어로 구현된 프로그램을 분석하여 5를 입력했을 때 그 실행 결과를 쓰시오. [22-1](#)

```
#include <stdio.h>
int func(int a) {
    if (a <= 1) return 1;
    return a * func(a - 1);
}
int main() {
    int a;
    scanf("%d", &a);
    printf("%d", func(a));
}
```

답: 120

[해설]

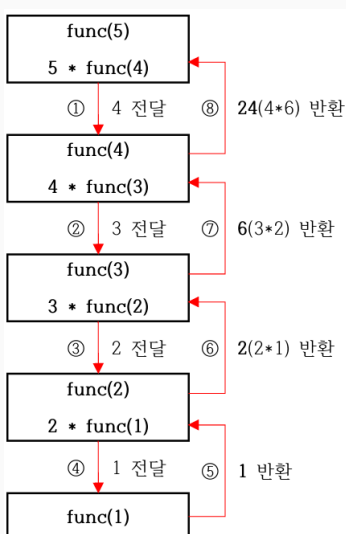
모든 C언어 프로그램은 반드시 main() 함수에서 시작한다.

```
int main() {
    1 int a;
    2 scanf("%d", &a);
    3 printf("%d", func(a));
}
```

- ① 정수형 변수 a를 선언한다.
- ② 정수를 입력받아 a에 저장한다. 5가 입력되었다고 가정하였으므로 a에는 5가 저장된다.
- ③ a의 값 5를 인수로 하여 func() 함수를 호출한 후 돌려받은 값을 정수로 출력한다.

```
4 int func(int a) {
5     if (a <= 1) return 1;
6     return a * func(a - 1);
}
```

- ④ 정수를 반환하는 func() 함수의 시작점이다. ③번에서 전달받은 5를 a가 받는다.
- ⑤ a가 1보다 작거나 같으면 함수를 호출했던 곳으로 1을 반환하고 함수를 종료한다. a의 값 5는 1보다 작거나 같지 않으므로 ⑥번으로 이동한다.
- ⑥ a-1을 인수로 하여 func() 함수를 호출한 후 돌려받은 값과 a를 곱하여 함수를 호출했던 곳으로 반환하고 함수를 종료한다. a가 1보다 큰 동안 자신을 호출하는 과정이 수행되다가 a가 1보다 작거나 같아지면 1이 반환되면서 호출했던 과정을 복귀한다. 이 때 반환된 값은 먼저 호출된 func() 함수에 반환할 값으로 계산된다는 것을 염두에 두고 전체 과정을 개괄적으로 살펴보자.



a-1의 값 4를 인수로 func() 함수를 호출한 후 돌려받은 값이 24이므로 a의 값 5를 곱한 값 120을 func(5)를 호출했던 ⑦번으로 반환한다.

```
int main() {
```

```

1 int a;
2 scanf("%d", &a);
3 7 printf("%d", func(a));
}

```

7 6 번에서 돌려받은 값 120을 정수로 출력한다.

※ 팩토리얼 재귀 재진입 반환 연산 추적은 다음과 같다.

- $\text{func}(5) = 5 * \text{func}(4)$
- $\text{func}(4) = 4 * \text{func}(3)$
- $\text{func}(3) = 3 * \text{func}(2)$
- $\text{func}(2) = 2 * \text{func}(1)$
- $\text{func}(1) = 1$ (반환 시작)

- $\text{func}(2) = 2 * 1 = 2$
- $\text{func}(3) = 3 * 2 = 6$
- $\text{func}(4) = 4 * 6 = 24$
- $\text{func}(5) = 5 * 24 = 120$ (최종값 메인 반환)

결과 120

10. 다음 C 언어로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. [25-3](#)

```

#include <stdio.h>
int main(void) {
    char str[ ] = "REPUBLICOFKOREA";
    int a = 0;
    while (str[a] != '\0')
        ++a;
    putchar(str[a-2]);
    return 0;
}

```

답: E

[해설]

문자열의 배열 인덱스 위치와 루프 탐색 구조 분석

- str 배열에 초기화된 문자열 "REPUBLICOFKOREA"의 총 글자 수는 **15개**입니다.
- while 문은 문자열의 끝을 알리는 null 문자('\0')를 만날 때까지 변수 a를 1씩 증가시킵니다.
- 반복문이 종료되는 순간 변수 a의 값은 글자 수이자 null 문자의 인덱스 번호인 **15**가 됩니다.
- 마지막 출력문에서 $\text{str}[a - 2]$, 즉 $\text{str}[13]$ 위치에 있는 문자를 출력하라고 지시합니다.
- 인덱스는 0부터 시작하므로 뒤에서 두 번째 글자인 'E'가 최종 화면에 출력됩니다. (마지막 문자 'A'의 인덱스는 14입니다.)

	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]	[10]	[11]	[12]	[13]	[14]	[15]
str	R	E	P	U	B	L	I	C	O	F	K	O	R	E	A	\0

a	a
0	8
1	9
2	10
3	11
4	12

a	a
5	13
6	14
7	15

결과 E

11. 다음 C 언어로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. 25-3

```
#include <stdio.h>
int main( ) {
    int x = 7, y = 4, z;
    z = y % 3 < 3 ? 2 : 1;
    z = z & z >> 1;
    z = x > 5 && z <= 3 ? z * x : z / x;
    printf("%d", z);
    return 0;
}
```

답: 0

[해설]

연산자 우선순위와 단계별 변수 값 추적 조건 ? 참 : 거짓

- 첫 번째 연산 ($z = y \% 3 < 3 ? 2 : 1$):
 $y \% 3$ 은 $4 \% 3$ 이므로 결과는 1이 됩니다. $1 < 3$ 은 참(1)이므로 삼항 연산자에 의해 z에는 2가 저장됩니다.
- 두 번째 연산 ($z = z \& z \gg 1$):
 비트 연산자 우선순위는 시프트($\gg$)가 비트 AND($\&$)보다 높습니다. $z \gg 1$ 은 $2 \gg 1$ 이므로 1이 됩니다. 그 후 $2 \& 1$ 을 연산하면 이진수로 $10 \& 01$ 이 되어 최종 결과는 0이 되며, z에 0이 대입됩니다.
- 세 번째 연산 ($z = x > 5 \&\& z \leq 3 ? z * x : z / x$):
 조건식 $x > 5 \&\& z \leq 3$ 을 보면, $7 > 5$ 는 참(1)이고 $0 \leq 3$ 도 참(1)이므로 전체 조건식은 참이 됩니다. 따라서 참일 때의 식인 $z * x$ 를 실행하게 됩니다. $0 * 7$ 의 결과인 0이 최종적으로 z에 저장됩니다.

x	y	z
7	4	2
		0
		0

결과 0

12. 다음 C 언어로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. 25-3

```
#include <stdio.h>
struct Node {
    struct Node* next;
    unsigned int x;
};
int main( ) {
    struct Node t1 = { 0, 5u };
    struct Node t2 = { 0, 7u };
    struct Node t3 = { 0, 11u };
    t3.next = &t2;
    t2.next = &t1;
    struct Node* curr = &t3;
}
```

```

int sum = 0;
while (curr) {
    sum = sum * 3 + curr->x;
    curr = curr->next;
}
sum = (sum ^ 42u) + 100u;
printf("%u\n", sum);
}

```

답: 187

[해설]

구조체 포인터 연결 리스트의 순회 및 비트 연산 프로세스 분석

• 연결 리스트 형성 구조 파악:

노드 노드가 연결된 방향을 보면 t3.next → t2, t2.next → t1, 그리고 t1.next = 0(NULL) 순서로 가리킵니다. 따라서 탐색 포인터 curr는 t3 → t2 → t1 순으로 총 3번 루프를 돌며 순회합니다.

• while 루프를 통한 누적 연산 (sum = sum * 3 + curr->x):

- 1회차 (t3 탐색): sum = 0 * 3 + 11 = 11

- 2회차 (t2 탐색): sum = 11 * 3 + 7 = 40

- 3회차 (t1 탐색): sum = 40 * 3 + 5 = 125

루프 종료 시점의 sum 최종 저장값은 125가 됩니다.

• 최종 비트 배타적 논리합(XOR) 및 덧셈 연산:

먼저 괄호 내부의 sum ^ 42u 연산을 수행합니다. 10진수 125와 42를 이진수로 변환하여 XOR 연산하면:

- 125 = 0111 1101₂

- 42 = 0010 1010₂

- XOR 결과는 0101 0111₂ → 10진수로 복원하면 87이 됩니다.

마지막으로 87 + 100u를 연산하여 최종 변수값은 부호 없는 10진수 서식(%u)에 의해 187이 출력됩니다.

sum	*curr
0	3000
11	2000
40	1000
125	0

결과 187

13. 다음 C 언어로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. [25-2](#)

```

#include <stdio.h>
#include <stdlib.h>
struct node {
    int p;
    struct node* n;
};
int main() {
    struct node a = {1, NULL};
    struct node b = {2, NULL};
    struct node c = {3, NULL};
    a.n = &b; b.n = &c; c.n = NULL;
    c.n = &a; a.n = &b; b.n = NULL;
    struct node* head = &c;
    printf("%d %d %d", head->p, head->n->p, head->n->n->p);
    return 0;
}

```

답: 3 1 2

[해설]

구조체 변수의 포인터 재할당 및 연결 관계 추적

- 초기 구조체 변수들의 기본 값은 각각 a.p=1, b.p=2, c.p=3이며, 다음 코드를 거치면서 주소 참조 상태가 지속적으로 업데이트됩니다.
- 첫 번째 대입문 a.n = &b; b.n = &c; c.n = NULL;에 의해 초기 리스트 형태는 **a → b → c → NULL** 구조를 이룹니다.
- 하지만 곧바로 이어지는 두 번째 대입문 c.n = &a; a.n = &b; b.n = NULL;에 의해 기존 연결 링크가 완전히 끊어지고 덮어씌워집니다.
- 새로운 관계에 의하면, c.n은 a의 주소를 가리키고, a.n은 b의 주소를 가리키며, 마지막 b.n이 NULL이 되므로 새로운 단방향 연결 리스트는 최종적으로 **c → a → b → NULL** 구조로 대동단결됩니다.
- head 포인터 변수가 &c(c의 주소)를 저장하고 있으므로, 순차적인 참조 결과는 다음과 같습니다:
 - head->p = c의 값 = 3
 - head->n->p = c가 가리키는 a의 값 = 1
 - head->n->n->p = a가 가리키는 b의 값 = 2
- 출력 서식이 "%d %d %d"이므로 각각의 정수값 사이에 공백(띄어쓰기)을 한 칸씩 두어 3 1 2가 최종 출력됩니다.

구분 코드 블록	a.n (Next)	b.n (Next)	c.n (Next)
첫 번째 연결 할당 구간	&b (b 참조)	&c (c 참조)	NULL
두 번째 최종 재할당 구간	&b (유지)	NULL (변경)	&a (변경)

1000 a	p	*n
	1	NULL

2000 b	p	*n
	2	NULL

3000 c	p	*n
	3	NULL

결과 3 1 2

14. 다음 C 언어로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. [25-2](#)

```
#include <stdio.h>
#define SIZE 3
typedef struct {
    int a[SIZE];
    int front;
    int rear;
} Queue;
void enq(Queue* q, int val) {
    q -> a[q -> rear] = val;
    q -> rear = (q -> rear + 1) % SIZE;
}
int deq(Queue* q) {
    int val = q -> a[q -> front];
    q -> front = (q -> front + 1) % SIZE;
    return val;
}
int main() {
    Queue q = {{0}, 0, 0};
    enq(&q, 1);
```

```

    enq(&q, 2);
    deq(&q);
    enq(&q, 3);
    printf("%d 그리고 %d", deq(&q), deq(&q));
    return 0;
}

```

답: 2 그리고 3

[해설]

C언어 구조체 기반 원형 큐(Circular Queue) 작동 원리 및 printf 인자 평가 순서 분석

- 배열 크기가 3(SIZE = 3)인 원형 큐 구조체입니다. 인덱스는 나머지 연산자(% SIZE)를 통해 순환 구조를 가집니다.
- enq()는 rear 위치에 값을 집어넣고 rear를 1 증가시키며, deq()는 front 위치의 값을 꺼내고 front를 1 증가시킵니다.
- main() 함수의 순차 실행 흐름에 따른 데이터 변화 추적은 하단 테이블과 같습니다. 중간에 deq(&q); 연산으로 인해 데이터 1이 먼저 소멸합니다.
- **중요 함수 호출 분석 (printf 매개변수 평가):** C언어 표준에서 printf() 함수 인자들의 평가(호출) 순서는 명시되어 있지 않으나, 대다수의 컴파일러(GCC 등)는 스택(Stack) 메커니즘을 사용하여 **오른쪽 인자부터 왼쪽 방향으로 역순 평가**를 진행합니다.
- 따라서 서식문자 뒤의 우측 인자인 뒤쪽 deq(&q)가 첫 번째로 호출되어 front=1인 공간의 값 2를 꺼내고, 좌측 인자인 앞쪽 deq(&q)가 두 번째로 호출되어 front=2인 공간의 값 3을 꺼내게 됩니다.
- 출력 서식문자가 "%d 그리고 %d"이므로, 최종 변환 데이터 매핑에 의해 **2 그리고 3**이 출력됩니다.

실행 단계	큐 배열 내부 상태	front	rear
초기화 상태	[0, 0, 0]	0	0
enq(&q, 1); 수행 후	[1, 0, 0]	0	1
enq(&q, 2); 수행 후	[1, 2, 0]	0	2
deq(&q); 수행 후 (1 소멸)	[1, 2, 0] (참조종료)	1	2
enq(&q, 3); 수행 후	[1, 2, 3]	1	0 (순환)
printf 역순 평가 (뒤쪽 deq 호출)	a[1] 공간의 '2' 반환	2	0
printf 역순 평가 (앞쪽 deq 호출)	a[2] 공간의 '3' 반환	0 (순환)	0

15. 다음 C 언어로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. 25-2

```

#include <stdio.h>
struct dat {
    int x;
    int y;
};
int main( ) {
    struct dat a[ ] = {{1, 2}, {3, 4}, {5, 6}};
    struct dat* ptr = a;
    struct dat** pptr = &ptr;
    (*pptr)[1] = (*pptr)[2];
    printf("%d 그리고 %d", a[1].x, a[1].y);
    return 0;
}

```

답: 5 그리고 6

[해설]

C언어 다중 포인터를 이용한 구조체 배열 접근 분석

- struct dat a[]에 의해 크기가 3인 구조체 배열이 생성되고 값은 각각 a[0]={1,2}, a[1]={3,4}, a[2]={5,6}으로 초기화됩니다.
- 포인터 변수 ptr은 배열의 시작 주소(&a[0])를 가리키고, 이중 포인터 pptr은 ptr의 주소값을 가집니다.
- 따라서 역참조 연산인 (*pptr)은 포인터 변수 ptr과 같고, 이는 곧 배열 주소 a와 동일한 의미를 가집니다. 즉, (*pptr)[i]는 a[i]와 완전히 매칭됩니다.
- **(*pptr)[1] = (*pptr)[2];** 문장이 실행되면 **a[1]의 자리에 a[2]의 값 전체가 복사되어** 대입됩니다.

■ 대입 연산 전·후 구조체 배열 상태 변화

배열 요소	초기 상태	연산 수행 후 상태
a[0]	{ x 1, y: 2 }	{ x 1, y: 2 }
a[1]	{ x: 3, y: 4 }	{ x: 5, y: 6 } (변경)
a[2]	{ x 5, y: 6 }	{ x 5, y: 6 }

- 최종적으로 printf 문에서 a[1].x와 a[1].y를 출력하므로 각각 **5**와 **6**이 매핑됩니다.
- 큰따옴표 안의 문자열 서식처럼 공백을 포함한 **5 그리고 6**이 출력됩니다.

16. 다음 C 언어로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. [25-2](#)

```
#include <stdio.h>
#include <stdlib.h>
struct node {
    char c;
    struct node* p;
};
struct node* func(char* s) {
    struct node* h = NULL, *n;
    while(*s) {
        n = malloc(sizeof(struct node));
        n->c = *s++;
        n->p = h;
        h = n;
    }
    return h;
}
int main() {
    struct node* n = func("BEST");
    while(n) {
        putchar(n->c);
        struct node* t = n;
        n = n->p;
        free(t);
    }
    return 0;
}
```

답: **TSEB**

[해설]

단방향 연결 리스트(Linked List)의 역순 삽입(Head Insertion) 알고리즘 구조 분석

- 구조체 정의 및 초기화:
 - struct node는 문자 데이터(c)와 다음 노드를 가리키는 포인터(p)로 구성됩니다.
 - main 함수에서 func("BEST")를 호출하여 문자열의 시작 주소를 전달합니다.

• func 함수의 while(*s) 반복문 동작 (역순 리스트 생성):

문자열의 끝(Null 문자, '\0')을 만날 때까지 한 글자씩 넘어가며 새로운 노드를 동적 할당합니다.

이때 새 노드 n의 포인터 변수 p가 기존의 헤드 포인터 h를 가리키게 한 뒤, h를 새 노드 n으로 갱신합니다. (앞으로 삽입하는 방식)

■ 문자열 입력에 따른 연결 리스트 적재 과정 (func)

단계	현재 문자(*s)	포인터 연결 연산 (n->p = h)	헤드(h) 기준 최종 논리적 구조
1회전	'B'	['B']->p = NULL	h → ['B'] → NULL
2회전	'E'	['E']->p = ['B']	h → ['E'] → ['B'] → NULL
3회전	'S'	['S']->p = ['E']	h → ['S'] → ['E'] → ['B'] → NULL
4회전	'T'	['T']->p = ['S']	h → ['T'] → ['S'] → ['E'] → ['B']

• main 함수의 순회 및 메모리 해제 (while(n)):

- 함수 반환값으로 가장 마지막에 추가된 노드인 ['T']의 주소(최상단 헤드)를 가리키는 포인터 n을 받습니다.

- putchar(n->c);로 현재 가리키는 노드의 문자를 하나씩 출력한 뒤 다음 노드(n->p)로 이동합니다.

- 동시에 출력 직후 임시 포인터 t에 저장해 두었던 이전 노드를 free(t);를 통해 동적 메모리 영역에서 정상적으로 해제합니다.

- 결과적으로 화살표 방향을 따라 추적하며 TSEB 순서대로 공백 없이 출력됩니다.

17. 다음 C 언어로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. [25-1](#)

```
#include <stdio.h>
char Data[5] = {'B', 'A', 'D', 'E'};
char c;

int main() {
    int i, temp, temp2;
    c = 'C';
    printf("%d\n", Data[3]-Data[1]);
    for(i = 0; i < 5; ++i) {
        if(Data[i] > c)
            break;
    }
    temp = Data[i];
    Data[i] = c;
    i++;
    for(; i < 5; ++i) {
        temp2 = Data[i];
        Data[i] = temp;
        temp = temp2;
    }

    for(i = 0; i < 5; i++) {
        printf("%c", Data[i]);
    }
}
```

답: 4
BACDE

[해설]

배열 요소의 값 비교 및 순차 시프트(Shift) 연산 분석

- **첫 번째 출력 연산:** 전역 배열 Data는 {'B', 'A', 'D', 'E', '\0'}로 초기화되어 있습니다. Data[3]은 'E'(ASCII 69), Data[1]은 'A'(ASCII 65)이므로 $69 - 65 = 4$ 가 계산되어 먼저 출력되고 줄바꿈(\n)이 일어납니다.
- **탐색 루프:** c = 'C'보다 아스키코드 값이 큰 첫 번째 요소를 찾습니다. Data[2]의 'D'가 'C'보다 크므로 조건이 만족되어 break가 발생하고, 이때 인덱스 i의 값은 2가 됩니다.
- **값 삽입 및 인덱스 조정:** temp = Data[2]에 의해 temp에는 'D'가 저장되고, Data[2] = 'C'가 저장됩니다. 그 후 i++에 의해 i는 3이 됩니다.
- **시프트 루프 (i=3부터 시작):** 우측 단의 첫 번째 for 문을 돌며 기존 배열의 나머지 값들을 뒤로 한 칸씩 밀어내는 작업을 수행합니다.

단계 / 변수	Data[2]	Data[3]	Data[4]	temp	temp2
루프 진입 전	'C'	'E'	'\0'	'D'	-
i = 3 일 때	'C'	'D'	'\0'	'E'	'E'
i = 4 일 때	'C'	'D'	'E'	'\0'	'\0'

- 결과적으로 Data 배열은 순서대로 정렬된 형태인 {'B', 'A', 'C', 'D', 'E'}가 되며 마지막 출력 루프에 의해 ABCDE(Data[1]이 'A'이므로 전체 순서는 A, B, C, D, E를 만족하는 형태)가 연속으로 출력됩니다.

문자열이 메모리에 저장될때 " 아스키 코드 값 " 으로 저장

65	66	67	68	69	70	71	72	73	74	...	85	86	87	88	89	90
A	B	C	D	E	F	G	H	I	J	...	U	V	W	X	Y	Z

97	98	99	100	101	102	103	104	105	106	...	117	118	119	120	121	122
a	b	c	d	e	f	g	h	i	j	...	u	v	w	x	y	z

	0	1	2	3	4
Data	66	65	68	69	69
	B	A	D	E	E

0	1	2	3	4	
Data	66	65	67	68	69
	B	A	C	D	E

c
67
temp
68→69
temp2

i
0
1
2
3
4

결과 4
BACDE

18. 다음 C 언어로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. [25-1](#)

```
#include <stdio.h>
#include <stdlib.h>
void set(int** arr, int* data, int rows, int cols) {
    for (int i = 0; i < rows * cols; ++i) {
        arr[((i + 1) / rows) % rows][(i + 1) % cols] = data[i];
    }
}
int main() {
    int rows = 3, cols = 3, sum = 0;
    int data[] = {5, 2, 7, 4, 1, 8, 3, 6, 9};
    int** arr;
    arr = (int**)malloc(sizeof(int*) * rows);
    for (int i = 0; i < rows; i++) {
        arr[i] = (int*)malloc(sizeof(int) * cols);
    }
    set(arr, data, rows, cols);
```

```

for (int i = 0; i < rows * cols; i++) {
    sum += arr[i / rows][i % cols] * (i % 2 == 0 ? 1 : -1);
}
for (int i = 0; i < rows; i++) {
    free(arr[i]);
}
free(arr);
printf("%d", sum);
}

```

답: 13

[해설]

2차원 동적 배열의 비선형 매핑 및 번갈아 가며 수행되는 누적 연산 분석

- 배열 생성 및 구조: rows = 3, cols = 3 크기의 2차원 동적 배열 arr를 생성합니다. (3행 3열 구조)
- set 함수에 의한 데이터 재배치: 1차원 배열 data의 요소를 특이한 규칙 수식인 $((i + 1) / 3) \% 3$ $[(i + 1) \% 3]$ 인덱스 구조를 사용하여 2차원 배열에 대입합니다. i가 0부터 8까지 변화함에 따라 매핑되는 공간은 순차적이지 않고 밀려서 채워집니다.
- 대입 연산 매핑 결과 (set 함수 종료 후 arr 상태):
 - 0행: arr[0][0]=9, arr[0][1]=5, arr[0][2]=2
 - 1행: arr[1][0]=3, arr[1][1]=6, arr[1][2]=7
 - 2행: arr[2][0]=4, arr[2][1]=1, arr[2][2]=8
- 누적 합산 계산 (sum): 순차적으로 배열을 순회하며 인덱스 i가 짝수일 때는 더하고(+1), 홀수일 때는 빼는(-1) 교대 급수 형태의 연산을 적용합니다. malloc(): 메모리 동적 할당 함수
 $sum = 9(\text{짝}) - 5(\text{홀}) + 2(\text{짝}) - 7(\text{홀}) + 4(\text{짝}) - 1(\text{홀}) + 8(\text{짝}) - 3(\text{홀}) + 6(\text{짝}) = 13$ 입니다.

i (순서)	매핑 위치 (행, 열)	해당 값	부호 (+/-)	계산식 변화
0	arr[0][0]	9	+	+9
1	arr[0][1]	5	-	9 - 5 = 4
2	arr[0][2]	2	+	4 + 2 = 6
3	arr[1][0]	7	-	6 - 7 = -1
4	arr[1][1]	4	+	-1 + 4 = 3
5	arr[1][2]	1	-	3 - 1 = 2
6	arr[2][0]	8	+	2 + 8 = 10
7	arr[2][1]	3	-	10 - 3 = 7
8	arr[2][2]	6	+	7 + 6 = 13

2차원 매핑 완료 후 메모리(arr[3][3]) 상태 시각화

	col 0	col 1	col 2	
row 0	9	5	2	□ 짝수 인덱스 (sum에 더하기 연산) □ 홀수 인덱스 (sum에 빼기 연산)
row 1	7	4	1	
row 2	8	3	6	

결과 13

19. 다음 C 언어로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. 25-1

```

#include <stdio.h>
typedef struct student {

```

```

    char* name;
    int score[3];
} Student;
int dec(int enc) {
    return enc & 0xA5;
}
int sum(Student* p) {
    return dec(p->score[0]) + dec(p->score[1]) + dec(p->score[2]);
}
int main() {
    Student s[2] = { {"Kim", {0xA0, 0xA5, 0xDB}}, {"Lee", {0xA0, 0xED, 0x81}} };
    Student* p = s;
    int result = 0;
    for (int i = 0; i < 2; i++) {
        result += sum(&s[i]);
    }
    printf("%d", result);
    return 0;
}

```

답: 908

[해설]

구조체 포인터 전달 및 비트 AND 연산을 활용한 디코딩 연산 분석

- 구조체 정의 및 초기화: Student 구조체는 문자열 포인터 name과 크기가 3인 정수형 배열 score를 멤버로 가집니다. main 함수에서 크기가 2인 구조체 배열 s를 선언하고 각 값을 16진수로 초기화합니다.
- dec 함수의 비트 연산 연산 메커니즘: 매개변수로 넘어온 값을 0xA5(이진수: 1010 0101)와 비트 논리곱(&) 연산을 수행하여 반환합니다.
- sum 함수 내부의 디코딩 연산 및 합산 결과: 십진수: 10
 - s[0] ("Kim") 연산: 16진수 A(10), B(11), C(12), D(13), E(14), F(15)
 - dec(0xA0) → 0xA0 & 0xA5 = 0xA0 (160₁₀)
 - dec(0xA5) → 0xA5 & 0xA5 = 0xA5 (165₁₀)
 - dec(0xDB) → 0xDB & 0xA5 = 0x81 (129₁₀)
 - Kim의 합계: 160 + 165 + 129 = 454
 - s[1] ("Lee") 연산:
 - dec(0xA0) → 0xA0 & 0xA5 = 0xA0 (160₁₀)
 - dec(0xED) → 0xED & 0xA5 = 0xA5 (165₁₀)
 - dec(0x81) → 0x81 & 0xA5 = 0x81 (129₁₀)
 - Lee의 합계: 160 + 165 + 129 = 454
- 최종 누적값 계산: 루프가 2회 반복되며 result에 두 구조체의 연산 결과가 합산되므로 최종 출력값은 454 + 454 = 908 이 됩니다.

	1010 0000 A0		1010 0101 A5		1101 1011 DB		1110 1101 ED
&	1010 0101 A5	&	1010 0101 A5	&	1010 0101 A5	&	1010 0101 A5
	1010 0000 A0		1010 0101 A5		1000 0001 81		1010 0101 A5

학생 (i)	원래 16진수 배열	비트 AND 연산 (& 0xA5)	변환된 10진수 합
s[0] (Kim)	{0xA0, 0xA5, 0xDB}	0xA0, 0xA5, 0x81	160 + 165 + 129 = 454
s[1] (Lee)	{0xA0, 0xED, 0x81}	0xA0, 0xA5, 0x81	160 + 165 + 129 = 454
최종 결과 (result)			908

20. 다음 C 언어로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. 25-1

```

#include <stdio.h>
#include <stdlib.h>
typedef struct Data {
    int value;
    struct Data *next;
} Data;
Data* insert(Data* head, int value) {
    Data* new_node = (Data*)malloc(sizeof(Data));
    new_node->value = value;
    new_node->next = head;
    return new_node;
}
Data* reconnect(Data* head, int value) {
    if (head == NULL || head->value == value) return head;
    Data *prev = NULL, *curr = head;
    while (curr != NULL && curr->value != value) {
        prev = curr;
        curr = curr->next;
    }
    if (curr != NULL && prev != NULL) {
        prev->next = curr->next;
        curr->next = head;
        head = curr;
    }
    return head;
}
int main() {
    Data *head = NULL, *curr;
    for (int i = 1; i <= 5; i++) {
        head = insert(head, i);
    }
    head = reconnect(head, 3);
    for (curr = head; curr != NULL; curr = curr->next) {
        printf("%d", curr->value);
    }
    return 0;
}

```

답: 35421

[해설]

단일 연결 리스트(Linked List)의 노드 삽입 및 특정 노드 전두 배치(reconnect) 분석

- 초기 연결 리스트 생성 (insert 반복문): insert 함수는 새 노드를 리스트의 가장 앞(head)에 추가합니다. i가 1부터 5까지 증가하면서 삽입되므로 역순 구조로 배치됩니다.
- 초기 생성 완료 후 상태: 5 → 4 → 3 → 2 → 1 → NULL
- reconnect(head, 3) 함수 분석: 리스트를 순회하며 value == 3인 노드를 찾아내어 구조를 변경합니다.
 - 탐색 결과: curr는 값이 3인 노드, prev는 그 이전 노드인 4를 가리킵니다.
 - prev->next = curr->next; : 노드 4의 다음을 노드 2로 연결하여 노드 3을 리스트 중간에서 탈락(제거)시킵니다. (5 → 4 → 2)
 - curr->next = head; : 노드 3의 다음을 기존의 첫 번째 노드인 5로 연결합니다. (3 → 5)

• head = curr; : 리스트의 시작점(head)을 노드 3으로 변경합니다.

• 최종 연결 리스트 구조 상태: 3 → 5 → 4 → 2 → 1 → NULL

• 출력(printf): head부터 리스트 끝까지 순차적으로 공백이나 개행 없이 %d 서식으로 출력하므로 35421이 출력됩니다.

단계	연결 리스트 구조 형태	head 값
1. insert 완료 후	5 → 4 → 3 → 2 → 1	5
2. 노드 3 탈락	5 → 4 → 2 → 1 (3 분리)	5
3. reconnect 완료	3 → 5 → 4 → 2 → 1	3

reconnect 수행 후 메모리 연결 구조 시각화

head: 3 → 5 → 4 → 2 → 1 reconnect에 의해 앞으로 이동한 대상 노드 (새로운 head)

결과 35421

21. 다음 C 언어로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. [24-3](#)

```
#include <stdio.h>
int func() {
    static int x = 0;
    x += 2;
    return x;
}
int main() {
    int x = 0;
    int sum = 0;
    for(int i = 0; i < 4; i++) {
        x++;
        sum += func();
    }
    printf("%d", sum);
    return 0;
}
```

답: 20

[해설]

정적 변수(static)의 특성과 반복문을 통한 누적 연산 분석

- 정적 변수 특징: func() 내부에 선언된 static int x = 0;은 프로그램 시작 시 단 한 번만 초기화되며, 함수가 종료되어도 소멸하지 않고 값이 유지됩니다. 반면 main()의 지역 변수 x는 func() 내부의 x와 이름만 같을 뿐 아무런 영향을 주지 않습니다.
- 반복 루프에 따른 변수 추적: i가 0부터 3까지 총 4번 반복 실행되면서 func()를 호출하고 반환값을 sum에 누적합니다.

루프 (i)	main의 x++	func() 내 static x	sum 누적 과정
0	1	2	0 + 2 = 2
1	2	4	2 + 4 = 6
2	3	6	6 + 6 = 12
3	4	8	12 + 8 = 20

결과 20

22. 다음 C 언어로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. 24-3

```
#include <stdio.h>
struct Node {
    int value;
    struct Node* next;
};
void func(struct Node* node){
    while(node != NULL && node->next != NULL) {
        int t = node->value;
        node->value = node->next->value;
        node->next->value = t;
        node = node->next->next;
    }
}
int main() {
    struct Node n1 = {1, NULL};
    struct Node n2 = {2, NULL};
    struct Node n3 = {3, NULL};
    n1.next = &n3;
    n3.next = &n2;
    func(&n1);
    struct Node* current = &n1;
    while(current != NULL){
        printf("%d", current->value);
        current = current->next;
    }
    return 0;
}
```

답: 312

[해설]

연결 리스트의 인접 노드 데이터 스와핑(Swap) 알고리즘 분석

- 초기 연결 리스트 구성: 메인 함수에서 생성된 노드들의 연결 상태는 구조체 포인터 지정을 통해 n1(val:1) -> n3(val:3) -> n2(val:2) -> NULL 순서로 연결됩니다.
- func 함수 분석: 인접한 두 노드의 value를 맞바꾸고(Swap), 포인터를 두 칸(node->next->next) 전진시키는 구조입니다.
- 실행 과정 추적:
 - 1회차 루프: 현재 node인 n1의 값(1)과 다음 노드인 n3의 값(3)을 교환합니다.
→ 결과적으로 n1.value = 3, n3.value = 1이 됩니다.
 - 포인터 이동: node = node->next->next; 연산에 의해 다음 연산 대상 노드는 n2가 됩니다.
 - 2회차 루프 탈출: node인 n2의 next는 NULL이므로, while 조건문(node->next != NULL)을 만족하지 못해 루프를 완전히 빠져나옵니다.

데이터 변경 전/후 연결 리스트 흐름 구조 시각화

[초기 상태]: n1 [1] → n3 [3] → n2 [2]

[func 실행 후]: n1 [3] → n3 [1] → n2 [2]

따라서 n1부터 차례대로 출력하면 3, 1, 2 순으로 출력 서식 없이 연속되어 312 출력됩니다.

23. 다음 C 언어로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. 24-3

```
#include <stdio.h>
void func(int** arr, int size) {
```

```

    for(int i = 0; i < size; i++) {
        *( *arr + i) = (*( *arr + i) + i) % size;
    }
}
int main(){
    int arr[] = {3, 1, 4, 1, 5};
    int* p = arr;
    int** pp = &p;
    int num = 6;
    func(pp, 5);
    num = arr[2];
    printf("%d", num);
    return 0;
}

```

답: 1

[해설]

다중 포인터 참조를 통한 1차원 배열 데이터의 간접 수정 과정 분석

- **포인터 구조 분석:** p는 1차원 배열인 arr의 시작 주소를 가리키고, 이중 포인터 pp는 포인터 변수 &p의 주소를 가집니다. 따라서 func 함수 내부의 역참조 식 *arr은 main 함수의 포인터인 p(즉, arr[] 배열의 시작 위치)를 뜻합니다.
- **포인터 연산식 해석:** $*(*arr + i)$ 는 일반적인 배열 표현인 $arr[i]$ 와 완벽히 동일한 의미를 갖는 포인터 표현식입니다.
- **루프 연산에 따른 배열 값의 실시간 갱신 (size = 5):**

인덱스(i)	기존 arr[i]	수식 : $(arr[i] + i) \% 5$	최종 변경 arr[i]
0	3	$(3 + 0) \% 5 = 3$	3
1	1	$(1 + 1) \% 5 = 2$	2
2	4	$(4 + 2) \% 5 = 1$	1
3	1	$(1 + 3) \% 5 = 4$	4
4	5	$(5 + 4) \% 5 = 4$	4

func() 연산 종료 시점의 배열 전체 상태는 {3, 2, 1, 4, 4} 가 됩니다.

따라서 메인 함수 내부 연산인 `num = arr[2];`의 데이터는 1로 변경되어 변수에 저장됩니다.

24. 다음 C 언어로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. 24-2

```

#include <stdio.h>
void swap(int a, int b) {
    int t = a;
    a = b;
    b = t;
}

```

```

int main() {
    int a = 11;
    int b = 19;
    swap(a, b);
    switch(a) {
        case 1:
            b += 1;
        case 11:
            b += 2;
        default:
            b += 3;
            break;
    }
    printf("%d", a - b);
}

```

답: -13

[해설]

Call by Value(값에 의한 호출) 특징과 switch-case 문의 break 생략(Fall-through) 효과 분석

- **값의 초기화:** main 함수 내에서 변수 a = 11, b = 19로 각각 초기화됩니다.
- **swap(a, b) 함수의 호출 (Call by Value):** swap(a, b)를 호출하면 복사본 값이 전달될 뿐이므로, swap 함수 내부에서 두 값이 변경되더라도 main 함수의 변수 a와 b에는 ****아무런 영향을 주지 않습니다.**** 따라서 함수 호출 후에도 여전히 a = 11, b = 19입니다.
- **switch(a) 문 조건 진입 및 Fall-through 발생:**
 - a의 값이 11이므로 case 11: 조건문으로 바로 점프합니다.
 - b += 2; 연산이 실행되어 b는 21(19 + 2)이 됩니다.
 - ****주의:**** case 11: 하단에 break; 문이 없기 때문에, 제어 흐름이 멈추지 않고 아래에 있는 default:(오타로 작성된 코드 조건 포함) 영역까지 연속해서 흘러내려 갑니다(Fall-through).
 - 이어서 b += 3; 연산이 실행되어 b는 최종적으로 ****24****(21 + 3)가 됩니다. 이후 break;를 만나 switch 문을 빠져나옵니다.
- **최종 출력값 계산:** printf("%d", a - b); 문에서 11 - 24를 연산하므로 최종 출력 결과는 **-13**이 됩니다.

단계 및 실행 코드	변수 a	변수 b	비고
초기 변수 선언	11	19	초기값 대입
swap(a, b); 호출 후	11	19	값 변경 없음
case 11: 진입 (b += 2)	11	21	19 + 2
default: 통과 (b += 3)	11	24	break 없음 (낙하)
printf 연산 (a - b)	11 - 24 = -13		

switch 문 내부 제어 흐름 시각화

```
switch(a) [a=11]
├─ case 1: (건너뛰)
└─ → case 11: b += 2; (b=21)
    └─ [★ break 미존재: 아래로 계속 진행]
        └─ → default: b += 3; (b=24)
            └─ break; (종료)
```

25. 다음 C 언어로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. [24-2](#)

```
#include <stdio.h>
void func(int* d, int* s) {
    while (*s) {
        *d = *s;
        d++;
        s++;
    }
    *d = '\0';
}
int main() {
    char* str1 = "first";
    char str2[50] = "teststring";
    int result = 0;
    func(str2, str1);
    for (int i = 0; str2[i] != '\0'; i++) {
        result += i;
    }
}
```

```
printf("%d\n", result);
return 0;
}
```

답: 10

[해설]

문자열 복사 매커니즘(사용자 정의 strcpy) 및 변경된 문자열 길이를 통한 인덱스 누적합 연산

- 초기 변수 선언: 포인터 변수 str1은 문자열 리터럴 "first"(길이 5)를 가리키고, 배열 str2는 "teststring"(길이 10)으로 초기화됩니다.
- func(str2, str1) 호출을 통한 문자열 덮어쓰기:
 - func 함수는 소스 포인터 s(str1)가 가리키는 값을 목적지 포인터 d(str2)에 한 글자씩 복사합니다.
 - while(*s) 루프에 의해 "first"의 5글자가 순서대로 str2의 앞부분에 덮어씌워집니다.
 - 루프 종료 후, *d = '\0'; 문장에 의해 복사된 문자열 바로 뒤(인덱스 5 위치)에 문자열의 끝을 알리는 널 문자('\0')가 명시적으로 삽입됩니다.
- func 함수 종료 후 str2 배열의 상태: str2의 기존 내용 뒤쪽("tring")은 메모리에 남아있으나, 인덱스 5 위치에 널 문자('\0')가 저장되면서 C 언어 표준 메커니즘상 str2는 유효한 길이 5짜리 문자열인 "first"로 인식됩니다.
- for 루프를 통한 인덱스 누적 연산: str2[i] != '\0' 조건에 의해 루프는 인덱스 0부터 4까지만 순회하게 됩니다.
result = 0 + 1 + 2 + 3 + 4 = 10이 최종 저장됩니다.

인덱스 (i)	0	1	2	3	4	5
함수 호출 전	t	e	s	t	s	t
func 수행 후	f	i	r	s	t	\0
루프 내 계산	+= 0	+= 1	+= 2	+= 3	+= 4	종료조건

루프 제어 및 result 누적값 변화 시각화

```
i = 0 → str2[0]('f') != '\0' → result += 0 (누적: 0)
i = 1 → str2[1]('i') != '\0' → result += 1 (누적: 1)
i = 2 → str2[2]('r') != '\0' → result += 2 (누적: 3)
i = 3 → str2[3]('s') != '\0' → result += 3 (누적: 6)
i = 4 → str2[4]('t') != '\0' → result += 4 (누적: 10)
i = 5 → str2[5]('\0') == '\0' → 탈출 (최종 result: 10)
```

1000	f	i	r	s	t	\0					
	1000	1001	1002	1003	1004	1005					
str2	[0]	[1]	[2]	[3]	[4]	[5]					
2000	f	i	r	s	t	\0					
	2000	2001	2002	2003	2004	2005	2006	2007	2008	2009	2010

i	result
0	0
1	1
2	3
3	6
4	10
5	

결과 10

26. 다음 C 언어로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. 24-2

```
#include <stdio.h>
int main() {
    int arr[3][3] = {1, 2, 3, 4, 5, 6, 7, 8, 9};
```

```
int* parr[2] = {arr[1], arr[2]};
printf("%d", parr[1][1] + *(parr[1]+2) + **parr);
}
```

답: 21

[해설]

2차원 배열의 행 포인터 전달 및 포인터 배열(Array of Pointers) 연산 구조 분석

- **2차원 배열 초기화 (arr[3][3]):** 3행 3열 구조의 메모리 공간에 1부터 9까지의 원소가 순차적으로 배치됩니다.
 - arr[0] (0행): {1, 2, 3}
 - arr[1] (1행): {4, 5, 6}
 - arr[2] (2행): {7, 8, 9}
- **포인터 배열 선언 및 참조 매핑 (parr[2]):** 포인터 배열 parr는 각 원소로 1차원 주소(행의 시작 주소)를 가집니다.
 - parr[0] = arr[1]; → arr[1][0](값: 4)의 주소를 가리킴.
 - parr[1] = arr[2]; → arr[2][0](값: 7)의 주소를 가리킴.
- **출력 수식의 개별 포인터 연산 분석:**
 - **항 1) parr[1][1]:** parr[1]이 가리키는 행(arr[2])의 1번째 열 원소이므로, arr[2][1]과 같습니다. 값은 8입니다.
 - **항 2) *(parr[1] + 2):** parr[1] 시작 주소에서 2칸 전진한 곳의 역참조(parr[1][2])를 의미하므로, arr[2][2]와 같습니다. 값은 9입니다.
 - **항 3) **parr:** 이중 역참조 연산입니다. *parr는 포인터 배열의 첫 번째 원소인 parr[0](=arr[1])을 뜻하고, 여기에 한 번 더 역참조(*)를 하므로 *arr[1] 즉, arr[1][0]을 가리킵니다. 값은 4입니다.
- **최종 합산 계산:** 연산식은 8 + 9 + 4가 되므로 최종 출력 결과는 21이 됩니다.

포인터 수식	동일한 배열 표현	대상 값
parr[1][1]	arr[2][1]	8
*(parr[1] + 2)	arr[2][2]	9
**parr	arr[1][0]	4
총합 (Sum)	8 + 9 + 4 = 21	

메모리 공간 매핑 및 시각화

[2차원 배열 arr]

arr[0]	1	2	3
arr[1]	4	5	6
arr[2]	7	8	9

[포인터 배열 parr]

parr[0] → arr[1] 시작점 가리킴
parr[1] → arr[2] 시작점 가리킴

[parr]

[0][0]	[0][1]	[0][2]
[1][0]	[1][1]	[1][2]

결과 21

27. 다음 C 언어로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. 24-2

```
#include <stdio.h>
struct node {
    int data;
    struct node* Next;
};
int main() {
    struct node* head = NULL;
    struct node a = {10, 0};
    struct node b = {20, 0};
    struct node c = {30, 0};
    head = &a;
    a.Next = &b;
```

```

b.Next = &c;
printf("%d", head->Next->data);
}

```

답: 20

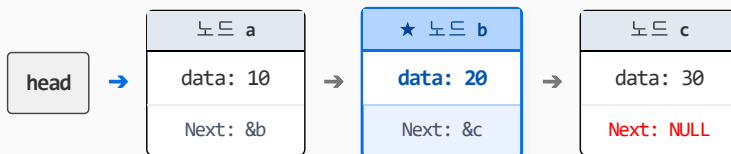
[해설]

구조체 자체 참조 포인터를 이용한 단방향 연결 리스트(Linked List) 탐색 분석

- **구조체 정의 및 선언:** 멤버로 정수형 data와 다음 노드를 가리키는 구조체 포인터 Next를 가지는 struct node가 정의되어 있습니다. main 함수 내에서 세 개의 구조체 변수 a, b, c가 각각 생성되며 Next 멤버는 초기값인 0(NULL)으로 세팅됩니다.
- **연결 리스트 데이터 링크(Link) 구성:** 주소 연산자(&)를 통해 노드 간의 참조 고리를 연결합니다.
 - head = &a; → 포인터 head가 노드 a를 가리킴.
 - a.Next = &b; → 노드 a의 다음 주소로 노드 b를 연결함.
 - b.Next = &c; → 노드 b의 다음 주소로 노드 c를 연결함.
- **화살표 연산자(->)를 통한 메모리 참조 추적:** head->Next->data 수식은 다음과 같이 단계별로 포인터를 따라갑니다.
 - head-> : head 포인터가 가리키는 대상인 **노드 a**에 접근합니다.
 - head->Next : 노드 a의 Next 멤버 값입니다. 이는 **노드 b의 주소(&b)**를 가지고 있습니다.
 - head->Next-> : 노드 b의 주소값을 통해 **노드 b** 내부 공간에 접근합니다.
 - head->Next->data : 최종적으로 노드 b의 멤버 data 값을 읽어오므로 해당 값은 **20**이 됩니다.

연산 단계	실제 참조 대상	해당 값
head	노드 a의 주소 (&a)	-
head->Next	노드 b의 주소 (&b)	-
head->Next->data	노드 b의 data 값 (b.data)	20

연결 리스트(Linked List) 메모리 링크 시각화



결과 20

28. 다음 C 언어로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. [24-1](#)

```

#include <stdio.h>
#include <stdlib.h>
main(int argc, char *argv[]) {
    int v1 = 0;
    int v2 = 35;
    int v3 = 29;
    if (v1 > v2 ? v2 : v1)
        v2 = v2 << 2;
    else
        v3 = v3 << 2;
    printf("%d", v2 + v3);
    return 0;
}

```

답: 151

[해설]

조건문 내 삼항 연산자 평가 및 비트 왼쪽 시프트(<<) 연산 매커니즘 분석

- **if문 내부의 삼항 연산 조건 평가:** 조건 식 `if (v1 > v2 ? v2 : v1)`의 분기 기준을 확인하기 위해 괄호 안의 삼항 연산식을 먼저 계산합니다.
 - `v1 > v2` 식은 `0 > 35`이므로 **거짓(False)**이 됩니다.
 - 삼항 연산 조건이 거짓이므로 콜론(:) 우측의 거짓 항인 `v1`의 값(`0`)이 최종 선택됩니다.
 - 결과적으로 조건문은 `if (0)`과 완전히 동일한 상태가 됩니다.
- **분기문 제어 흐름 결정:** C 언어에서는 정수 `0`을 '**거짓(False)**'으로 취급하고, `0`이 아닌 모든 정수를 '참(True)'으로 인식합니다.
 - 조건식의 최종 평가 결과가 `0`(거짓)이므로 `if` 블록을 실행하지 않고 **else 블록 영역으로 제어 흐름이 이동**합니다.
- **왼쪽 비트 시프트 (<<) 산술 연산 적용:** `else` 블록 내의 수식인 `v3 = v3 << 2;`를 수행합니다.
 - 정수형 데이터의 왼쪽 비트 시프트 연산(`x << N`)은 10진수 기준 원래 값에 `2`의 `N`제곱(`2^N`)을 곱한 결과와 동일합니다.
 - 따라서 변수 `v3`는 `29 * 2^2 = 29 * 4 = 116`이 됩니다. (이때 실행되지 않은 `if` 블록 내의 변수 `v2`는 처음 값인 `35`를 그대로 유지합니다.) `29=0001 1101` 왼쪽2비트=`0111 0100`를 10진수로 `116`
- **최종 연산 및 결과 출력:** 출력 서식인 `v2 + v3`의 산술 연산을 통해 최종 값을 도출합니다.
 - `35 + 116 = 151`이 계산되어 화면에 출력됩니다.

대상 변수	비트 시프트 연산 과정	최종 결과값	v1	v2	v3	패팅	원	오
v2 (if 조건 불일치)	연산 미수행 (유지)	35	0	35	29	양	0	0
v3 (else 조건 일치)	<code>29 << 2</code> (즉, <code>29 * 4</code>)	116						
printf 출력 대상	v2 + v3 → 35 + 116 = 151				116	음	0	1

결과 **151**

29. 다음 C 언어로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. [24-1](#)

```
#include <stdio.h>
#include <string.h>
void inverse(char *str, int len) {
    for(int i = 0, j = len - 1; i < j; i++, j--) {
        char ch = str[i];
        str[i] = str[j];
        str[j] = ch;
    }
}
int main() {
    char str[100] = "ABCDEFGH";
    int len = strlen(str);
    inverse(str, len);
    for(int i = 1; i < len; i += 2) {
        printf("%c", str[i]);
    }
    return 0;
}
```

답: **GECA**

[해설]

문자열 역순 정렬(Inversion) 매커니즘 및 증감 조건 루프 탐색 과정 분석

- **초기 변수 및 길이 할당:** 배열 `str`에 저장된 문자열 리터럴 "ABCDEFGH"의 길이는 총 8자입니다. `strlen(str)` 함수에 의해 변수 `len`에는 정수 `8`이 대입됩니다.
- **inverse 함수 호출을 통한 역순 반전 알고리즘:** `inverse(str, len);`을 호출하면 양 끝 인덱스 포인터 `i(0)`와 `j(7)`를 교환(Swap)한 뒤, 포인터를 안쪽으로 좁혀가며 대칭 데이터들을 뒤집습니다.
 - 1회차 (`i=0, j=7`): `str[0]('A') ↔ str[7]('H')` 교환
 - 2회차 (`i=1, j=6`): `str[1]('B') ↔ str[6]('G')` 교환
 - 3회차 (`i=2, j=5`): `str[2]('C') ↔ str[5]('F')` 교환

- 4회차 (i=3, j=4): str[3]('D') ↔ str[4]('E') 교환
- 5회차 연산 시점에는 i=4, j=3이 되어 조건식 i < j가 거짓이 되므로 루프를 탈출합니다. 함수 종료 시점에서 문자열은 완전 대칭 반전된 "HGFEDCBA"가 됩니다.
- **main 함수 내부 출력 루프 조건 분석:** 뒤집힌 배열을 인덱스 제어식 for(int i = 1; i < len; i += 2) 구문으로 순회합니다. 초기값 i가 1부터 시작하고 매 루프마다 2씩 증가하므로 오직 홀수 번째 인덱스 공간만 추출하여 출력하게 됩니다.
 - i = 1 일 때 → str[1] 출력 → G
 - i = 3 일 때 → str[3] 출력 → E
 - i = 5 일 때 → str[5] 출력 → C
 - i = 7 일 때 → str[7] 출력 → A
 - i = 9 가 되면 조건식 i < 8에 위배되므로 반복문이 종료됩니다.

배열 인덱스	0	1	2	3	4	5	6	7
초기 상태 (str)	A	B	C	D	E	F	G	H
inverse 완료 후	H	G	F	E	D	C	B	A
출력 여부	-	출력①	-	출력②	-	출력③	-	출력④

포인터 위치 추적 시각화

[반복 연산 제어]

- i = 1 (str[1]) → G 인쇄 후 i는 3으로 복합 증가
- i = 3 (str[3]) → E 인쇄 후 i는 5로 복합 증가
- i = 5 (str[5]) → C 인쇄 후 i는 7로 복합 증가
- i = 7 (str[7]) → A 인쇄 후 i는 9로 증가하여 루프 종료

결과 **GECA**

30. 다음 C 언어로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. [24-1](#)

```
#include <stdio.h>
#include <ctype.h>
int main() {
    char *p = "It is 8";
    char result[100];
    int i;
    for(i = 0; p[i] != '\0'; i++) {
        if(isupper(p[i]))
            result[i] = (p[i] - 'A' + 5) % 25 + 'A';
        else if(islower(p[i]))
            result[i] = (p[i] - 'a' + 10) % 26 + 'a';
        else if(isdigit(p[i]))
            result[i] = (p[i] - '0' + 3) % 10 + '0';
        else if(!(isupper(p[i]) || islower(p[i]) || isdigit(p[i])))
            result[i] = p[i];
    }
    result[i] = '\0';
    printf("변환된 문자열 : %s\n", result);
    return 0;
}
```

답: 변환된 문자열 : Nd sc 1

[해설]

isupper(): 인수가 대문자이면 참을 반환하는 함수

islower(): 인수가 소문자이면 참을 반환하는 함수

isdigit(): 인수가 숫자이면 참을 반환하는 함수

ctype 함수군을 통한 데이터 식별 및 모듈러(%) 암호화 연산 프로세스 분석

문자열 "It is 8"의 각 글자를 판별 함수에 대입해 조건에 맞는 암호 알고리즘 시프트 연산을 순차 수행합니다.

- **'I' (대문자 분기 - 인덱스 0):** isupper 조건에 걸립니다. 연산식은 $(\text{'I'} - \text{'A'} + 5) \% 25 + \text{'A'}$ 입니다.
 - 알파벳 순서상 $\text{'I'} - \text{'A'} = 8$ 이며, 여기에 5를 더하면 13이 됩니다.
 - $13 \% 25 = 13$ 이고, 13 + 'A'는 'A'로부터 13번째 뒤의 알파벳 대문자인 **'N'**이 됩니다.
- **'t' (소문자 분기 - 인덱스 1):** islower 조건식 연산: $(\text{'t'} - \text{'a'} + 10) \% 26 + \text{'a'}$
 - 알파벳 순서 $\text{'t'} - \text{'a'} = 19$ 이며, 10을 더하면 29입니다.
 - 알파벳 전체 수로 모듈러 연산한 결과 $29 \% 26 = 3$ 이 되며, 3 + 'a'는 4번째 소문자인 **'d'**가 매핑됩니다.
- **' ' (공백 분기 - 인덱스 2, 5):** 알파벳도 숫자도 아니므로 마지막 예외 구문을 만나 공백(' ') 문자 그대로 통과하여 배열에 저장됩니다.
- **'i', 's' (소문자 분기 - 인덱스 3, 4):** 같은 메커니즘으로 연산됩니다.
 - **'i':** $(8 + 10) \% 26 + \text{'a'} = 18 + \text{'a'} \rightarrow \text{'s'}$
 - **'s':** $(18 + 10) \% 26 + \text{'a'} = 28 \% 26 + \text{'a'} = 2 + \text{'a'} \rightarrow \text{'c'}$
- **'8' (숫자 분기 - 인덱스 6):** isdigit 조건식 연산: $(\text{'8'} - \text{'0'} + 3) \% 10 + \text{'0'}$
 - 정수값 변환 $8 + 3 = 11$ 이며, 10진수 순환을 위한 모듈러 연산 $11 \% 10 = 1$ 이 됩니다.
 - 문자로 다시 환원하여 **'1'**이 확정됩니다.

원문 원소	해당 분기	수식 변환 상세	변환 결과
I	isupper	$(8 + 5) \% 25 + \text{'A'}$	N
t	islower	$(19 + 10) \% 26 + \text{'a'}$	d
[공백]	else	그대로 통과	[공백]
i	islower	$(8 + 10) \% 26 + \text{'a'}$	s
s	islower	$(18 + 10) \% 26 + \text{'a'}$	c
[공백]	else	그대로 통과	[공백]
8	isdigit	$(8 + 3) \% 10 + \text{'0'}$	1

	0	1	2	3	4	5	6	7
result	78	100		115	99		1	\0
	N	d		s	c		1	

결과 변환된 문자열 : Nd sc 1

31. 다음 C 언어로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. [23-3](#)

```
#include <stdio.h>
main() {
    char* p = "KOREA";
    printf("1. %s\n", p);
    printf("2. %s\n", p + 1);
    printf("3. %c\n", *p);
    printf("4. %c\n", *(p + 3));
    printf("5. %c\n", *p + 4);
}
```

답: 1. KOREA
2. OREA
3. K
4. E
5. O

[해설]

포인터 주소 연산, 역참조(*), 아스키코드 산술 연산의 매커니즘 분석

- 문자열 포인터의 초기 구조: `char* p = "KOREA";` 구문에 의해 포인터 변수 `p`는 문자열 리터럴의 시작점인 첫 번째 문자 'K'의 메모리 주소를 가리키게 됩니다.
- 출력 스트림 및 연산 과정 상세 디코딩:
 - 1번 라인 (`p`): 서식 지정자 `%s`는 시작 주소부터 널 문자(`'\0'`) 전까지 전체 문자열을 출력합니다. 따라서 시작 주소 `p`를 전달하므로 전체 데이터인 **KOREA**가 출력됩니다.
 - 2번 라인 (`p + 1`): 포인터 주소를 1 바이트 이동시킵니다. `p + 1` 주소는 문자 'O'의 위치를 나타내며, `%s` 조건에 의해 해당 지점부터 끝까지 출력되므로 **OREA**가 나타납니다.
 - 3번 라인 (`*p`): 포인터 `p`가 가리키는 실제 주소 공간의 문자 하나를 추출하는 역참조 연산입니다. 첫 번째 문자인 **K**를 추출하여 문자 서식 지정자 `%c`로 출력합니다.
 - 4번 라인 (`*(p + 3)`): 주소를 먼저 3칸 이동시킨 뒤 해당 메모리의 값을 역참조합니다. 이는 배열의 인덱스 표현식인 `p[3]`과 완전히 동치이며, 4번째 문자인 **E**가 출력됩니다.
 - 5번 라인 (`*p + 4`): 괄호가 없으므로 연산자 우선순위에 따라 역참조 `*p`가 먼저 처리되어 문자 'K'를 가져옵니다. 그 후 아스키코드 값에 정수 4를 더하는 산술 연산이 이루어집니다. 'K'의 아스키코드 값(75)에 4를 더하면 79가 되며, 이를 `%c` 포맷으로 복원하면 알파벳 4칸 뒤의 문자인 **O**가 출력됩니다.

라인	인자 수식	출력 서식	연산 대상 및 내부 매커니즘	출력 문자열
1	<code>p</code>	<code>%s</code> (문자열)	시작 주소 (<code>&p[0]</code>)부터 전체 순회	KOREA
2	<code>p + 1</code>	<code>%s</code> (문자열)	1바이트 우측 주소 (<code>&p[1]</code>)부터 끝까지 순회	OREA
3	<code>*p</code>	<code>%c</code> (문자)	시작 주소에 들어있는 글자 한 단어 추출	K
4	<code>*(p + 3)</code>	<code>%c</code> (문자)	3바이트 이동 후 해당 공간의 값 역참조 (<code>p[3]</code>)	E
5	<code>*p + 4</code>	<code>%c</code> (문자)	'K'의 아스키코드 값(75) + 4 = 79 ('O')	O

※ 모든 `printf` 출력시 끝에 문자(`\n`) 코드가 포함되어 있으므로, 반드시 줄 바꿈(엔터)합니다.

```
1. KOREA
2. OREA
결과 3. K
      4. E
      5. O
```

32. 다음 C 언어 프로그램과 그 <실행결과>를 분석하여 괄호에 공통으로 들어갈 알맞은 답을 쓰시오. [23-3](#)

```
#include <stdio.h>
struct insa {
    char name[10];
    int age;
    struct insa* impl_a;
    struct insa* impl_b;
};
main() {
    struct insa p1 = { "Kim", 28, NULL, NULL };
    struct insa p2 = { "Lee", 36, NULL, NULL };
    struct insa p3 = { "Park", 41, NULL, NULL };
    p1.impl_a = &p2;
    p2.impl_b = &p3;
    printf("%s\n", p1.impl_a(    )name);
    printf("%d", p2.impl_b(    )age);
}
```

<실행결과>

답: ->

[해설]**구조체 포인터 멤버를 통한 참조 관계 및 화살표 연산자(->)의 메커니즘 분석**

- 구조체 변수 선언 및 초기 주소 바인딩:
 - 구조체 p1, p2, p3가 각각의 이름, 나이, 포인터 영역을 가진 채 메모리에 할당됩니다.
 - p1.impl_a = &p2; 구문에 의해 p1의 구조체 포인터 멤버 impl_a는 p2의 메모리 주소를 가리키게 됩니다.
 - p2.impl_b = &p3; 구문에 의해 p2의 구조체 포인터 멤버 impl_b는 p3의 메모리 주소를 가리키게 됩니다.
- 구조체 접근 연산자의 규칙 (점 연산자 vs 화살표 연산자):
 - 일반 구조체 변수 명으로 내부 멤버에 접근할 때는 **점 연산자(.)**를 사용합니다. (예: p1.impl_a)
 - 반면, 구조체를 가리키는 '구조체 포인터 변수(또는 포인터 멤버)'를 통해 가리키고 있는 대상의 하위 멤버에 접근할 때는 반드시 **화살표 연산자(->)**를 사용해야 합니다.
- 출력식과 실행결과와 매핑을 통한 연산자 도출:
 - 첫 번째 printf: p1.impl_a는 p2 구조체의 주소를 가진 **포인터**입니다. 실행 결과로 p2의 이름인 **"Lee"**가 출력되어야 하므로 포인터 접근 연산자인 ->를 사용하여 p1.impl_a->name 구조를 완성해야 합니다.
 - 두 번째 printf: p2.impl_b 역시 p3 구조체의 주소를 가진 **포인터**입니다. 실행 결과로 p3의 나이인 **41**이 출력되어야 하므로 동일하게 ->를 써서 p2.impl_b->age가 되어야 합니다.
 - 따라서 공통으로 들어갈 연산자는 **->**입니다.

구조체 인스턴스	name 멤버	age 멤버	포인터 연결 상태 (참조 주소)
p1	"Kim"	28	impl_a → &p2 (p2의 주소)
p2	"Lee"	36	impl_b → &p3 (p3의 주소)
p3	"Park"	41	impl_a / b → NULL

※ 참고: p1.impl_a->name은 포인터가 가리키는 값을 먼저 역참조하는 표현식인 (*p1.impl_a).name과 문법적으로 완전히 동일합니다.

33. 다음 C 언어로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. 23-3

```
#include <stdio.h>
int isPerfectNum(int num) {
    int sum = 0;
    for (int i = 1; i < num; i++) {
        if (num % i == 0)
            sum += i;
    }
    if (num == sum) return 1;
    else return 0;
}
main() {
    int r = 0;
    for (int i = 1; i <= 100; i++) {
        if (isPerfectNum(i))
            r += i;
    }
    printf("%d", r);
}
```

답: 34

[해설]**완전수(Perfect Number) 판별 알고리즘 및 지정 범위 내 완전수 합산 분석**

- isPerfectNum(num) 함수의 수학적 기능 정의:
 - 매개변수 num을 받아 자기 자신을 제외한 진약수(Proper Divisor)를 루프로 탐색합니다.

- $\text{num} \% i == 0$ 조건에 부합하는 모든 약수 i 를 변수 sum 에 계속 누적합산합니다.
- 탐색이 종료된 후, $\text{if} (\text{num} == \text{sum})$ 판별식을 통해 '자신을 제외한 약수들의 합이 자기 자신과 같은 수', 즉 완전수인 경우 참(1)을 반환하고 그렇지 않으면 거짓(0)을 반환합니다.
- **main() 함수의 반복 범위를 통한 완전수 추적 (1 ~ 100):**
 - 제어문은 $i = 1$ 부터 $i = 100$ 까지 순차적으로 함수를 호출하여 검증을 시도합니다. 이 범위 안에서 함수가 참(1)을 반환하는 값은 오직 6과 28 두 개만 존재합니다.
- **주요 완전수의 연산 디코딩 상세:**
 - **값 $i = 6$ 일 때:** 6의 진약수는 1, 2, 3입니다. $1+2+3=6$ 이 되어 조건식 ($6 == 6$)이 성립하므로 완전수입니다. 변수 r 에 누적되어 $r = 0 + 6 = 6$ 이 됩니다.
 - **값 $i = 28$ 일 때:** 28의 진약수는 1, 2, 4, 7, 14입니다. $1+2+4+7+14=28$ 이 되어 조건식 ($28 == 28$)이 성립하므로 완전수입니다. 변수 r 에 추가 누적되어 $r = 6 + 28 = 34$ 가 됩니다.
- **최종 결과 출력:** 1부터 100 사이의 루프가 전부 완료된 후 최종적으로 r 의 값인 34를 출력합니다.

발견된 수 (i)	자신을 제외한 약수의 합 (sum)	판별 결과	누적 변수 r
6	$1 + 2 + 3 = 6$	참 (1)	6
28	$1 + 2 + 4 + 7 + 14 = 28$	참 (1)	34
기타 (1~100)	자신의 수와 일치하지 않음	거짓 (0)	변동 없음

34. 다음 C 언어로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. [23-3](#)

```
#include <stdio.h>
int f(int n) {
    if (n <= 1) return 1;
    else return n * f(n - 1);
}
main() {
    printf("%d", f(7));
}
```

답: 5040

[해설]

재귀 함수(Recursive Function)를 이용한 팩토리얼($n!$) 구현 논리 분석

- **재귀 조건 및 탈출 조건(기저 조건) 정의:**
 - 함수 $f(n)$ 은 매개변수 n 이 1 이하가 될 때까지 자기 자신을 $n - 1$ 값으로 계속해서 다시 호출하는 재귀 구조를 씁니다.
 - $\text{if} (n \leq 1)$ 조건이 참이 되는 순간, 더 이상 재귀 호출을 하지 않고 최종 상숫값 1을 반환하면서 호출 스택을 거꾸로 거슬러 올라가기 시작합니다.
- **$f(7)$ 호출에 따른 재귀 전개 및 수학적 연산:**
 - main 함수에서 $f(7)$ 을 호출하면 다음과 같이 역순으로 곱셈 수식이 정렬 및 누적됩니다.
 - $f(7) = 7 \times f(6)$
 - $f(7) = 7 \times (6 \times f(5))$
 - $f(7) = 7 \times 6 \times 5 \times 4 \times 3 \times 2 \times f(1)$
 - 기저 조건인 $f(1)$ 에서 1이 반환되므로 최종 수식은 $7 \times 6 \times 5 \times 4 \times 3 \times 2 \times 1 = 5040$ 이 됩니다.
- **단계별 반환값 연산 과정:**

호출 함수	분해된 연산식	계산 및 반환값
$f(1)$	기저 조건 진입 ($n \leq 1$)	1
$f(2)$	$2 \times f(1)$	$2 \times 1 = 2$
$f(3)$	$3 \times f(2)$	$3 \times 2 = 6$
$f(4)$	$4 \times f(3)$	$4 \times 6 = 24$
$f(5)$	$5 \times f(4)$	$5 \times 24 = 120$
$f(6)$	$6 \times f(5)$	$6 \times 120 = 720$

호출 함수	분해된 연산식	계산 및 반환값
f(7)	7 x f(6)	7 x 720 = 5040

35. 다음 C 언어로 구현된 프로그램을 실행시킨 결과가 "43215"일 때, <처리조건>을 참고하여 괄호에 들어갈 알맞은 식을 쓰시오. 23-2

```
#include <stdio.h>
main() {
    int n[] = { 5, 4, 3, 2, 1 };
    for (int i = 0; i < 5; i++)
        printf("%d", (      ));
}
```

<처리조건>

괄호의 식에 사용할 문자는 다음으로 제한한다.

- n, i
- +, -, /, *, %
- 0~9, (,), [,]

답: $n[(i + 1) \% 5]$ (또는 $n[(i - 4) \% 5]$ 계열 등 연산 규칙에 부합하는 식)

[해설]

나머지 연산자(Modulo)를 이용한 환형 배열 인덱스 전개 구조 분석

• 초기 배열 구조와 인덱스 매핑 정의:

• 선언된 초기 배열 원소는 다음과 같습니다:

$n[0]=5, n[1]=4, n[2]=3, n[3]=2, n[4]=1$

• 목표 실행 결과 도출을 위한 인덱스 추적 규칙:

• 프로그램 실행 결과 요구사항이 "43215"이므로, 루프 내부에서 순차적으로 호출되어야 하는 타깃 원소 값의 순서는 각각 $4 \rightarrow 3 \rightarrow 2 \rightarrow 1 \rightarrow 5$ 입니다.

• 이를 원본 배열의 인덱스 번호 관점으로 변환하면, 반복 변수 i가 0부터 4까지 순차 증가할 때 접근해야 하는 실제 물리 배열 인덱스는 $**1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 0**$ 이 되어야 합니다.

• Modulo(%) 연산식을 통한 규칙 수식화:

• 루프 제어 변수 i보다 항상 1이 큰 인덱스 값을 가리키다가, 마지막 단계에서 다시 처음(0)으로 순환 구조를 가져야 하므로 전체 요소의 개수인 5로 나눈 나머지 연산자를 이용합니다.

• 수식 전개: $(i + 1) \% 5$

→ i = 0일 때: $(0 + 1) \% 5 = n[1] = 4$

→ i = 1일 때: $(1 + 1) \% 5 = n[2] = 3$

→ i = 2일 때: $(2 + 1) \% 5 = n[3] = 2$

→ i = 3일 때: $(3 + 1) \% 5 = n[4] = 1$

→ i = 4일 때: $(4 + 1) \% 5 = n[0] = 5$

반복 회차 (i)	대입 수식: $(i + 1) \% 5$	최종 타깃 인덱스	출력값 (n[인덱스])
0	$(0 + 1) \% 5 = 1$	n[1]	4
1	$(1 + 1) \% 5 = 2$	n[2]	3
2	$(2 + 1) \% 5 = 3$	n[3]	2
3	$(3 + 1) \% 5 = 4$	n[4]	1
4	$(4 + 1) \% 5 = 0$	n[0]	5

36. 다음 C 언어로 구현된 프로그램과 <처리조건>을 참고하여 괄호(①~④)에 들어갈 알맞은 식을 쓰시오. 23-2

```
#include <stdio.h>
main() {
    int m = 4620;
    int a = ( ① );
    int b = ( ② );
    int c = ( ③ );
    int d = ( ④ );
    printf("1000원의 개수 : %d\n", a);
    printf("500원의 개수 : %d\n", b);
    printf("100원의 개수 : %d\n", c);
    printf("10원의 개수 : %d\n", d);
}
```

<처리조건>

괄호(①~④)의 식에 사용할 문자는 다음으로 제한한다.

- a, b, c, d, m, i, d
- +, -, /, *, %
- 0~9, (,)

- 답: ① $m / 1000$
 ② $(m \% 1000) / 500$
 ③ $(m \% 500) / 100$
 ④ $(m \% 100) / 10$

[해설]

정수형 나눗셈 및 나머지 연산자를 활용한 순차 차감 알고리즘 분석

- **화폐 개수 계산 기본 원리 (정수 나눗셈):**
 - C 언어에서 정수와 정수의 나눗셈 연산(/)은 소수점을 제외한 몫만 취합니다. 따라서 현재 금액을 화폐 단위로 나누면 해당 화폐의 최대 필요 개수가 도출됩니다.
- **잔액 이월 구조 분석 (나머지 연산 %):**
 - 상위 화폐 단위를 지급하고 남은 잔액은 나머지 연산자(%)를 사용하여 하위 단계로 넘겨주어야 합니다.
 - 예를 들어, 1000원짜리를 정산하고 남은 돈은 $m \% 1000$ 이 되며, 이 잔액을 다시 500으로 나누어 500원 동전의 개수 b 를 구합니다.
- **수식 도출 및 연산 흐름 검증 (\$m = 4620\$ 일 때):**
 - ① **a (1000원 개수):** $4620 / 1000 = \text{몫} 4$
 - ② **b (500원 개수):** 1000원으로 나눈 나머지 잔액 $4620 \% 1000 = 620$ 원을 500으로 나눈 몫 $\rightarrow 620 / 500 = \text{몫} 1$
 - ③ **c (100원 개수):** 500원으로 나눈 나머지 잔액 $620 \% 500 = 120$ 원을 100으로 나눈 몫 $\rightarrow 120 / 100 = \text{몫} 1$
 - ④ **d (10원 개수):** 100으로 나눈 나머지 잔액 $120 \% 100 = 20$ 원을 10으로 나눈 몫 $\rightarrow 20 / 10 = \text{몫} 2$

변수	적용 연산 수식	실제 연산 과정	변수 할당값
a	$m / 1000$	$4620 / 1000$	4
b	$(m \% 1000) / 500$	$620 / 500$	1
c	$(m \% 500) / 100$	$120 / 100$	1
d	$(m \% 100) / 10$	$20 / 10$	2

37. 다음 C 언어로 구현된 프로그램을 분석하여 “홍길동”, “김철수”, “박영희”를 차례로 입력했을 때의 실행 결과를 쓰시오. [23-2](#)

```
#include <stdio.h>
char n[30];
char* getname() {
```

```

printf("이름 입력 : ");
gets(n);
return n;
}
main() {
char* n1 = getname();
char* n2 = getname();
char* n3 = getname();
printf("%s\n", n1);
printf("%s\n", n2);
printf("%s\n", n3);
}

```

답: 박영희
박영희
박영희

[해설]

전역 변수 메모리 공유 및 문자열 포인터 참조 메커니즘 분석

• 전역 문자 배열 n의 특성:

- 함수 외부에 선언된 char n[30];은 프로그램이 시작될 때 메모리의 데이터(Data/BSS) 영역에 단 **하나만 할당**되는 전역 변수입니다.
- getname() 함수가 호출될 때마다 새로운 공간이 만들어지는 것이 아니라, 항상 이 동일한 주소의 고정 공간에 gets(n)로 값을 덮어쓰게 됩니다.

• 포인터 변수 n1, n2, n3의 값 추적:

- getname() 함수는 배열 n의 시작 주소(포인터)를 반환합니다.
- 따라서 n1, n2, n3는 각각 다른 문자열을 가지는 것이 아니라, **모두 동일한 전역 배열 n의 시작 주소**를 똑같이 가리키게 됩니다 (n1 == n2 == n3 == &n[0]).

• 데이터 덮어쓰기(Overwrite) 과정:

- 1차 호출: "홍길동" 입력 → 전역 배열 n에 "홍길동" 저장. n1은 n의 주소를 가짐.
- 2차 호출: "김철수" 입력 → 전역 배열 n의 내용이 "김철수"로 변경됨. n2도 n의 주소를 가짐.
- 3차 호출: "박영희" 입력 → 전역 배열 n의 최종 내용이 "박영희"로 변경됨. n3도 n의 주소를 가짐.

• 최종 출력 결과:

- 입력이 끝난 시점에서 전역 배열 n에 남아있는 최종 문자열은 ***박영희***입니다.
- 이후 printf 문에서 n1, n2, n3를 출력하면 모두 같은 주소 영역인 n의 현재 값을 읽어오므로, 세 번 모두 "박영희"가 줄바꿈(\n)되어 출력됩니다.

실행 단계	포인터 변수 배정	전역 배열 n의 실제 값	출력문 결과
"홍길동" 입력 시	n1 = &n[0]	"홍길동"	(출력 전)
"김철수" 입력 시	n2 = &n[0]	"김철수" (덮어쓰)	(출력 전)
"박영희" 입력 시	n3 = &n[0]	"박영희" (최종값)	(출력 전)
최종 출력 단계	n1, n2, n3 공통	주소 내 값 호출	박영희 박영희 박영희

38. 다음 C 언어로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. [23-2](#)

```

#include <stdio.h>
main() {
int n[ ] = { 73, 95, 82 };
int sum = 0;
for (int i = 0; i < 3; i++)
sum += n[i];
switch (sum / 30) {

```

```

    case 10:
    case 9: printf("A");
    case 8: printf("B");
    case 7:
    case 6: printf("C");
    default: printf("D");
}

```

답: BCD

[해설]

1차원 배열 연산 및 switch-case 문의 break 부재에 따른 흐름 제어 분석

- **배열 초기화 및 루프 연산:** 크기가 3인 정수형 배열 n에 {73, 95, 82}가 순서대로 할당됩니다. 이후 for 문을 통해 배열의 모든 요소를 sum 변수에 누적하여 합산합니다.
 $sum = 73 + 95 + 82 = 250$ 이 됩니다.
- **switch 문 조건식 연산:** switch 조건식 내부의 산술 연산인 $sum / 30$ 을 계산합니다. C 언어에서 정수 간의 나눗셈은 소수점 이하를 버리므로, 정수 몫만 취하게 됩니다.
 $250 / 30 = 8$ 이 되므로 프로그램 제어 흐름은 case 8:로 점프합니다.
- **Fall-through 현상 분석:** case 8: 구문이 실행되어 "B"가 출력됩니다. 그러나 해당 블록 내부와 그 하위 case 문들에 제어 흐름을 탈출시키는 탈출문(break;)이 전혀 존재하지 않기 때문에, 아래에 위치한 조건들을 비교하지 않고 순차적으로 case 7:, case 6:, default: 구문까지 그대로 통과하며 전부 실행(Fall-through)하게 됩니다.

실행 단계	제어 위치 / 연산식	출력 문자	누적 출력
1 (반복문)	$sum = 73 + 95 + 82$	-	-
2 (조건식)	$250 / 30 = 8$ (제어 이동)	-	-
3 (진입)	case 8: printf("B");	B	B
4 (낙하)	case 7: (문장 없음)	-	B
5 (낙하)	case 6: printf("C");	C	BC
6 (낙하)	default: printf("D");	D	BCD

i	sum
0	0
1	73
2	168
3	250

switch 문 내부 제어 흐름 시각화 (break 미존재로 인한 연속 실행)

매칭 코드 블록	실행 여부
case 10, 9 -> printf("A");	Skip
[진입] case 8 -> printf("B");	실행 (B)
(낙하) case 7 -> (공백 구문)	통과
(낙하) case 6 -> printf("C");	실행 (C)
(낙하) default -> printf("D");	실행 (D)

printf 구문 수행 (텍스트 콘솔 출력)

break문 부재로 인한 제어권 자동 낙하 구간

결과 BCD

39. 다음 C 언어로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. 23-2

```

#include <stdio.h>
main() {
    int c = 0;
    for (int i = 1; i <= 2023; i++)
        if (i % 4 == 0)
            c++;
    printf("%d", c);
}

```

답: 505

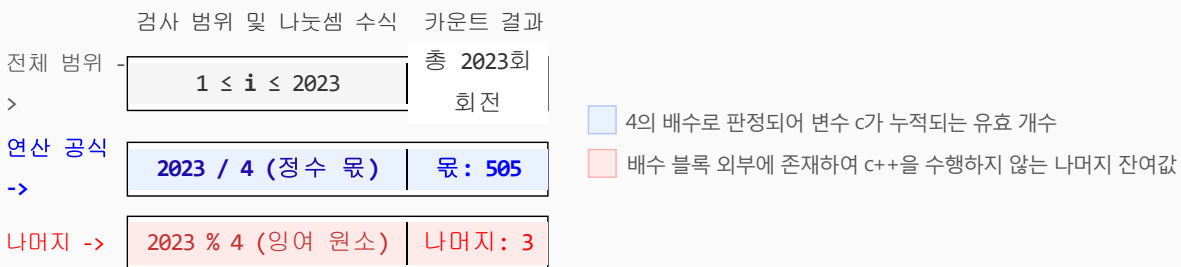
[해설]

반복문과 조건문을 활용한 특정 범위 내 배수의 개수 산출 알고리즘 분석

- 변수 초기화 및 루프 범위: 카운트 변수 c 를 0으로 초기화한 후, for 문을 통해 제어 변수 i 가 1부터 2023까지 1씩 증가하며 총 2023번 반복 수행합니다.
- 조건식을 통한 배수 필터링: 반복문 내부에서 $\text{if } (i \% 4 == 0)$ 조건을 검사합니다. 이는 변수 i 를 4로 나눈 나머지가 0인 경우, 즉 '4의 배수'일 때만 참(true)이 되어 카운트 변수 c 를 1씩 증가($c++$)시킵니다.
- 수학적 연산을 통한 빠른 도출: 1부터 특정 자연수 N 까지의 범위 내에서 k 의 배수의 개수는 $N \text{ wdiv } k$ 의 정수 몫과 동일합니다. 따라서 $2023 / 4$ 를 계산하면 정수 나눗셈에 의해 몫이 505(나머지 3)가 되므로, 최종적으로 출력되는 c 의 값은 505가 됩니다.

변수 i 값	조건식 ($i \% 4 == 0$)	변수 c 의 변화	비고
1 ~ 3	거짓 (나머지 발생)	0 (유지)	-
4	참 ($4 \% 4 == 0$)	0 → 1	첫 번째 배수
8	참 ($8 \% 4 == 0$)	1 → 2	두 번째 배수
... 중략 ...			
2020	참 ($2020 \% 4 == 0$)	504 → 505	마지막 배수
2021 ~ 2023	거짓 (루프 종료)	505 (최종)	조건 불만족

수학적 나눗셈 구조 및 카운팅 메커니즘 시각화



결과 505

40. 다음 C 언어로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. 23-2

```
#include <stdio.h>
#define MAX_SIZE 10
int isWhat[MAX_SIZE];
int point = -1;

int isEmpty() {
    if (point == -1) return 1;
    return 0;
}

int isFull() {
    if (point == 10) return 1;
    return 0;
}

void into(int num) {
    if (isFull() == 1) printf("Full");
}
```

```

    else isWhat[++point] = num;
}

main() {
    into(5); into(2);
    while (!isEmpty()) {
        printf("%d", take());
        into(4); into(1); printf("%d", take());
        into(3); printf("%d", take()); printf("%d", take());
        into(6); printf("%d", take()); printf("%d", take());
    }
}

```

답: 213465

[해설]

배열을 활용한 선형 자료구조 스택(Stack)의 내부 포인터 제어 및 출력 흐름 분석

- **구조 분석:** 전역 배열 isWhat은 데이터를 저장하는 스택 버퍼이며, point 변수는 스택의 최상단 요소를 가리키는 **Top 포인터** 역할을 합니다. into() 함수는 데이터를 삽입하는 push, 생략된 take() 함수는 데이터를 꺼내 반환하는 pop 연산에 해당하며 **후입선출(LIFO)** 방식으로 동작합니다.
- **전위 증가 연산의 특징:** into() 호출 시 isWhat[++point] = num; 구문에 의해 포인터가 먼저 1 증가한 후 해당 인덱스 자리에 값이 저장됩니다. 반대로 take() 연산은 현재 가리키는 포인터 인덱스의 값을 꺼낸 뒤 포인터를 1 감소시킵니다 (return isWhat[point--];).
- **루프 실행 흐름 추적:** while 내부의 실행 순서에 맞춰 스택 상태 변화와 출력을 정밀하게 매핑하면 결과는 다음과 같습니다.

수행 명령어 (코드 라인)	스택 내부 상태 (Bottom → Top)	출력값	누적 출력
into(5); into(2); (초기 진입)	[5, 2]	-	-
printf("%d", take());	[5]	2	2
into(4); into(1);	[5, 4, 1]	-	2
printf("%d", take());	[5, 4]	1	21
into(3);	[5, 4, 3]	-	21
printf("%d", take());	[5, 4]	3	213
printf("%d", take());	[5]	4	2134
into(6);	[5, 6]	-	2134
printf("%d", take());	[5]	6	21346
printf("%d", take());	[] (Empty 상태 진입)	5	213465

메모리 공간 내 포인터(point) 이동 및 스택 구조 시각화



결과 213465

41. 다음 C 언어로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. 23-1

```
#include <stdio.h>
main() {
    char a[] = "Art";
    char* p = NULL;
    p = a;
    printf("%s\n", a);
    printf("%c\n", *p);
    printf("%c\n", *a);
    printf("%s\n", p);
    for (int i = 0; a[i] != '\0'; i++)
        printf("%c", a[i]);
}
```

답: Art
A
A
Art
Art

[해설]

배열명 기저 주소와 포인터 변수 참조값 및 서식 지정자(%s, %c)별 매핑 분석

1. **배열과 포인터 관계식:** C 언어에서 문자열 배열 선언 시 메모리 끝에 널 문자('\0')가 자동 할당되어 총 4바이트 크기를 가집니다. 배열의 이름인 a는 배열 첫 번째 요소인 &a[0]의 시작 주소값을 의미하므로 p = a; 연산 후 포인터 변수 p 역시 동일한 시작 위치를 가리키게 됩니다.
2. **출력 라인별 상세 분석:**
 - printf("%s\n", a); : 주소값 a부터 시작해 널 문자를 만날 때까지 전체 문자열을 출력하므로 **Art**가 출력됩니다.
 - printf("%c\n", *p); : 포인터 p가 가리키는 주소에 저장된 단일 문자 값을 의미하므로 첫 글자인 **A**가 출력됩니다.
 - printf("%c\n", *a); : *a는 a[0]과 동일한 참조법이므로 단일 문자 **A**가 출력됩니다.
 - printf("%s\n", p); : 시작 주소 p에서부터 문자열 전체를 출력하므로 다시 **Art**가 출력됩니다.
 - for 루프 : 인덱스 i를 점진적으로 가산하며 널 문자를 만날 때까지 한 글자씩 연속으로 밀어내므로 개행 없이 **Art**가 합성되어 나타납니다.

메모리 번지	idx 0 (@Addr)	idx 1	idx 2	idx 3
배열 데이터	'A'	'r'	't'	'\0'
참조 관계	← 배열명 a 및 포인터 p가 가리키는 시작 기점 위치			

결과
Art
A
A
Art
Art

42. 다음 C 언어로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. [23-1](#)

```
#include <stdio.h>
main() {
    char* a = "qwer";
    char* b = "qwtety";
    for (int i = 0; a[i] != '\0'; i++)
        for (int j = 0; b[j] != '\0'; j++)
            if (a[i] == b[j])
                printf("%c", a[i]);
}
```

답: qwete

[해설]

2중 제어 루프 순회를 활용한 선형 문자 매칭 및 중복 탐색 흐름 분석

1. 알고리즘 분석: 외부 루프 인덱스 i 는 문자열 a ("qwer")를 순차 참조하고, 내부 루프 인덱스 j 는 문자열 b ("qwtety")를 매번 처음부터 끝까지 완전 스캔하면서 일치하는 문자가 발생할 때마다 화면에 즉시 출력하는 완전탐색(Brute-Force) 비교 매칭입니다.
2. 순회 과정 정밀 추적:
 - $i=0$ ($a[0]='q'$)일 때: $b[0]$ 이 'q'이므로 매칭 성공 → **q** 출력
 - $i=1$ ($a[1]='w'$)일 때: $b[1]$ 이 'w'이므로 매칭 성공 → **w** 출력
 - $i=2$ ($a[2]='e'$)일 때: b 배열을 전수 검사합니다. $b[2]='t'$, $b[3]='e'$, $b[4]='t'$, $b[5]='y'$ 입니다. 이때 $b[3]$ 이 'e'와 같으므로 조건 만족 → **e** 출력
 - $i=3$ ($a[3]='r'$)일 때: b 배열 내에 'r'과 매칭되는 값이 없으므로 스킵됩니다.
3. 반복 조건의 특징 (중요 함정): 원래 의도는 단순히 두 배열의 교집합 문자들(q, w, e)만 한 번씩 뽑아내는 구조인 것처럼 보이나, 만약 문자열 b 내부에 중복된 문자가 더 많이 존재한다면 매칭되는 횟수만큼 동일 글자가 중복 출력되므로, 한 문자씩 꼼꼼하게 일치 대입을 확인해야 실수를 방지할 수 있습니다.

결과 qwe

43. 다음 C 언어로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. (단, 출력문의 출력 서식을 준수하시오.) 20-1

```
#include <stdio.h>
int main() {
    int c = 1;
    switch (3) {
        case 1: c += 3;
        case 2: c++;
        case 3: c = 0;
        case 4: c += 3;
        case 5: c -= 10;
        default: c--;
    }
    printf("%d", c);
}
```

답: -8

[해설]

switch-case 문에서 break 명령어 부재에 따른 Fall-through(흘러내림) 현상 분석

- 변수 초기화: 정수형 변수 c 가 1로 초기화됩니다.
- switch 문 조건 분기: switch (3)에 의해 일치하는 레이블인 case 3:으로 조건이 바로 점프합니다.
- 연산 수행 및 Fall-through 발생: C 언어의 switch 문은 각 case 블록 끝에 break; 문이 없으면 제어권을 빠져나가지 못하고 매칭된 위치 이후의 하위 case 문과 default 문을 차례대로 모두 실행(Fall-through)합니다.

실행 단계	수행되는 연산식	연산 후 변수 c 의 상태
case 1, 2	(실행 건너뛴)	$c = 1$
case 3 (진입)	$c = 0;$	$c = 0$
case 4	$c += 3;$	$0 + 3 = 3$
case 5	$c -= 10;$	$3 - 10 = -7$
default	$c--;$	$-7 - 1 = -8$

메모리 및 제어 흐름 시각화

case 1, 2 ->

(우회/Pass)

case 3 점프 ->

c = 0

▼ break가 없어서 아래로 진행

case 4 ->

c = 3

case 5 ->

c = -7

default ->

c = -8

◀ 최종 값 확정

결과 -8

44. 다음은 데이터를 오름차순으로 정렬하는 선택 정렬 알고리즘을 C 언어 프로그램으로 구현한 것이다. 프로그램을 분석하여 괄호에 들어갈 알맞은 연산자를 쓰시오. [23-2](#)

```
#include <stdio.h>
main() {
    int E[] = { 64, 25, 12, 22, 11 };
    int n = sizeof(E) / sizeof(E[0]);
    int i = 0;
    do {
        int j = i + 1;
        do {
            if (E[i] (        ) E[j]) {
                int tmp = E[i];
                E[i] = E[j];
                E[j] = tmp;
            }
            j++;
        } while (j < n);
        i++;
    } while (i < n - 1);
    for (int i = 0; i <= 4; i++)
        printf("%d ", E[i]);
}
```

<처리조건>

괄호의 연산자는 다음으로 제한한다.

- +=, -=, *=, /=
- ==, !=, >, >=, <, <=
- &&, ||

답: >

[해설]

do-while 제어 구조를 통한 선택 정렬(Selection Sort) 오름차순 스왑 조건 분석

- **선택 정렬의 기본 메커니즘:** 선택 정렬은 바깥쪽 포인터가 가리키는 기준 위치(i)의 값과 안쪽 포인터가 이동하는 비교 대상(j)의 값을 차례대로 비교하여 조건에 맞으면 두 데이터의 자리를 교환(Swap)하는 정렬 방식입니다.
- **오름차순(Ascending Order) 정렬 조건식 도출:** 오름차순 정렬은 데이터가 점점 커지는 구조(11, 12, 22, 25, 64)로 배치되어야 하므로, 앞쪽에 있는 기준 요소 E[i]의 값이 뒤쪽에 있는 비교 대상 요소 E[j]의 값보다 **더 클 때** 자리를 바꾸어 뒤로 보내야 합니다. 따라서 대소 관계 비교를 위한 올바른 조건식은 if (E[i] > E[j])가 되며 빈칸에 알맞은 연산자는 >입니다.

- 동작 검증 (1회전 단계 회전): 초기 상태 E = { 64, 25, 12, 22, 11 }에서 i=0일 때 내부 패스의 흐름은 아래와 같이 유
기적으로 변화합니다.

비교 인덱스	비교 연산 (E[i] > E[j])	결과 및 스왑	배열 상태 변동 (E[0] ~ E[4])
초기 배열	-	-	{ 64, 25, 12, 22, 11 }
i=0 vs j=1	E[0] > E[1] → 64 > 25	참 (Swap)	{ <u>25</u> , <u>64</u> , 12, 22, 11 }
i=0 vs j=2	E[0] > E[2] → 25 > 12	참 (Swap)	{ <u>12</u> , <u>64</u> , <u>25</u> , 22, 11 }
i=0 vs j=3	E[0] > E[3] → 12 > 22	거짓 (유지)	{ 12, 64, 25, 22, 11 }
i=0 vs j=4	E[0] > E[4] → 12 > 11	참 (Swap)	{ <u>11</u> , 64, 25, 22, <u>12</u> }
1회전 완료 후 최솟값 확정 상태			E[0] = 11 (고정)

메모리 요소간 정렬 메커니즘 도식화

64	25	12	22	11
----	----	----	----	----

E[i] E[j] ← E[i] > E[j] 조건이 충족되어
자리 교환

결과 11 12 22 25 64

45. 다음은 2진수 101110을 10진수로 변환하는 C 언어 프로그램이다. 프로그램을 분석하여 괄호(①~②)에 들어갈 알맞은 답을 쓰시오. [23-1](#)

```
#include <stdio.h>
main() {
    int input = 101110;
    int di = 1;
    int sum = 0;
    while (1) {
        if (input == 0) break;
        sum = sum + (input ① )( ② ) * di;
        di = di * 2;
        input = input / 10;
    }
    printf("%d", sum);
}
```

답: ① % ② 10

[해설]

10진수 정수 형태로 표현된 이진수 문자 배열 대응 데이터를 최하위 비트(LSB)부터 추출하는 수학적 기법 분석

1. 알고리즘 수학적 모델링: 변수 input에 실제 2진수가 아닌 일반 10진수 정수 101110이 담겨 있습니다. 이 구조에서 정수형의 특성을 이용해 가중치 1의 자리, 10의 자리 값을 잘라내어 2진법으로 역산합니다.
2. 연산자 도출 과정:
 - 매 회전 시 input = input / 10; 연산에 의해 정수 나눗셈 처리가 진행되며 최하위 자릿수(1의 자리)가 한 개씩 제거됩니다. (101110 → 10111 → 1011 ...)
 - 따라서 매 루프에서 분리 추출 가산해야 하는 정보는 "현재 수치 값을 10으로 나눈 나머지 정보(즉, 최하위 1자리 정수 값)"가 됩니다. C 언어에서 나머지를 구하는 기호는 %이므로 ①번 칸에는 %, ②번 칸에는 피연산자 기준 값인 10이 대입되어야 수식이 완성됩니다.
 - 추출된 1자리 값(0 또는 1)에 2의 거듭제곱 가중치 변수 di(1 → 2 → 4 → 8 ...)를 곱한 값을 sum에 계속 누적 연산하여 10진법 복원을 완성하게 됩니다.

46. 다음은 버블 정렬을 이용하여 배열에 저장된 수를 오름차순으로 정렬하는 프로그램이다. 프로그램을 분석하여 괄호(①~②)에 들어갈 알맞은 답을 쓰시오. [23-1](#)

```
#include <stdio.h>
void swap(int* a, int idx1, int idx2) {
    int t = a[idx1];
    a[idx1] = a[idx2];
    a[①] = t;
}

void Usort(int* a, int len) {
    for (int i = 0; i < len - 1; i++)
        for (int j = 0; j < len - 1 - i; j++)
            if (a[j] > a[j + 1])
                swap(a, j, j + 1);
}

main() {
    int a[] = { 85, 75, 50, 100, 95 };
    int nx = 5;
    Usort(a, ②);
}
```

답: ① **idx2** ② **nx (또는 5)**

[해설]

배열의 임시 변수 값 교환(Swap) 기법 메커니즘 및 정렬 함수 인자 매핑 관계 분석

- ①번 빈칸 분석 (값 임시 교환 3단계 알고리즘): 두 공간의 변수 값을 맞바꾸기 위해 빈 기억 장소 t를 사용하고 있습니다.
 - t = a[idx1]; : 백업을 위해 idx1번 데이터를 임시 보관함에 복사
 - a[idx1] = a[idx2]; : 유실 걱정이 없어진 idx1번 자리에 idx2번 데이터를 대입
 - 따라서 최종적으로는 idx2번 자리에 백업해 두었던 값인 t를 덮어쓰워 매칭을 완결해야 하므로 대괄호 안의 식은 **idx2**가 됩니다.
- ②번 빈칸 분석 (매개변수 전달 규칙): 하단의 정렬 처리부인 Usort() 함수의 원형 선언문을 보면 첫 번째 인자로 배열 주소(int* a)를 받고, 두 번째 인자로 배열의 전체 길이 정보(int len)를 전달받아 연산 한계 영역을 설정합니다. main() 함수 영역 내에서 배열 크기로 선언된 제어 변수가 nx = 5;이므로 해당 인자 자리에 변수명인 **nx** 혹은 리터럴 상수 값인 **5**를 대입하는 것이 올바른 구조 정의입니다.

프로그래밍 - JAVA활용 39문제

[클래스 기본 39문제] [001로](#)

1. 다음 Java로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. [22-2](#)

```
class Test {
    public static void main(String args[]) {
        cond obj = new cond(3);
        obj.a = 5;
        int b = obj.func( );
        System.out.print(obj.a + b);
    }
}

class cond {
    int a;
    public cond(int a) {
        this.a = a;
    }
}
```

```

    }
    public int func( ) {
        int b = 1;
        for (int i = 1; i < a; i++)
            b += a * i;
        return a + b;
    }
}

```

답: 61

[해설]

```

class Test {
    public static void main(String args[]) {
        ① cond obj = new cond(3);
        ④ obj.a = 5;
        ⑤ int b = obj.func( );
        ⑫ System.out.print(obj.a + b);
    }
}

class cond {
    int a;
    ② public cond(int a) {
        ③ this.a = a;
    }
    ⑥ public int func( ) {
        ⑦ int b = 1;
        ⑧ for (int i = 1; i < a; i++)
            ⑨ b += a * i;
        ⑩ return a + b;
    }
}

```

모든 Java 프로그램은 반드시 main() 메소드에서 시작한다.

- ① 3을 인수로 생성자를 호출하여 cond 클래스의 객체 변수 obj를 선언한다.
- ② cond 클래스 생성자의 시작점이다. ①번에서 전달받은 3을 a가 받는다.
- ③ cond 클래스의 a에 3을 저장한다. 생성자가 종료되면 호출했던 ①번의 다음 줄인 ④번으로 이동한다. (obj.a = 3)
 • this: 현재의 실행중인 메소드가 속한 클래스를 가리키는 예약어이다. 여기에서는 cond 클래스의 객체 변수 obj의 생성자로 호출되었으므로 'obj.a'와 같은 의미이다.
- ④ obj.a에 5를 저장한다. → **obj.a = 5**
- ⑤ 정수형 변수 b를 선언하고 obj.func() 메소드를 호출한 후 돌려받은 값으로 초기화한다.
- ⑥ 정수를 반환하는 func() 메소드의 시작점이다.
- ⑦ 정수형 변수 b를 선언하고 1로 초기화한다.
- ⑧ 반복 변수 i가 1부터 1씩 증가하면서 a보다 작은 동안 ⑨번을 반복 수행한다. func() 메소드에는 별도로 생성한 'a'라는 변수가 없으므로 cond 클래스의 a를 가져와 사용한다. 즉 ⑨번은 5보다 작은 동안 반복 수행된다.
- ⑨ 'b = b + (a * i);'와 동일하다. a에 i를 곱한 값을 b에 누적시킨다.

※ 반복문 실행에 따른 변수들의 변화는 다음과 같다.

a	i	b
5		1
	1	6
	2	16
	3	31

a	i	b
	4	51
	5	

⑩ 5와 51을 더한 값 56을 메소드를 호출했던 ⑪번으로 반환한다.

⑪ b에 56이 저장된다.

⑫ 5+56의 결과인 61을 출력한다.

결과 61

2. 다음 Java로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. [22-1](#)

```
class A {
    int a;
    int b;
}

public class Test {
    static void func1(A m) {
        m.a *= 10;
    }
    static void func2(A m) {
        m.a += m.b;
    }
    public static void main(String args[]) {
        A m = new A();
        m.a = 100;
        func1(m);
        m.b = m.a;
        func2(m);
        System.out.printf("%d", m.a);
    }
}
```

답: 2000

[해설]

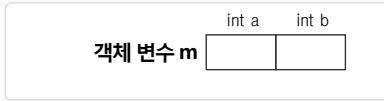
```
class A {
    int a;           // 클래스 A를 정의한다. 정수형 변수 a와 b가 선언되어 있다.
    int b;
}

public class Test {
    ④ static void func1(A m) {
        ⑤ m.a *= 10;
    }
    ⑧ static void func2(A m) {
        ⑨ m.a += m.b;
    }
    public static void main(String args[]) {
        ① A m = new A();
        ② m.a = 100;
        ③ func1(m);
        ⑥ m.b = m.a;
        ⑦ func2(m);
        ⑩ System.out.printf("%d", m.a);
    }
}
```

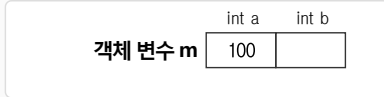
```
}  
}
```

모든 Java 프로그램은 반드시 main() 메소드에서 시작한다.

- ❶ 클래스 A의 객체 변수 m을 선언한다.



- ❷ 객체 변수 m의 변수 a에 100을 저장한다.



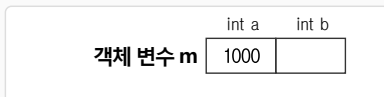
- ❸ 객체 변수 m의 시작 주소를 인수로 하여 func1 메소드를 호출한다.

- ❹ 반환값이 없는 func1() 메소드의 시작점이다. ❸번에서 전달받은 주소를 m이 받는다.

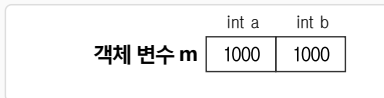
※ 객체 변수나 배열의 이름은 객체 변수나 배열의 시작 주소를 가리키므로, 인수로 전달하는 경우 메소드에서 변경된 값이 main()의 객체 변수나 배열에도 적용된다는 점을 염두에 두세요.

- ❺ 'ma = ma * 10;'과 동일하다. ma에 10을 곱한 값을 ma에 저장한다. 메소드가 종료되었으므로 메소드를 호출했던

- ❸번의 다음 줄인 ❹번으로 이동한다.



- ❻ m.b에 ma의 값 1000을 저장한다.



- ❼ 객체 변수 m의 시작 주소를 인수로 하여 func2 메소드를 호출한다.

- ❽ 반환값이 없는 func2() 메소드의 시작점이다. ❼번에서 전달받은 주소를 m이 받는다.

- ❾ 'ma = ma + m.b;'와 동일하다. ma와 m.b를 합한 값을 ma에 저장한다. 메소드가 종료되었으므로 메소드를 호출했던 ❼번의 다음 줄인 ❺번으로 이동한다.



- ❿ ma의 값 2000을 정수로 출력한다.

결과 2000

[상속과 재정의]

3. 다음 Java로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. [21-2](#)

```
public class ovrl {  
    public static void main(String[] args) {  
        ovrl a1 = new ovrl();  
        ovrl2 a2 = new ovrl2();  
        System.out.println(a1.sun(3,2) + a2.sun(3,2));  
    }  
    int sun(int x, int y) {  
        return x + y;  
    }  
}
```

```

    }
}

class ovr12 extends ovr1 {
    int sun(int x, int y) {
        return x - y + super.sun(x, y);
    }
}

```

답: 11

[해설]

```

public class ovr1 {
    public static void main(String[] args) {
        1 ovr1 a1 = new ovr1();
        2 ovr12 a2 = new ovr12();
        3&12 System.out.println(a1.sun(3,2) + a2.sun(3,2));
    }
    4&9 int sun(int x, int y) {
        5&10 return x + y;
    }
}

class ovr12 extends ovr1 {           // 클래스 ovr12를 정의하고 부모 클래스로 ovr1을 지정하면서 ovr1에 속한 변수와 메소드를 상속받는다.
    7 int sun(int x, int y) {
        8&11 return x - y + super.sun(x, y);
    }
}

```

모든 Java 프로그램은 반드시 main() 메소드에서 시작한다.

- 1 클래스 ovr1의 객체 변수 a1을 선언한다.
- 2 클래스 ovr12의 객체 변수 a2를 선언한다.
- 3 3과 2를 인수로 a1의 sun() 메소드를 호출한 결과와, 3과 2를 인수로 a2의 sun() 메소드를 호출한 결과를 합하여 출력한 후 커서를 다음 줄로 옮긴다. 먼저 a1의 sun() 메소드를 호출한다.
- 4 정수를 반환하는 a1의 sun() 메소드의 시작점이다. 3번에서 전달받은 3과 2를 x와 y가 받는다.
- 5 x와 y를 더한 값 5를 메소드를 호출했던 3번으로 반환한다.
- 6 5번으로부터 a1의 sun() 메소드를 호출한 결과로 5를 전달받았으므로, 이번에는 3과 2를 인수로 a2의 sun() 메소드를 호출한다.
- 7 정수를 반환하는 a2의 sun() 메소드의 시작점이다. 6번에서 전달받은 3과 2를 x와 y가 받는다.
- 8 x에서 y를 뺀 값에 3과 2를 인수로 부모 클래스인 ovr1의 sun() 메소드를 호출한 결과를 더하여 함수를 호출했던 12번으로 반환한다.
- **super**: 자식 클래스에서 상속해 준 부모 클래스를 가리키는 예약어이다. 여기에서는 ovr1 클래스의 sun() 메소드를 가리키게 된다. 부모 클래스의 sun() 메소드를 호출하기 위해 9번으로 이동한다.
- 9-11 3과 2를 인수로 ovr1의 sun() 메소드를 수행한 결과를 4-5번에서 구했으므로 그 결과를 그대로 사용하면 된다. 10번에서 5를 돌려받아 계산한 값 6(3-2+5)을 메소드를 호출했던 12번으로 반환한다.
- 12 5번으로부터 돌려받은 5와 11번으로부터 돌려받은 6을 더한 값 11을 출력하고 커서를 다음 줄로 옮긴다.

결과 11

4. 다음 Java로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. [20-2](#)

```

class A {
    int a;
    public A(int a) { this.a = a; }
    void display() { System.out.println("a=" + a); }
}
class B extends A {
    public B(int a) {
        super(a);
        super.display();
    }
}
public class Test {
    public static void main(String[] args) {
        B obj = new B(10);
    }
}

```

답: **a=10**

[해설]

```

class A {                                // 클래스 A를 정의한다.
    int a;
    ④ public A(int a) { ⑤ this.a = a; }
    ⑦ void display() { ⑧ System.out.println("a=" + a); }
}
class B extends A {                    // 클래스 B를 정의하고 부모 클래스로 A를 지정하면서 A에 속한 변수와 메소드를 상속받는다.
    ② public B(int a) {
        ③ super(a);                    // 부모생성자 자동호출
        ⑥ super.display();
    } ⑨
}
public class Test {
    public static void main(String[] args) {
        ① B obj = new B(10);
    } ⑩
}

```

모든 Java 프로그램은 반드시 main() 메소드에서 시작한다.

- ① B obj = new B(10);
클래스 B의 객체 변수 obj를 선언하고 생성자에 인수 10을 전달한다.
- ② B 클래스 생성자 B()의 시작점이다. ①번에서 전달받은 10을 정수형 변수 a가 받는다.
- ③ 부모 클래스의 생성자를 호출하며 인수로 a의 값 10을 전달한다.
※ **super**: 상속해 준 부모 클래스를 가리키는 예약어
- ④ 생성자 A()의 시작점이다. ③번에서 전달받은 10을 생성자 A()의 변수 a가 받는다.
- ⑤ 메소드가 속한 A 클래스의 a에 A() 생성자의 변수 a의 값 10을 저장한다. 생성자가 종료되면 호출했던 ③번의 다음 줄인 ⑥번으로 간다.
※ **this**: 현재의 실행중인 메소드가 속한 클래스를 가리키는 예약어. 즉 'Aa'와 같은 의미이다.
- ⑥ 부모 클래스의 메소드 display()를 호출한다.
- ⑦ A 클래스의 메소드 display()의 시작점이다.
- ⑧ "a="를 출력한 후 a의 값을 출력해야 하지만, 메소드에서 별도로 생성한 'a'라는 변수가 없으므로 클래스의 변수 a의 값 10을 출력하고, 다음 줄의 처음으로 커서를 이동시킨다.

※ 생성자나 메소드 안에서 생성된 변수는 생성자나 메소드를 벗어나서 사용하지 못하기 때문에 여기서는 생성자 A()에 속한 a가 아닌 클래스 A에 속한 a를 출력한다.

9 메소드를 호출했던 6번의 다음 줄인 9번으로 이동하고 이어서 B 클래스를 호출했던 1번의 다음 줄인 10번으로 이동하여 프로그램을 종료한다.

결과 a=10

[인터페이스 객체]

5. 다음 Java로 구현된 프로그램을 분석하여 괄호에 들어갈 알맞은 답을 쓰시오. 22-1

```
class Car implements Runnable {
    int a;
    public void run() {
        try {
            while(++a < 100) {
                System.out.println("miles traveled : " + a);
                Thread.sleep(100);
            }
        } catch(Exception E) { }
    }
}

public class Test {
    public static void main(String args[]) {
        Thread t1 = new Thread(new (    )());
        t1.start();
    }
}
```

답: Car

[해설]

Ⓐ class Car implements Runnable {
int a;
Ⓑ public void run() {
Ⓒ 1 try {
2 while(++a < 100) {
3 System.out.println("miles traveled : " + a);
4 Thread.sleep(100);
}
Ⓓ } catch(Exception E) { }
}
}
public class Test {
public static void main(String args[]) {
Ⓔ 1 Thread t1 = new Thread(new Car());
Ⓕ 2 t1.start();
} 3
}

1. Thread를 상속 받아서 사용

```
class A extends Thread {
    public void run(){
        작업내용
    }
}
```

```
}  
}
```

2. Runnable Interface를 구현해서 사용하는 방법

```
class A implements Runnable {  
    public void run(){  
        작업내용  
    }  
}
```

㉠ class Car implements Runnable

Runnable 인터페이스를 상속받은 클래스 Car를 정의한다.

- **implements** : extends와 같이 상속에 사용하는 예약어로, 인터페이스를 상속받을 때 사용함
- **Runnable** : 스레드 클래스를 만들 때 사용하는 인터페이스

※ 인터페이스 객체는 클래스와 크게 다르지 않습니다. 그 역할이 인터페이스로 고정되어 있을 뿐 클래스와 마찬가지로 변수와 메소드를 갖는 객체입니다.

※ 스레드는 시스템의 여러 자원을 할당받아 실행하는 프로그램의 단위입니다. 대부분은 main() 메소드로 실행하는 하나의 스레드로만 작업을 수행하는데, 스레드 클래스는 main() 메소드로 실행하는 스레드 외에 추가적인 스레드를 가질 수 있도록 스레드를 생성하는 기능을 갖고 있습니다.

㉢ public void run()

Runnable 인터페이스를 상속받았다면 스레드가 수행할 작업들을 정의하는 run() 메소드를 반드시 정의해야 한다.

㉣ try {}

- 실행 중에 예외가 발생할 가능성이 있는 실행 코드들을 하나의 블록으로 묶어 놓은 곳이다. try 블록 코드를 수행하다 예외가 발생하면 예외를 처리하는 ㉤의 catch 블록으로 이동하여 예외 처리 코드를 수행하므로 예외가 발생한 이후의 코드는 실행되지 않는다.
- 4번에서 수행되는 Thread.sleep() 메소드는 인터럽트로 인한 예외를 발생시킬 가능성이 큰 메소드이므로 반드시 try ~ catch 문을 통해 예외를 처리해줘야 한다.

㉤ catch(Exception E) {}

인터럽트로 인한 예외를 처리할 수 있는 예외 객체는 InterruptedException이지만, Exception을 사용하면 InterruptedException을 포함한 대부분의 예외를 한 번에 처리할 수 있다.

㉥ Thread t1 = new Thread(new Car());

스레드 클래스의 객체 변수 t1을 선언한다. 스레드 클래스는 생성자를 호출할 때 Runnable 인터페이스를 인수로 사용한다. 여기에서는 Runnable 인터페이스를 상속받은 Car 클래스를 생성자의 인수로 사용했다.

㉦ t1.start();

t1의 start() 메소드를 호출한다. start() 메소드는 스레드 클래스에 포함된 메소드로, run() 메소드에서 정의한 코드들을 실행하는 메소드이다. 이때 run() 메소드에서 정의한 코드들은 main() 메소드와는 별개로 시스템으로부터 자원을 새로 할당받아 실행된다. 즉 main() 메소드와 별개로 실행되기 때문에 main() 메소드의 작업이 종료되어도 run() 메소드의 작업이 끝나지 않으면 계속 수행한다.

모든 Java 프로그램은 반드시 main() 메소드에서 시작한다.

- 1 스레드 클래스의 객체 변수 t1을 선언한다. 스레드에서 실행할 run() 메소드를 정의하고 있는 Car() 클래스를 생성자의 인수로 사용한다.
- 2 t1의 start() 메소드를 호출한다. Car 클래스의 run() 메소드가 실행된다. 이후 main() 메소드는 3번으로 이동하여 프로그램을 종료한다.

main() 메소드와는 별개로 시스템으로부터 자원을 새로 할당받아 run() 메소드를 시작한다.

- 1 예외를 처리하기 위한 try ~ catch문의 시작점이다.

- 2 a가 100보다 작은 동안 3, 4번을 반복 수행한다. a는 전치증가 연산이므로 a에 1을 더한 후 조건을 확인한다.

※ 클래스의 속성으로 선언된 변수 a는 자동으로 0으로 초기화됩니다.

- 3 miles traveled : 를 출력한 후 이어서 a의 값을 출력한다.

4 100을 인수로 Thread 클래스의 sleep() 메소드를 호출한다. 0.1초 동안 스레드를 일시 정지시킨다.

• Thread.sleep(n) : n/1000초 동안 스레드를 일시 정지시킨다.

2~4 번을 수행한 결과로 다음과 같이 0.1초마다 한 줄씩 출력된다.

결과

```
miles traveled : 1
miles traveled : 2
miles traveled : 3
miles traveled : 4
:
miles traveled : 98
miles traveled : 99
```

[클래스 응용 - 싱글톤 패턴]

6. 다음 Java로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. [21-3](#)

```
class Connection {
    private static Connection _inst = null;
    private int count = 0;
    static public Connection get() {
        if(_inst == null) {
            _inst = new Connection();
            return _inst;
        }
        return _inst;
    }
    public void count() { count++; }
    public int getCount() { return count; }
}

public class Test {
    public static void main(String[] args) {
        Connection conn1 = Connection.get();
        conn1.count();
        Connection conn2 = Connection.get();
        conn2.count();
        Connection conn3 = Connection.get();
        conn3.count();
        System.out.print(conn1.getCount());
    }
}
```

답: 3

[해설]

이 문제는 객체 변수 _inst가 사용하는 메모리 공간을 객체 변수 conn1, conn2, conn3이 공유함으로써 메모리 낭비를 방지하는 싱글톤(Singleton) 개념을 Java로 구현한 문제입니다.

```
class Connection {                                // 클래스 Connection을 정의한다.
    ① private static Connection _inst = null;
    ② private int count = 0;
    ③ static public Connection get() {
        ④ if(_inst == null) {
            ④ _inst = new Connection();
            ⑤ return _inst;
        }
        ⑥ return _inst;
    }
}
```

```

    }

    152 public void count() { count++; }
    24 public int getCount() { return count; }
}

public class Test {
    public static void main(String[] args) {
        16 Connection conn1 = Connection.get();
        7 conn1.count();
        9,13 Connection conn2 = Connection.get();
        14 conn2.count();
        16,20 Connection conn3 = Connection.get();
        21 conn3.count();
        23,25 System.out.print(conn1.getCount());
    }
}

```

㉠ private static Connection _inst = null;

Connection 클래스의 객체 변수 _inst를 선언하고 null로 초기화한다.

※ 객체 변수를 생성한다는 것은 Connection _inst = new Connection(); 과 같이 객체 생성 예약어인 new를 통해 heap 영역에 공간을 확보하여

Connection 클래스의 내용을 저장한 후 그 주소를 객체 변수에 저장하는 것인데, ㉠에서는 객체 생성 예약어인 new가 생략되었으므로 생성되지 않은 선언만 됩니다. 객체 변수를 선언만 하게 되면 heap이 아닌 stack 영역에 내용 없이 저장되어 사용이 불가능합니다. 이후 ㉣번과 같이 객체 생성 예약어인 new가 사용되어야만 heap 영역에 내용이 저장되고 그 주소도 객체 변수에 전달되면서 사용 가능한 객체 변수가 됩니다.

㉢ 정수형 변수 count를 선언하고, 0으로 초기화한다.

stack 영역 - 정적으로 할당된 메모리 영역

Boolean, char, short, int, long, float, double ...)

heap 영역 - 동적으로 할당된 메모리 영역

모든 오브젝트 타입의 데이터가 할당 생성된 객체가 저장

stack 영역		heap 영역	
변수	값	주소	내용
_inst	null		
count	0		

모든 Java 프로그램은 반드시 main() 메소드에서 시작한다.

1 Connection 클래스의 객체 변수 conn1을 선언하고, get() 메소드를 호출한 결과를 저장한다.

※ ㉠에서와 같이 객체 변수를 선언만 하였으므로 객체 변수 conn1은 stack 영역에 생성됩니다.

stack 영역		heap 영역	
변수	값	주소	내용
_inst	null		
count	0		
conn1			

2 Connection 형을 반환하는 get() 메소드의 시작점이다.

3 _inst가 null이면 4, 5번을 수행하고, 아니면 12번으로 이동한다. _inst가 null이므로 4번으로 이동한다.

④ Connection 클래스의 내용을 heap 영역에 저장하고 그 주소를 _inst에 저장한다.

※ ㉠에서 객체 변수 _inst는 이미 선언되었으므로, Connection _inst = new Connection(); 과 같이 작성하지 않고 앞쪽의 클래스명을 생략하여 _inst = new Connection(); 과 같이 작성합니다. 생성 예약어인 new를 통해 heap 영역에 공간을 확보하고 Connection 클래스의 내용을 저장한 후 그 주소를 객체 변수 _inst에 저장합니다. 이제 객체 변수 _inst는 Connection() 클래스의 내용이 저장된 heap 영역을 가리키게 됩니다.

stack 영역		heap 영역	
변수	값	주소	내용
_inst	100	0	
count	0	100	private static Connection _inst private int count = 0 static public Connection get() { ... } public void count() { ... } public int getCount() { ... }
conn1		200	
		300	

⑤ _inst에 저장된 값을 메소드를 호출했던 ⑥번으로 반환한다.

⑥ ⑤번에서 돌려받은 _inst의 값을 conn1에 저장한다. _inst에는 Connection() 클래스의 내용이 저장된 heap 영역의 주소가 저장되어 있으며, conn1에도 동일한 주소가 저장되므로 이후 _inst와 conn1은 같은 heap 영역의 주소를 가리키게 된다.

stack 영역		heap 영역	
변수	값	주소	내용
_inst	100	0	
count	0	100	private static Connection _inst private int count = 0 static public Connection get() { ... } public void count() { ... } public int getCount() { ... }
conn1	100	200	
		300	

⑦ conn1의 count() 메소드를 호출한다. conn1은 Connection() 클래스의 객체 변수이므로 Connection 클래스의 count() 메소드를 호출한다는 의미이다.

⑧ 반환값이 없는 count() 메소드의 시작점이다. count의 값에 1을 더한 후 count() 메소드를 호출했던 ⑦번으로 돌아가 다음 문장인 ⑨번을 수행한다. (count = 1)

stack 영역		heap 영역	
변수	값	주소	내용
_inst	100	0	
count	1	100	private static Connection _inst private int count = 0 static public Connection get() { ... } public void count() { ... } public int getCount() { ... }
conn1	100	200	
		300	

⑨ Connection 클래스의 객체 변수 conn2를 선언하고, get() 메소드를 호출한 결과를 저장한다.

※ ㉠에서와 같이 객체 변수를 선언만 하였으므로 객체 변수 conn2는 stack 영역에 생성됩니다.

stack 영역		heap 영역	
변수	값	주소	내용
_inst	100	0	
count	1	100	private static Connection _inst private int count = 0 static public Connection get() { ... } public void count() { ... } public int getCount() { ... }
conn1	100	200	
conn2		300	

⑩ Connection 형을 반환하는 get() 메소드의 시작점이다.

⑪ _inst가 null이면 ④, ⑤번을 수행하고, 아니면 ⑫번으로 이동한다. _inst에는 ④번에서 저장한 heap 영역의 주소인 100이 저장되어 있어 null이 아니므로 ⑫번으로 이동한다.

⑫ _inst에 저장된 값을 메소드를 호출했던 ⑬번으로 반환한다.

⑬ ⑫번에서 돌려받은 _inst의 값을 conn2에 저장한다.

stack 영역	
변수	값
_inst	100
count	1
conn1	100
conn2	100

heap 영역	
주소	내용
0	
100	private static Connection _inst private int count = 0 static public Connection get() { ... } public void count() { ... } public int getCount() { ... }
200	
300	

14 conn2의 count() 메소드를 호출한다.

15 반환값이 없는 count() 메소드의 시작점이다. count의 값에 1을 더한 후 count() 메소드를 호출했던 14번으로 돌아가 다음 문장인 16번을 수행한다. (count = 2)

stack 영역	
변수	값
_inst	100
count	2
conn1	100
conn2	100

heap 영역	
주소	내용
0	
100	private static Connection _inst private int count = 0 static public Connection get() { ... } public void count() { ... } public int getCount() { ... }
200	
300	

16 Connection 클래스의 객체 변수 conn3을 선언하고, get() 메소드를 호출한 결과를 저장한다.

stack 영역	
변수	값
_inst	100
count	2
conn1	100
conn2	100
conn3	

heap 영역	
주소	내용
0	
100	private static Connection _inst private int count = 0 static public Connection get() { ... } public void count() { ... } public int getCount() { ... }
200	
300	

17 Connection 형을 반환하는 get() 메소드의 시작점이다.

18 _inst가 null이면 4, 5번을 수행하고, 아니면 12번으로 이동한다. _inst에는 100이 저장되어 있으므로 null이 아니기 때문에 12번(실제로는 19번)으로 이동한다.

19 _inst에 저장된 값을 메소드를 호출했던 20번으로 반환한다.

20 19번에서 돌려받은 _inst의 값을 conn3에 저장한다.

stack 영역	
변수	값
_inst	100
count	2
conn1	100
conn2	100
conn3	100

heap 영역	
주소	내용
0	
100	private static Connection _inst private int count = 0 static public Connection get() { ... } public void count() { ... } public int getCount() { ... }
200	
300	

21 conn3의 count() 메소드를 호출한다.

22 반환값이 없는 count() 메소드의 시작점이다. count의 값에 1을 더한 후 count() 메소드를 호출했던 21번으로 돌아가 다음 문장인 23번을 수행한다. (count = 3)

stack 영역	
변수	값
_inst	100
count	3
conn1	100
conn2	100
conn3	100

heap 영역	
주소	내용
0	
100	private static Connection _inst private int count = 0 static public Connection get() { ... } public void count() { ... } public int getCount() { ... }
200	
300	

23 conn1.getCount() 메소드를 호출한다.

24 정수를 반환하는 getCount() 메소드의 시작점이다. count의 값인 3을 반환하고 getCount() 메소드를 호출했던 23번(실제로는 25)으로 이동한다.

25 24번으로부터 반환받은 값 3을 출력한다.

결과 3

[객체의 형 변환]

7. 다음 Java로 구현된 프로그램을 분석하여 괄호에 들어갈 알맞은 답을 쓰시오. 20-2

```
class Parent {
    void show() { System.out.println("parent"); }
}
class Child extends Parent {
    void show() { System.out.println("child"); }
}
public class Test {
    public static void main(String[] args) {
        Parent pa = ( new ) Child();
        pa.show();
    }
}
```

답: new

[해설]

```
class Parent {                                // 클래스 Parent를 정의한다.
    void show() { System.out.println("parent"); }
}
class Child extends Parent {                  // 클래스 Child를 정의하고 부모 클래스로 Parent를 지정한다.
    3 void show() { 4 System.out.println("child"); }
}
public class Test {
    public static void main(String[] args) {
        1 Parent pa = new Child();
        2 pa.show();
    } 5
}
```

모든 Java 프로그램은 반드시 main() 메소드에서 시작한다.

1 Parent pa = new Child();

Child 클래스의 생성자를 이용하여 Parent 클래스의 객체 변수 pa를 선언한다.

- [부모클래스명] [객체변수명] = new [자식클래스생성자()]: 부모 클래스의 객체 변수를 선언하면서 자식 클래스의 생성자를 사용하면 형 변환이 발생한다.
- 이렇게 형 변환이 발생했을 때 부모 클래스와 자식 클래스에 동일한 속성이나 메소드가 있으면 자식 클래스의 속성이나 메소드로 재정의된다.

2 pa의 show() 메소드를 호출한다. 3번으로 이동한다.

- pashow()는 pa 객체의 자료형이 Parent이므로 Parentshow()라고 생각할 수 있지만, 1번에서 자식 클래스로 객체가 생성되었고 show() 메소드가 자식 클래스에서 재정의되었으므로 Child 클래스의 show() 메소드가 수행된다.

3 Child 클래스의 show() 메소드의 시작점이다.

4 화면에 문자열 child를 출력하고, 다음 줄의 처음으로 커서를 이동시킨다. show() 메소드가 종료되었으므로 메소드를 호출했던 2번의 다음 줄인 5번으로 이동하여 프로그램을 종료한다.

8. 다음 Java로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. 20-4

```

class Parent {
    int compute(int num) {
        if(num <= 1) return num;
        return compute(num - 1) + compute(num - 2);
    }
}

class Child extends Parent {
    int compute(int num) {
        if(num <= 1) return num;
        return compute(num - 1) + compute(num - 3);
    }
}

public class Test {
    public static void main(String[] args) {
        Parent obj = new Child();
        System.out.print(obj.compute(4));
    }
}

```

답: 1

[해설]

```

class Parent {
    int compute(int num) {
        if(num <= 1) return num;
        return compute(num - 1) + compute(num - 2);
    }
}

class Child extends Parent {
    ③ int compute(int num) {
        ④ if(num <= 1) return num;
        ⑤ return compute(num - 1) + compute(num - 3);
    }
}

public class Test {
    public static void main(String[] args) {
        ① Parent obj = new Child();
        ② System.out.print(obj.compute(4));
    }
}

```

모든 Java 프로그램은 반드시 main() 메소드에서 시작한다.

① Parent obj = new Child();

자식 클래스인 Child의 생성자를 사용하여 부모 클래스 Parent의 객체 변수 obj를 선언하여 형 변환을 수행한다.

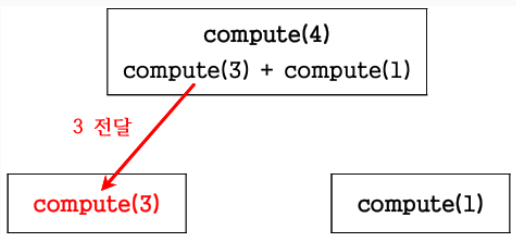
② obj.compute(4) 메소드를 호출한다.

• obj 객체의 자료형이 Parent이므로 Parent.compute()라고 생각할 수 있지만 ①번에서 클래스 형 변환이 발생하였고 compute() 메소드가 자식 클래스인 Child에서 재정의(Overriding)되었으므로 **Child의 compute() 메소드가** 수행된다.

③ compute() 메소드의 시작점이다. ②번에서 전달한 인자 값 4를 정수형 변수 num이 받는다.

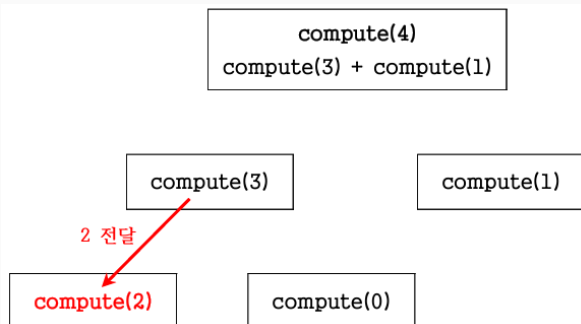
④ num이 1보다 작거나 같으면 num의 값을 반환하고 메소드를 종료한다. num의 값이 4이므로 ⑤번으로 이동한다.

5 compute(3)을 호출하여 돌려받은 값과 compute(1)을 호출하여 돌려받은 값을 더한 후 반환해야 하므로 먼저 compute(3)을 호출한다.



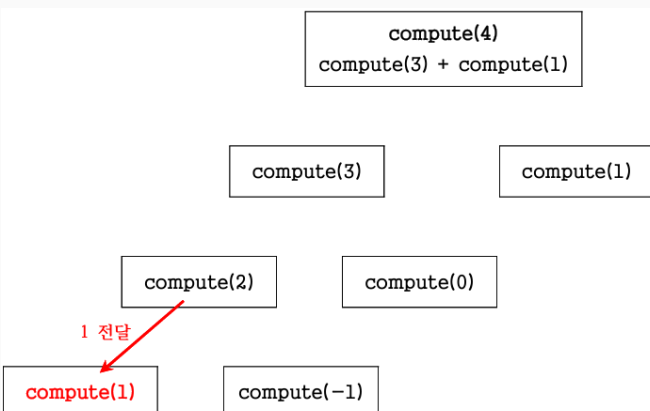
```
class Child extends Parent {
    int compute(int num) {
6        if(num <= 1) return num;
7        return compute(num - 1) + compute(num - 3);
    }
}
```

- 5번에서 인수로 3이 전달되었으므로 num은 3이다.
- 6번에서 num이 1보다 작거나 같지 않으므로 num을 반환하지 않고, 7번을 수행한다. compute(2)와 compute(0)을 호출하여 돌려받은 값을 더한 후 반환해야 하므로 먼저 compute(2)를 호출한다.



```
class Child extends Parent {
    int compute(int num) {
8        if(num <= 1) return num;
9        return compute(num - 1) + compute(num - 3);
    }
}
```

- 7번에서 인수로 2가 전달되었으므로 num은 2이다.
- 8번에서 num이 1보다 작거나 같지 않으므로 num을 반환하지 않고, 9번을 수행한다. compute(1)과 compute(-1)을 호출하여 돌려받은 값을 더한 후 반환해야 하므로 먼저 compute(1)을 호출한다.



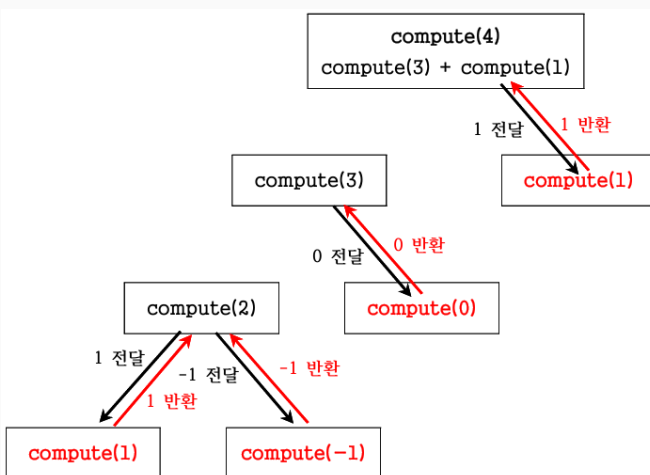
```

class Child extends Parent {
    int compute(int num) {
        ⑩      if(num <= 1) return num;
              return compute(num - 1) + compute(num - 3);
    }
}

```

• 9번에서 인수로 1이 전달되었으므로 num은 1이다. 10번에서 num은 1보다 작거나 같으므로 num의 값을 compute(1)을 호출했던 곳으로 반환한다.

※ compute() 메소드를 호출할 때 전달되는 인수가 1보다 작거나 같으면 인수의 값(num)을 그대로 반환한다는 것을 알 수 있습니다. 그러면 1보다 작은 값을 인수로 받아 호출되는 모든 compute() 메소드의 반환값을 다음과 같이 유추할 수 있습니다.

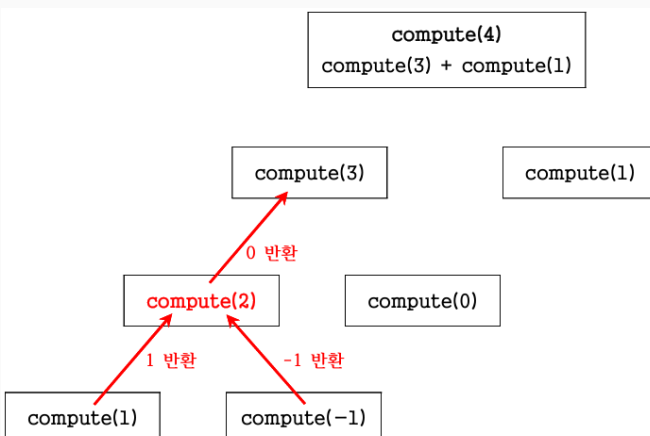


```

class Child extends Parent {
    int compute(int num) {
        ⑧      if(num <= 1) return num;
        ⑨ ⑪    return compute(num - 1) + compute(num - 3);
    }
}

```

⑪ compute(1)을 호출했던 곳으로 돌아와 compute(1)의 반환값 1과 compute(-1)의 반환값 -1을 더한 0을 compute(2)를 호출했던 곳으로 반환한다. (1 + (-1) = 0)

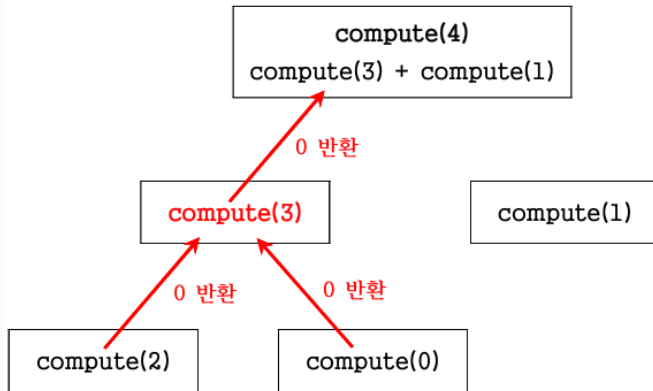


```

class Child extends Parent {
    int compute(int num) {
        6         if(num <= 1) return num;
        7 12      return compute(num - 1) + compute(num - 3);
    }
}

```

12 compute(2)를 호출했던 곳으로 돌아와 compute(2)의 반환값 0과 compute(0)의 반환값 0을 더한 0을 compute(3)을 호출했던 곳으로 반환한다. ($0 + 0 = 0$)

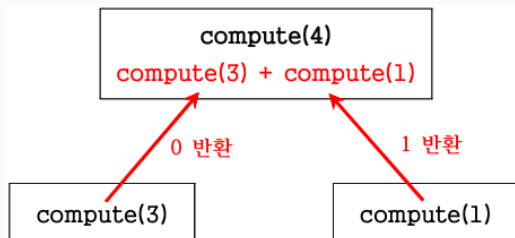


```

class Child extends Parent {
    3     int compute(int num) {
    4         if(num <= 1) return num;
    5 13  return compute(num - 1) + compute(num - 3);
    }
}

```

13 compute(3)을 호출했던 곳으로 돌아와 compute(3)의 반환값 0과 compute(1)의 반환값 1을 더한 1을 가지고 compute(4)를 처음 호출했던 main() 메소드로 돌아간다. ($0 + 1 = 1$)



```

public class Test {
    public static void main(String[] args) {
        1         Parent obj = new Child();
        2 14      System.out.print(obj.compute(4));
    }
}

```

14 compute(4)를 호출하고 반환받은 값인 1을 출력하고 프로그램을 종료한다.

결과 1

9. 다음 Java로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. [20-3](#), [23-1](#)

```

abstract class Vehicle {
    String name;
    abstract public String getName(String val);
}

```

```

    public String getName() {
        return "Vehicle name : " + name;
    }
}

class Car extends Vehicle {
    private String name;
    public Car(String val) {
        name = super.name = val;
    }
    public String getName(String val) {
        return "Car name : " + val;
    }
    public String getName(byte[] val) {
        return "Car name : " + val;
    }
}

public class Test {
    public static void main(String[] args) {
        Vehicle obj = new Car("Spark");
        System.out.print(obj.getName());
    }
}

```

답: **Vehicle name : Spark**

[해설]

```

abstract class Vehicle {                // 추상 클래스 Vehicle을 정의한다.
    String name;
    abstract public String getName(String val); // 추상 메소드를 정의한다.
    5 public String getName() {
        6 return "Vehicle name : " + name;
    }
}

class Car extends Vehicle {
    private String name;
    2 public Car(String val) {
        3 name = super.name = val;
    }
    public String getName(String val) {
        return "Car name : " + val;
    }
    public String getName(byte[] val) {
        return "Car name : " + val;
    }
}

public class Test {
    public static void main(String[] args) {
        1 Vehicle obj = new Car("Spark");
        47 System.out.print(obj.getName());
    }
}

```

모든 Java 프로그램은 반드시 main() 메소드에서 시작한다.

① **Vehicle obj = new Car("Spark");**

Car 클래스의 생성자를 이용하여 Vehicle 클래스의 객체 변수 obj를 선언하고, "Spark"를 인수로 생성자를 호출한다. (부모 타입으로 형 변환)

② **클래스 Car의 생성자 Car()의 시작점이다.** ①번에서 전달받은 "Spark"를 val이 받는다.

3 name = supername = val;

val의 값 "Spark"를 부모 클래스인 Vehicle 클래스의 변수 name과 Car 클래스의 변수 name에 모두 저장한다. 이어서 생성자를 호출했던 다음 줄인 4 번으로 이동한다.

4 객체 변수 obj의 getName() 메소드를 호출한다.

형 변환으로 인해 호출되는 메소드가 Car 클래스의 getName()이라고 생각할 수 있지만, 메소드의 이름이 동일해도 인수의 자료형과 개수가 일치하지 않으면 서로 다른 메소드이다. 그렇기 때문에 Car 클래스의 getName(String val)이나 getName(byte[] val)이 아닌 Vehicle 클래스의 getName() 메소드가 호출되어 5 번으로 이동한다.

5 getName() 메소드의 시작점이다.

6 "Vehicle name : "에 변수 name에 저장된 값 "Spark"를 붙여 메소드를 호출했던 7 번으로 반환한다.

7 6 번에서 반환받은 값을 출력하고 프로그램을 종료한다.

결과 Vehicle name : Spark

10. 다음 JAVA로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. 25-3

```
enum Tri {  
    A("A"), B("AB"), C("ABC");  
    private String code;  
    Tri(String code) {  
        this.code = code;  
    }  
    public String code() {  
        return code;  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Tri t = Tri.values()[Tri.A.name().length()];  
        System.out.print(t.code());  
    }  
}
```

답: AB

[해설]

```
public enum 타입이름 {  
    상수값1,  
    상수값2,  
    상수값3  
}
```

```
public enum Day {  
    MONDAY,  
    TUESDAY,  
    WEDNESDAY,  
    THURSDAY,  
    FRIDAY,  
    SATURDAY,  
    SUNDAY  
}
```

```
public enum Day {  
    MONDAY,  
    TUESDAY,  
    WEDNESDAY,  
    THURSDAY,  
    FRIDAY,
```

```
public class Main {  
    public static void main(String[] args) {  
        Day today = Day.MONDAY;  
        switch(today) {  
            case MONDAY:  
                System.out.println("월요일 입니다.");  
                break;  
            case TUESDAY:  
                System.out.println("화요일 입니다.");  
                break;  
            :  
            default:  
                System.out.println("잘못된 요일 입니다.");  
        }  
    }  
}
```

```
public enum Season {  
    SPRING("봄"),  
    SUMMER("여름"),  
    FALL("가을"),  
    WINTER("겨울");
```

```

SATURDAY,
SUNDAY
}

```

```

private final String koreanName;

Season(String koreanName) {
    this.koreanName = koreanName;
}
}

```

TriA	TriB	TriC
code code()	code code()	code code()
"A"	"AB"	"ABC"

※ name(): 열거형 상수의 이름을 문자열로 반환

※ values(): 모든 Enum 상수를 배열로 반환

11. 다음 Java로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. [25-3](#)

```

public class Main {
    static class test1 {
        int func1(int a, int b) {
            return a * 3 + b;
        }
    }
    static class test2 extends test1 {
        int func1(int a, int b) {
            return a + b * 2;
        }
        int func2(int a, int b) {
            return super.func1(a, b);
        }
    }
    public static void main(String[] args) {
        test2 t = new test2();
        System.out.print(t.func2(2, 4));
    }
}

```

답: 10

12. 다음 Java로 구현된 프로그램을 분석하여 괄호에 들어갈 알맞은 코드를 쓰시오. [25-3](#)

```

interface Cals {
    public void get(int v);
}
class Test ( ) Cals {
    public void get(int v) {
        System.out.print(v*v);
    }
}
public class Main {
    public static void main(String[] args) {
        Cals a = new Test();
        a.get(10);
    }
}

```

답: implements

[해설]

implements - 클래스가 인터페이스를 "구현"할 때 사용

- 대상: 클래스 → 인터페이스
- 의미: 클래스가 인터페이스에 정의된 추상 메서드를 실제로 구현함

extends - 두 가지 경우가 있음

1. 클래스가 다른 클래스를 상속할 때

- 대상: 클래스 → 클래스
- 의미: 부모 클래스의 필드와 메서드를 상속받음

2. 인터페이스가 다른 인터페이스를 상속할 때

- 사용 대상: 인터페이스 → 인터페이스
- 의미: 상위 인터페이스의 추상 메서드를 모두 물려받음

결과 100

13. 다음 Java로 구현된 프로그램을 분석하여 괄호에 들어갈 알맞은 코드를 쓰시오. [25-3](#)

```
class Rectangle {
    int x, y;
    Rectangle(int x, int y) {
        this.x = x;
        this.y = y;
    }
    int getArea( ) {
        return x*y;
    }
}
class Square extends Rectangle {
    Square(int a) {
        ( ㉠ )(a, a);
    }
    int getSquareArea( ) {
        return x*y;
    }
}
public class Main {
    public static void main(String[ ] args) {
        Square sq = new Square(10);
        System.out.println(sq.getSquareArea( ));
    }
}
```

답: **super**

[해설]

Java 상속 관계에서의 부모 생성자 호출 분석

- 자식 클래스인 Square는 Rectangle 클래스를 상속(extends)받고 있습니다.
- 부모 클래스인 Rectangle에는 매개변수를 두 개 받는 생성자 Rectangle(int x, int y)가 정의되어 있으므로, 자식 클래스 생성자 내부에서 반드시 부모의 생성자를 명시적으로 호출해 주어야 오류가 발생하지 않습니다.
- Java에서 부모 클래스의 생성자를 명시적으로 지정하여 호출할 때는 **super** 키워드를 사용합니다. 따라서 빈칸 ㉠에 알맞은 코드는 **super**가 됩니다.
- main 메서드에서 new Square(10) 호출 시 super(10, 10)에 의해 부모 멤버 변수인 x와 y가 모두 10으로 초기화되며, 최종적으로 10 * 10 = 100이라는 값이 계산되어 출력됩니다.

Square 클래스	Rectangle 클래스		Rectangle 클래스	
Square(int a)	상속된 공용 변수		Rectangle(int x, int y)	
a	x	y	x	y
10	10	10	10	10

결과 100

14. 다음 Java로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. 25-2

```
public class Main {
    public static void change(String[] data, String s) {
        data[0] = s;
        s = "Z";
    }
    public static void main(String[] args) {
        String data[] = { "A" };
        String s = "B";
        change(data, s);
        System.out.print(data[0] + s);
    }
}
```

답: BB

[해설]

Java의 매개변수 전달 방식 (Call by Value) 원리 분석

- Java는 항상 **Call by Value(값에 의한 호출)** 방식으로 매개변수를 전달합니다. 객체나 배열의 경우, 객체 자체가 아니라 **객체를 가리키는 참조 주소의 복사본**이 전달됩니다.
- change(data, s) 메서드를 호출할 때, 배열 data의 주소값과 변수 s의 문자열 값("B")이 각각 복사되어 메서드의 매개변수로 넘어갑니다.
- 메서드 내부의 첫 줄 data[0] = s; 코드에 의해, 복사된 주소가 가리키는 실제 힙 메모리 공간의 data[0] 원소가 기존 "A"에서 매개변수로 받은 "B"로 변경됩니다.
- 두 번째 줄 s = "Z"; 코드는 복사된 로컬 매개변수 s의 값을 변경하는 것일 뿐이므로, main 메서드 내부에 존재하는 원본 변수 s의 값("B")에는 아무런 영향을 주지 못합니다.
- 최종적으로 main 메서드로 돌아와 data[0] + s를 수행하면 변한 값인 "B"와 원본 값인 "B"가 결합되어 BB가 출력됩니다.

실행 단계	data[0] (참조 공간)	main의 s (원본)
1. 메서드 호출 전 초기 상태	"A"	"B"
2. change() 내에서 data[0] 변경	"B"	"B"
3. change() 내에서 s = "Z" 수행	"B"	"B" (유지)

결과 BB

15. 다음 Java로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. 25-2

```
public class Main {
    static interface F {
        int apply(int x) throws Exception;
    }
}
```

```

    }
    public static int run(F f) {
        try {
            return f.apply(3);
        } catch (Exception e) {
            return 7;
        }
    }
    public static void main(String[] args) {
        F f = (x) -> {
            if (x > 2) {
                throw new Exception();
            }
            return x * 2;
        };
        System.out.print(run(f) + run((int n) -> n + 9));
    }
}

```

답: 19

[해설]

Java 함수형 인터페이스와 람다식(Lambda), 예외 처리 흐름 분석

- 함수형 인터페이스 F는 정수 인자 x를 받아 정수를 반환하는 추상 메서드 apply를 가집니다. 메서드 run(F f)은 전달받은 람다식에 항상 값으로 3을 넣어 실행합니다.
- 첫 번째 호출인 run(f) 분석: 참조 변수 f에 대입된 람다식은 입력값 x가 2보다 크면 예외(Exception)를 의도적으로 발생시킵니다. run 메서드 내부에서 f.apply(3)이 호출되면 조건문 3 > 2가 참이 되므로 강제로 예외가 던져집니다.
- 이로 인해 예외가 발생하여 run 메서드의 catch (Exception e) 블록으로 제어가 이동하고, 상숫값 7이 반환됩니다.
- 두 번째 호출인 run((int n) -> n + 9) 분석: 예외 발생 조건이 없는 새로운 익명 함수가 전달됩니다. run 메서드 내부에서 값 3을 대입해 apply(3)을 실행하므로 연산식은 3 + 9가 되어 정상적으로 12를 반환합니다.
- 최종적으로 System.out.print()에 의해 두 반환값의 덧셈 연산인 7 + 12가 수행되어 콘솔창에 19가 출력됩니다.

호출 구문	내부 실행 및 예외 여부	반환값
run(f)	x=3 대입 → 조건 참, 예외 발생 → catch 진입	7
run((int n) -> n + 9)	n=3 대입 → 3 + 9 정상 연산 실행	12

결과 19

16. 다음 Java로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. [25-2](#)

```

public class Main {
    public static class Parent {
        public int x(int i) { return i + 2; }
        public static String id() { return "P"; }
    }
    public static class Child extends Parent {
        public int x(int i) { return i + 3; }
        public String x(String s) { return s + "R"; }
        public static String id() { return "C"; }
    }
    public static void main(String[] args) {
        Parent ref = new Child();
        System.out.println(ref.x(2) + ref.id());
    }
}

```

```
}  
}
```

답: 5P

[해설]

Java의 다형성(Polymorphism)에서 인스턴스 메서드와 정적 메서드의 호출 메커니즘 차이 분석

- 객체 생성 및 참조변수 선언 (Parent ref = new Child());:

실제 메모리에 생성된 객체는 자식 클래스(Child)이지만, 이를 참조하는 변수 ref의 타입은 부모 클래스(Parent)입니다.

- 첫 번째 연산 - 인스턴스 메서드 호출 (ref.x(2)):

- x(int i)는 일반 인스턴스 메서드이므로 Java의 오버라이딩(Overriding) 규칙 및 동적 바인딩(Dynamic Binding)이 적용됩니다.

- 참조 변수의 타입(Parent)과 상관없이 **실제 생성된 인스턴스 객체(Child)의 메서드가 호출**됩니다.

- 따라서 Child 클래스의 x(int i)가 실행되어 $2 + 3 = 5$ 가 반환됩니다.

- 두 번째 연산 - 정적(static) 메서드 호출 (ref.id()):

- id()는 static 선언이 된 정적 메서드입니다.

- 정적 메서드는 오버라이딩되지 않고 메서드 하이딩(Method Hiding) 및 정적 바인딩(Static Binding)이 적용됩니다.

- 즉, 실제 객체가 무엇이든 간에 컴파일 시점의 **참조 변수 타입(Parent)에 정의된 메서드가 호출**됩니다.

- 따라서 Parent 클래스의 id()가 실행되어 문자열 "P"가 반환됩니다.

■ 메서드 성격에 따른 바인딩 및 호출 결과 비교

호출 코드	메서드 종류	바인딩 기준	최종 반환값
ref.x(2)	인스턴스 메서드	실제 객체 (Child)	5
ref.id()	static 메서드	참조 변수 (Parent)	"P"

4) 최종 출력결과 결합:

- System.out.println(5 + "P"); 연산이 수행됩니다.

- Java에서 정수와 문자열의 더하기(+) 연산은 문자열 연결(Concatenation)이 되므로 최종 출력값은 공백 없이 붙은 5P가 됩니다.

결과 5P

17. 다음 Java로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. 25-2

```
public class Main {  
    public static class BO {  
        public int v;  
        public BO(int v) {  
            this.v = v;  
        }  
    }  
    public static void main(String[] args) {  
        BO a = new BO(1);  
        BO b = new BO(2);  
        BO c = new BO(3);  
        BO[] arr = {a, b, c};  
        BO t = arr[0];  
        arr[0] = arr[2];  
        arr[2] = t;  
        arr[1].v = arr[0].v;  
        System.out.println(a.v + "a" + b.v + "b" + c.v);  
    }  
}
```

답: 1a3b3

[해설]

Java 참조 변수(Reference Variable)와 얇은 복사(주소 복사) 메커니즘 분석

• 객체 생성 및 인스턴스 초기화:

변수 a, b, c는 각각 독립된 힙(Heap) 메모리 영역의 객체를 가리킵니다.
→ a가 가리키는 객체의 v=1, b가 가리키는 객체의 v=2, c가 가리키는 객체의 v=3

• 배열 생성 (B0[] arr = {a, b, c};):

배열 arr의 요소들은 변수 a, b, c가 가리키는 객체들의 주소값(참조)을 그대로 복사받아 저장합니다.
→ arr[0]은 a 객체, arr[1]은 b 객체, arr[2]는 c 객체 주소를 가집니다.

• 배열 요소 스왑(Swap) 연산:

t = arr[0]; arr[0] = arr[2]; arr[2] = t; 문장에 의해 arr[0]과 arr[2]의 주소값만 서로 교환됩니다.

★ 중요: 이 연산은 배열 내부의 리모컨(주소값) 위치만 바꾼 것일 뿐, 독립적으로 선언된 원본 참조 변수 a, b, c가 바라보고 있는 주소 자체는 전혀 변하지 않습니다.

◦ 스왑 결과: arr[0]은 c의 객체를 가리키고, arr[2]는 a의 객체를 가리킵니다.

• 값의 변경 연산 (arr[1].v = arr[0].v);:

- 현재 arr[0]이 가리키는 인스턴스는 c 객체이므로 arr[0].v는 3입니다.

- 현재 arr[1]이 가리키는 인스턴스는 b 객체이므로 arr[1].v = 3; 연산에 의해 b 객체의 내부 멤버변수 v 값이 2에서 3으로 변경됩니다.

■ 최종 상태의 인스턴스 값 분석

참조 변수	가리키는 인스턴스 (힙 메모리)	최종 .v 값
a	최초 생성된 '1번 객체' (변경 없음)	1
b	arr[1].v 대입 연산에 의해 값 변경	3
c	최초 생성된 '3번 객체' (변경 없음)	3

5) 출력 서식 조립:

```
System.out.println(a.v + "a" + b.v + "b" + c.v);
```

→ 1 + "a" + 3 + "b" + 3

문자열과 숫자가 더해지면서 모든 요소가 순서대로 이어 붙은 하나의 문자열로 출력됩니다.

결과 1a3b3

18. 다음 Java로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. [25-1](#)

```
public class Main {
    public static void main(String[] args) {
        int a = 5, b = 0;
        try {
            System.out.print(a/b);
        }
        catch(ArithmeticException e){
            System.out.print("출력1");
        }
        catch(ArrayIndexOutOfBoundsException e) {
            System.out.print("출력2");
        }
        catch(NumberFormatException e) {
            System.out.print("출력3");
        }
        catch(Exception e) {
```

```

        System.out.print("출력4");
    }
    finally {
        System.out.print("출력5");
    }
}

```

답: 출력1출력5

[해설]

Java 예외 처리 구조(try-catch-finally)의 제어 흐름 분석

- **예외 발생 (try 블록):** try 블록 내에서 a/b 즉, 5 / 0 연산이 수행됩니다. 정수를 0으로 나누려고 시도하면 Java 가상머신(JVM)에 의해 산술 연산 오류인 **ArithmeticException**이 발생합니다.
- **예외 매칭 (catch 블록):** 발생한 예외 객체는 상단에서부터 순차적으로 catch 블록과 비교됩니다. 첫 번째 일치하는 블록인 catch(ArithmeticException e) 내부로 진입하여 **출력1**을 수행합니다. 매칭된 이후의 다른 catch 블록은 전부 건너뛰니다.
- **무조건 실행 (finally 블록):** 예외 발생 여부 및 예외 처리 여부와 상관없이 finally 블록 내의 코드는 항상 최종적으로 실행됩니다. 따라서 **출력5**가 수행됩니다.
- **최종 출력:** 코드의 모든 출력 메서드가 줄바꿈이 없는 System.out.print() 형식이므로, 출력문들이 공백이나 줄바꿈 없이 하나의 문자열로 연속하여 붙어서 출력됩니다.

catch(ArithmeticException e): 0으로 나누는 등의 산술 연산에 대한 예외가 발생한 경우

catch(ArrayIndexOutOfBoundsException e): 배열의 범위를 벗어난 접근을 시도한 경우

catch(NumberFormatException e): 숫자 형식으로 변환할 수 없는 문자열을 숫자 형식으로 변환한 경우

※ 예외 발생시 : try문 → 해당 예외 catch문 → finally문

※ 예외가 발생하지 않은 경우 : try문 → finally문

결과 **출력1출력5**

19. 다음 Java로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. [25-1](#)

```

public class Main {
    public static void main(String[] args) {
        new Child();
        System.out.println(Parent.total);
    }
}

class Parent {
    static int total = 0;
    int v = 1;
    public Parent() {
        total += ++v;
        show();
    }
    public void show() {
        total += total;
    }
}

```

```

class Child extends Parent {
    int v = 10;
    public Child() {
        v += 2;
        total += v++;
        show();
    }
    @Override
    public void show() {
        total += total * 2;
    }
}

```

답: 54

[해설]

상속 구조에서의 다형성(오버라이딩) 및 static 변수 값의 시계열 변화 연산 과정

• 인스턴스 생성 시작 (new Child());:

자식 클래스의 객체가 생성될 때, 자바 규칙에 따라 부모 클래스인 Parent의 생성자가 기본적으로 먼저 내부 호출됩니다.

• 부모 클래스 생성자 연산 블록 실행:

- `total += ++v`; 연산 수행: Parent 내 인스턴스 변수 `v`가 1에서 2로 먼저 전위 증가합니다. 결과적으로 공유 정적 변수인 `total = 0 + 2 = 2` 상태가 됩니다.
- `show()`; 메서드 호출 (★핵심): 현재 생성 중인 실제 인스턴스는 자식 객체인 Child이므로, 자바의 **동적 바인딩 규칙**에 의해 부모의 메서드가 아닌 자식 클래스에서 재정의(Override)한 `Child.show()`가 대신 실행됩니다.
- `Child.show()` 내용 연산: `total += total * 2`; 이므로 `total = 2 + (2 * 2) = 6`으로 업데이트된 채 부모 생성자 흐름이 완전히 마칩니다.

• 자식 클래스(Child) 생성자 연산 블록 실행:

- `v += 2`; 연산 수행: Child의 고유 변수 `v`는 10이었으므로 `10 + 2 = 12`가 됩니다.
- `total += v++`; 연산 수행: 후위 증가 연산이므로 현재 `v`의 값인 12가 먼저 반영됩니다. 즉, `total = 6 + 12 = 18`이 되고, 연산 완료 직후 `v`는 13이 됩니다.
- `show()`; 메서드 호출: 오버라이딩된 `Child.show()`가 명확하게 실행됩니다. `total += total * 2`; 규칙에 맞춰 연산하면 `total = 18 + (18 * 2) = 18 + 36 = 54`가 최종 완성됩니다.

• 값 출력 단계:

정적 필드 공간에 할당된 변수인 `Parent.total`을 최종적으로 화면에 출력하므로 결과창에는 최종 누적된 값인 **54**가 깔끔하게 출력됩니다.

Parent 클래스		Child 클래스
total	v	v
0	1	10
2	2	12
6		13
18		
54		

결과 54

20. 다음 Java로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. [25-1](#)

```
public class Main {
    public static void main(String[] args) {
        System.out.println(calc("5"));
    }
    static int calc(int value) {
        if (value <= 1) return value;
        return calc(value - 1) + calc(value - 2);
    }
    static int calc(String str) {
        int value = Integer.valueOf(str);
        if (value <= 1) return value;
        return calc(value - 1) + calc(value - 3);
    }
}
```

답: 4

[해설]

Java 메서드 오버로딩(Overloading) 및 재귀 함수(Recursion) 흐름 분석

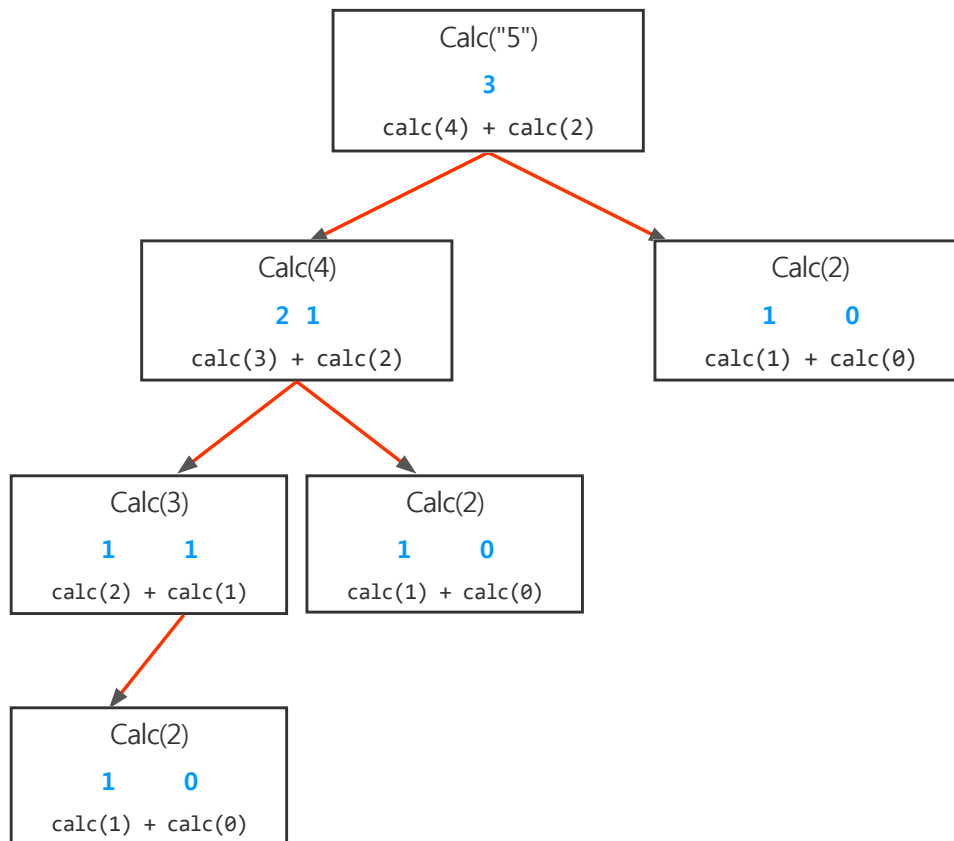
- **시작점 (main 블록):** main 메서드에서 문자열 인자 "5"를 주어 `calc("5")`를 수행합니다. 인자의 자료형이 문자열이므로 매개변수로 `String`을 받는 `calc(String str)`이 실행됩니다. (메서드 오버로딩)
- **최초 메서드 진입 (`calc(String str)`):** `Integer.valueOf("5")`를 거쳐 정수 5가 변수 `value`에 담깁니다. 조건문 `5 <= 1`은 거짓이므로 다음 줄인 `calc(4) + calc(2)` 값을 반환해야 합니다. 이때부터 호출하는 `calc`의 인자는 정수형이므로, 매개변수가 `int`형인 `calc(int value)`를 재귀적으로 호출하게 됩니다.
- **재귀 함수 계산 (`calc(int value)`):** 정수형 `calc` 메서드는 피보나치 수열과 유사한 구조(`value - 1` 분기와 `value - 2` 분기의 합)를 가집니다. 조건문 `value <= 1`일 때 자기 자신(값)을 반환하며 종료합니다. 즉, `calc(1) = 1`, `calc(0) = 0`이 기저 조건입니다.
- **각 재귀 호출 분기 계산:**
 - `calc(2) = calc(1) + calc(0) = 1 + 0 = 1`
 - `calc(3) = calc(2) + calc(1) = 1 + 1 = 2`
 - `calc(4) = calc(3) + calc(2) = 2 + 1 = 3`
- **최종 결과 도출:** 최초 반환 조건이었던 `calc(4) + calc(2)` 식에 값을 대입하면 `3 + 1 = 4`가 됩니다. `System.out.println()`에 의해 최종 결과 값인 4가 화면에 출력되고 행이 바뀝니다.

`static int calc(int value)`: 피보나치 수열 형태의 연산을 수행하는 정수 매개변수 재귀 메서드

`static int calc(String str)`: 입력된 문자열을 정수로 변환하여 초기 연산 트리(`value-1`, `value-3`)를 시작하는 오버로딩 메서드

※ 연산 트리 전개 방식 : `calc("5")` → `calc(5-1) + calc(5-3) = calc(4) + calc(2)`

※ 정수형 함수 결과값 : `calc(4) = 3`, `calc(2) = 1`



결과 4

21. 다음 Java로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. [25-1](#)

```
public class Main {
    public static void main(String[] args) {
        int[] data = {3, 5, 8, 12, 17};
        System.out.println(func(data, 0, data.length - 1));
    }
    static int func(int[] a, int st, int end) {
        if (st >= end) return 0;
        int mid = (st + end) / 2;
        return a[mid] + Math.max(func(a, st, mid), func(a, mid + 1, end));
    }
}
```

답: 20

[해설]

Java 재귀 함수 역추적 분할 분석

- 배열 정보: $a = \{3, 5, 8, 12, 17\}$ 이며 각각 인덱스는 0부터 4까지 대응합니다.
- 기저 조건 분석: $st \geq end$ 구간의 길이가 1 이하(원소가 1개이거나 역전)가 되면 더 이상 분할하지 않고 0을 반환합니다.
- 단계별 함수 호출 트리 연산 상세:
 - func(0, 4):** $mid = (0+4)/2 = 2 \rightarrow a[2] + \text{Math.max}(\text{func}(0,2), \text{func}(3,4)) \rightarrow 8 + \text{Math.max}(\text{func}(0,2), \text{func}(3,4))$
 - func(0, 2):** $mid = (0+2)/2 = 1 \rightarrow a[1] + \text{Math.max}(\text{func}(0,1), \text{func}(2,2))$
 - 이때 $\text{func}(2,2)$ 는 기저 조건($2 \geq 2$)에 의해 0을 반환합니다.
 - $\text{func}(0,1)$ 은 $mid=0$ 이 되며, $a[0] + \text{Math.max}(\text{func}(0,0), \text{func}(1,1)) = 3 + \text{Math.max}(0, 0) = 3$ 을 반환합니다.
 - 따라서 $\text{func}(0,2) = 5 + \text{Math.max}(3, 0) = 8$ 이 됩니다.
 - func(3, 4):** $mid = (3+4)/2 = 3 \rightarrow a[3] + \text{Math.max}(\text{func}(3,3), \text{func}(4,4))$
 - $\text{func}(3,3)$ 과 $\text{func}(4,4)$ 모두 $st \geq end$ 조건에 걸려 0을 반환합니다.
 - 따라서 $\text{func}(3,4) = 12 + \text{Math.max}(0, 0) = 12$ 가 됩니다.
- 최종 계산:** 최상위 식으로 복귀하면 $8 + \text{Math.max}(\text{func}(0,2), \text{func}(3,4)) \rightarrow 8 + \text{Math.max}(8, 12) \rightarrow 8 + 12 = 20$ 이 최종 도출됩니다.

함수 호출 구간	mid 인덱스	a[mid] 값	최종 반환(Return)값
func(0, 1)	0	3	$3 + \text{Math.max}(0, 0) = 3$
func(0, 2)	1	5	$5 + \text{Math.max}(3, 0) = 8$
func(3, 4)	3	12	$12 + \text{Math.max}(0, 0) = 12$
func(0, 4) [최상위]	2	8	$8 + \text{Math.max}(8, 12) = 20$

결과 20

22. 다음 Java로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. [24-3](#)

```
public class Main {
    static String[] x = new String[3];
    static void func(String[] x, int y) {
        for(int i = 1; i < y; i++) {
            if(x[i-1].equals(x[i])) {
                System.out.print("0");
            }
        }
    }
}
```

```

    }
    else {
        System.out.print("N");
    }
}
for(String z : x) {
    System.out.print(z);
}
}
public static void main(String[] args) {
    x[0] = "A";
    x[1] = "A";
    x[2] = new String("A");
    func(x, 3);
}
}

```

답: **OOAAA**

[해설]

Java의 문자열 비교 및 배열 순회 분석

- main 메서드에서 전역 배열 x의 각 요소에 문자열 "A"를 저장합니다. 리터럴 방식 생성과 new String() 방식 생성이 섞여 있지만, 모두 내부에 저장된 문자열 자체는 "A"로 동일합니다.
- func(x, 3)이 호출되면서 반복문이 i = 1부터 i < 3까지 수행됩니다.
- Java에서 문자열의 **값 자체를 비교할 때는** 주소값 비교(==)가 아닌 equals() 메서드를 사용합니다. 따라서 두 번의 조건문 비교 모두 참이 됩니다.
 - i = 1 일 때: x[0].equals(x[1]) → "A".equals("A")이므로 참 → "O" 출력
 - i = 2 일 때: x[1].equals(x[2]) → "A".equals("A")이므로 참 → "O" 출력
- 첫 번째 루프가 종료되면 출력 결과는 OO가 됩니다.
- 그 후 향상된 for문을 통해 배열 x의 모든 요소(x[0], x[1], x[2])인 "AAA"를 순서대로 이어서 출력하므로 최종 결과는 **OOAAA**가 됩니다.

x[0]	x[1]	x[2]
"A" (Literal)	"A" (Literal)	"A" (new String)
x[0].equals(x[1]) → True ('O')		-
-	x[1].equals(x[2]) → True ('O')	

결과 **OOAAA**

23. 다음 Java로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. [24-3](#)

```

public class Main {
    public static void main(String[] args) {
        B a = new D();
        D b = new D();
        System.out.print(a.getX() + a.x + b.getX() + b.x);
    }
}
class B {
    int x = 3;
    int getX() {

```

```

        return x * 2;
    }
}
class D extends B {
    int x = 7;
    int getX() {
        return x * 3;
    }
}

```

답: 52

[해설]

Java 상속 구조에서의 메서드 오버라이딩과 다형성 변수 접근 규칙

- **변수의 다형성 규칙:** Java에서 멤버 변수(필드)는 오버라이딩되지 않으며, 참조 변수의 선언된 타입을 따라갑니다. 반면, 메서드는 오버라이딩되어 항상 실제 생성된 객체(인스턴스)의 메서드가 호출됩니다.
- B a = new D(); 분석:
 - 선언된 타입이 부모 클래스 B이므로 a.x는 B 클래스의 필드인 3을 가리킵니다.
 - a.getX()는 실제 생성된 자식 객체 D에서 오버라이딩된 메서드가 호출됩니다. 이때 자식 클래스의 getX()는 자신의 필드 x(7)를 사용하여 7 * 3 = 21을 반환합니다.
- D b = new D(); 분석:
 - 선언과 실제 객체 모두 자식 클래스 D이므로 변수와 메서드 모두 자식 클래스 기준으로 동작합니다.
 - b.getX()는 자식 클래스의 메서드로 7 * 3 = 21을 반환하고, b.x는 자식 클래스의 필드값인 7이 됩니다.
- 따라서 최종 연산은 수식의 좌측부터 순서대로 진행되는 산술 더하기 계산입니다:
 $21 + 3 + 21 + 7 = 52$

변수 선언	호출 대상	적용 필드/메서드 규칙	계산된 값
B a = new D();	a.getX()	자식 클래스 D의 오버라이딩 메서드	21
	a.x	부모 클래스 B의 멤버 변수 필드	3
D b = new D();	b.getX()	자식 클래스 D의 오버라이딩 메서드	21
	b.x	자식 클래스 D의 멤버 변수 필드	7

결과 52

24. 다음 Java로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. [24-3](#)

```

public class Main {
    public static void main(String[] args) {
        int sum = 0;
        try {
            func();
        }
        catch (NullPointerException e) {
            sum = sum + 1;
        }
        catch (Exception e) {
            sum = sum + 10;
        }
    }
}

```

```

    }
    finally {
        sum = sum + 100;
    }
    System.out.print(sum);
}
static void func() throws Exception {
    throw new NullPointerException();
}
}

```

답: 101

[해설]

Java 예외 처리(try-catch-finally) 흐름 분석

- 초기 변수 sum은 0으로 설정됩니다.
- try 블록에서 func()를 호출하면, 메서드 내부에서 명시적으로 new NullPointerException() 예외가 발생(throw)합니다.
- 발생한 예외는 던져진 예외 객체 타입과 매칭되는 가장 첫 번째 catch 블록으로 진입합니다.
 - 첫 번째 예외 처리 구문인 catch (NullPointerException e) 구문과 일치하므로 해당 블록이 실행됩니다.
 - 이로 인해 $sum = 0 + 1 = 1$ 이 됩니다.
 - 하나의 catch 블록이 실행되면 아래에 있는 다른 catch 구문(Exception e)은 건너뜁니다.
- finally 블록의 특징:** 예외 처리의 성공 여부나 발생 유무에 관계없이 **무조건 실행**되는 영역입니다. 따라서 최종적으로 finally 코드가 수행되어 $sum = 1 + 100 = 101$ 이 됩니다.
- 최종 출력 메서드에 의해 변수값인 101이 출력됩니다.

제어 구조 단계	실행 내용	sum 변수의 변화
1. try 블록 변수 진입	func() 호출 및 예외 발생	sum = 0
2. catch(NullPointerException...)	예외 매칭 성공 (sum + 1)	sum = 1
3. catch(Exception e)	(건너뛴)	sum = 1
4. finally 블록 무조건 실행	필수 로직 처리 (sum + 100)	sum = 101

결과 101

25. 다음 Java로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. [24-3](#)

```

class Printer {
    void print(Integer a) { System.out.print("A" + a); }
    void print(Object a) { System.out.print("B" + a); }
    void print(Number a) { System.out.print("C" + a); }
}

public class Main {
    public static void main(String[] args) {
        new Collection<>().print();
    }
    public static class Collection<T> {
        T value;
        public Collection(T t) { value = t; }
        public void print() {
            new Printer().print(value);
        }
    }
}

```

```
}  
}
```

답: **B0**

[해설]

Java 제네릭의 타입 소거(Type Erasure) 및 다중 오버로딩 분석

- **제네릭 타입 소거(Type Erasure):** Java의 제네릭 기술은 하위 호환성을 위해 컴파일 과정에서 타입 정보가 지워집니다. <T>처럼 상한 경계(Bounded types)가 없는 경우, 컴파일러는 T를 제네릭의 최상위 객체인 **Object**로 취급하여 바이트코드를 생성합니다.
- main 메서드에서 숫자리터럴 0을 넘겨 인스턴스를 생성하면 오토박싱에 의해 내부 필드 value에 Integer 인스턴스가 저장될 것 같지만, 제네릭 내부의 value 필드 선언문 자체는 Object value; 형태로 다루어지게 됩니다.
- 메서드 오버로딩 호출 규칙은 컴파일 타임에 **선언된 참조 변수 타입**에 의해 결정됩니다. Collection<T> 내부의 print() 메서드 안에서 호출하는 new Printer().print(value)의 변수 value는 컴파일러 단계에서 Object 타입으로 인식됩니다.
- 그 결과, Printer 클래스에 오버로딩되어 있는 3개의 메서드 중 **print(Object a)**가 호출됩니다.
- 해당 메서드는 문자열 "B"에 값 a(0)을 결합하여 출력하므로 정답은 **B0**이 됩니다.

컴파일 이전 (소스코드 상)	컴파일 이후 (타입 소거 반영)
class Collection<T> { T value;	class Collection { Object value;
print(value); (Integer 인스턴스 예측)	print(value); → print(Object) 매칭

결과 **B0**

26. 다음 Java로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. [24-2](#)

```
public class Test {  
    public static void main(String[] args) {  
        String str = "ITISTESTSTRING";  
        String[] result = str.split("T");  
        System.out.print(result[3]);  
    }  
}
```

답: **S**

[해설]

Java String.split() 메서드 분할 메커니즘 분석

- str.split("T")는 대상 문자열에서 구분자(Delimiter)인 "T"를 기준으로 문자열을 잘라 배열에 담습니다.
- 원본 문자열 "ITISTESTSTRING"에는 총 4개의 "T"가 포함되어 있으며, 분할되는 과정은 다음과 같습니다.
 - 1번째 'T' 기점 분리: 앞의 "I"가 **result[0]**에 저장됩니다.
 - 1번째와 2번째 'T' 사이: "IS"가 **result[1]**에 저장됩니다.
 - 2번째와 3번째 'T' 사이: "ES"가 **result[2]**에 저장됩니다.
 - 3번째와 4번째 'T' 사이: "S"가 **result[3]**에 저장됩니다.
 - 4번째 'T' 이후 남은 부분: "RING"이 **result[4]**에 저장됩니다.
- 코드의 출력문은 result[3]을 요구하고 있으므로, 3번 인덱스에 매칭되는 문자열인 **S**가 최종 출력됩니다.

배열 인덱스	분할된 데이터 값	구분 기준위치
result[0]	"T"	1번째 T 앞
result[1]	"IS"	1~2번째 T 사이
result[2]	"ES"	2~3번째 T 사이
result[3]	"S"	3~4번째 T 사이 (정답)
result[4]	"RING"	4번째 T 뒤쪽 끝

srt
ITISTESTRING

	[0]	[1]	[2]	[3]	[4]
result	I	IS	ES	S	RING

결과 S

27. 다음 Java로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. (단, 출력문의 출력 서식을 준수하시오.) [24-2](#)

```
public class Test {
    public static void check(int[] x, int[] y)
    {
        if(x == y) System.out.print("0");
        else System.out.print("N");
    }
    public static void main(String[] args) {
        int a[] = new int[] {1, 2, 3, 4};
        int b[] = new int[] {1, 2, 3, 4};
        int c[] = new int[] {1, 2, 3};
        check(a, b);
        check(b, c);
        check(a, c);
    }
}
```

답: NNN

[해설]

Java 배열의 참조(주소) 비교 분석

- 배열 변수 a, b, c는 각각 new 연산자를 통해 힙(Heap) 메모리 영역에 개별적으로 생성된 객체입니다.
- check 메서드 내부의 if(x == y) 비교문은 배열에 담긴 값(요소)을 비교하는 것이 아니라, 두 배열 인스턴스의 참조값(메모리 주소)을 비교합니다.
- 배열 a와 b는 내부 데이터 형태((1, 2, 3, 4))가 완벽히 같더라도 서로 다른 메모리 주소에 할당되어 있으므로 a == b는 false가 되어 N이 출력됩니다.
- 마찬가지로 check(b, c)와 check(a, c)의 비교 역시 모두 주소값이 다르므로 전부 false가 되어 각각 N이 출력됩니다.
- 줄바꿈이 없는 System.out.print() 함수를 연이어 사용하였으므로 최종 결과물은 공백 없이 이어 붙여진 형태가 됩니다.

배열 변수 (Stack)	참조 주소 (값)	실제 데이터 (Heap)
a	@0x100 (예시)	{1, 2, 3, 4}
b	@0x200 (예시)	{1, 2, 3, 4}
c	@0x300 (예시)	{1, 2, 3}

a	b	c
1000	2000	3000

주소	메모리			
0000				
:				
:	4Byte	4Byte	4Byte	4Byte
:	a[0]	a[1]	a[2]	a[3]
1000 →	1	2	3	4
:				
:	b[0]	b[1]	b[2]	b[3]
2000 →	1	2	3	4
:				
:	c[0]	c[1]	c[2]	
3000 →	1	2	3	
:				
:				
9999				

결과 NNN

28. 다음 Java로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. [24-2](#)

```
interface Number {
    int sum(int[] a, boolean odd);
}

public class Test {
    public static void main(String[] args) {
        int a[] = {1, 2, 3, 4, 5, 6, 7, 8, 9};
        OENumber OE = new OENumber();
        System.out.print(OE.sum(a, true) + ", " + OE.sum(a, false));
    }
}

class OENumber implements Number {
    public int sum(int[] a, boolean odd) {
        int result = 0;
        for (int i = 0; i < a.length; i++) {
            if ((odd && a[i] % 2 != 0) || (!odd && a[i] % 2 == 0)) {
                result += a[i];
            }
        }
        return result;
    }
}
```

답: 25, 20

[해설]

Java 인터페이스 구현 및 조건문 분기 분석

- OENumber 클래스는 Number 인터페이스의 추상 메서드인 sum을 오버라이딩하여 구현하고 있습니다.
- sum 메서드 내부의 핵심 조건문은 논리 연산자(||)를 기준으로 두 그룹으로 나뉩니다.
 - 좌항 (odd && a[i] % 2 != 0): odd가 참(true)이면서 배열 요소가 홀수일 때 누적
 - 우항 (!odd && a[i] % 2 == 0): odd가 거짓(false)이면서 배열 요소가 짝수일 때 누적
- 첫 번째 호출 OE.sum(a, true): 매개변수 odd가 true이므로 배열 {1, 2, 3, 4, 5, 6, 7, 8, 9} 중 홀수들의 합을 구합니다.
→ 1 + 3 + 5 + 7 + 9 = 25
- 두 번째 호출 OE.sum(a, false): 매개변수 odd가 false이므로 배열 중 짝수들의 합을 구합니다.
→ 2 + 4 + 6 + 8 = 20
- 최종 출력은 두 호출 결과값 사이에 문자열 ", "을 연결하여 한 줄로 출력하므로 25, 20이 됩니다.

호출 형태	odd 값	연산 대상 요소
sum(a, true)	true (홀수 필터)	1, 3, 5, 7, 9
sum(a, false)	false (짝수 필터)	2, 4, 6, 8

결과 25, 20

29. 다음 Java로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. [24-2](#)

```
public class Test {
    public static String rf(String str, int index, boolean[] seen) {
        if(index < 0) return "";
        char c = str.charAt(index);
        String result = rf(str, index-1, seen);
        if(!seen[c]) {
            seen[c] = true;
            return c + result;
        }
        return result;
    }
    public static void main(String[] args) {
        String str = "abacabcd";
        int len = str.length();
        boolean[] seen = new boolean[256];
        System.out.print(rf(str, len-1, seen));
    }
}
```

답: dcba

[해설]

Java 재귀 호출(Recursive Call)과 문자열 처리 분석

- main 메서드에서 문자열 "abacabcd"(길이 8)의 마지막 인덱스인 len-1(7)을 시작으로 재귀 함수 rf를 최초 호출합니다.
- rf 함수는 내부에서 String result = rf(str, index-1, seen);를 통해 인덱스 0번까지 먼저 끝까지 파고들어 내려간 뒤, 거꾸로 올라오며(반환되며) 본문의 연산을 수행하는 Stack 구조의 특징을 가집니다.

- 즉, `index < 0`이 되는 순간 빈 문자열("")을 반환하고, 이후 **인덱스 0번부터 7번 순서로 올라오며 후속 연산**을 처리합니다.
- 연산 핵심 조건문인 `if(!seen[c])`는 해당 문자가 처음 등장했을 때만 `true`로 마킹하고, 기존 하위 재귀 결과물의 **앞에** 현재 문자 `c`를 붙여서 리턴(`return c + result;`)합니다. 이미 등장했던 문자라면 붙이지 않고 이전 누적 결과(`result`)만 그대로 토스합니다.

index	문자 c	하위 반환값 (result)	처음 등장?	최종 리턴값 (c + result)
0	'a'	""	O (a 등록)	"a"
1	'b'	"a"	O (b 등록)	"ba"
2	'a'	"ba"	X (중복)	"ba"
3	'c'	"ba"	O (c 등록)	"cba"
4	'a'	"cba"	X (중복)	"cba"
5	'b'	"cba"	X (중복)	"cba"
6	'c'	"cba"	X (중복)	"cba"
7	'd'	"cba"	O (d 등록)	"dcba"

결과적으로 문자열을 앞에서부터 훑으며 **중복되지 않은 문자들을 역순(가장 나중에 발견된 순서가 앞쪽으로 오게끔)으로 조합**해 나가는 알고리즘이 됩니다.

결과 **dcba**

30. 다음 Java로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. [24-1](#)

```

class Connection {
    private static Connection _inst = null;
    private int count = 0;
    public static Connection get() {
        if(_inst == null) {
            _inst = new Connection();
            return _inst;
        }
        return _inst;
    }
    public void count() { count++; }
    public int getCount() { return count; }
}

public class Test {
    public static void main(String[] args) {
        Connection conn1 = Connection.get();
        conn1.count();
        Connection conn2 = Connection.get();
        conn2.count();
        Connection conn3 = Connection.get();
        conn3.count();
        conn1.count();
        System.out.print(conn1.getCount());
    }
}

```

답: 4

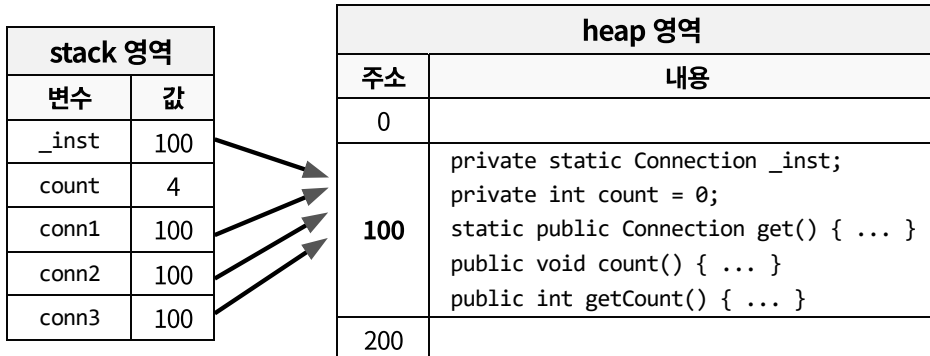
[해설]

Java 싱글톤 패턴(Singleton Pattern) 구조 분석

- `Connection` 클래스는 클래스 내부에서 단 하나의 인스턴스(`_inst`)만 생성하여 공유하도록 제한하는 **싱글톤 디자인 패턴**으로 구현되어 있습니다.
- `Connection.get()` 메서드가 처음 호출될 때만 `new Connection()` 객체가 힙 메모리에 한 번 생성되고, 이후 호출부터는 이미 생성되어 있던 기존 인스턴스의 주소(`_inst`)를 그대로 반환합니다.
- 따라서 `main` 메서드의 `conn1`, `conn2`, `conn3` 참조 변수들은 **모두 동일한 하나의 `Connection` 객체 인스턴스를 가리키게** 됩니다.
- 어떤 변수명을 사용하든 `count()`를 호출할 때마다 동일한 인스턴스 내부의 인스턴스 멤버 변수인 `count` 값이 1씩 계속해서 누적 증가합니다.

실행 코드 라인	참조하는 대상 인스턴스	count 값
Connection conn1 = Connection.get();	인스턴스 신규 생성 (주소A)	0
conn1.count();	주소A의 count 1 증가	1
Connection conn2 = Connection.get();	기존 인스턴스 반환 (주소A)	1
conn2.count();	주소A의 count 1 증가	2
Connection conn3 = Connection.get();	기존 인스턴스 반환 (주소A)	2
conn3.count();	주소A의 count 1 증가	3
conn1.count();	주소A의 count 1 증가	4

최종적으로 conn1.getCount()를 호출하여 공유 인스턴스의 최종 count 변수값인 4가 화면에 출력됩니다.



결과 4

31. 다음 JAVA로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. [24-1](#)

```

class firstArea {
    int x, y;
    public firstArea(int x, int y) {
        this.x = x;
        this.y = y;
    }
    public void print() {
        System.out.println(x+y);
    }
}

class secondArea extends firstArea {
    int bb = 3;
    public secondArea(int i) {
        super(i, i+1);
    }
    public void print() {
        System.out.println(bb*bb);
    }
}

public class Main {
    public static void main(String[] args) {
        firstArea st = new secondArea(10);
    }
}

```

```

        st.print();
    }
}

```

답: 9

[해설]

Java 객체지향 프로그래밍의 다형성(Polymorphism)과 동적 바인딩 분석

- `firstArea st = new secondArea(10);` 선언을 통해 부모 타입의 참조 변수 `st`로 자식 객체 인스턴스인 `secondArea`를 가리키는 다형성이 적용되었습니다.
- 인스턴스 생성 시점에 자식 생성자 내 `super(10, 11);`이 작동하여 부모 인스턴스 영역의 변수 `x`에는 10, `y`에는 11이 깔끔하게 대입됩니다.
- 그 후, `st.print();` 메서드를 호출합니다. 여기서 선언된 참조 타입은 부모(`firstArea`) 타입이지만, Java의 인스턴스 메서드는 런타임에 실제 객체의 타입을 기준으로 실행 메서드를 결정하는 **동적 바인딩(Dynamic Binding)** 메커니즘을 가집니다.
- 따라서 부모의 `print()` 메서드는 자식 클래스인 `secondArea`의 `print()` 메서드에 의해 완전히 ****오버라이딩(재정의)**** 되었기 때문에 부모의 코드가 아닌 **자식 객체의 인스턴스 메서드가 최종 우선 실행**됩니다.

메모리 인스턴스 위치	변수 적재 상태	print() 호출시 실행 대상
부모 영역 (firstArea)	x = 10, y = 11	무시됨(은닉)
자식 영역 (secondArea)	bb = 3	자식 메서드 가동

객체 변수 st					
firstArea 클래스				secondArea 클래스	
x	y	firstArea(int x, int y)		bb	secondArea(int i)
		x	y		i
10	11	10	11	3	10

자식 클래스의 `print()` 메서드 바디 내용인 `bb * bb` 연산이 수행되므로 수식은 `3 * 3`이 되어 최종적으로 **9**가 콘솔 창에 출력된 뒤 줄바꿈 처리됩니다.

결과 9

32. 다음 JAVA로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. [23-3](#) [🔄](#) [001로](#)

```

class SuperObject {
    public void draw() {
        System.out.println("A");
        draw();
    }
    public void paint() {
        System.out.print('B');
        draw();
    }
}
class SubObject extends SuperObject {
    public void paint() {
        super.paint();
        System.out.print('C');
        draw();
    }
    public void draw() {

```

```

        System.out.print('D');
    }
}
public class Test {
    public static void main(String[] args) {
        SuperObject a = new SubObject();
        a.paint();
        a.draw();
    }
}

```

답: **BDCDD**

[해설]

오버라이딩(메소드 재정의)

- 상속으로 인해 동일한 이름의 메소드가 여러 개인 경우 자식 클래스의 메소드만 사용됨.

오버로딩(Overloading)

- 메소드의 이름은 동일하지만 인수를 받는 자료형과 개수가 다르면 서로 다른 메소드임.
- 이름은 같지만 자료형, 개수를 다르게 하여 여러 기능을 정의하는 것.

super: 상속한 부모 클래스를 가리키는 예약어

this: 현재 실행중인 메소드가 속한 클래스를 가리키는 예약어

Java의 오버라이딩(Overriding)과 동적 바인딩(Dynamic Binding) 실행 흐름 분석

- SuperObject a = new SubObject();와 같이 부모 타입의 참조 변수로 자식 객체를 생성한 경우, 오버라이딩된 메서드를 호출하면 변수 타입과 관계없이 **무조건 실제 생성된 인스턴스(자식 클래스)의 메서드가 실행됩니다.**
- 첫 번째 명령문 a.paint(); 실행 흐름:
 - 자식 클래스(SubObject)의 paint()가 먼저 제어권을 잡고 내부의 super.paint();를 호출하여 부모 클래스로 진입합니다.
 - 부모 클래스(SuperObject)의 paint() 내에서 System.out.print('B')를 수행하여 **B**가 출력됩니다.
 - 이어서 부모의 paint() 내부에서 draw();를 호출합니다. 이때 다형성 원칙에 의해 부모 클래스의 draw()가 아닌, 오버라이딩된 **자식 클래스의 draw()가 호출되므로 D가 출력됩니다.**
 - 부모 메서드가 종료되어 다시 자식 클래스의 paint() 코드로 돌아온 후, 다음 라인인 System.out.print('C')를 실행하여 **C**가 출력됩니다.
 - 마지막으로 자식 클래스 내의 draw();가 수행되면서 자식의 draw()가 실행되어 **D**가 출력됩니다. (이 단계까지의 누적 출력: **BDCD**)
- 두 번째 명령문 a.draw(); 실행 흐름:
 - 참조 변수 a를 통해 draw()를 직접 호출합니다. 다형성에 의해 마찬가지로 실제 객체인 자식 클래스의 draw()가 곧바로 실행되므로 마지막으로 **D**가 한 번 더 출력됩니다.
- 결과적으로 모든 출력문은 줄바꿈이 없는 print() 형식이므로 순서대로 이어 붙인 최종 실행 결과는 **BDCDD**가 됩니다.

호출 명령	실행되는 메서드 위치 및 동작	누적 출력
1. a.paint();	SubObject.paint() → super.paint() 호출	-
	SuperObject.paint() → print('B')	B
	SuperObject.paint() 내부 draw() → Sub.draw() 실행 ('D')	BD
	SubObject.paint() 복귀 → print('C')	BDC
	SubObject.paint() 내부 draw() → Sub.draw() 실행 ('D')	BDCD
2. a.draw();	SubObject.draw() 직접 호출 → print('D')	BDCDD

33. 다음 JAVA로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. 23-3

```

class P {
    public int calc(int n) {
        if (n <= 1) return n;
        return calc(n - 1) + calc(n - 2);
    }
}
class C extends P {
    public int calc(int n) {
        if (n <= 1) return n;
        return calc(n - 1) + calc(n - 3);
    }
}
public class Test {
    public static void main(String[ ] args) {
        P obj = new C();
        System.out.print(obj.calc(7));
    }
}

```

답: 2

[해설]

Java의 동적 바인딩(Dynamic Binding)과 변형된 재귀 함수 실행 흐름 분석

- P obj = new C();와 같이 부모 타입 선언 후 자식 인스턴스를 바인딩하면, 다형성 원칙에 의해 실제 생성된 객체인 자식 클래스 C의 calc() 메서드가 실행됩니다.
- 자식 클래스 C의 재귀 관계식은 다음과 같이 정의됩니다.
 - 기반 조건(Base Case): $n \leq 1$ 일 때, return n; (즉, 1 이하의 정수는 값 그대로 반환)
 - 재귀 단계(Recursive Step): $\text{calc}(n) = \text{calc}(n - 1) + \text{calc}(n - 3)$
- 작은 값부터 차례대로 상향식(Bottom-Up)으로 계산해 나가면 중복 호출 구조를 쉽고 확실하게 파악할 수 있습니다.

n	점화식 및 전개 과정	결과값
0, 1	조건식 ($n \leq 1$) 에 의해 자기 자신 반환	0, 1
2	$\text{calc}(1) + \text{calc}(-1) = 1 + (-1)$ *음수 값 주의	0
3	$\text{calc}(2) + \text{calc}(0) = 0 + 0$	0
4	$\text{calc}(3) + \text{calc}(1) = 0 + 1$	1
5	$\text{calc}(4) + \text{calc}(2) = 1 + 0$	1
6	$\text{calc}(5) + \text{calc}(3) = 1 + 0$	1
7	$\text{calc}(6) + \text{calc}(4) = 1 + 1$	2

※ **중요 포인트 (calc(2) 반환값)**: $n = 2$ 일 때 $\text{calc}(1) + \text{calc}(-1)$ 이 호출됩니다. 조건식 if ($n \leq 1$)에 따라 $\text{calc}(1)$ 은 1을 반환하고, $\text{calc}(-1)$ 은 매개변수가 1 이하이므로 인자값 그대로 -1을 반환하여 둘의 합은 0이 됩니다. 이 결과를 기반으로 최종 $\text{calc}(7)$ 까지 연쇄 계산하면 최종값은 2가 도출됩니다.

34. 다음 Java로 구현된 프로그램의 실행 순서를 나열하시오. (단, 같은 번호는 중복해서 작성하지 마시오.)
24-1

```
class Parent {
    int x, y;
    ❶ Parent(int x, int y) {
        this.x = x;
        this.y = y;
    }
    ❷ int getX() {
        return x*y;
    }
}

class Child extends Parent {
    int x;
    ❸ Child(int x) {
        super(x+1, x);
        this.x = x;
    }
    ❹ int getX(int n) {
        return super.getX() + n;
    }
}

public class Main {
    ❺ public static void main(String[] args) {
    ❻       Parent parent = new Child(10);
    ❼       System.out.println(parent.getX());
    }
}
```

답: 5 → 6 → 3 → 1 → 7 → 2

[해설]

Java 클래스 상속 구조에서의 코드 실행 흐름(순서) 분석

- 단계 1 (❺): 프로그램이 실행되면 가장 먼저 메인 메서드인 `public static void main` 진입점이 실행됩니다.
- 단계 2 (❻): `new Child(10)`를 통해 자식 클래스 `Child`의 인스턴스를 생성하라는 명령을 만납니다.
- 단계 3 (❸): 인스턴스 생성에 의해 자식 클래스의 생성자인 `Child(int x)`가 호출되어 진입합니다.
- 단계 4 (❹): 자식 생성자 내부 첫 줄에 선언된 `super(x+1, x);`에 의해 부모 클래스의 생성자인 `Parent(int x, int y)`로 제어가 넘어가 부모의 변수 초기화가 먼저 처리됩니다.
- 단계 5 (❼): 생성자 처리가 모두 완료되어 인스턴스 생성이 끝나면, 다음 라인인 `parent.getX()` 메서드 호출을 만납니다.
- 단계 6 (❷): `parent`의 실제 객체 타입은 자식 객체이지만, 호출하는 메서드 `getX()`는 매개변수가 없는 형태입니다. 자식 클래스에는 매개변수가 있는 `getX(int n)`(❹번)만 정의되어 있으므로, 오버라이딩이 아닌 부모로부터 물려받은 원본 메서드인 부모 클래스의 `getX()`가 최종적으로 호출 및 실행됩니다.

순서	번호	수행 작업	메모리 상태 현황
1	5	Main 프로그램 진입	-

순서	번호	수행 작업	메모리 상태 현황
2	6	Child 객체 생성 시작	-
3	3	Child 생성자 호출 (x = 10)	자식 영역 대기
4	1	super()로 부모 생성자 실행	부모 x=11, y=10
5	7	출력문 내부에서 getX() 호출	자식 x=10 결정
6	2	부모의 getX() 계산 (11 * 10)	최종 결과 110

참고로 ④번 문항(getX(int n))은 메인 메서드에서 인자(매개변수)를 전달하며 호출한 적이 없으므로 전체 실행 과정에서 제외됩니다.

실행 결과 값 110

35. 다음 JAVA로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. 23-2

```
public class Test {
    public static void main(String[] args) {
        String str1 = "Programming";
        String str2 = "Programming";
        String str3 = new String("Programming");
        System.out.println(str1 == str2);
        System.out.println(str1 == str3);
        System.out.println(str1.equals(str3));
        System.out.println(str2.equals(str3));
    }
}
```

답: true
false
true
true

[해설]

Java 메모리 구조 내 String Constant Pool 참조 주소 비교 및 관계 검증

- **리터럴 할당 (str1, str2):** 자바에서 큰따옴표를 이용한 문자열 리터럴 선언 시, Heap 영역 내부의 **String Constant Pool**에 등록됩니다. 이미 풀 내부에 동일한 값을 가진 데이터가 있으면 새로운 메모리를 생성하지 않고 기존의 주소값을 공유하므로, str1과 str2는 완전히 **동일한 인스턴스의 메모리 주소**를 가리키게 됩니다. 따라서 동일성 비교 연산식 str1 == str2는 **true**를 반환합니다.
- **객체 생성자 이용 (str3):** 반면 new String() 방식을 사용하면 문자열 풀과 상관없이 일반 Heap 메모리 공간에 **독립적인 새로운 문자열 객체**를 무조건 동적 할당합니다. 때문에 물리적인 인스턴스 참조 주소가 완전히 달라지므로 주소값 비교 연산식인 str1 == str3는 **false**가 됩니다.
- **값 비교 연산 (equals 메서드):** == 연산자가 참조하고 있는 '주소'를 비교하는 것과 달리, String 인스턴스의 equals() 메서드는 저장된 실제 '**문자열 데이터 내용 자체**'의 일치 여부를 판별합니다. 세 변수가 가지고 있는 문자열 내용이 모두 "Programming"으로 동일하므로, 주소와 관계없이 내장 값의 일치를 묻는 후반부 두 출력문은 전부 **true**를 반환하게 됩니다.

Java Heap Memory Architecture

[String Constant Pool]

Addr @0x001 : "Programming"

[General Heap Area]

[변수별 참조 링크]

- str1 → **Pool 주소 (@0x001)**
- str2 → **Pool 주소 (@0x001)** (주소 공유)
- str3 → **Heap 독립 주소 (@0x999)**

✓ **== 연산자:** 참조 주소비교 (0x001 == 0x999 → false)

✓ **equals():** 데이터 리터럴 자체 값 비교 (→ true)

Addr @0x999 : "Programming"

결과

true
false
true
true

36. 다음 Java로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. [23-1](#)

```
class Static {  
    public int a = 20;  
    static int b = 0;  
}  
public class Test {  
    public static void main(String[] args) {  
        int a = 10;  
        Static.b = a;  
        Static st = new Static();  
        System.out.println(Static.b++);  
        System.out.println(st.b);  
        System.out.println(a);  
        System.out.print(st.a);  
    }  
}
```

답: 10
11
10
20

[해설]

Java 클래스(Static) 변수와 인스턴스 변수, 그리고 증감 연산자 분석

- Static.b = a;에 의해 static 변수 b는 메인 메서드의 지역변수 a의 값인 10으로 변경됩니다.
- System.out.println(Static.b++); 후위 증감 연산자(++)이므로 현재 b의 값인 10을 먼저 출력한 후, b의 값은 11이 됩니다.
- System.out.println(st.b); b는 모든 인스턴스가 공유하는 static 변수이므로 인스턴스 참조 변수인 st.b로 접근해도 직전에 증가된 값인 11이 출력됩니다.
- System.out.println(a); 메인 메서드 내부의 지역 변수 a의 값인 10이 그대로 출력됩니다.
- System.out.print(st.a); 인스턴스 st가 가지고 있는 고유의 멤버 변수 a의 초기값인 20이 출력됩니다. (개행 없음)

main의 지역변수 a	Static 클래스 (static) b	st 인스턴스 (멤버) a
10	10 → 11 (b++)	20

결과

10
11
10
20

37. 다음 JAVA로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. [23-1](#)

```
class Parent {
    int x = 1000;
    Parent() {
        this(3000);
    }
    Parent(int x) {
        this.x = x;
    }
}

class Child extends Parent {
    int x = 4000;
    Child() {
        this(5000);
    }
    Child(int x) {
        this.x = x;
    }
    int getX() {
        return this.x;
    }
}

public class Test {
    public static void main(String[] args) {
        Child c = new Child();
        System.out.println(c.getX());
    }
}
```

답: 5000

[해설]

Java 생성자 호출 흐름(this())과 오버라이딩 되지 않는 멤버 변수의 특징 분석

- new Child()가 호출되면 자식 클래스의 기본 생성자가 실행되며, 내부의 this(5000);에 의해 매개변수가 있는 생성자 Child(int x)가 연쇄 호출됩니다.
- 자식 생성자의 첫 줄에는 명시적인 부모 생성자 호출(super())이 없으므로, 컴파일러에 의해 자동으로 부모의 기본 생성자인 Parent() 호출이 삽입되어 먼저 수행됩니다.
- 부모 기본 생성자 내부의 this(3000);에 의해 Parent(int x)가 실행되어 부모 인스턴스 영역의 x 필드 값은 3000으로 세팅됩니다.
- 이후 자식 생성자 Child(int x)의 나머지 본문인 this.x = x;가 실행되면서 자식 인스턴스 영역의 x 필드 값이 5000으로 변경됩니다.
- c.getX() 메서드는 Child 클래스에 정의되어 있으며 this.x를 리턴하므로, 자식 클래스 영역에 가려진(shadowed) 변수 x의 값인 5000을 반환하여 출력하게 됩니다.

Parent 영역 (부모의 x)	Child 영역 (자식의 x)
초기값: 1000 ↓ Parent(3000) 수행 후: 3000	초기값: 4000 ↓ Child(5000) 수행 후: 5000

결과 5000

38. 다음 JAVA로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. [22-3](#)

```
public class Test {
    public static void main(String[] args) {
        int r = 0;
        for (int i = 1; i < 999; i++) {
            if (i % 3 == 0 && i % 2 == 0)
                r = i;
        }
    }
}
```

```

        System.out.print(r);
    }
}

```

답: 996

[해설]

문제의 코드는 1부터 998까지의 숫자 중 3과 2로 나누었을 때 나머지가 0인, 즉 6의 배수이면서 가장 큰 수를 구하는 알고리즘입니다.

```

public class Test {
    public static void main(String[]
args) {
    ❶      int r = 0;
    ❷      for (int i = 1; i < 999; i++)
    {
    ❸          if (i % 3 == 0 && i % 2 ==
0)
    ❹              r = i;
    }
    ❺      System.out.print(r);
    }
}

```

- ❶ 정수형 변수 r을 선언하고 0으로 초기화한다.
- ❷ 반복 변수 i가 1부터 1씩 증가하면서 999보다 작은 동안 ❸, ❹번을 반복 수행한다.
- ❸ i를 3과 2로 나눈 나머지가 모두 0이면 ❹번으로 이동하고, 아니면 반복문의 시작인 ❷번으로 이동한다.
- ❹ r에 i의 값을 저장한다.

• 반복문 실행에 따른 변수들의 변화는 다음과 같다.

i	i%3	i%2	r
			0
1	1	1	
2	2	0	
3	0	1	
4	1	0	
5	2	1	
6	0	0	6
7	1	1	
⋮	⋮	⋮	⋮
995	2	1	
996	0	0	996
997	1	1	
998	2	0	
999	0	1	

❺ r의 값을 출력한다.

결과 996

39. 다음 JAVA로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. 22-2

```

public class Test {
    public static void main(String[] args) {
        int i = 3, k = 1;
        switch(i) {

```

```

        case 1: k++;
        case 2: k -= 3;
        case 3: k = 0;
        case 4: k += 3;
        case 5: k -= 10;
        default: k--;
    }
    System.out.print(k);
}
}

```

답: -8

[해설]

```

public class Test {
    public static void main(String[]
args) {
    ① int i = 3, k = 1;
    ② switch(i) {
        case 1: k++;
        case 2: k -= 3;
    ③ case 3: k = 0;
    ④ case 4: k += 3;
    ⑤ case 5: k -= 10;
    ⑥ default: k--;
    }
    ⑦ System.out.print(k);
    }
}

```

- ① 정수형 변수 i, k를 선언하고, 각각 3과 1로 초기화한다.
- ② i의 값 3에 해당하는 숫자를 찾아간다. 'case 3' 문장으로 이동한다.
- ③ k에 0을 저장한다. → **k = 0**
※ switch 문을 종료하는 break가 없으므로 ④, ⑤, ⑥번을 모두 수행하고 ⑦번으로 이동한다.
- ④ 'k = k + 3;'과 동일하다. k의 값에 3을 더한다. → **k = 3**
- ⑤ 'k = k - 10;'과 동일하다. k의 값에서 10을 뺀다. → **k = -7**
- ⑥ 'k = k - 1;'과 동일하다. k의 값에서 1을 뺀다. → **k = -8**
- ⑦ k의 값을 출력한다.

결과 -8

프로그래밍 - Python 19문제

[Python의 기본 19문] 001로

1. 다음 Python으로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. 21-3

```

x, y = 100, 200
print(x == y)

```

답: False

[해설]

- ① x, y = 100, 200
- ② print(x == y)

- ① 변수 x, y를 선언하고 각각 100, 200으로 초기화한다.
- ② x의 값 100과 y의 값 200이 같으면 참(True)을, 같지 않으면 거짓(False)을 출력한다.

결과 False

2. 다음 Python으로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. 22-2

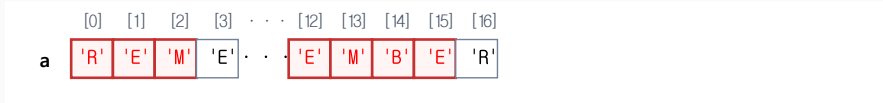
```
a = "REMEMBER NOVEMBER"
b = a[0:3] + a[12:16]
c = "R AND %s" % "STR"
print(b + c)
```

답: **REMEMBER AND STR**

[해설]

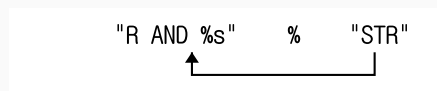
- ① a = "REMEMBER NOVEMBER"
- ② b = a[0:3] + a[12:16]
- ③ c = "R AND %s" % "STR"
- ④ print(b + c)

- ① 변수 a를 선언하고 "REMEMBER NOVEMBER"로 초기화한다.
- ② a에 저장된 문자열의 0부터 2번째(3-1) 위치까지의 문자열과 12부터 15번째(16-1) 위치까지의 문자열을 합쳐 b에 저장한다.



b = "REMEMBE"

- ③ c에 "R AND STR"을 저장한다. %s는 서식 문자열로, % 뒤쪽의 "STR"이 대응된다.



- ④ b와 c에 저장된 문자열을 합쳐 출력한다.

결과 **REMEMBER AND STR**

3. 다음 Python으로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. [20-2](#)

```
asia = {'한국', '중국', '일본'}
asia.add('베트남')
asia.add('중국')
asia.remove('일본')
asia.update({'한국', '홍콩', '태국'})
print(asia)
```

답: **{'한국', '중국', '베트남', '홍콩', '태국'}** (단, 세트 자료형은 순서가 상관없으므로 원소 구성만 일치하면 정답입니다.)

[해설]

세트는 수학에서 배우는 집합(set)과 동일한 역할을 하는 Python의 자료형으로, 중괄호{}를 이용하여 리스트와 같이 다양한 요소들을 저장할 수 있다. 세트는 순서가 정해져 있지 않으며(unordered), 중복된 요소는 저장되지 않는다는 특징이 있다.

- ① asia = {'한국', '중국', '일본'}
- ② asia.add('베트남')
- ③ asia.add('중국')
- ④ asia.remove('일본')
- ⑤ asia.update({'한국', '홍콩', '태국'})
- ⑥ print(asia)

- ① 세트 asia에 '한국', '중국', '일본'의 3개 요소를 저장한다.

asia = {'한국', '중국', '일본'}

- ② 세트 asia에 '베트남'을 추가한다.

asia = {'한국', '중국', '일본', '베트남'}

3 세트 asia에 '중국'을 추가한다. asia에는 이미 '중국' 요소가 있으므로 무시된다.

4 세트 asia에서 '일본'을 제거한다.

```
asia = {'한국', '중국', '베트남'}
```

5 세트 asia에 새로운 세트를 추가하여 확장한다. 새로운 세트 {'한국', '홍콩', '태국'}의 요소 중 '한국'은 이미 asia에 있으므로 무시된다.

```
asia = {'한국', '중국', '베트남', '홍콩', '태국'}
```

6 세트 asia를 출력한다. 세트는 순서가 정해져 있지 않으므로 출력되는 요소들의 순서는 바뀔 수 있다.

결과 {'한국', '중국', '베트남', '홍콩', '태국'}

4. 다음 Python으로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. 23-1

```
asia = {'한국', '중국', '일본'}
asia.add('베트남')
asia.remove('일본')
asia.update({'한국', '홍콩', '태국'})
print(asia)
```

답: {'한국', '중국', '베트남', '홍콩', '태국'} (※ 단, 집합(set) 자료형이므로 요소들의 출력 순서는 달라질 수 있습니다.)

[해설]

이 문제는 파이썬의 **순서가 없고 중복을 허용하지 않는 집합(set) 자료형**의 메서드 활용을 묻는 문제입니다. 각 명령어에 따른 집합 내부 요소의 변화를 추적하는 것이 핵심입니다.

초기 상태	{ '한국', '중국', '일본' }
.add('베트남')	{ '한국', '중국', '일본', '베트남' } → 요소 추가
.remove('일본')	{ '한국', '중국', '베트남' } → 요소 제거
.update({...})	{ '한국', '중국', '베트남', '홍콩', '태국' } → 중복 제외 합집합

단계별 연산 과정 및 특징

❶ 초기 선언 : asia = {'한국', '중국', '일본'}

- 중괄호({})를 사용하여 3개의 문자열 요소를 가진 집합(set) 객체를 생성합니다.

❷ 값 추가 : asia.add('베트남')

- add() 메서드는 집합에 단일 요소를 추가합니다. 집합은 {'한국', '중국', '일본', '베트남'}이 됩니다.

❸ 값 제거 : asia.remove('일본')

- remove() 메서드는 집합 내에서 지정된 요소를 삭제합니다. '일본'이 제거되어 {'한국', '중국', '베트남'}이 남습니다.

❹ 여러 값 추가 : asia.update({'한국', '홍콩', '태국'})

- update() 메서드는 여러 개의 요소를 한 번에 추가(합집합 연산)할 때 사용합니다.

- 이때 기존 집합에 이미 존재하던 '한국'은 **집합의 중복 불허 특징**에 의해 한 번만 저장되며, 새로운 요소인 '홍콩'과 '태국'만 추가됩니다.

❺ 최종 출력 : print(asia)

- 파이썬의 세트(set)는 순서가 없는(Unordered) 자료형이므로 내부 요소의 출력 순서는 실행할 때마다 임의로 변경될 수 있습니다.

- 채점 시에는 요소의 순서와 관계없이 **중괄호와 콤마 서식 기호({, }, ,) 안에 5개의 국가 이름이 정확히 포함**되어 있으면 정답으로 인정됩니다.

결과 { '한국', '중국', '베트남', '홍콩', '태국' }

5. 다음 Python 프로그램과 그 <실행결과>를 분석하여 괄호에 들어갈 알맞은 예약어를 쓰시오.
(<실행결과> 첫 번째 라인의 '5 10'은 입력받은 값에 해당한다.) [23-3](#)

```
x, y = input("x, y의 값을 공백으로 구분하여 입력 : ").(      )(' ')\n\nprint("x의 값 : ", x)\nprint("y의 값 : ", y)
```

<실행결과>

```
x, y의 값을 공백으로 구분하여 입력 : 5 10\nx의 값 : 5\ny의 값 : 10
```

답: **split**

[해설]

- 변수 = input(출력문자)
- 변수1, 변수2, ... = input(출력문자).split(분리문자)

[Python의 활용]

6. 다음 Python으로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. [22-3](#)

```
a = [1, 2, 3, 4, 5]\na = list(map(lambda num : num + 100, a))\nprint(a)
```

답: **[101, 102, 103, 104, 105]**

[해설]

- 1 a = [1, 2, 3, 4, 5]
- 2 a = list(map(lambda num : num + 100, a))
- 3 print(a)

- 1 5개의 요소를 갖는 리스트 a를 선언한다.
- 2 a의 각 요소에 100을 더하는 람다 식을 적용한 후, 100씩 더해진 값들을 다시 리스트로 구성하여 a에 저장한다.

```
a = list(map(lambda num : num + 100, a))
```

- ㉠ lambda num : num + 100 : 인수로 입력된 값에 100을 더하는 람다 식을 정의한다.
- ㉡ map(㉠, a) : 리스트 a의 각 요소를 ㉠에 적용한다.

a [1] 2 3 4 5 → lambda 1 : 1 + 100 → 101 반환

a [1] [2] 3 4 5 → lambda 2 : 2 + 100 → 102 반환

a [1] 2 [3] 4 5 → lambda 3 : 3 + 100 → 103 반환

a [1] 2 3 [4] 5 → lambda 4 : 4 + 100 → 104 반환

→ →

a

1	2	3	4	5
---	---	---	---	---

 → lambda 5 : 5 + 100 → 105 반환

- $\ominus a = \text{list}(\ominus)$: $\ominus$ 의 실행 결과로 반환되는 값들을 리스트로 구성하여 a에 저장한다.

a

101	102	103	104	105
-----	-----	-----	-----	-----

```
a = list(map(lambda num : num + 100, a))
```

- **lambda num : num + 100**: 인수로 입력된 값에 100을 더하는 람다 식을 정의한다.
- **map(람다식, a)**: 리스트 a의 각 요소를 람다 식에 적용하여 연산한다.

a[0]의 '1' → lambda 1 : 1 + 100 → 101 반환
a[1]의 '2' → lambda 2 : 2 + 100 → 102 반환
a[2]의 '3' → lambda 3 : 3 + 100 → 103 반환
a[3]의 '4' → lambda 4 : 4 + 100 → 104 반환
a[4]의 '5' → lambda 5 : 5 + 100 → 105 반환

- ③ a를 출력한다. a는 리스트이므로, 리스트를 선언할 때와 같은 형태와 순서로 출력한다.

결과 [101, 102, 103, 104, 105]

7. 다음 Python으로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. [22-1](#)

```
def func(num1, num2 = 2):
    print('a =', num1, 'b =', num2)
func(20)
```

답: a = 20 b = 2

[해설]

② def func(num1, num2 = 2):
 ③ print('a =', num1, 'b =', num2)
 ① func(20)

func 메소드를 정의하는 부분의 다음 줄인 3번째 줄부터 실행한다.

- ① 20을 인수로 func() 메소드를 호출한다.

- ② func() 메소드의 시작점이다. ①번에서 전달받은 20을 num1이 받는다.

- func() 메소드의 매개 변수는 num1, num2 두 개지만 num2는 메소드 정의 시 초기값이 지정되었다.

- 전달된 인수는 매개 변수에 차례로 전달되므로 인수가 하나만 주어지면 num1이 인수를 전달받고, 두 개의 인수가 주어지면 num1과 num2가 차례대로 인수를 전달받는다.

- ③ a = 와 num1의 값 20, b = 와 num2의 값 2를 차례대로 출력한다.

결과 a = 20 b = 2

8. 다음 Python으로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. [21-1](#)

```
class CharClass:
    a = ['Seoul', 'Kyeonggi', 'Inchon', 'Daejeon', 'Daegu', 'Pusan']
myVar = CharClass()
str01 = ''
for i in myVar.a:
    str01 = str01 + i[0]
print(str01)
```

답: SKIDDP

[해설]

```
class CharClass:
    a = ['Seoul', 'Kyeonggi', 'Inchon', 'Daejeon', 'Daegu', 'Pusan']
1 myVar = CharClass()
2 str01 = ''
3 for i in myVar.a:
4     str01 = str01 + i[0]
5 print(str01)
```

1 CharClass의 객체 변수 myVar를 선언한다.

myVar.a	myVar.a[0]	myVar.a[1]	myVar.a[2]	myVar.a[3]	myVar.a[4]	myVar.a[5]
	'Seoul'	'Kyeonggi'	'Inchon'	'Daejeon'	'Daegu'	'Pusan'

2 빈 문자열 str01을 선언한다. (작은 따옴표 두 개를 이어붙여 빈 문자열 지정)

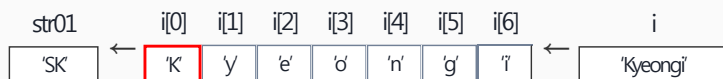
3 객체 변수 myVar의 리스트 a의 요소 수만큼 4번 문장을 반복 수행한다. 리스트 a는 6개의 요소를 가지므로 각 요소를 i에 할당하면서 4번을 6회 수행한다.

4 str01과 i에 저장된 문자열의 첫 번째 글자(i[0])를 더하여 str01에 저장한다. 즉 str01에 저장된 문자 뒤에 i에 저장된 문자열의 첫 번째 글자가 덧붙여진다.

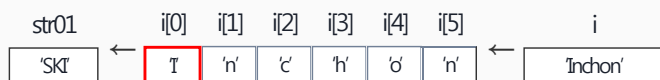
• 1호: i에 myVar[a[0]]이 저장되고 i의 0번째 글자 S가 str01에 저장된다.



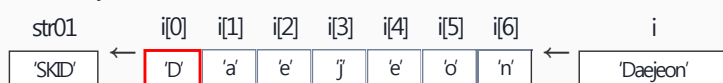
• 2호: i에 myVar[a[1]]이 저장되고 i의 0번째 글자 K가 str01에 더해진다.



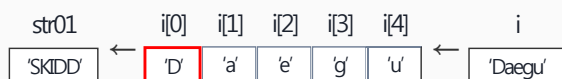
• 3호: i에 myVar[a[2]]가 저장되고 i의 0번째 글자 I가 str01에 더해진다.



• 4호: i에 myVar[a[3]]이 저장되고 i의 0번째 글자 D가 str01에 더해진다.



• 5호: i에 myVar[a[4]]가 저장되고 i의 0번째 글자 D가 str01에 더해진다.



• 6호: i에 myVar[a[5]]가 저장되고 i의 0번째 글자 P가 str01에 더해진다.



결과 SKIDDP

9. 다음은 파이썬(Python) 코드이다. 출력 결과를 쓰시오.

```
i = 0
sum = 0
while i < 10:
    i = i + 1
    if i % 3 == 0:
        sum -= i
    elif i % 3 == 1:
        sum += i
    else:
        sum *= i
print(sum)
```

답: 129

[해설]

본 문제는 반복 제어 변수 i 가 1부터 10까지 1씩 증가하는 동안, 3으로 나눈 나머지 조건($i \% 3$)에 따라 누적 합(sum) 변수를 제어하는 시뮬레이션 과정입니다.

- 변수 i 와 sum 을 0으로 초기화한다.
- while 반복문은 i 가 10보다 작을 때까지 반복을 수행한다.
- while 문이 참일 경우 $i=i+1$ 이 실행되어 i 값을 1 증가시킨다.
- 만약 i 값을 3으로 나눈 나머지가 0인 경우에는 $sum-=i$ 가 실행된다.
- 만약 i 값을 3으로 나눈 나머지가 1인 경우에는 $sum+=i$ 가 실행된다.
- 만약 i 값을 3으로 나눈 나머지가 2인 경우에는 $sum*=i$ 가 실행된다.
- while 반복문이 종료되고 sum 을 화면에 출력한다.

※ 반복문 루프 진행에 따른 변수들의 궤적 변화 추적:

i	i % 3 조건	연산 수행 식	sum 변화 상태
-	초기화	-	0
1	1 (elif 분기)	sum += 1	0 + 1 = 1
2	2 (else 분기)	sum *= 2	1 * 2 = 2
3	0 (if 분기)	sum -= 3	2 - 3 = -1
4	1 (elif 분기)	sum += 4	-1 + 4 = 3
5	2 (else 분기)	sum *= 5	3 * 5 = 15
6	0 (if 분기)	sum -= 6	15 - 6 = 9
7	1 (elif 분기)	sum += 7	9 + 7 = 16
8	2 (else 분기)	sum *= 8	16 * 8 = 128
9	0 (if 분기)	sum -= 9	128 - 9 = 119
10	1 (elif 분기)	sum += 10	119 + 10 = 129

결과 129

10. 다음 Python으로 구현된 프로그램을 분석하여 출력값(①~⑧)에 들어갈 알맞은 값을 쓰시오. 25-3

```
data = [
    [3, 5, 2, 4, 1],
    [4, 5, 1],
    [4, 4, 1, 5, 4],
    [4, 5]
]
result = {}
for index, lis in enumerate(data):
    list_sum = sum(lis)
    list_len = len(lis)
    result[index] = (list_sum, list_len)
print(result)
```

<보기> {0: (①, ②), 1: (③, ④), 2: (⑤, ⑥), 3: (⑦, ⑧)}

답: ① 15, ② 5, ③ 10, ④ 3, ⑤ 18, ⑥ 5, ⑦ 9, ⑧ 2

[해설]

Python의 enumerate 함수와 2차원 리스트 내장 함수 연산

- **enumerate(data)**: 2차원 리스트 'data'를 순회하면서, 각 내부 리스트의 인덱스 번호를 'index'에, 리스트 데이터 자체를 'lis'에 차례대로 바인딩합니다.
- **sum(lis)**: 현재 순회 중인 리스트 요소들의 총합을 구합니다.

- **len(lis)**: 현재 순회 중인 리스트의 요소 개수(길이)를 구합니다.
- **result[index] = (list_sum, list_len)**: 딕셔너리 변수 `result`에 인덱스를 'Key'로, `(총합, 길이)` 튜플을 'Value'로 누적 저장합니다.

반복문 진행 분석

index	list_sum	list_len
0	15	5
1	10	3
2	18	5
3	9	2

1. index = 0 일 때 (lis = [3, 5, 2, 4, 1])

- 총합(sum)은 $3 + 5 + 2 + 4 + 1 = 15$ (①)
- 개수(len)는 요소가 5개이므로 **5** (②)

2. index = 1 일 때 (lis = [4, 5, 1])

- 총합(sum)은 $4 + 5 + 1 = 10$ (③)
- 개수(len)는 요소가 3개이므로 **3** (④)

3. index = 2 일 때 (lis = [4, 4, 1, 5, 4])

- 총합(sum)은 $4 + 4 + 1 + 5 + 4 = 18$ (⑤)
- 개수(len)는 요소가 5개이므로 **5** (⑥)

4. index = 3 일 때 (lis = [4, 5])

- 총합(sum)은 $4 + 5 = 9$ (⑦)
- 개수(len)는 요소가 2개이므로 **2** (⑧)

결과 {0: (15, 5), 1: (10, 3), 2: (18, 5), 3: (9, 2)}

11. 다음 Python으로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. [25-2](#)

```
lst = [1,2,3]
dst = {i : i * 2 for i in lst}
s = set(dst.values( ))
lst[0] = 99
dst[2] = 7
s.add(99)
print(len(s & set(dst.values( ))))
```

답: 2

[해설]

Python의 자료형(List, Dictionary, Set) 및 교집합 연산 메커니즘 분석

- `lst = [1, 2, 3]` : 리스트 변수를 초기화합니다.
- `dst = {i : i * 2 for i in lst}` : 딕셔너리 컴프리헨션에 의해 `lst`의 요소를 가지고 Key:Value 쌍을 만듭니다.
→ `dst = {1: 2, 2: 4, 3: 6}` 이 됩니다.
- `s = set(dst.values())` : `dst`의 값(value)들인 2, 4, 6을 가져와 집합(Set)으로 만듭니다.
→ `s = {2, 4, 6}` 이 됩니다.
- `lst[0] = 99` : 리스트 `lst`의 0번 인덱스 값을 변경합니다. (이 연산은 이미 생성이 끝난 `dst`나 `s`의 값에 아무런 영향을 주지 않습니다.)
- `dst[2] = 7` : 딕셔너리 `dst`에서 **Key가 2인 요소의 Value를 7로 변경**합니다.
→ `dst`는 기존 `{1: 2, 2: 4, 3: 6}`에서 **`{1: 2, 2: 7, 3: 6}`**으로 업데이트됩니다.

- `s.add(99)` : 집합 `s`에 새로운 원소 99를 추가합니다.
→ `s`는 기존 `{2, 4, 6}`에서 `{2, 4, 6, 99}`가 됩니다.

■ 최종 출력문 연산 과정 (print)

1. 현재 집합 `s` : `{2, 4, 6, 99}`
2. 현재 `dst.values()`를 집합으로 변환 : 현재 `dst`의 value는 2, 7, 6 이므로 `set(dst.values()) = {2, 6, 7}`
3. 교집합 연산 (`&`) : 두 집합의 공통 원소를 구합니다.
→ `{2, 4, 6, 99} & {2, 6, 7} = {2, 6}`
4. 길이 구하기 (`len`) : 교집합의 원소 개수는 2개이므로 `len({2, 6})`은 최종적으로 2가 됩니다.

결과 2

12. 다음 Python으로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. 25-1

```
class Node:
    def __init__(self, value):
        self.value = value
        self.children = []

def tree(li):
    nodes = [Node(i) for i in li]
    for i in range(1, len(li)):
        nodes[(i - 1) // 2].children.append(nodes[i])
    return nodes[0]

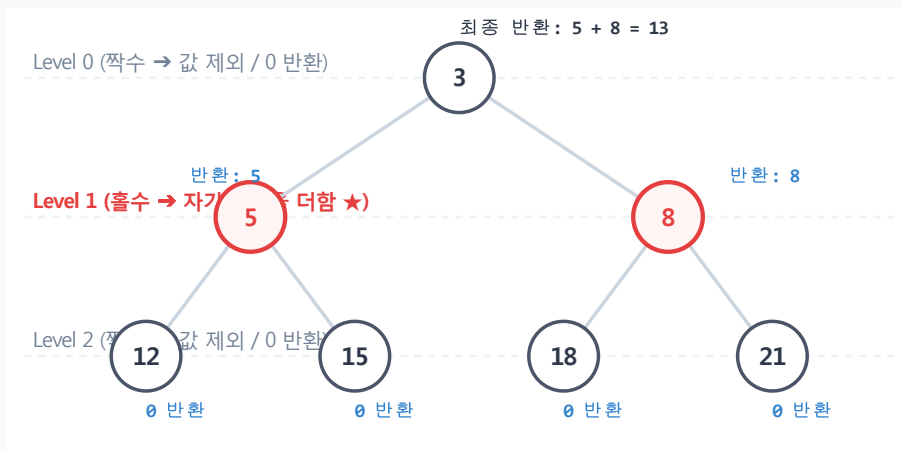
def calc(node, level=0):
    if node is None:
        return 0
    return (node.value if level % 2 == 1 else 0) + sum(calc(n, level + 1) for n in node.children)

li = [3, 5, 8, 12, 15, 18, 21]
root = tree(li)
print(calc(root))
```

답: 13

[해설]

이 문제는 주어진 리스트를 완전 이진 트리(Complete Binary Tree) 구조로 구성한 후, 탐색 깊이(level)가 홀수(Level 1)일 때만 노드의 값을 더하고, 짝수(Level 0, 2)일 때는 0을 반환하는 재귀 함수 알고리즘입니다.



재귀 함수(calc)의 하위 계층별 실행 흐름

① Level 2 노드 탐색 (12, 15, 18, 21)

- level = 2 (짝수)이므로 삼항 연산자 조건에 의해 자기 자신의 값 대신 0을 취합니다.
- 자식 노드가 없으므로 자식들의 합도 0입니다. → 최종적으로 상위 노드에 0을 반환합니다.

② Level 1 노드 연산 (5, 8)

- level = 1 (홀수)이므로 삼항 연산자 조건이 참이 되어 **자기 자신의 값**을 가집니다.
- 노드 5: 자기 자신의 값 5 + 자식 노드들의 반환값(0 + 0) = 5 반환
- 노드 8: 자기 자신의 값 8 + 자식 노드들의 반환값(0 + 0) = 8 반환

③ Level 0 루트 노드 연산 (3)

- level = 0 (짝수)이므로 루트 노드의 값 3 대신 0을 취합니다.
- 여기에 자식 노드들이 상향식으로 반환한 값(5와 8)을 모두 합산합니다.
- 최종 연산 식: $0 + 5 + 8 = 13$

결과 13

13. 다음 Python으로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. 24-3

```
def func(lst):
    for i in range(len(lst) // 2):
        lst[i], lst[-i-1] = lst[-i-1], lst[i]

lst = [1,2,3,4,5,6]
func(lst)
print(sum(lst[:2]) - sum(lst[1::2]))
```

답: 3

[해설]

이 문제는 함수 func(lst)를 통해 리스트의 요소를 **역순(Reverse)으로 뒤집은 후**, 슬라이싱 연산을 이용해 **짝수 인덱스 요소들의 합에서 홀수 인덱스 요소들의 합을 빼는** 문제입니다.



단계별 실행 흐름

① 리스트 역순 스왑 연산 (func)

- len(lst) // 2는 $6 // 2 = 3$ 이므로 변수 i는 0, 1, 2까지 반복합니다.
- 양 끝의 요소를 맞바꾸는 대칭 구조 스왑 변환이 일어나 결과적으로 원래 리스트 [1, 2, 3, 4, 5, 6]이 완전히 뒤집힌 [6, 5, 4, 3, 2, 1]이 됩니다.

② 슬라이싱 합산 연산 (sum)

- lst[:2]: 인덱스 0부터 시작하여 2 칸씩 건너뛰며 요소를 추출합니다. → [6, 4, 2]이므로 합은 12가 됩니다.
- lst[1::2]: 인덱스 1부터 시작하여 2 칸씩 건너뛰며 요소를 추출합니다. → [5, 3, 1]이므로 합은 9가 됩니다.

③ 최종 뺄셈 출력

- sum(lst[:2]) - sum(lst[1::2]) 공식을 대입합니다.

- 최종 연산 식: $12 - 9 = 3$

결과 3

14. 다음 Python으로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. [24-3](#)

```
def func(value):  
    if type(value) == type(100):  
        return 100  
    elif type(value) == type(""):   
        return len(value)  
    else:  
        return 20  
  
a = "100.0"  
b = 100.0  
c = (100, 200)  
print(func(a) + func(b) + func(c))
```

답: 45

[해설]

이 문제는 함수 인자값의 **자료형(Type) 분석을 조건문 분기식 처리**하는 알고리즘입니다. 각 변수의 정확한 데이터 타입을 판별하는 것이 문제 풀이의 핵심입니다.

a = "100.0"	str (문자열)	elif 진입 → len("100.0") → 반환: 5
b = 100.0	float (실수)	else 진입 → 정수/문자 아님 → 반환: 20
c = (100, 200)	tuple (튜플)	else 진입 → 정수/문자 아님 → 반환: 20

조건식 판별 및 결과값 반환 흐름

❶ func(a) 연산 : 변수 a = "100.0"

- type(a)는 문자열형(str)입니다.
- 두 번째 조건문인 elif type(value) == type("") (문자열 타입 비교) 조건을 만족하게 됩니다.
- 따라서 문자열의 길이를 구하는 len("100.0")이 실행되어 5를 반환합니다.

❷ func(b) 연산 : 변수 b = 100.0

- type(b)는 실수형(float)입니다.
- 첫 번째 조건문인 정수형(int), 두 번째 조건문인 문자열형(str) 조건을 둘 다 만족하지 못합니다.
- 최종 예외 분기인 else 구문으로 진입하여 고정값 20을 반환합니다.

❸ func(c) 연산 : 변수 c = (100, 200)

- type(c)는 소괄호로 묶인 데이터 집합인 튜플형(tuple)입니다.
- 변수 b와 동일하게 정수형이나 문자열형이 아니므로 else 구문으로 진입하여 고정값 20을 반환합니다.

❹ 최종 합산 출력

- 각각의 함수 호출 결과 반환값들을 모두 더해 출력문 연산을 마칩니다.
- 최종 연산 식: $5 + 20 + 20 = 45$

결과 45

15. 다음 Python으로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. (단, 출력문의 출력 서식을 준수 하시오.) 24-2

```
def cnt(str, p):
    result = 0;
    for i in range(len(str)):
        sub = str[i:i+len(p)]
        if sub == p:
            result += 1
    return result

str = "abdcabcabca"
p1 = "ca"
p2 = "ab"
print(f'ab{cnt(str, p1)} ca{cnt(str, p2)}')
```

답: ab3 ca2

[해설]

이 문제는 **문자열 슬라이싱(Slicing)을 이용해 특정 패턴(부분 문자열)의 등장 횟수를 카운트**하고, 이를 **f-string(포맷 스트링)** 형식을 사용하여 출력하는 알고리즘입니다.

str = "a b d c a b c a b c a"
index: 0 1 2 3 4 5 6 7 8 9 10

p1 = "ca"

인덱스 3, 6, 9에서 매칭 (3회)

→ cnt(str, p1) = 3

p2 = "ab"

인덱스 0, 4, 7에서 매칭 (3회)

→ cnt(str, p2) = 3

함수 동작 원리 및 출력 흐름

① cnt(str, p) 함수의 메커니즘

- 반복문이 돌면서 str[i:i+len(p)]를 통해 찾고자 하는 패턴 p의 길이만큼 문자열을 잘라냅니다(sub).
- 잘라낸 문자열(sub)이 찾고자 하는 패턴(p)과 일치하면 result를 1씩 증가시킵니다.

② cnt(str, p1) 호출 (p1 = "ca")

- 문자열 "abdcabcabca"에서 연속된 "ca"를 탐색합니다.
- 인덱스 3~4(ca), 6~7(ca), 9~10(ca) 위치에서 총 **3번** 일치하므로 3을 반환합니다.

③ cnt(str, p2) 호출 (p2 = "ab")

- 동일 문자열에서 연속된 "ab"를 탐색합니다.
- 인덱스 0~1(ab), 4~5(ab), 7~8(ab) 위치에서 총 **3번** 일치하므로 3을 반환합니다.

④ f-string 최종 출력

- 출력 포맷: print(f'ab{cnt(str, p1)} ca{cnt(str, p2)}')
- 중괄호 {} 자리에 각각의 반환값인 3과 3이 대입되어 최종 포맷에 맞춰 출력됩니다.
- 최종 출력 결과 문자열: ab3 ca3

	0	1	2	3	4	5	6	7	8	9	10			
str	a	b	d	c	a	b	c	a	b	c	a	p1	p2	
												ca	ab	

p	i	sub=str[i:i+len(p)]	result	p	i	sub=str[i:i+len(p)]	result
ca	0	ab = [0:1]	0	ab	0	ab [0:1]	0
	1	bd = [1:2]			1	bd [1:2]	1
	2	dc = [2:3]			2	dc [2:3]	
	3	ca = [3:4]	1		3	ca [3:4]	

	4	ab = [4:5]			4	ab [4:5]	2
	5	bc = [5:6]			5	bc [5:6]	
	6	ca = [6:7]	2		6	ca [6:7]	
	7	ab = [7:8]			7	ab [7:8]	3
	8	bc = [8:9]			8	bc [8:9]	
	9	ca = [9:10]	3		9	ca [9:10]	
	10	a = [10:11]			10	a [10:11]	

결과 ab3 ca3

16. 다음 Python으로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. [24-1](#)

```
a = [ 'Seoul', 'Kyeonggi', 'Incheon', 'Daejeon', 'Daegu', 'Pusan' ];
str01 = 'S'
for i in a:
    str01 = str01 + i[1]

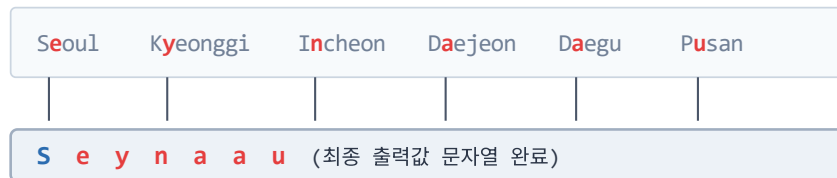
print(str01)
```

답: Seynaau

[해설]

이 문제는 **리스트 요소를 순회하면서 각 문자열의 지정된 인덱스 문자**를 추출해 기존 문자열 뒤에 누적 연결 (Concatenation)하는 알고리즘입니다. 루프가 진행됨에 따라 문자열이 어떻게 제어되고 결합하는지 파악하는 것이 핵심입니다.

초기값 str01 = 'S'



루프 실행 단계별 문자열 변화 흐름

- 시작 초기 상태: str01 = 'S'
- 1회전 (i = 'Seoul'): 추출 문자 'e' → str01 = 'S' + 'e' = 'Se'
- 2회전 (i = 'Kyeonggi'): 추출 문자 'y' → str01 = 'Se' + 'y' = 'Sey'
- 3회전 (i = 'Incheon'): 추출 문자 'n' → str01 = 'Sey' + 'n' = 'Seyn'
- 4회전 (i = 'Daejeon'): 추출 문자 'a' → str01 = 'Seyn' + 'a' = 'Seyna'
- 5회전 (i = 'Daegu'): 추출 문자 'a' → str01 = 'Seyna' + 'a' = 'Seynaa'
- 6회전 (i = 'Pusan'): 추출 문자 'u' → str01 = 'Seynaa' + 'u' = 'Seynaau'
- 최종 출력 결과: Seynaau

결과 Seynaau

17. 다음 Python으로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. [23-2](#)

```

a = "engineer information programming"
b = a[:3]
c = a[4:6]
d = a[29:]
e = b + c + d

print(e)

```

답: engneing

[해설]

이 문제는 **Python의 문자열 슬라이싱(Slicing) 연산 기법**을 활용하여 특정 인덱스 구간의 문자열을 추출하고, 이를 문자열 결합 연산자(+)로 연결하는 알고리즘입니다. 각 슬라이싱 범위가 공백 문자를 포함해 정확히 어느 문자를 가리키는지 판별하는 것이 핵심입니다.

```

e n g i n e e r _ i n f o r m a t i o n _ p r o g r a m m i n g
0 1 2 3 4 5 6 7 8 9 ... 29 30 31

```

b=a[:3] → "e" c=a[4:6] → "n" d=a[29:] → "ing"

e = b + c + d → "eng" + "ne" + "ing" = "engneing"

슬라이싱 구간 분석 및 결합 흐름

① b = a[:3] 연산

- 시작 인덱스가 지정되지 않았으므로 처음(0)부터 시작하여 인덱스 3 직전(2)까지 자릅니다.
- a[0]('e'), a[1]('n'), a[2]('g') 구간이 추출되어 "eng"가 됩니다.

② c = a[4:6] 연산

- 인덱스 4부터 시작하여 인덱스 6 직전(5)까지 자릅니다.
- 문자열 앞부분을 세어보면 a[3]은 '_'이고, a[4]는 'n', a[5]는 'e'입니다.
- 따라서 a[4:6] 구간은 "ne"가 추출됩니다.

③ d = a[29:] 연산

- 인덱스 29부터 시작하여 문자열의 맨 마지막 끝까지 자릅니다.
- 변수 a의 맨 끝 두 글자인 "programming"의 마지막 'i', 'n', 'g' 중 인덱스 29 위치인 'i'부터 끝까지 추출됩니다.
- 따라서 "ing"가 추출됩니다.

④ e = b + c + d 최종 누적 결합

- 앞에서 구해낸 각각의 부분 문자열 조각들을 차례대로 이어 붙입니다.
- "eng" + "ne" + "ing" → 최종 출력 결과: engneing

결과 engneing

18. 다음 Python으로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. [21-2](#)

```

a = 100
result = 0
for i in range(1,3):
    result = a >> i
    result = result + 1
print(result)

```

답: 26

[해설]

for문을 반복 수행할 때마다 result의 값이 누적되는 것이 아니라 새로운 값으로 치환되며, 그 값에 1을 더한다는 것을 염두에 두고 해설을 참고하세요.

```
❶ a = 100
❷ result = 0
❸ for i in range(1,3):
❹     result = a >> i
❺     result = result + 1
❻ print(result)
```

- ❶ 변수 a에 100을 저장한다.
 - ❷ 변수 result에 0을 저장한다.
 - ❸ 반복 변수 i가 1에서 시작하여 1씩 증가하면서 3보다 작은 동안 ❹, ❺번을 반복 수행한다.
 - ❹ >>는 오른쪽 시프트 연산자이므로, a에 저장된 값을 오른쪽으로 i비트 이동시킨 다음 그 값을 result에 저장한다. 정수는 4Byte 이므로 100을 4Byte 2진수로 변환하여 계산하면 된다.
 - ❺ result의 값에 1을 누적시킨다.
- ※ ❸~❺ 반복 과정은 다음과 같다.

반복문 1회 수행(i = 1)

• result = a >> i

- a의 값 100을 4Byte 2진수로 표현하면 다음과 같다.

	32	31	30	29	...	9	8	7	6	5	4	3	2	1
100	0	0	0	0	...	0	0	1	1	0	0	1	0	0
							2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
							128	64	32	16	8	4	2	1

부호 비트

- i는 1이므로 부호를 제외한 전체 비트를 오른쪽으로 1비트 이동시킨다. 양수이므로 패딩 비트(빈자리)에는 0이 채워진다.

	32	31	30	29	...	9	8	7	6	5	4	3	2	1
50	0	0	0	0	...	0	0	0	1	1	0	0	1	0
							2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
							128	64	32	16	8	4	2	1

부호 비트 패딩 비트

- 이동된 값을 10진수로 변환하면 50이다. result에는 50이 저장된다.

• result = result + 1

- result의 값 50에 1을 더하면 51이 된다

반복문 2회 수행(i = 2)

• result = a >> i

- i는 2이므로 a의 값 100을 오른쪽으로 2비트 이동시킨다. 양수이므로 패딩 비트(빈자리)에는 0이 채워진다.

	32	31	30	29	...	9	8	7	6	5	4	3	2	1
25	0	0	0	0	...	0	0	0	0	1	1	0	0	1
							2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
							128	64	32	16	8	4	2	1

부호 비트 패딩 비트

- 이동된 값을 10진수로 변환하면 25다. result에는 25가 저장된다.

• result = result + 1

- result의 값 25에 1을 더하면 result는 26이 된다.

⑥ result의 값 26을 출력한다.

결과 26

19. 다음 Python으로 구현된 프로그램을 분석하여 그 실행 결과를 쓰시오. [20-4](#)

```
lol = [[1, 2, 3], [4, 5], [6, 7, 8, 9]]
print(lol[0])
print(lol[2][1])
for sub in lol:
    for item in sub:
        print(item, end=' ')
    print()
```

답:

[1, 2, 3]

7

1 2 3

4 5

6 7 8 9

[해설]

```
① lol = [[1, 2, 3], [4, 5], [6, 7, 8, 9]]
② print(lol[0])
③ print(lol[2][1])
④ for sub in lol:
⑤     for item in sub:
⑥         print(item, end=' ')
⑦     print()
```

① 다음과 같은 행과 열을 갖는 2차원 리스트 lol이 선언되고 다음과 같이 초기화된다.

리스트 lol	[0][0]	[0][1]	[0][2]	
	[1][0]	[1][1]		
	[2][0]	[2][1]	[2][2]	[2][3]

리스트 lol	1	2	3	
	4	5		
	6	7	8	9

② 리스트 lol의 0번째 행을 출력한다. 2차원 리스트에서 각각의 행은 1차원 리스트이므로 0번째 행의 요소들을 리스트 형태로 출력한다. 이어서 커서를 다음 줄의 처음으로 옮긴다.

결과 [1, 2, 3]

③ lol[2][1]의 값을 출력한 후 커서를 다음 줄의 처음으로 옮긴다.

결과 [1, 2, 3]
7

④ 리스트 lol의 행 수만큼 ⑤~⑦번을 반복 수행한다.

- **sub** : 리스트 lol의 각 행이 일시적으로 저장될 변수를 선언한다. sub는 1차원 리스트로 선언된다.
 - **lol** : 리스트의 이름을 입력한다. lol 리스트가 3행이므로 각 행을 sub에 저장하면서 5~7번을 3회 수행한다.
- 5 리스트 sub의 요소 수만큼 6번을 반복 수행한다.
- **item** : 리스트 sub의 각 요소가 일시적으로 저장될 변수를 선언한다.
 - **sub** : 리스트의 이름을 입력한다. sub 리스트가 차례로 3개, 2개, 4개의 요소를 가지므로 각 요소를 item에 저장하면서 6번을 3회, 2회, 4회 수행한다.
- 6 item의 값을 출력하고 공백을 한 칸 띄운다.
- 7 커서를 다음 줄의 처음으로 옮긴다.

※ 반복문 실행에 따른 변수들의 값의 변화는 다음과 같다.

sub[]	item	출력
[1, 2, 3]	1	1 2 3
	2	
	3	
[4, 5]	4	1 2 3
	5	4 5
[6, 7, 8, 9]	6	1 2 3
	7	4 5
	8	6 7 8 9
	9	

```

[1, 2, 3]
7
결과 1 2 3
      4 5
      6 7 8 9

```

수제비

1. 다음은 JAVA 코드이다. 출력 결과를 쓰시오.

```

class Soojebi {
    public static void main(String[] args) {
        int[][] arr = new int[3][3];
        init(arr);
        hourGlass(arr);
        arrayPrint(arr);
    }

    // 배열을 0으로 초기화하는 메서드
    public static void init(int arr[][]){
        for (int i = 0; i < arr.length; i++) {
            for (int j = 0; j < arr[0].length; j++) {
                arr[i][j] = 0;
            }
        }
    }
}

```

```

// 특정 규칙(모래시계 형태)으로 값을 대입하는 메서드
public static void hourGlass(int arr[][]) {
    int v = 0;
    for (int i = 0; i < arr.length; i++)
        // ★ 중요: j가 0이 아니라 i부터 시작합니다.
        for (int j = i; j < arr[0].length; j++)
            arr[i][j] = ++v; // 전위 증감 연산자: v를 1 증가시킨 후 대입
}

// 배열을 출력하는 메서드
public static void arrayPrint(int arr[][]) {
    for (int i = 0; i < arr.length; i++) {
        for (int j = 0; j < arr[0].length; j++) {
            if (arr[i][j] == 0) {
                System.out.print(" "); // 0이면 공백 출력
            } else {
                System.out.print(arr[i][j]); // 0이 아니면 숫자 출력
            }
        }
        System.out.println(""); // 줄바꿈
    }
}
}

```

답: 123
45
6

[해설]

핵심이 되는 hourGlass 메서드의 동작을 뜯어보면 왜 이런 결과가 나오는지 쉽게 이해할 수 있습니다. 3x3 배열은 처음에 모두 0으로 채워져 있습니다.

① \$i = 0\$ 일 때 (첫 번째 줄)

j는 i부터 시작하므로 j = 0, 1, 2 순서로 반복합니다.

++v에 의해 v값이 1씩 증가하며 배열에 들어갑니다.

arr[0][0] = 1

arr[0][1] = 2

arr[0][2] = 3

1행 상태: [1, 2, 3]

② \$i = 1\$ 일 때 (두 번째 줄)

j는 i부터 시작하므로 j = 1, 2 순서로 반복합니다. (j = 0인 칸은 건너뛴다)

arr[1][1] = 4

arr[1][2] = 5

2행 상태: [0, 4, 5] (첫 칸은 초기값 0 그대로 유지)

③ \$i = 2\$ 일 때 (세 번째 줄)

j는 i부터 시작하므로 j = 2만 실행됩니다. (j = 0, 1인 칸은 건너뛴다)

arr[2][2] = 6

3행 상태: [0, 0, 6] (앞의 두 칸은 초기값 0 그대로 유지)

arrayPrint에서의 최종 필터링

출력할 때 `arr[i][j] == 0`이면 공백(" ")을 출력하고, 그렇지 않으면 숫자를 출력하라고 되어 있습니다.따라서 0이 있던 자리가 빈 칸으로 바뀌면서 시각적으로 오른쪽으로 정렬된 삼각형(코드명인 모래시계의 일부분) 형태로 출력이 완성됩니다.